Tema 2: Procesos

SISTEMAS OPERATIVOS

Índice

1. Introducción
2. Conceptos fundamentales
3. Implementación de procesos
4. Hilos (Threads)
5. Planificación de la CPU

Bibliografía

W. Stalling. **Sistemas Operativos**. 5ª Edición.

Capítulos 3, 4, 9 y 10.

Objetivos

LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- Define the term *process* and explain the relationship between processes and process control blocks.
- Explain the concept of a process state and discuss the state transitions the processes undergo.
- List and describe the purpose of the data structures and data structure elements used by an OS to manage processes.
- Assess the requirements for process control by the OS.
- Understand the issues involved in the execution of OS code.
- Describe the process management scheme for UNIX SVR4.

1. Definir que es un proceso
2. Explicar el conjunto de estados típicos desde su creación hasta su finalización
3. Ver las estructuras en memoria de un proceso y la información que guarda el SO
4. Definir que es un Hilo
5. Planificación: diferentes algoritmos

Introducción

- The OS must interleave the execution of multiple processes, to maximize processor utilization while providing reasonable response time.
- The OS must allocate resources to processes in conformance with a specific policy (e.g., certain functions or applications are of higher priority) while at the same time avoiding deadlock.[1]
- The OS may be required to support interprocess communication and user creation of processes, both of which may aid in the structuring of applications.

- Procesamiento concurrente:
  - Multiprogramación
  - Multiprocesamiento
  - Procesamiento distribuido
- Servicios del SO
  - Ejecución concurrente
  - Sincronización de procesos
  - Comunicación entre procesos

Los OS basados en Multiprogramación, Multiprocesamiento, Procesamiento distribuido usan el concepto de proceso.

- El OS se encarga de la **ejecución concurrente** de tareas para maximizar la utilización del procesado.
- El OS se encarga de gestionar que diferentes recursos se puedan usar de forma eficiente y sin bloqueos (**Sincronización de procesos**)
- El OS tiene que permitir la **creación y la comunicación entre procesos**

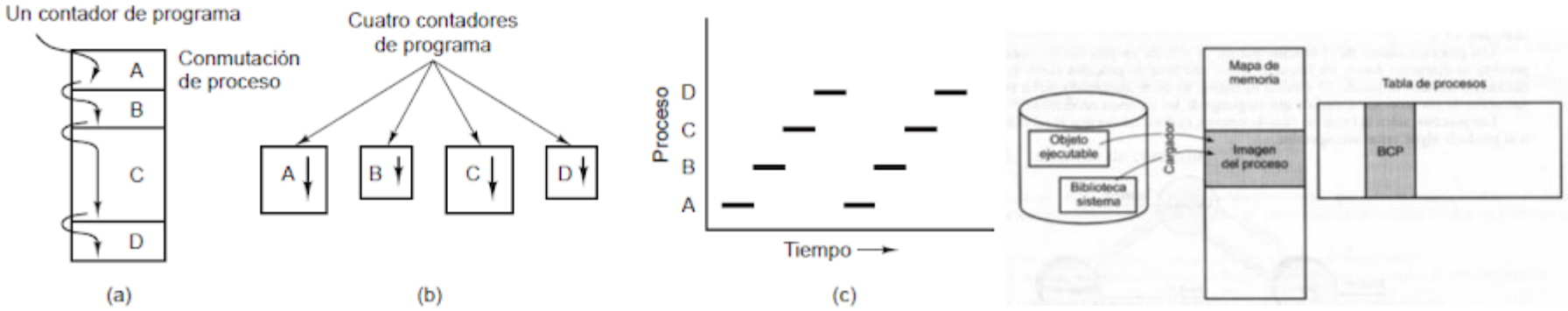


Figura 2-1. (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

Introducción

- The OS must interleave the execution of multiple processes, to maximize processor utilization while providing reasonable response time.
- The OS must allocate resources to processes in conformance with a specific policy (e.g., certain functions or applications are of higher priority) while at the same time avoiding deadlock.[1]
- The OS may be required to support interprocess communication and user creation of processes, both of which may aid in the structuring of applications.

- Procesamiento concurrente:
  - Multiprogramación
  - Multiprocesamiento
  - Procesamiento distribuido
- Servicios del SO
  - Ejecución concurrente
  - Sincronización de procesos
  - Comunicación entre procesos

Los OS basados en Multiprogramación, Multiprocesamiento, Procesamiento distribuido usan el concepto de proceso.

- El OS se encarga de la **ejecución concurrente** de tareas para maximizar la utilización del procesado.
- El OS se encarga de gestionar que diferentes recursos se puedan usar de forma eficiente y sin bloqueos (**Sincronización de procesos**)
- El OS tiene que permitir la **creación y la comunicación entre procesos**

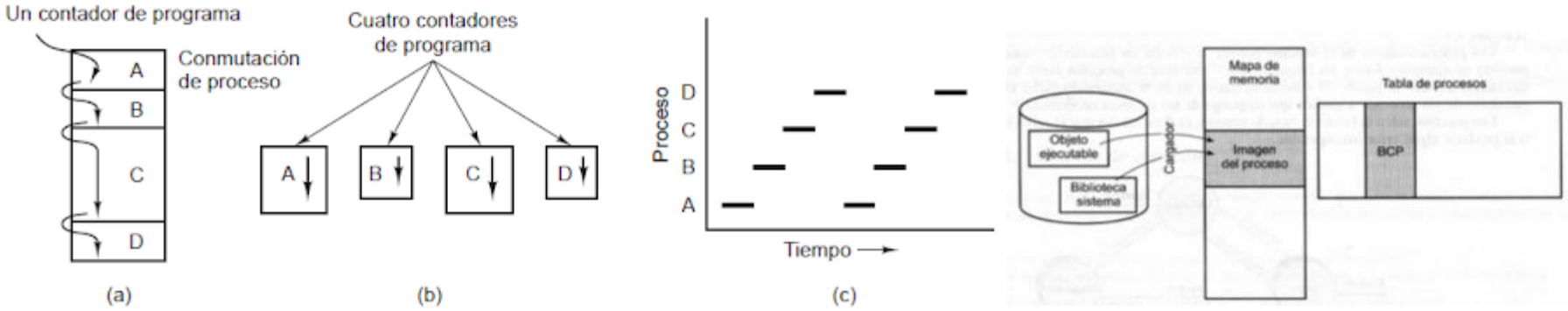


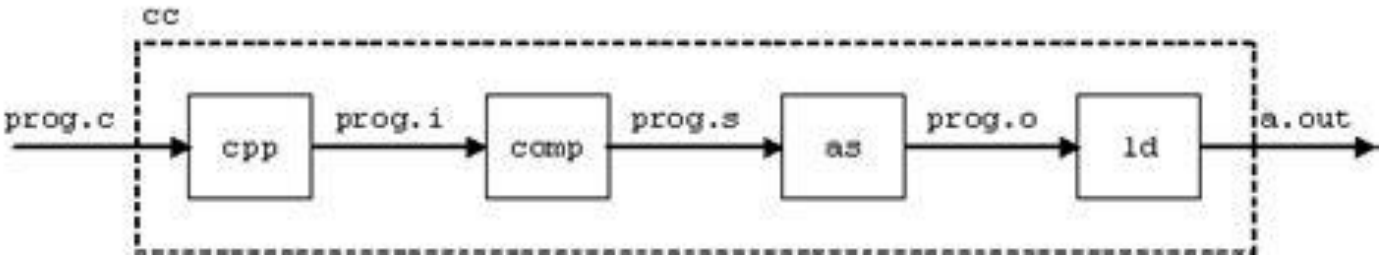
Figura 2-1. (a) Multiprogramación de cuatro programas. (b) Modelo conceptual de cuatro procesos secuenciales independientes. (c) Sólo hay un programa activo a la vez.

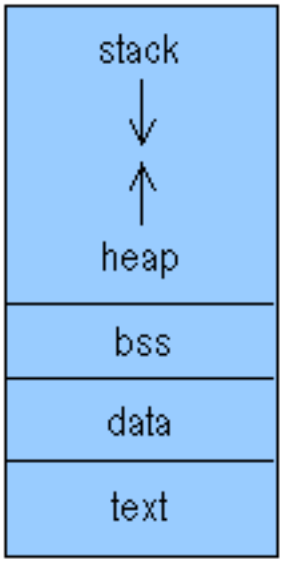
¿Qué es un proceso?

¿Qué es un programa?

Un programa es un conjunto de instrucciones de código

- Procesadores de Lenguaje



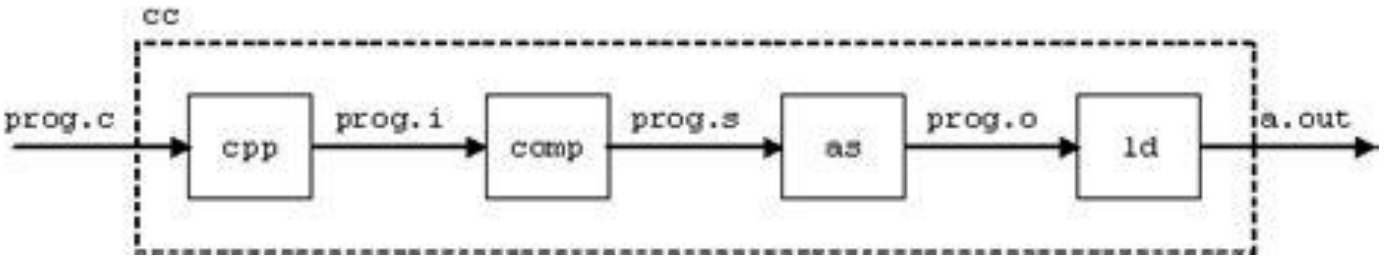


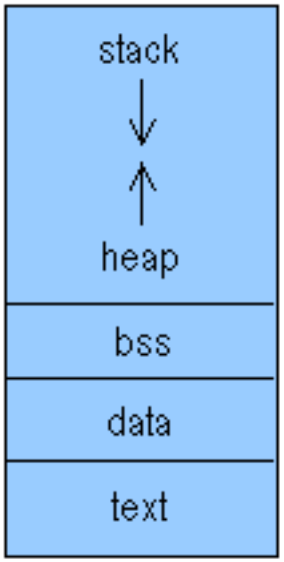
SRC PRE ASM OBJ ELF

¿Qué es un programa?

Un programa es un conjunto de instrucciones de código

- Procesadores de Lenguaje





SRC PRE ASM OBJ ELF

¿Programa = Proceso?

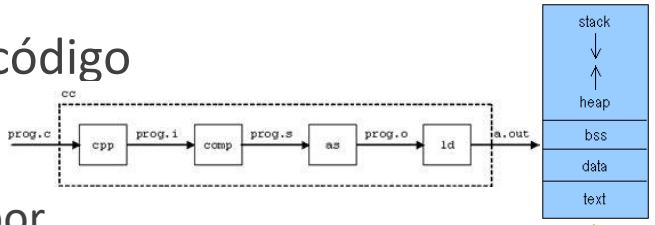
- Proceso: *Programa en ejecución*
- Proceso ≠ programa

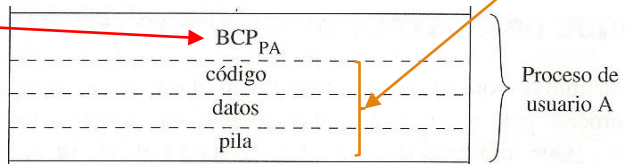
Proceso vs programa

Un programa es un conjunto de instrucciones de código

Proceso ≠ programa → Programa en ejecución

- Un proceso es una entidad dinámica representada por
  - ***bloque de control de procesos (PCB)***
    - el valor de un contador de programa (PC)
    - el contenido de los registros del procesador
  - una pila
  - el propio código
  - una sección destinada a almacenar datos





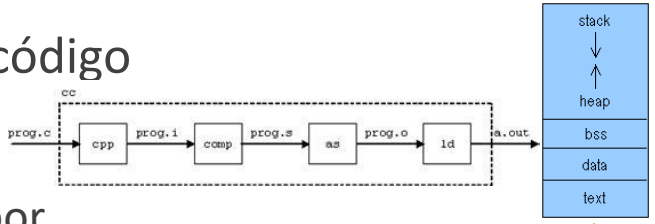
- Proceso: *Programa en ejecución*
- Proceso ≠ programa

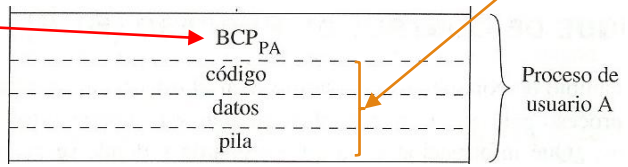
Proceso vs programa

Un programa es un conjunto de instrucciones de código

Proceso ≠ programa → Programa en ejecución

- Un proceso es una entidad dinámica representada por
  - ***bloque de control de procesos (PCB)***
    - el valor de un contador de programa (PC)
    - el contenido de los registros del procesador
  - una pila
  - el propio código
  - una sección destinada a almacenar datos

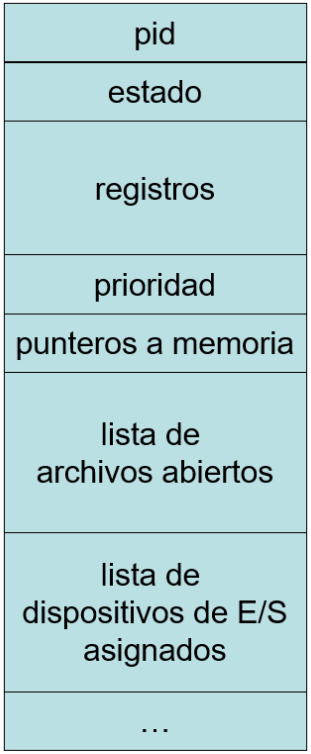




*El **bloque de control de procesos (PCB)** es la estructura de datos del sistema que mantiene toda la información sobre un proceso*

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.



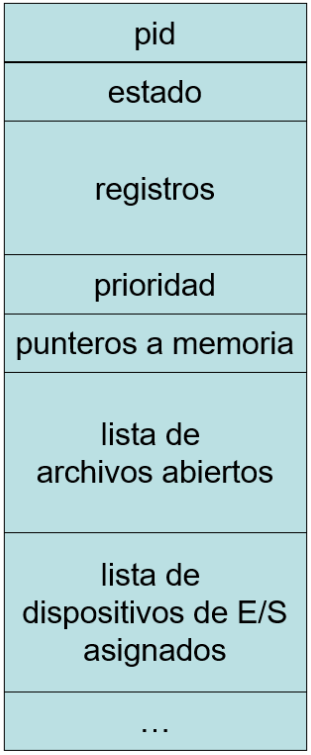
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)



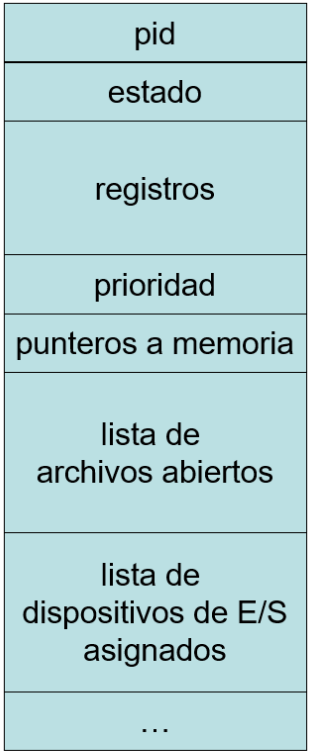
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**



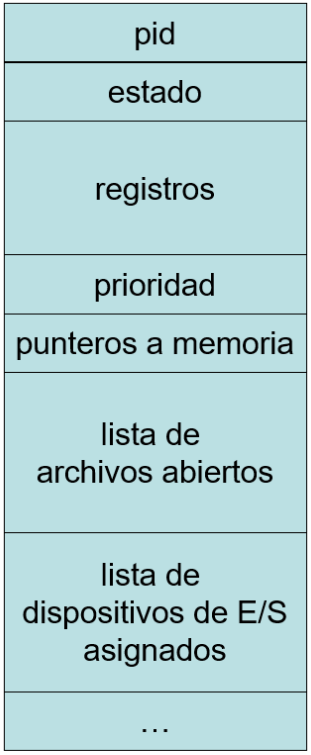
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …



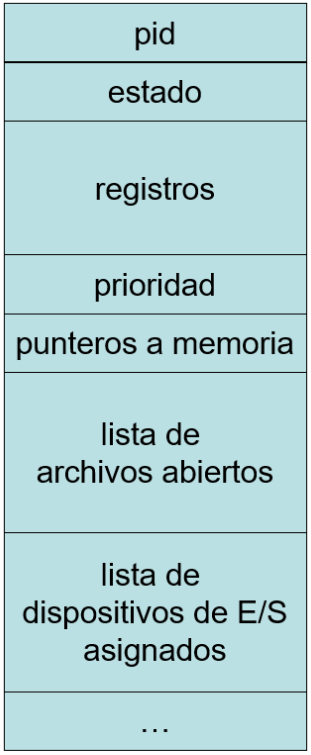
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …
- Información de planificación de la CPU
  - prioridad, …



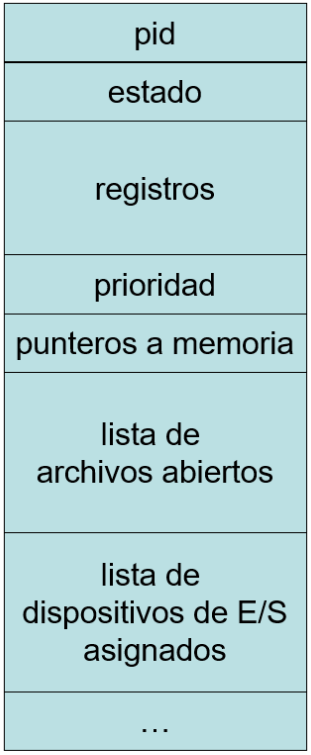
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …
- Información de planificación de la CPU
  - prioridad, …
- Información de gestión de memoria
  - punteros, tablas, registros, …



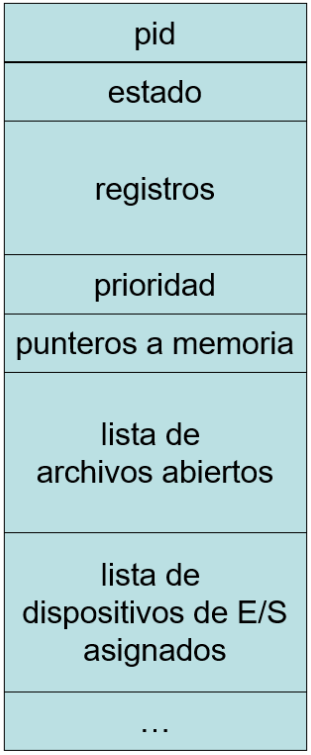
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …
- Información de planificación de la CPU
  - prioridad, …
- Información de gestión de memoria
  - punteros, tablas, registros, …
- Información contable
  - tiempo consumido, límites de tiempo, …



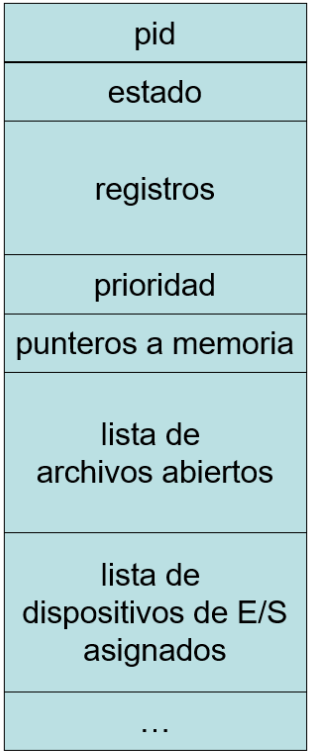
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …
- Información de planificación de la CPU
  - prioridad, …
- Información de gestión de memoria
  - punteros, tablas, registros, …
- Información contable
  - tiempo consumido, límites de tiempo, …
- Información de estado de E/S
  - lista de dispositivos de E/S, lista de archivos abiertos, …



PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …
- Información de planificación de la CPU
  - prioridad, …
- Información de gestión de memoria
  - punteros, tablas, registros, …
- Información contable
  - tiempo consumido, límites de tiempo, …
- Información de estado de E/S
  - lista de dispositivos de E/S, lista de archivos abiertos, …



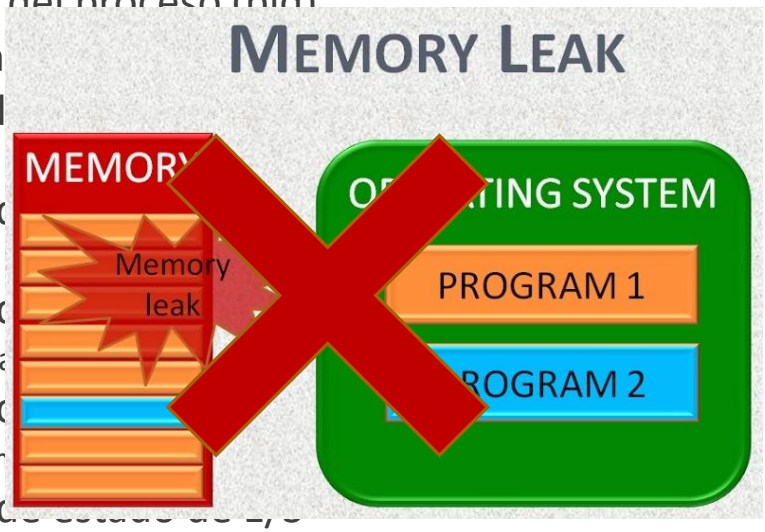
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un**
- Registros de l
  - CP, flags, …
- Información d
  - prioridad, …
- Información d
  - punteros, tabla
- Información d
  - tiempo consur
- Información d
  - lista de dispositivos de E/S, lista de archivos abiertos, …



Descripción completa y detallada

pid
estado
registros
prioridad
punteros a memoria
lista de archivos abiertos
lista de dispositivos de E/S asignados
…

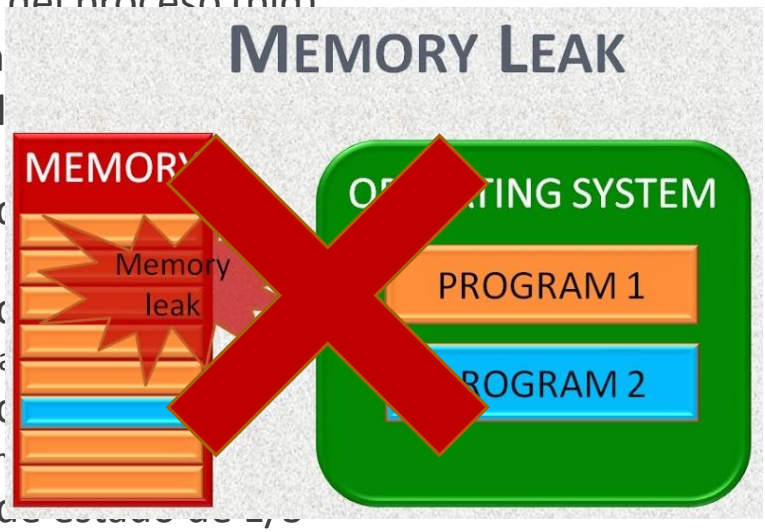
PCB de un proceso

Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

- Identificador del proceso (pid)
- **Estado de un**
- Registros de l
  - CP, flags, …
- Información
  - prioridad, …
- Información
  - punteros, tabla
- Información
  - tiempo consur
- Información
  - lista de dispositivos de E/S, lista de archivos abiertos, …



Descripción completa y detallada

pid
estado
registros
prioridad
punteros a memoria
lista de archivos abiertos
lista de dispositivos de E/S asignados
…

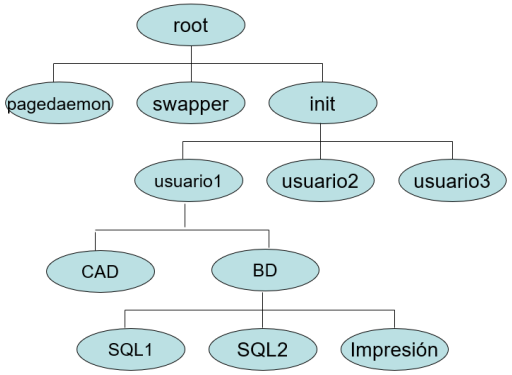
PCB de un proceso

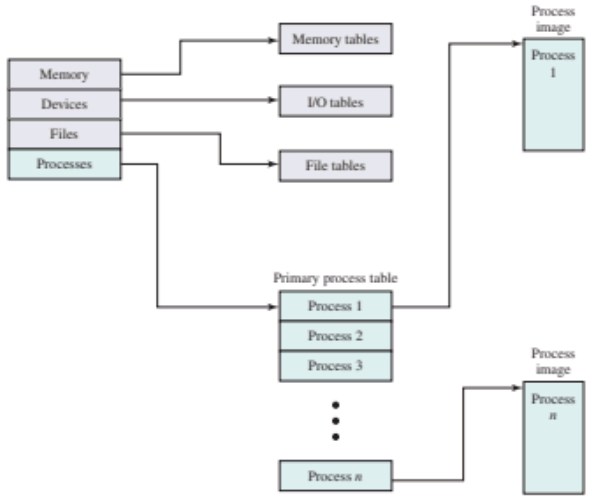
¿Cómo gestiona el SO todos los procesos?

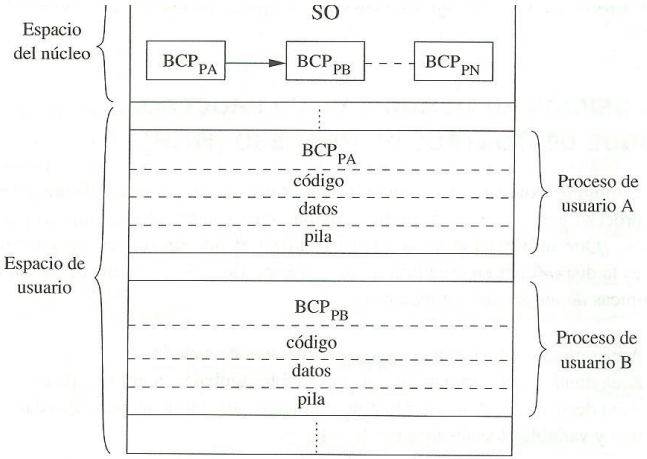
Radiografía en memoria

Hay dos zona en la memoria:

- **Espacio del núcleo**: donde se encuentra el código y las estructuras de datos del propio Sistema Operativo
  - Tablas de memoria (trazar bloques occ. Memorias)
  - Tablas de E/S (manejo de canales y el estado)
  - Tablas de Archivos
  - Tablas de Procesos
- **Espacio de usuario**: donde se encuentran los procesos de usuarios







Bloque de control de proceso

El PCB de cada proceso se crea cuando se crea un proceso y se destruye cuando el proceso termina.

Información que contiene:

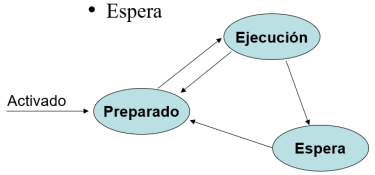
- Identificador del proceso (pid)
- **Estado de un proceso**
- Registros de la CPU
  - CP, flags, …
- Información de planificación de la CPU
  - prioridad, …
- Información de gestión de memoria
  - punteros, tablas, registros, …
- Información contable
  - tiempo consumido, límites de tiempo, …
- Información de estado de E/S
  - lista de dispositivos de E/S, lista de archivos abiertos, …



PCB de un proceso

Estados de un proceso

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



Al ser algo dinámico, un proceso pasa por diferentes estados que son gestionados por el Sistema Operativo. Los procesos se ejecutan hasta que:

- Interrupciones externas:
  - necesitan hacer E/S
  - o se les acaba la asignación temporal (quantum)
- Interrupciones internas:
  - se bloquean (wait)

Nuevo: se identifica con el nacimiento de un proceso, pero aún no dispone de los recursos mínimos para poder empezar a ejecutarse

Preparado:

- ya dispone de todo lo necesario para ejecutarse

Ejecución:

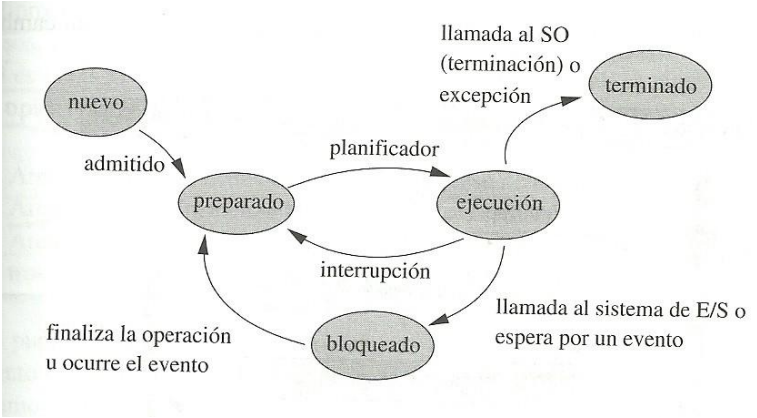
- en uso de la CPU

Bloqueado:

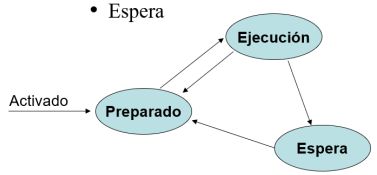
- el proceso espera la ocurrencia de algún evento

Terminado:

- el proceso no ejecuta más instrucciones



- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



Estados de un proceso

Al ser algo dinámico, un proceso pasa por diferentes estados que son gestionados por el Sistema Operativo. Los procesos se ejecutan hasta que:

- Interrupciones externas:
  - necesitan hacer E/S
  - o se les acaba la asignación temporal (quantum)
- Interrupciones internas:
  - se bloquean (wait)

Nuevo: se identifica con el nacimiento de un proceso, pero aún no dispone de los recursos mínimos para poder empezar a ejecutarse

Preparado:

- ya dispone de todo lo necesario para ejecutarse

Ejecución:

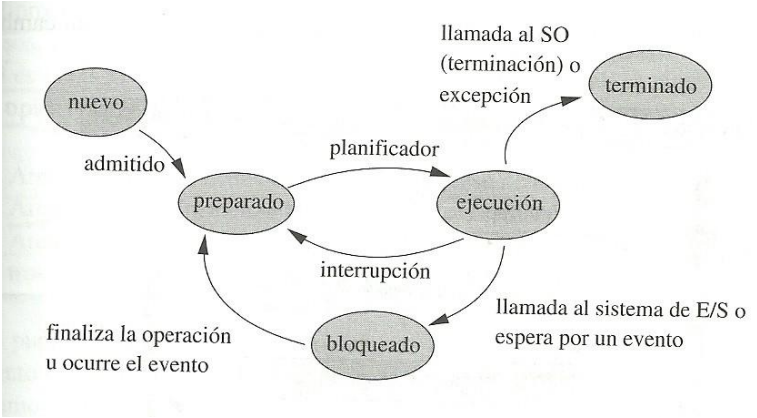
- en uso de la CPU

Bloqueado:

- el proceso espera la ocurrencia de algún evento

Terminado:

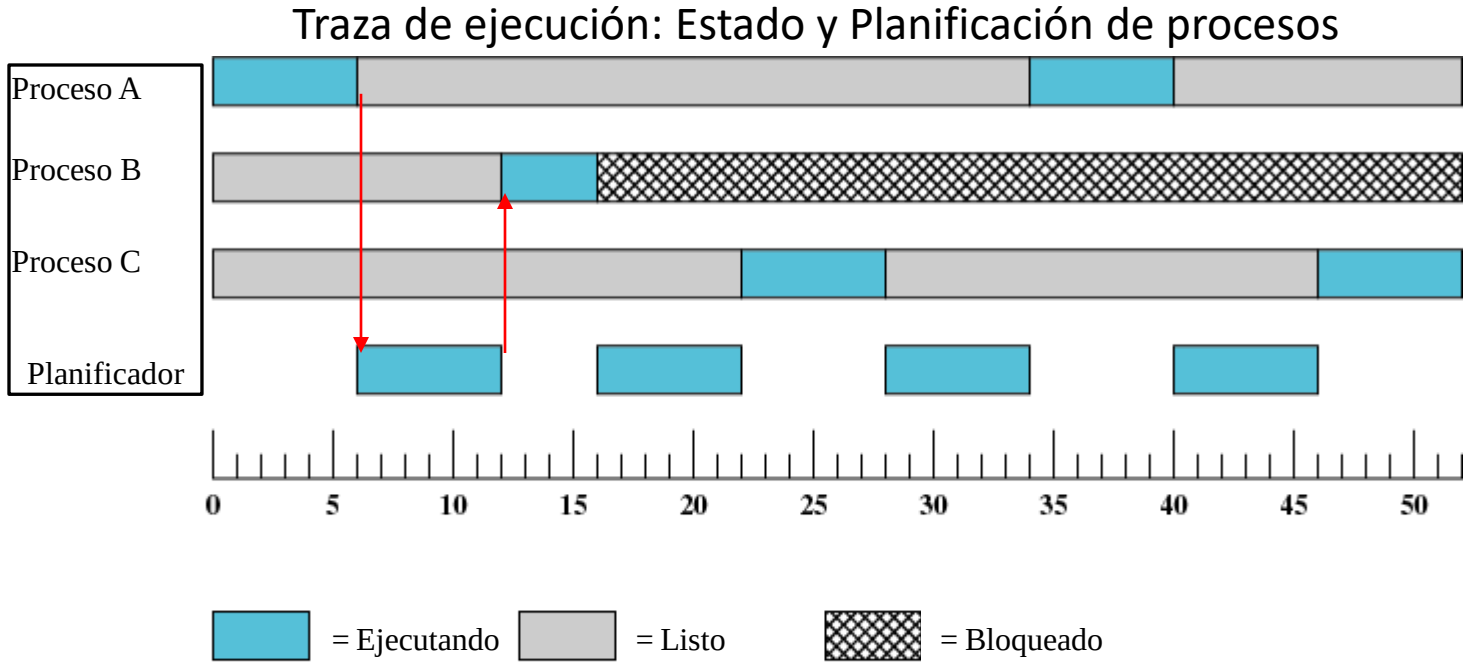
- el proceso no ejecuta más instrucciones



¿Cuál es la utilidad de los estados?

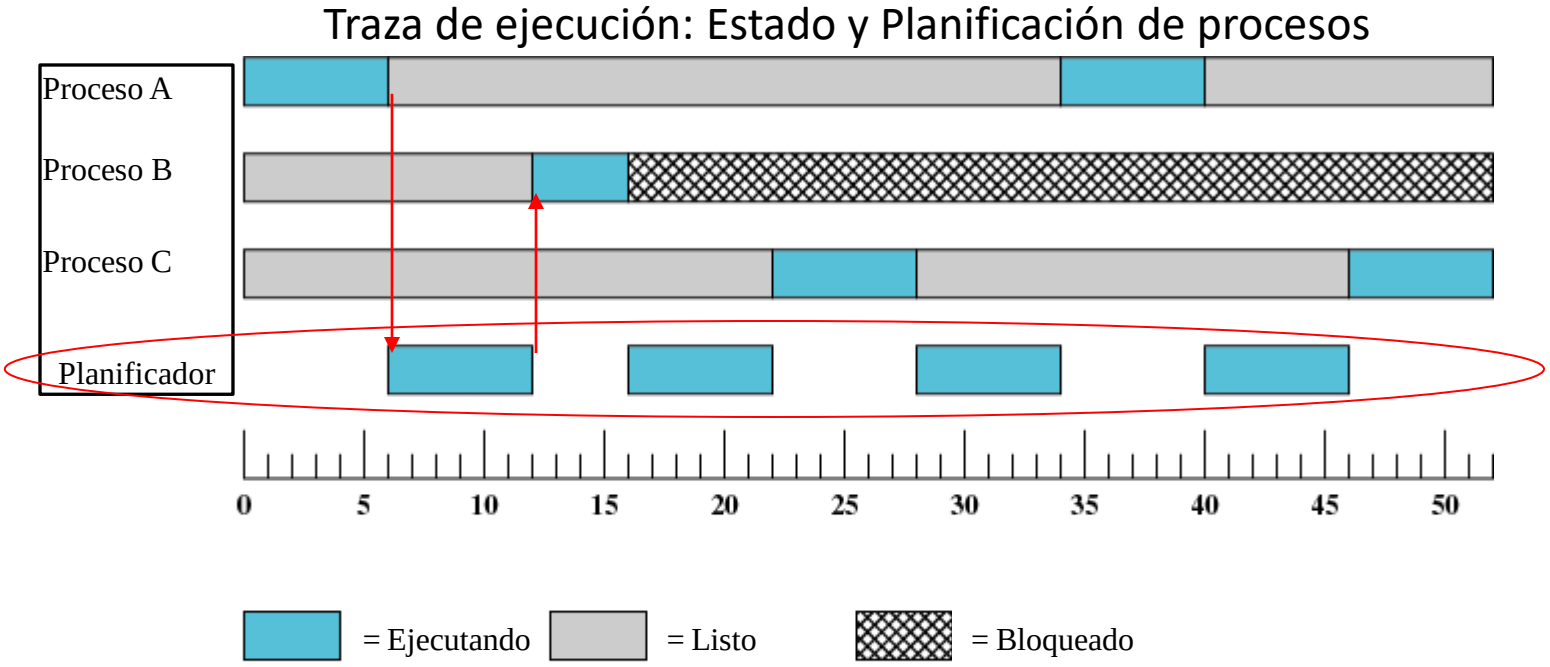
- Relación entre procesos:
  - Independientes
  - Cooperativos
  - Competitivos

Ciclo de vida de los procesos



- Relación entre procesos:
  - Independientes
  - Cooperativos
  - Competitivos

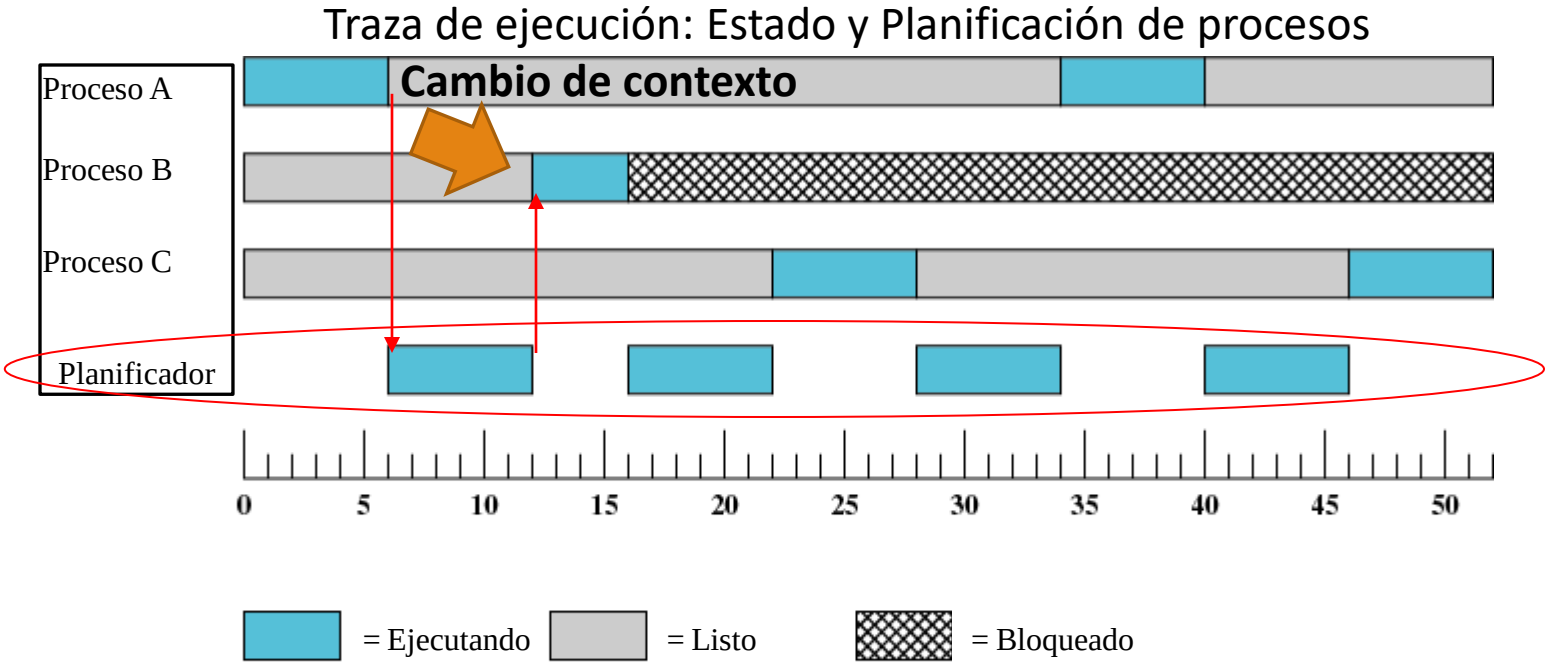
Ciclo de vida de los procesos



El módulo responsable de aplicar las diferentes políticas es el **Planificador** (**Scheduler**)

- Relación entre procesos:
  - Independientes
  - Cooperativos
  - Competitivos

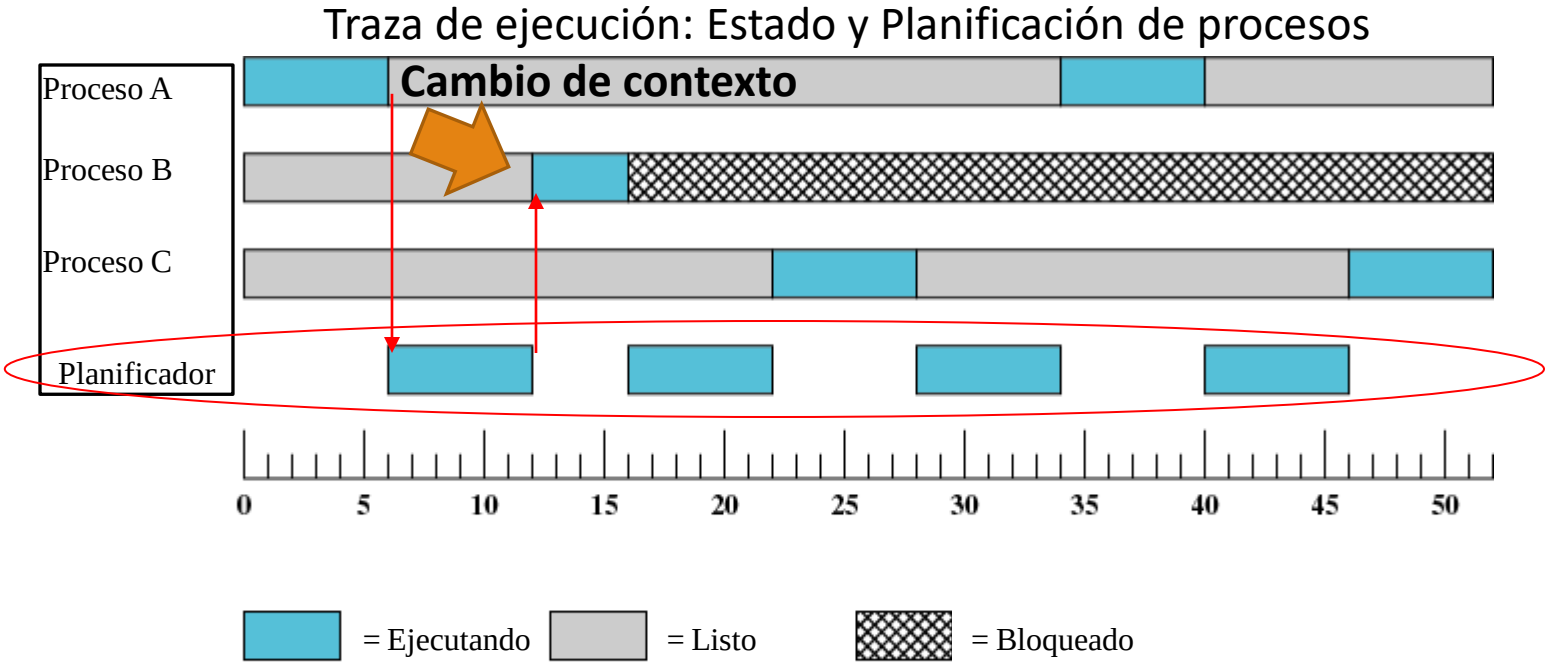
Ciclo de vida de los procesos



El módulo responsable de aplicar las diferentes políticas es el **Planificador** (**Scheduler**)
Cuando **el Sistema Operativo le quita la CPU** un proceso se denomina **Cambio de Contexto**

- Relación entre procesos:
  - Independientes
  - Cooperativos
  - Competitivos

Ciclo de vida de los procesos



El módulo responsable de aplicar las diferentes políticas es el **Planificador** (**Scheduler**)
Cuando **el Sistema Operativo le quita la CPU** un proceso se denomina **Cambio de Contexto**

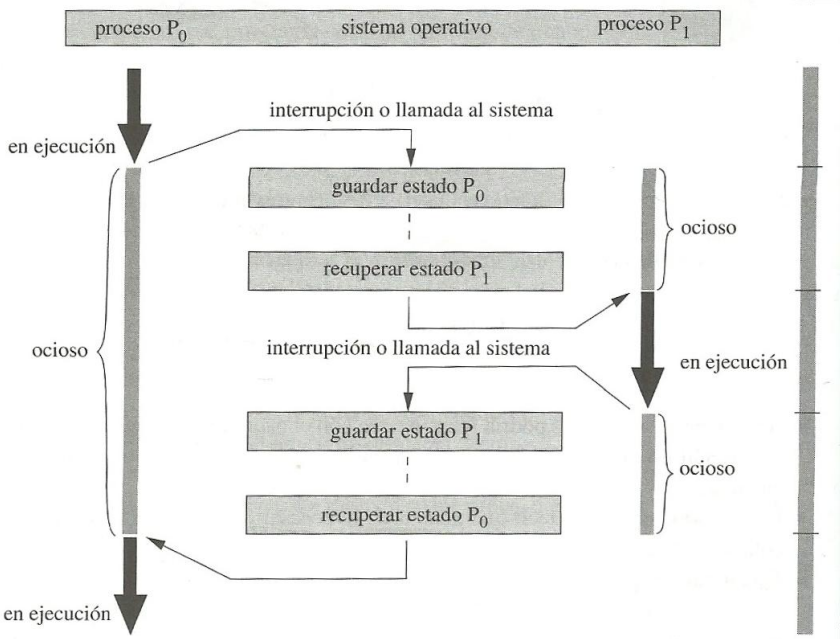
¿Qué es lo que hace un cambio de contexto?

- Cambio de contexto de un proceso
  - Motivos que lo provocan
    - Finalización normal de un proceso en ejecución
    - Espera de un proceso al realizar una operación de E/S
    - Existencia de una interrupción

Cambio de contexto

El SO es el encargado de desalojar un proceso de la CPU y asignársela a otro proceso.

- Finalización normal de la ejecución
- Espera al realizar una operación de E/S
- Existencia de una interrupción



- Cambio de contexto de un proceso
  - Motivos que lo provocan
    - Finalización normal de un proceso en ejecución
    - Espera de un proceso al realizar una operación de E/S
    - Existencia de una interrupción

Cambio de contexto

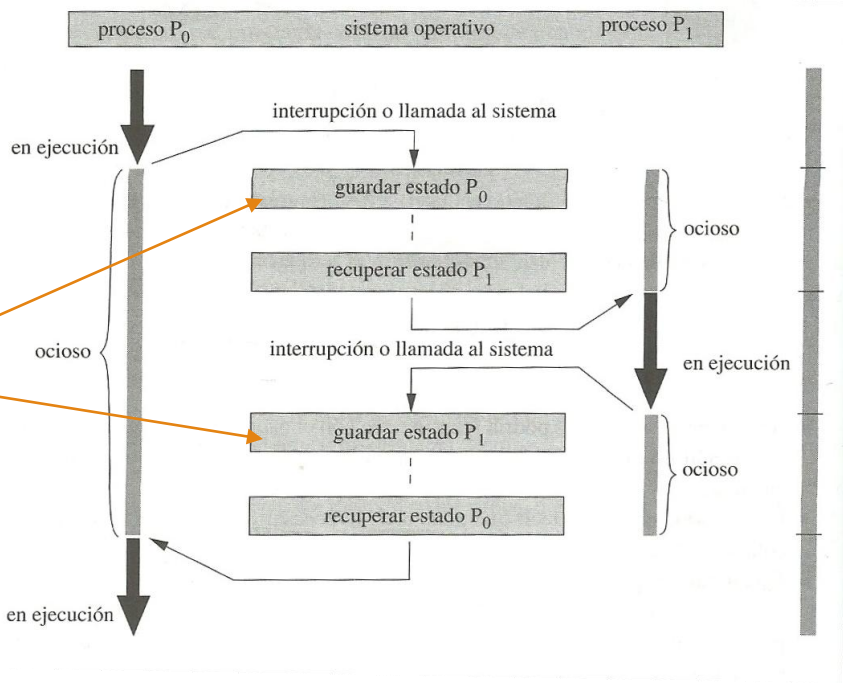
El SO es el encargado de desalojar un proceso de la CPU y asignársela a otro proceso.

- Finalización normal de la ejecución
- Espera al realizar una operación de E/S
- Existencia de una interrupción

El cambio de contexto requiere guardar el estado del proceso desalojado:

- el contador de programa,
- los registros del procesador,
- ficheros abiertos,
- certificados de seguridad,
- etc..

Restaurar el contexto del nuevo proceso desde su PCB



- Cambio de contexto de un proceso
  - Motivos que lo provocan
    - Finalización normal de un proceso en ejecución
    - Espera de un proceso al realizar una operación de E/S
    - Existencia de una interrupción

Cambio de contexto

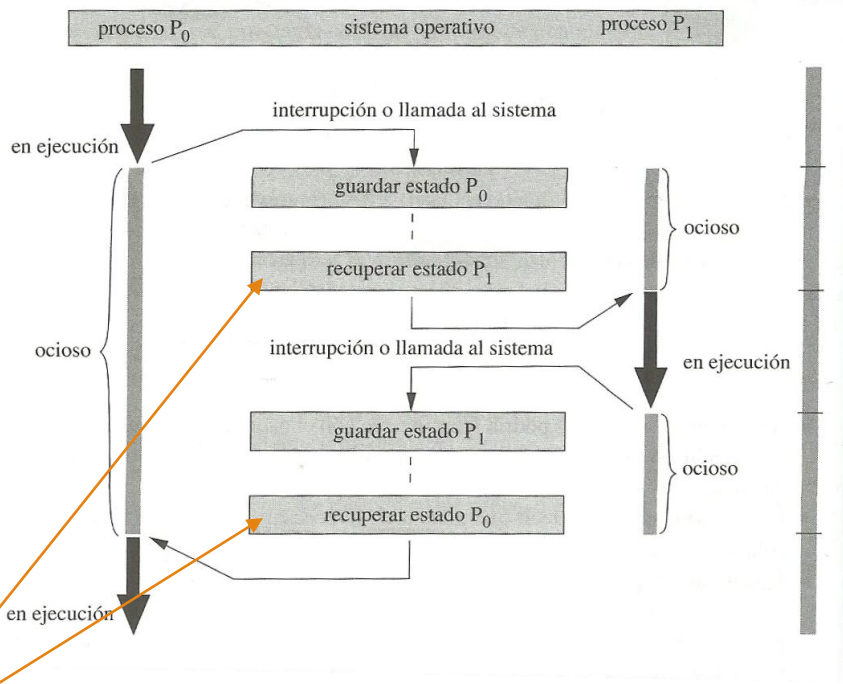
El SO es el encargado de desalojar un proceso de la CPU y asignársela a otro proceso.

- Finalización normal de la ejecución
- Espera al realizar una operación de E/S
- Existencia de una interrupción

El cambio de contexto requiere guardar el estado del proceso desalojado:

- el contador de programa,
- los registros del procesador,
- ficheros abiertos,
- certificados de seguridad,
- etc..

Restaurar el contexto del nuevo proceso desde su PCB



Cambio de contexto

- Cambio de contexto de un proceso
  - Motivos que lo provocan
    - Finalización normal de un proceso en ejecución
    - Espera de un proceso al realizar una operación de E/S
    - Existencia de una interrupción

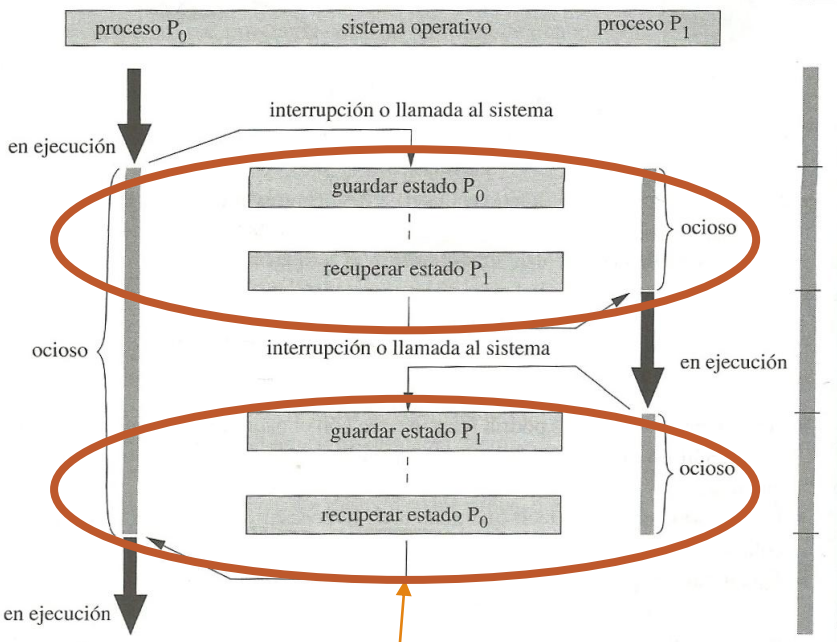
El SO es el encargado de desalojar un proceso de la CPU y asignársela a otro proceso.

- Finalización normal de la ejecución
- Espera al realizar una operación de E/S
- Existencia de una interrupción

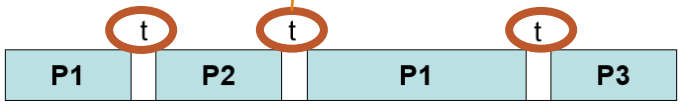
El cambio de contexto requiere guardar el estado del proceso desalojado:

- el contador de programa,
- los registros del procesador,
- ficheros abiertos,
- certificados de seguridad,
- etc..

Restaurar el contexto del nuevo proceso desde su PCB

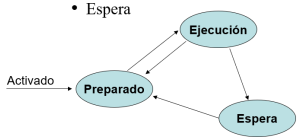


- Reducen la utilización de la CPU



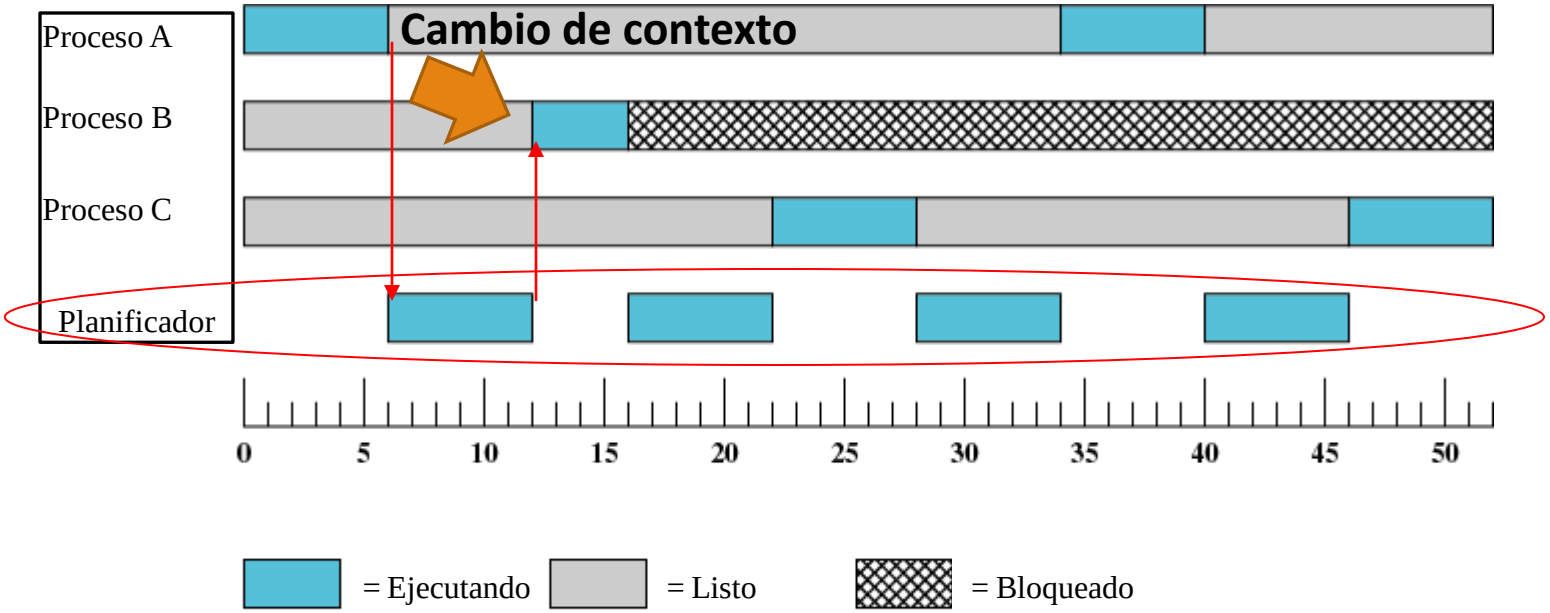
$$\text{Utilización} = \frac{T(P1) + T(P2) + T(P3)}{T(P1) + T(P2) + T(P3) + 3 \cdot t}$$

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

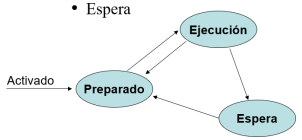


Ciclo de vida de los procesos

Traza de ejecución: Estado y Planificación de procesos

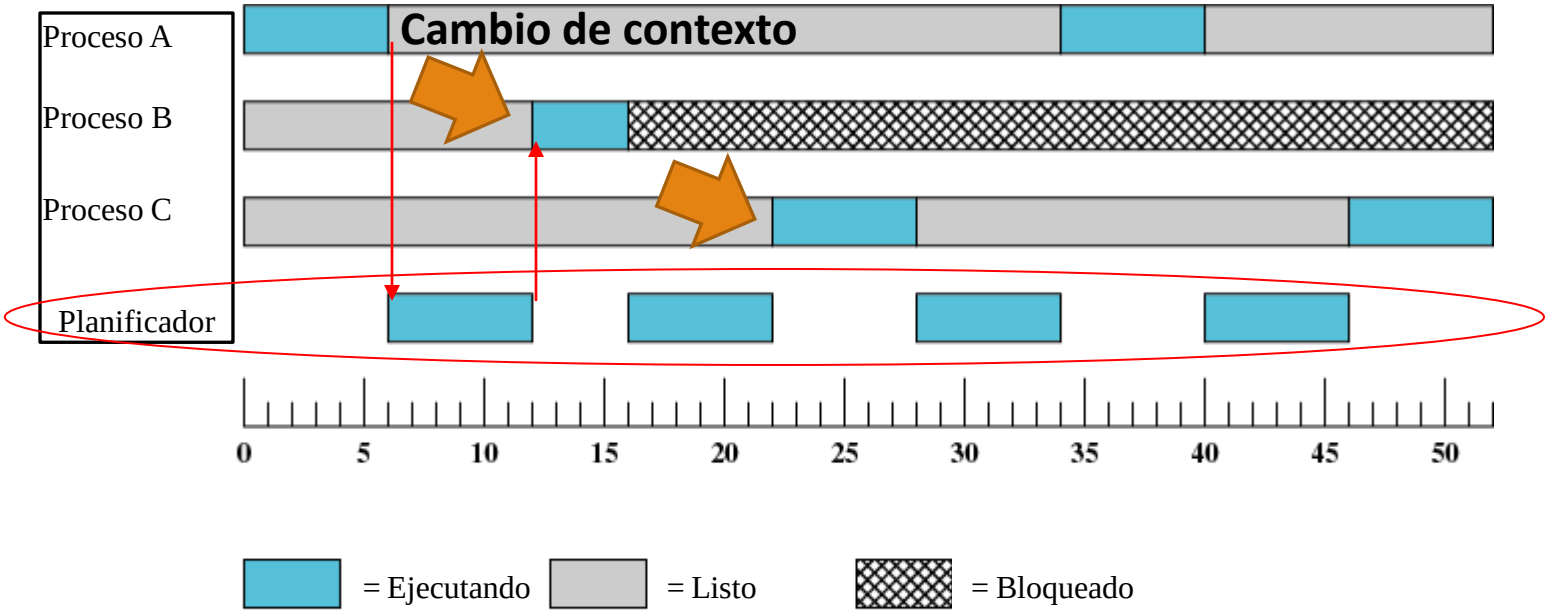


- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

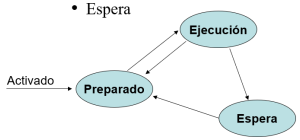


Ciclo de vida de los procesos

Traza de ejecución: Estado y Planificación de procesos

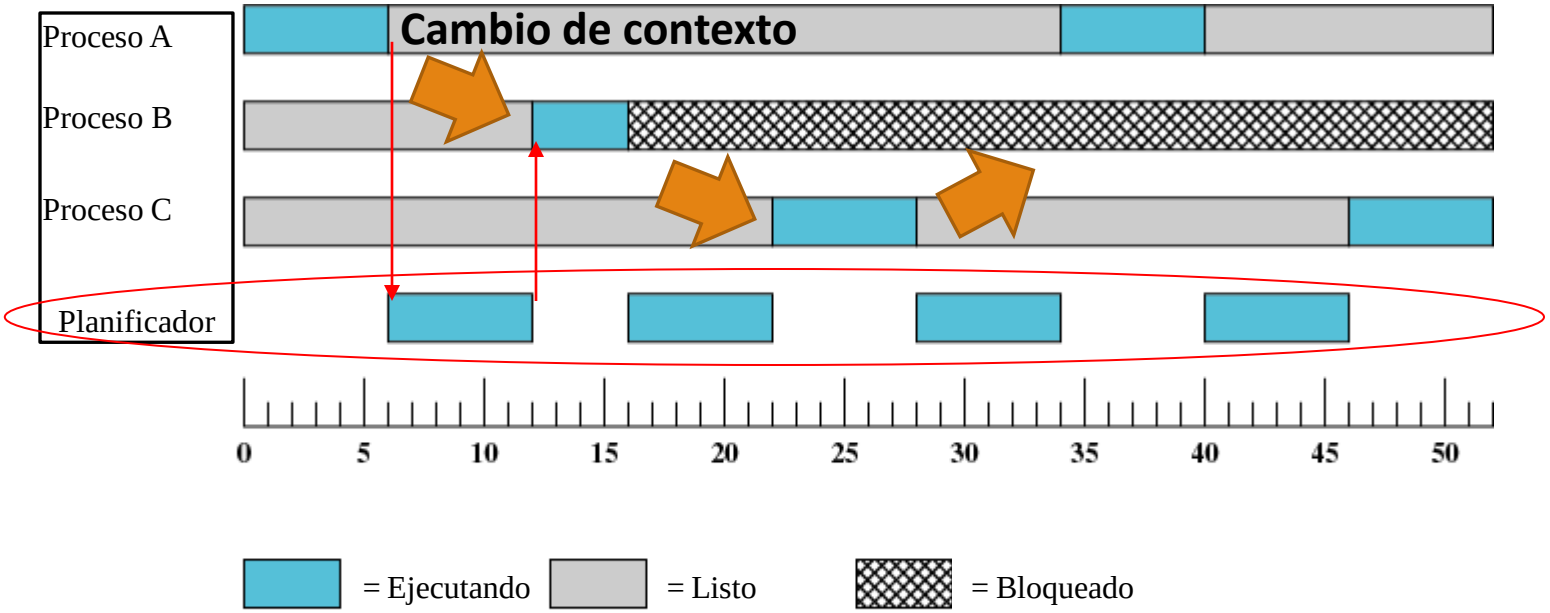


- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

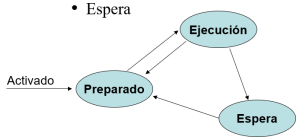


Ciclo de vida de los procesos

Traza de ejecución: Estado y Planificación de procesos

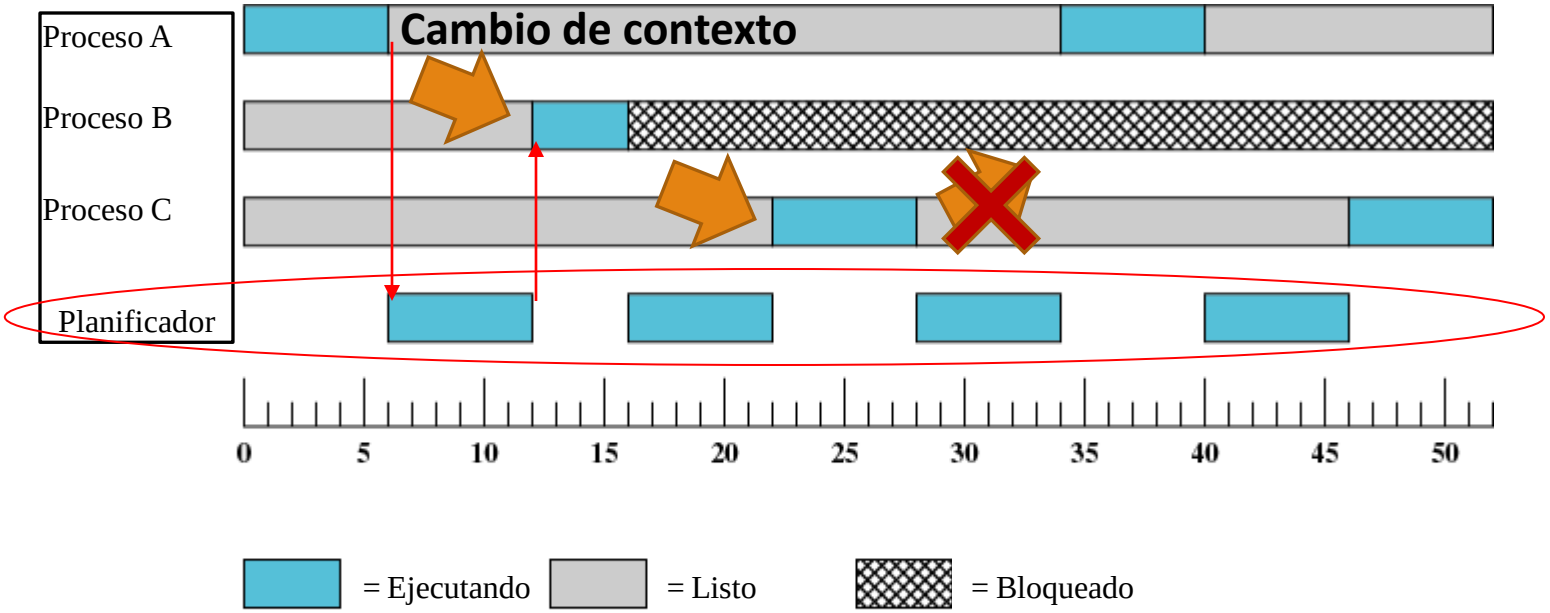


- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

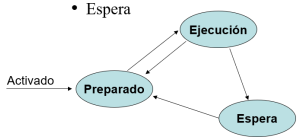


Ciclo de vida de los procesos

Traza de ejecución: Estado y Planificación de procesos

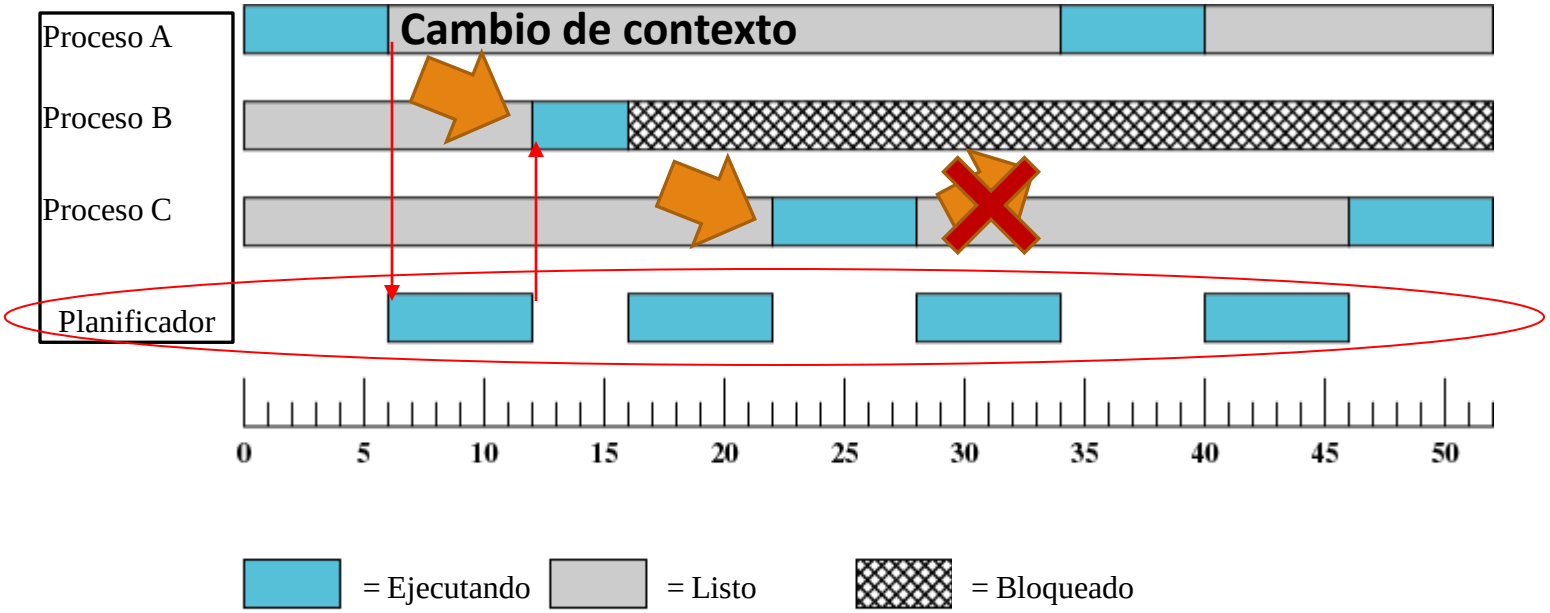


- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



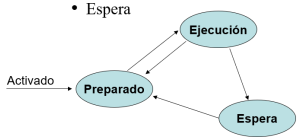
Ciclo de vida de los procesos

Traza de ejecución: Estado y Planificación de procesos



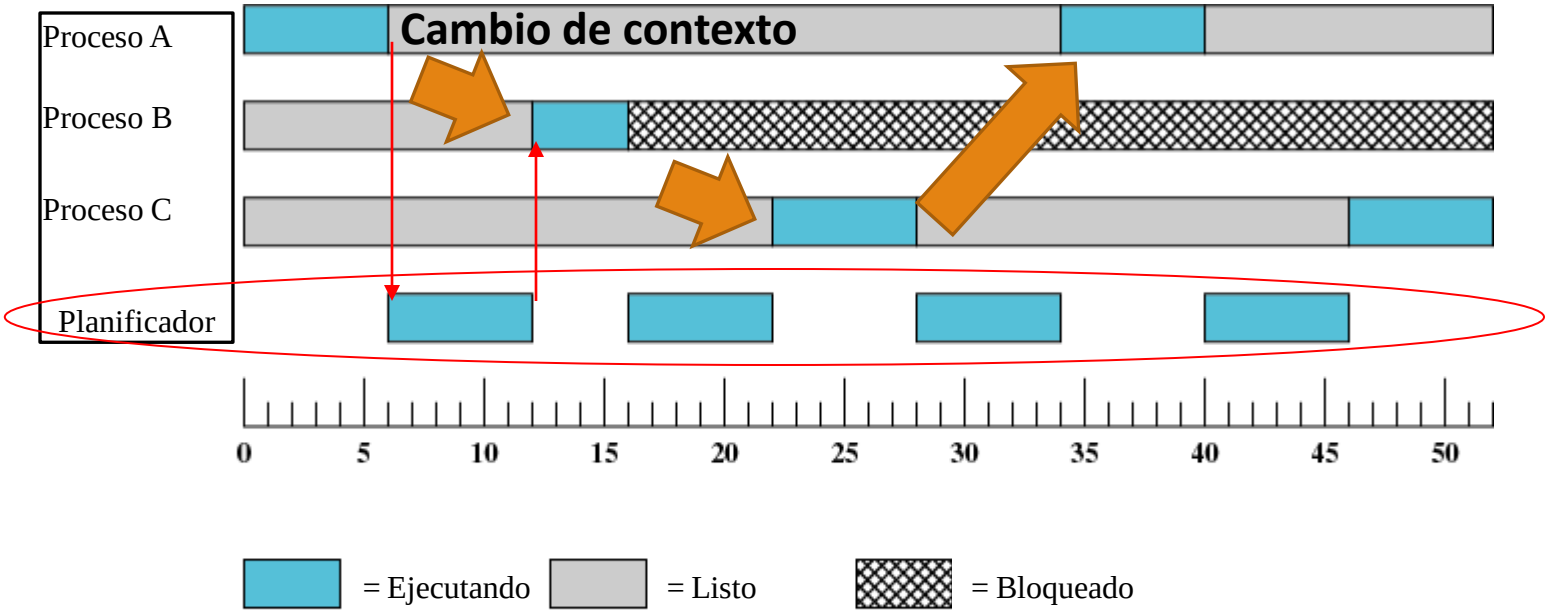
¿Cómo evita hacer una mala planificación?

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



Ciclo de vida de los procesos

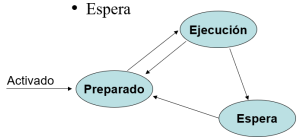
Traza de ejecución: Estado y Planificación de procesos



¿Cómo evita hacer una mala planificación?

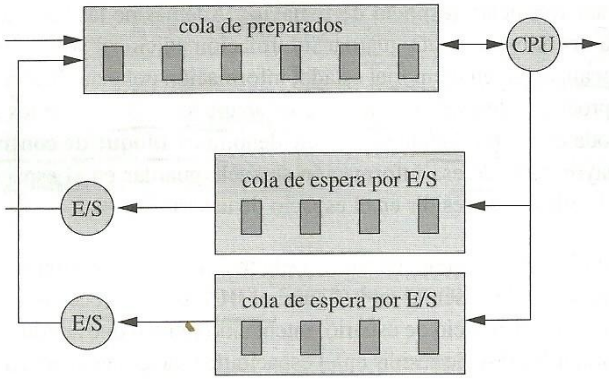
Implementación de colas Niveles de planificación

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



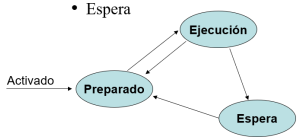
Es muy común que los Sistemas Operativos organicen los BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos



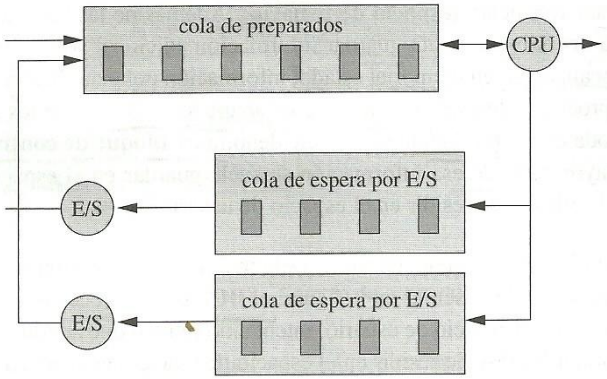
Implementación de colas Niveles de planificación

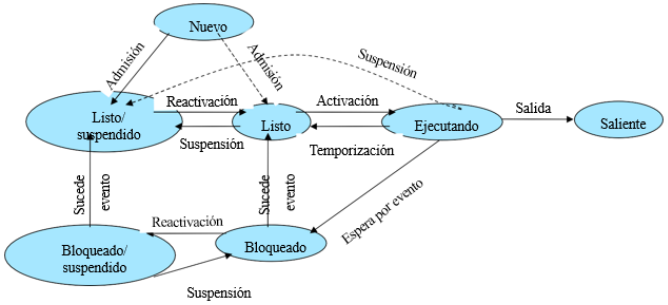
- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



Es muy común que los Sistemas Operativos organicen los BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos



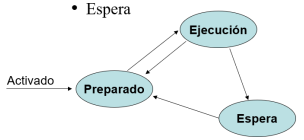


(b) Con dos estados Suspendidos

Diagrama de transición de estados de procesos con estado suspendidos

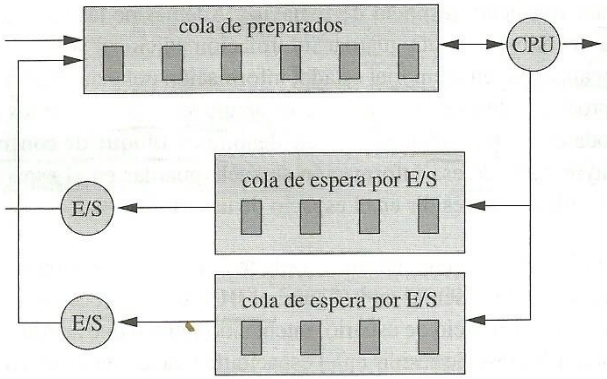
Implementación de colas Niveles de planificación

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

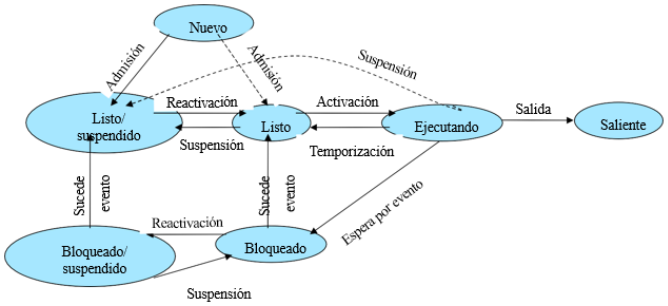


Es muy común que los Sistemas Operativos organicen los BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos



La planificación de procesos en el Sistema Operativo actúa en distintos niveles

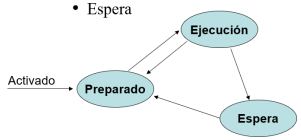


(b) Con dos estados Suspendidos

Diagrama de transición de estados de procesos con estado suspendidos

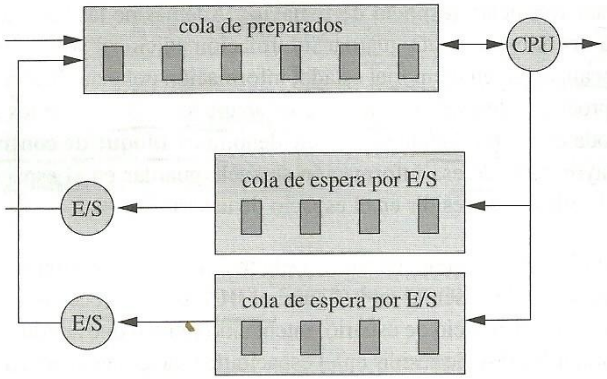
Implementación de colas Niveles de planificación

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



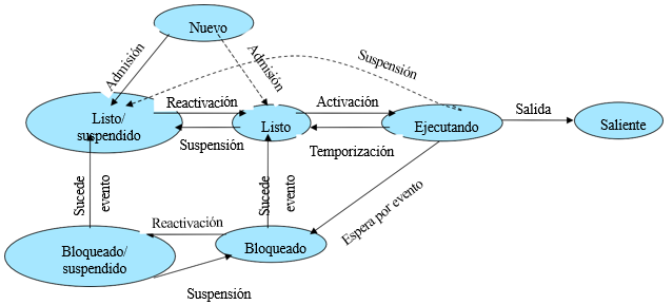
Es muy común que los Sistemas Operativos organicen los BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos



La planificación de procesos en el Sistema Operativo actúa en distintos niveles

- Planificador de largo plazo o de alto nivel (PLP)
- Planificador de medio plazo (PMP)
- Planificador de corto plazo o de bajo nivel (PCP)

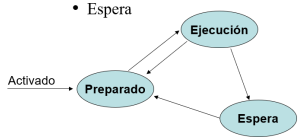


(b) Con dos estados Suspendidos

Diagrama de transición de estados de procesos con estado suspendidos

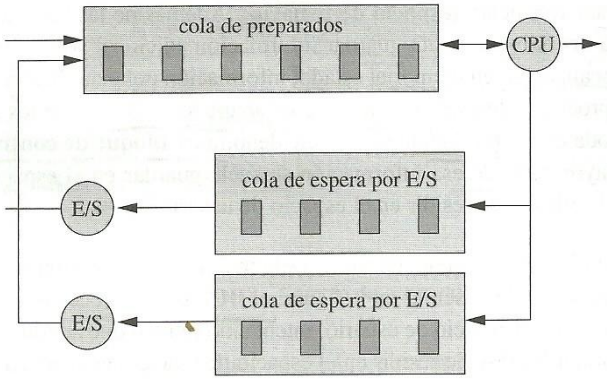
Implementación de colas Niveles de planificación

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



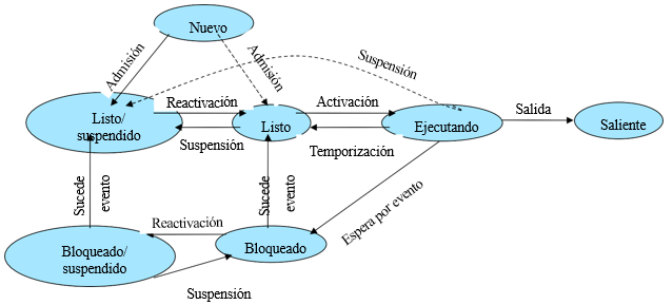
Es muy común que los Sistemas Operativos organicen los BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos



La planificación de procesos en el Sistema Operativo actúa en distintos niveles

- Planificador de largo plazo o de alto nivel (PLP)
  - Es el responsable de los **cambio de contexto**
  - Selecciona de entre los procesos que se encuentren **en el estado preparado**
- Planificador de medio plazo (PMP)
- Planificador de corto plazo o de bajo nivel (PCP)

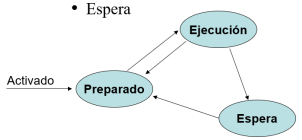


(b) Con dos estados Suspendidos

Diagrama de transición de estados de procesos con estado suspendidos

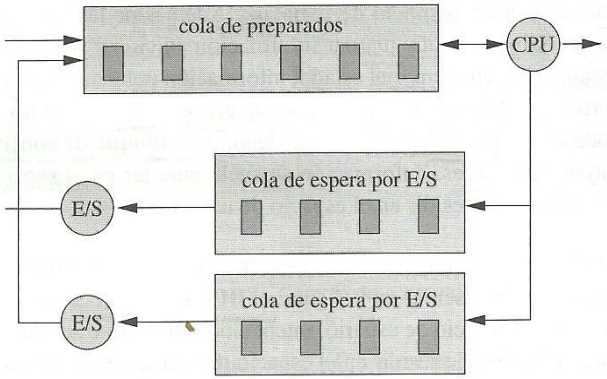
Implementación de colas Niveles de planificación

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



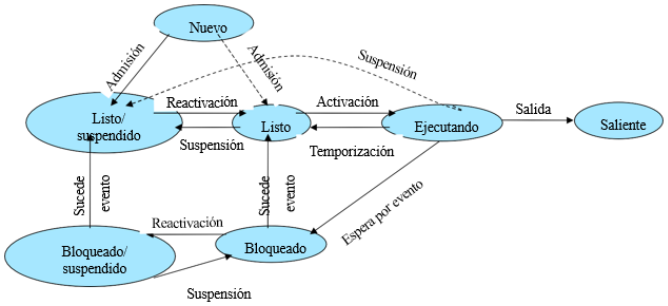
Es muy común que los Sistemas Operativos organicen lc BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos



La planificación de procesos en el Sistema Operativo actúa en distintos niveles

- Planificador de largo plazo o de alto nivel (PLP)
  - Es el responsable de los **cambio de contexto**
  - Selecciona de entre los procesos que se encuentren **en el estado preparado**
- Planificador de medio plazo (PMP)
  - Se encarga de gestionar situaciones en las que **el sistema se queda sin recursos**
  - La estrategia consiste en **desalojar de memoria principal** los procesos en estado bloqueado y **copiarlos en memoria secundaria** (**swapping**)
  - Aparece un nuevo estado posible para los procesos (suspendido)
- Planificador de corto plazo o de bajo nivel (PCP)

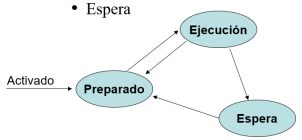


(b) Con dos estados Suspendidos

Diagrama de transición de estados de procesos con estado suspendidos

Implementación de colas Niveles de planificación

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

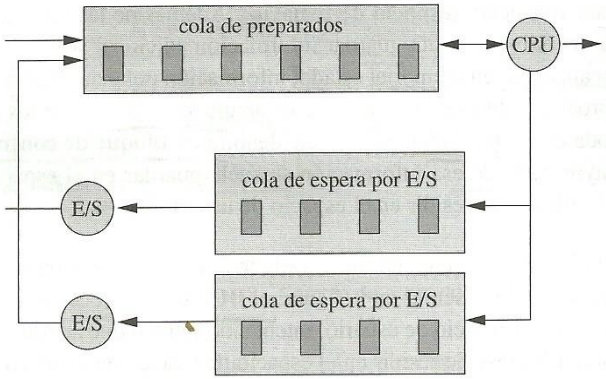


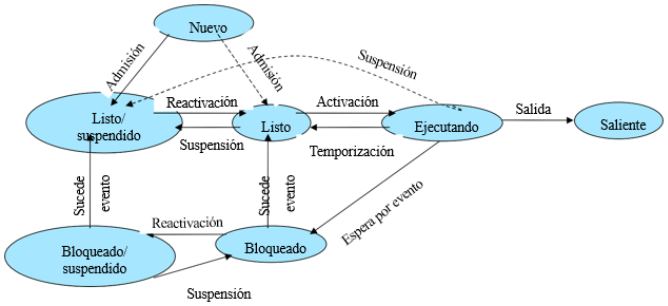
Es muy común que los Sistemas Operativos organicen los BCP en estructuras dinámicas

- Se mantiene un cola de procesos por cada estado de los procesos.
- Sistemas con varios procesadores
- Lista de espera por recursos

La planificación de procesos en el Sistema Operativo actúa en distintos niveles

- Planificador de largo plazo o de alto nivel (PLP)
  - Es el responsable de los **cambio de contexto**
  - Selecciona de entre los procesos que se encuentren **en el estado preparado**
- Planificador de medio plazo (PMP)
  - Se encarga de gestionar situaciones en las que **el sistema se queda sin recursos**
  - La estrategia consiste en **desalojar de memoria principal** los procesos en estado bloqueado y **copiarlos en memoria secundaria** (**swapping**)
  - Aparece un nuevo estado posible para los procesos (suspendido)
- Planificador de corto plazo o de bajo nivel (PCP)
  - Encargado de **suministrar nuevos procesos** a la cola de preparados
  - Alimenta al sistema para conseguir carga de trabajo, **evitando la sobresaturación**
    - carga del procesador, memoria ocupada, actividad de E/S.





(b) Con dos estados Suspendidos

Diagrama de transición de estados de procesos con estado suspendidos

Resumen

El SO es responsable del manejo de los procesos

El SO es responsable de la coordinación y manejo de los recursos para evitar conflictos

Las fuentes de interrupciones que alteran la planificación del SO pueden ser:

- Internas
  - syscalls, sleeps, …
- Externas
  - I/O

3.7 SUMMARY

The most fundamental concept in a modern OS is the process. The principal function of the OS is to create, manage, and terminate processes. While processes are active, the OS must see that each is allocated time for execution by the processor, coordinate their activities, manage conflicting demands, and allocate system resources to processes.

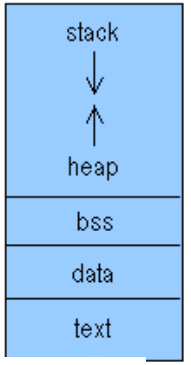
To perform its process management functions, the OS maintains a description of each process, or process image, which includes the address space within which the process executes, and a process control block. The latter contains all of the information that is required by the OS to manage the process, including its current state, resources allocated to it, priority, and other relevant data.

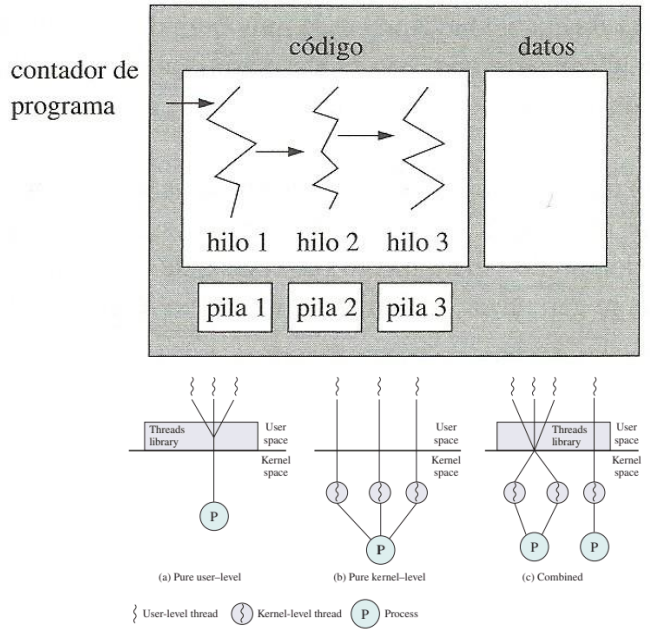
During its lifetime, a process moves among a number of states. The most important of these are Ready, Running, and Blocked. A ready process is one that is not currently executing, but that is ready to be executed as soon as the OS dispatches it. The running process is that process that is currently being executed by the processor. In a multiprocessor system, more than one process can be in this state. A blocked process is waiting for the completion of some event, such as an I/O operation.

A running process is interrupted either by an interrupt, which is an event that occurs outside the process and that is recognized by the processor, or by executing a supervisor call to the OS. In either case, the processor performs a mode switch, transferring control to an operating system routine. The OS, after it has completed necessary work, may resume the interrupted process or switch to some other process.

Procesos vs hilos

Hilo (thread) son flujos de control independientes dentro de un mismo proceso con pila, variables locales y PC propios.





Procesos vs hilos

Hilo (thread) son flujos de control independientes dentro de un mismo proceso con pila, variables locales y PC propios.

Ventajas

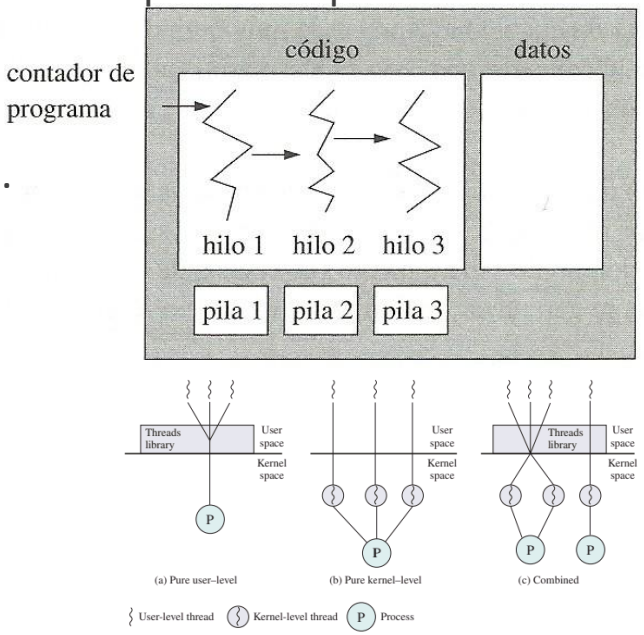
- Posibilidad de compartir une espacio de direcciones y recursos (datos y archivos)
- El tiempo empleado en el cambio de contexto es menor que en los procesos completos

Inconvenientes

- Sincronización entre los hilos y entre los procesos.
- Complejidad en la programación y depuración

Diseño de programas basados en hilos

- Aplicaciones con paralelismo potencial
- Existencia de numerosas tareas de E/S
- Existencia de eventos asíncronos



Procesos vs hilos

Hilo (thread) son flujos de control independientes dentro de un mismo proceso con pila, variables locales y PC propios.

Ventajas

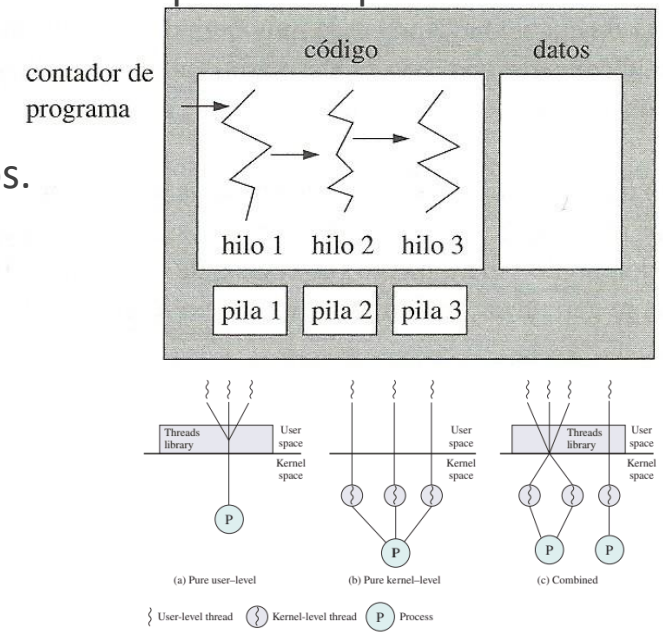
- Posibilidad de **compartir une espacio de direcciones y recursos** (datos y archivos)
- El tiempo empleado en el **cambio de contexto es menor** que en los procesos completos

Inconvenientes

- Sincronización entre los hilos y entre los procesos.
- Complejidad en la programación y depuración

Diseño de programas basados en hilos

- Aplicaciones con paralelismo potencial
- Existencia de numerosas tareas de E/S
- Existencia de eventos asíncronos



Procesos vs hilos

Hilo (thread) son flujos de control independientes dentro de un mismo proceso con pila, variables locales y PC propios.

Ventajas

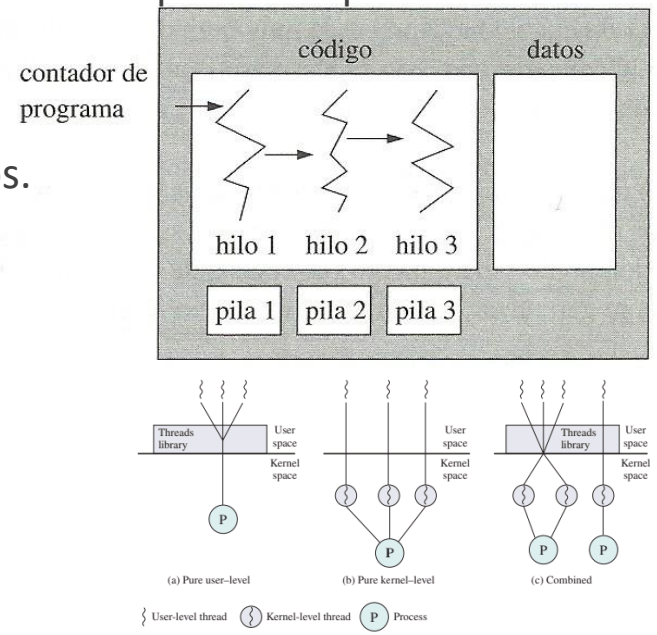
- Posibilidad de compartir une espacio de direcciones y recursos (datos y archivos)
- El tiempo empleado en el cambio de contexto es menor que en los procesos completos

Inconvenientes

- **Sincronización** entre los hilos y entre los procesos.
- **Complejidad** en la programación y depuración

Diseño de programas basados en hilos

- Aplicaciones con paralelismo potencial
- Existencia de numerosas tareas de E/S
- Existencia de eventos asíncronos



Procesos vs hilos

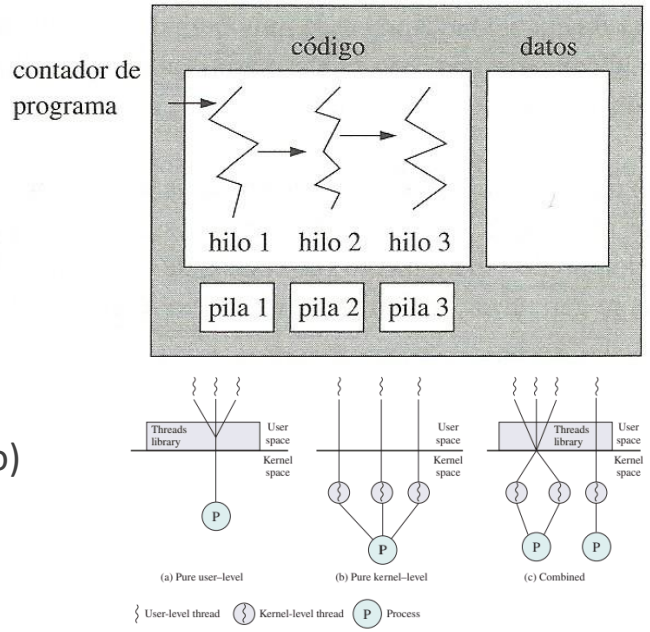
Hilo (thread) son flujos de control independientes dentro de un mismo proceso con pila, variables locales y PC propios.

Ventajas
- Posibilidad de compartir une espacio de direcciones y recursos (datos y archivos)
- El tiempo empleado en el cambio de contexto es menor que en los procesos completos

Inconvenientes
- Sincronización entre los hilos y entre los procesos.
- Complejidad en la programación y depuración

Diseño de programas basados en hilos
- Aplicaciones con paralelismo potencial
- Existencia de numerosas tareas de E/S
- Existencia de eventos asíncronos

El soporte a los hilos puede darlo:
- **El Sistema Operativo** (implementación a nivel de núcleo)
- **Bibliotecas/librerías a nivel de usuario** (Win32, POSIX)

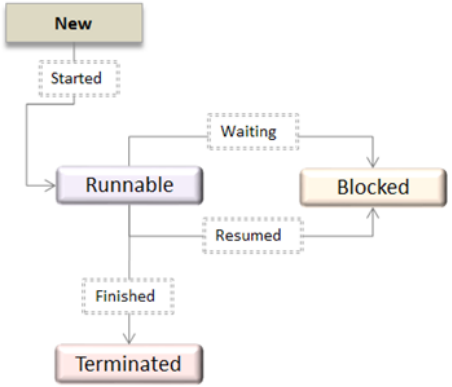


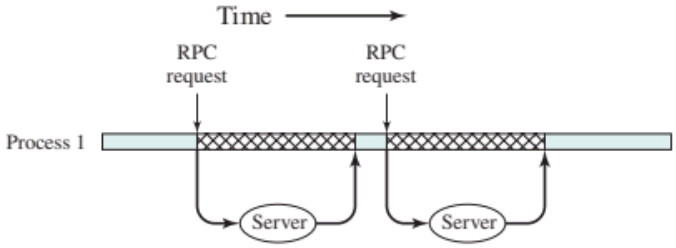
Estado de los hilos

1. **Spawn:** Typically, when a new process is spawned, a thread for that process is also spawned. Subsequently, a thread within a process may spawn another thread within the same process, providing an instruction pointer and arguments for the new thread. The new thread is provided with its own register context and stack space and placed on the Ready queue.
2. **Block:** When a thread needs to wait for an event, it will block (saving its user registers, program counter, and stack pointers). The processor may then turn to the execution of another ready thread in the same or a different process.
3. **Unblock:** When the event for which a thread is blocked occurs, the thread is moved to the Ready queue.
4. **Finish:** When a thread completes, its register context and stacks are deallocated.

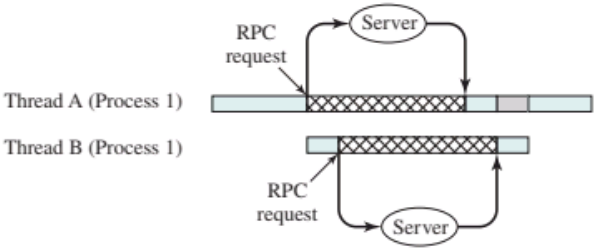
De forma análoga a los procesos los hilos constan de 4 estados:

- Emitidos o nuevos
- Bloqueados
- Desbloqueados
- Finalizados





(a) RPC using single thread



(b) RPC using one thread per server (on a uniprocessor)

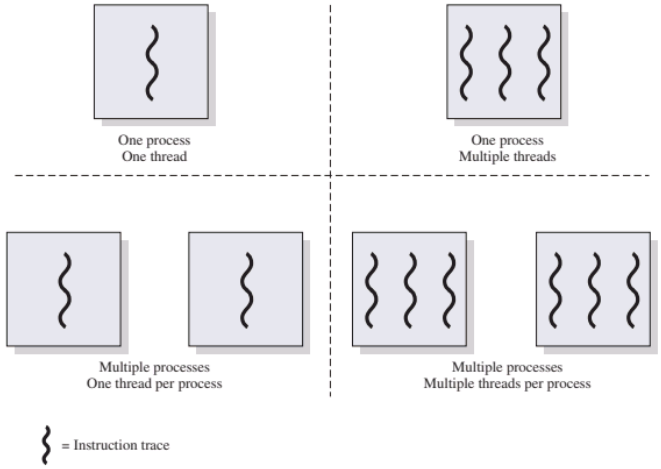
Blocked, waiting for response to RPC

Blocked, waiting for processor, which is in use by Thread B

Running

Hilos

En SO multiprocesos, varios hilos se pueden ejecutar dentro de un mismo proceso



4.9 SUMMARY

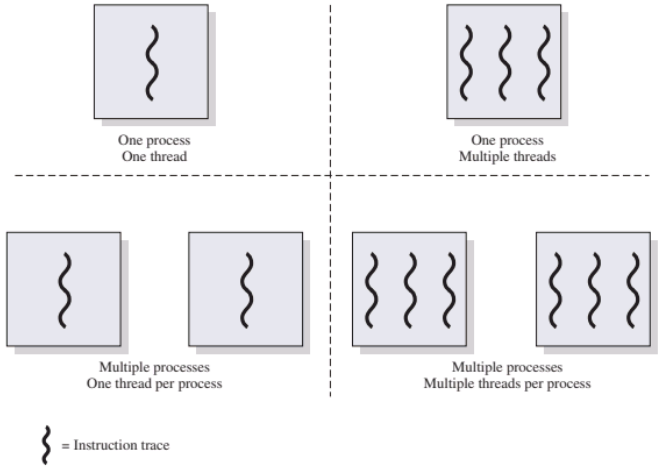
Some operating systems distinguish the concepts of process and thread, the former related to resource ownership, and the latter related to program execution. This approach may lead to improved efficiency and coding convenience. In a multithreaded system, multiple concurrent threads may be defined within a single process. This may be done using either user-level threads or kernel-level threads. User-level threads are unknown to the OS and are created and managed by a threads library that runs in the user space of a process. User-level threads are very efficient because a mode switch is not required to switch from one thread to another. However, only a single user-level thread within a process can execute at a time, and if one thread blocks, the entire process is blocked. Kernel-level threads are threads within a process that are maintained by the kernel. Because they are recognized by the kernel, multiple threads within the same process can execute in parallel on a multiprocessor and the blocking of a thread does not block the entire process. However, a mode switch is required to switch from one thread to another.

Hilos

En SO multiprocesos, varios hilos se pueden ejecutar dentro de un mismo proceso

Las librerías en el espacio de usuario se encargan de la planificación interna.

Los hilos en espacio de Kernel permiten la ejecución en múltiples procesadores



4.9 SUMMARY

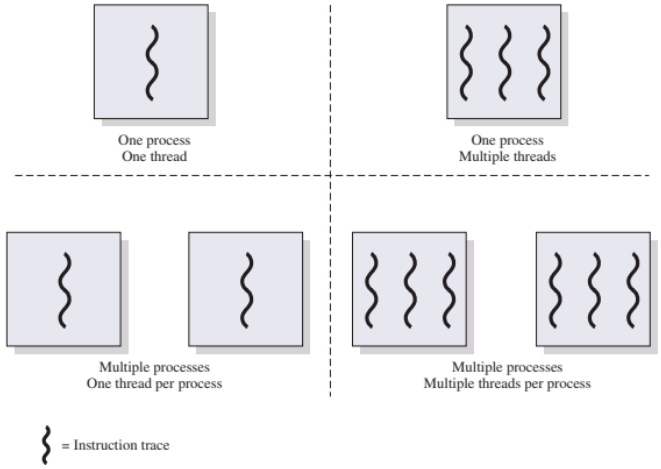
Some operating systems distinguish the concepts of process and thread, the former related to resource ownership, and the latter related to program execution. This approach may lead to improved efficiency and coding convenience. In a multithreaded system, multiple concurrent threads may be defined within a single process. This may be done using either user-level threads or kernel-level threads. User-level threads are unknown to the OS and are created and managed by a threads library that runs in the user space of a process. User-level threads are very efficient because a mode switch is not required to switch from one thread to another. However, only a single user-level thread within a process can execute at a time, and if one thread blocks, the entire process is blocked. Kernel-level threads are threads within a process that are maintained by the kernel. Because they are recognized by the kernel, multiple threads within the same process can execute in parallel on a multiprocessor and the blocking of a thread does not block the entire process. However, a mode switch is required to switch from one thread to another.

Hilos

En SO multiprocesos, varios hilos se pueden ejecutar dentro de un mismo proceso

Las librerías en el espacio de usuario se encargan de la planificación interna.

Los hilos en espacio de Kernel permiten la ejecución en múltiples procesadores**, pero tiene el sobrecoste de la sincronización.**



4.9 SUMMARY

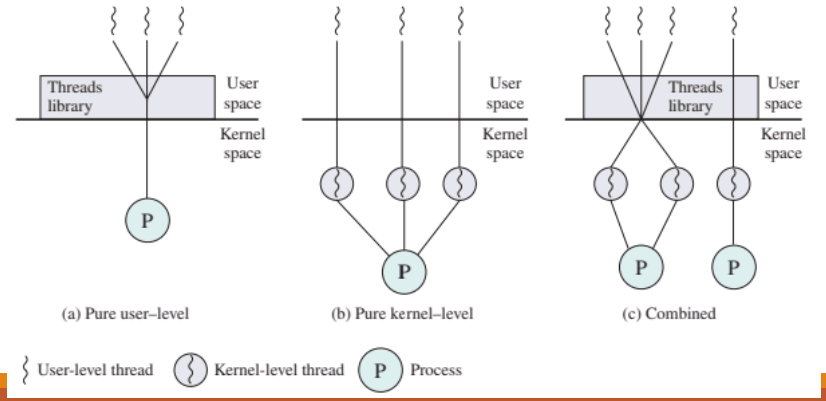
Some operating systems distinguish the concepts of process and thread, the former related to resource ownership, and the latter related to program execution. This approach may lead to improved efficiency and coding convenience. In a multithreaded system, multiple concurrent threads may be defined within a single process. This may be done using either user-level threads or kernel-level threads. User-level threads are unknown to the OS and are created and managed by a threads library that runs in the user space of a process. User-level threads are very efficient because a mode switch is not required to switch from one thread to another. However, only a single user-level thread within a process can execute at a time, and if one thread blocks, the entire process is blocked. Kernel-level threads are threads within a process that are maintained by the kernel. Because they are recognized by the kernel, multiple threads within the same process can execute in parallel on a multiprocessor and the blocking of a thread does not block the entire process. However, a mode switch is required to switch from one thread to another.

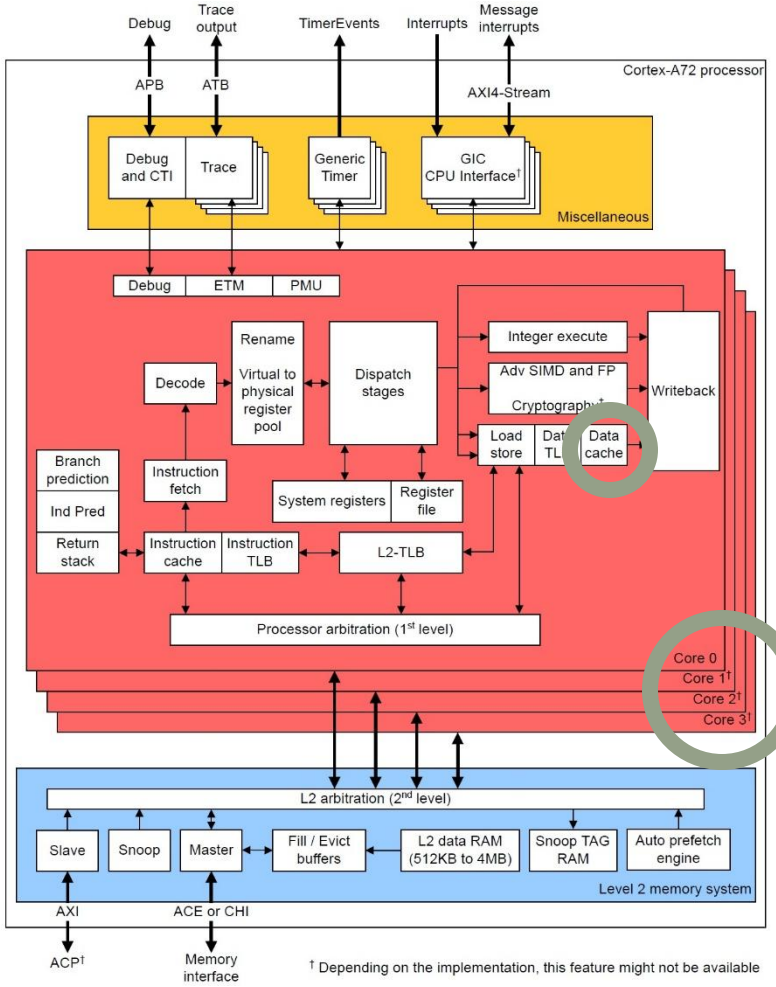
Hilos

En SO multiprocesos, varios hilos se pueden ejecutar dentro de un mismo proceso

Las librerías en el espacio de usuario se encargan de la planificación interna.

Los hilos en espacio de Kernel permiten la ejecución en múltiples procesadores**, pero tiene el sobrecoste de la sincronización.**



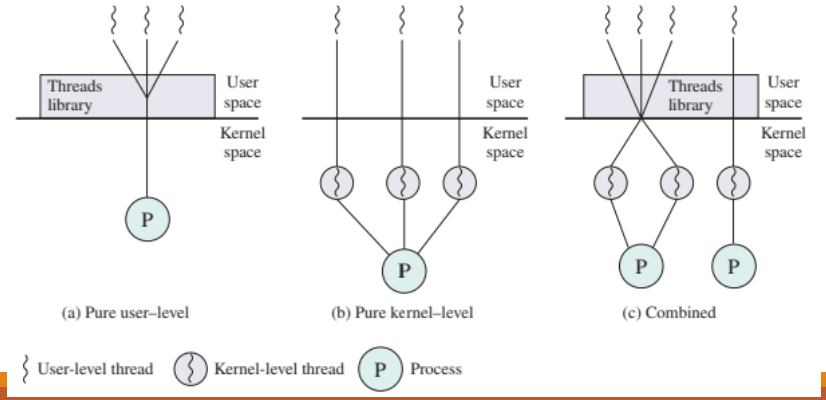


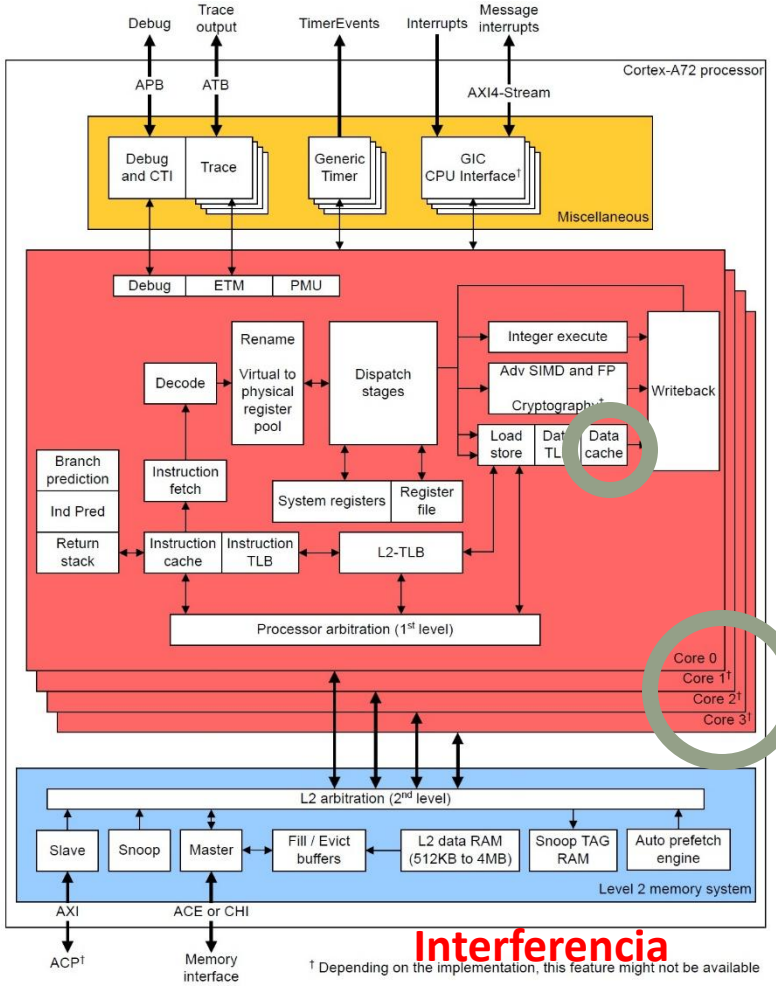
Hilos

En SO multiprocesos, varios hilos se pueden ejecutar dentro de un mismo proceso

Las librerías en el espacio de usuario se encargan de la planificación interna.

Los hilos en espacio de Kernel permiten la ejecución en múltiples procesadores**, pero tiene el sobrecoste de la sincronización.**





Interferencia

Tema 2: Planificación

SISTEMAS OPERATIVOS

Índice

1. Introducción
2. Conceptos fundamentales
3. Implementación de procesos
4. Hilos (Threads)
5. Planificación de la CPU

Bibliografía

W. Stalling. **Sistemas Operativos**. 5ª Edición.

Capítulos 3, 4, 9 y 10.

Objetivos

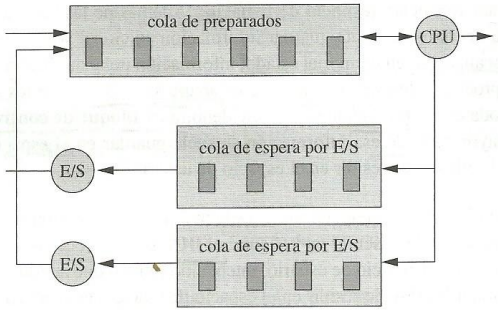
LEARNING OBJECTIVES

After studying this chapter, you should be able to:

- Define the term *process* and explain the relationship between processes and process control blocks.
- Explain the concept of a process state and discuss the state transitions the processes undergo.
- List and describe the purpose of the data structures and data structure elements used by an OS to manage processes.
- Assess the requirements for process control by the OS.
- Understand the issues involved in the execution of OS code.
- Describe the process management scheme for UNIX SVR4.

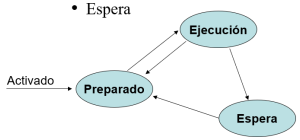
1. Definir que es un proceso
2. Explicar el conjunto de estados típicos desde su creación hasta su finalización
3. Ver las estructuras en memoria de un proceso y la información que guarda el SO
4. Definir que es un Hilo
5. Planificación: diferentes algoritmos

RECAP

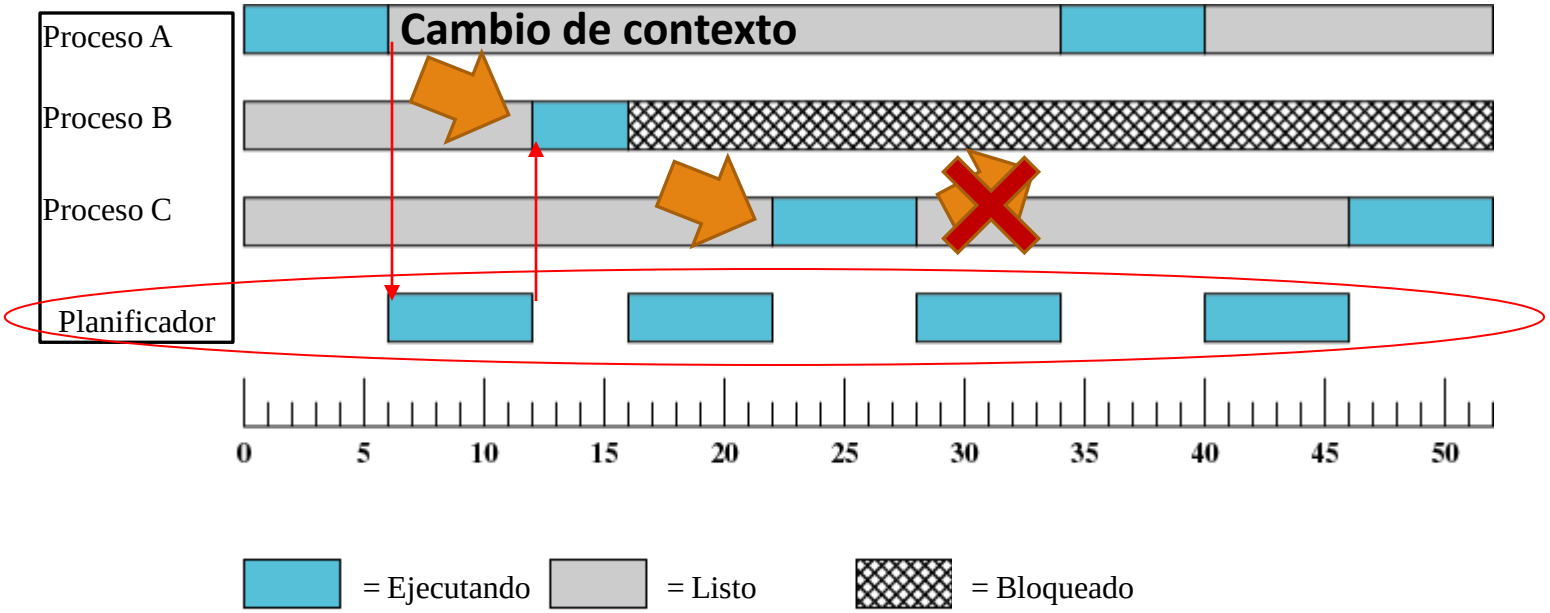


- Relación entre procesos:
  - Independientes
  - Cooperativos
  - Competitivos

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

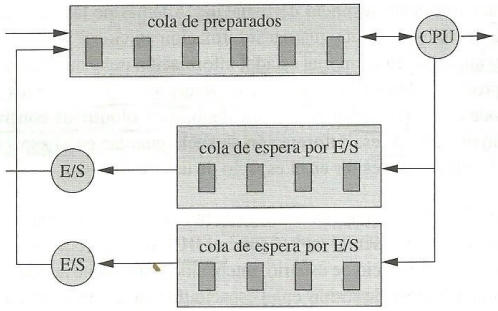


Traza de ejecución: Estado y Planificación de procesos



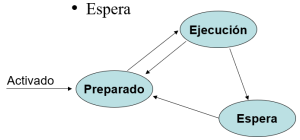
¿Cómo evita hacer una mala planificación?

RECAP

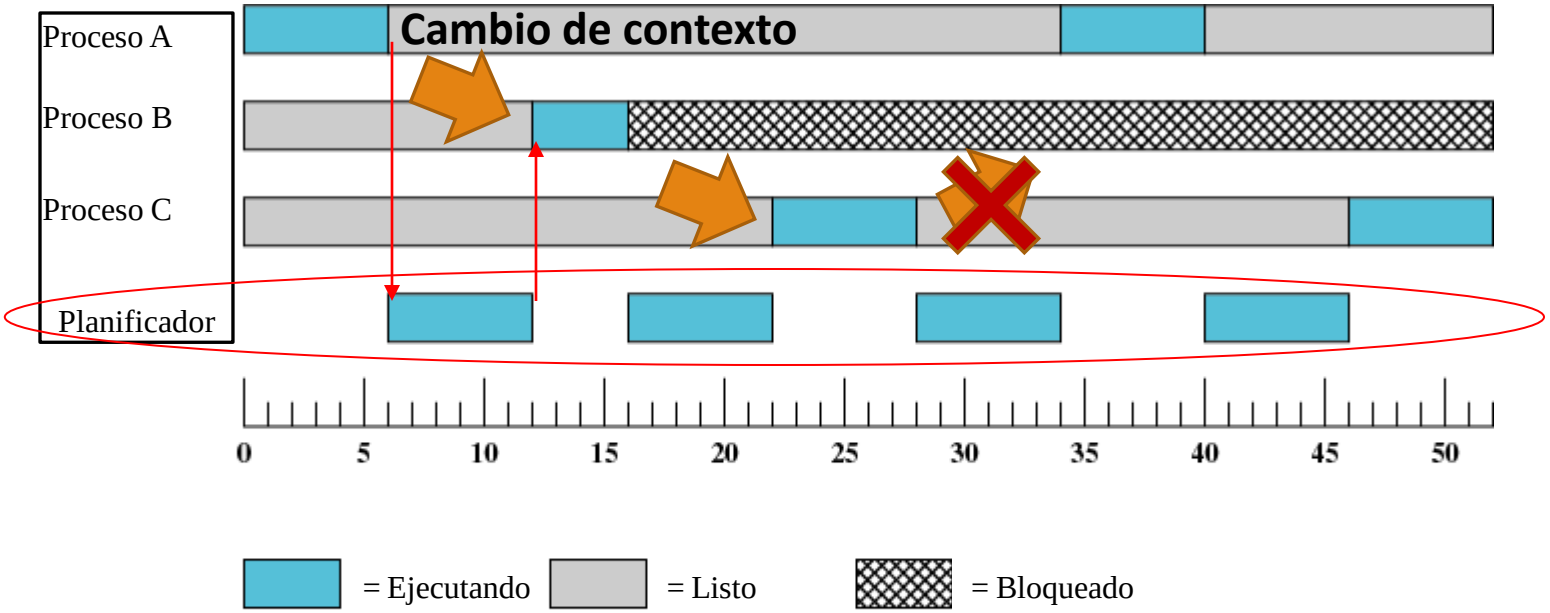


- Relación entre procesos:
  - Independientes
  - Cooperativos
  - Competitivos

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



Traza de ejecución: Estado y Planificación de procesos



¿Cómo evita hacer una mala planificación?

¿Cómo medir si es una buena/mala planificación?

¿Qué se busca a nivel de proceso y sistema?

Planificación
Criterios de rendimiento

Criterios de valoración de **los algoritmos de planificación**

Criterios importantes en la planificación de **procesos**:

Planificación
Criterios de rendimiento

Criterios de valoración de los algoritmos de planificación

- **Tiempo de espera** ⏬
- **Utilización de la CPU** ⏫
- **Tiempo de respuesta** ⏬

Criterios importantes en la planificación de procesos:

Planificación
Criterios de rendimiento

Criterios de valoración de los algoritmos de planificación

- Tiempo de espera ⇊
- Utilización de la CPU ⇈
- Tiempo de respuesta ⇊

Criterios importantes en la planificación de procesos:

- **Justicia**
  - trata de que no haya procesos perjudicados por la política
- **No inanición**
  - garantizar que todo proceso se ejecutará en un tiempo finito

Planificación
Criterios de rendimiento

Criterios de valoración de los algoritmos de planificación

- Tiempo de espera
- Utilización de la CPU
- Tiempo de respuesta

Criterios importantes en la planificación de procesos:

- **Justicia**
  - trata de que no haya procesos perjudicados por la política
- **No inanición**
  - garantizar que todo proceso se ejecutará en un tiempo finito



Métricas

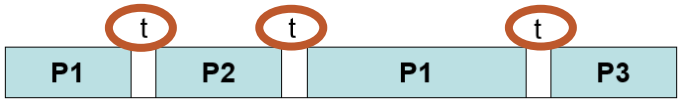
Magnitudes para la valoración de los algoritmos de planificación:

- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo

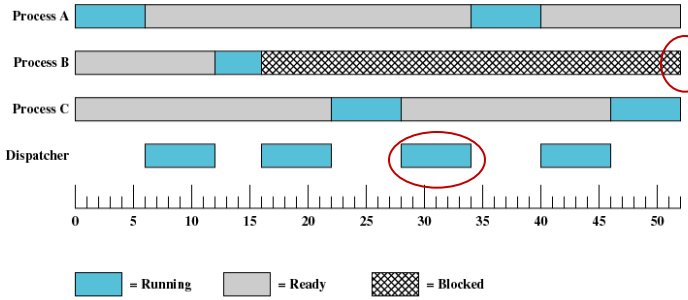
Métricas

Magnitudes para la valoración de los algoritmos de planificación:

- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo



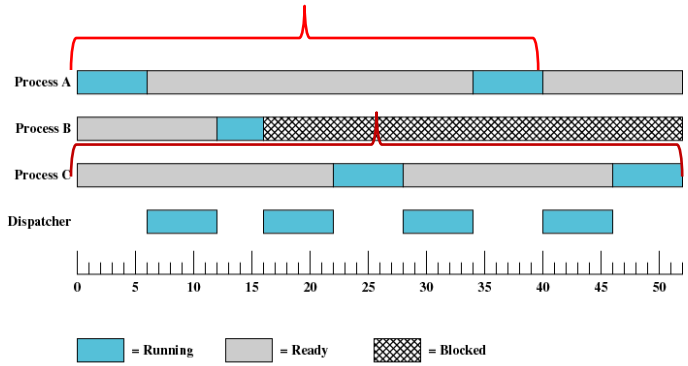
$$\text{Utilización} = \frac{T(P1) + T(P2) + T(P3)}{T(P1) + T(P2) + T(P3) + 3 \cdot t}$$



Métricas

Magnitudes para la valoración de los algoritmos de planificación:

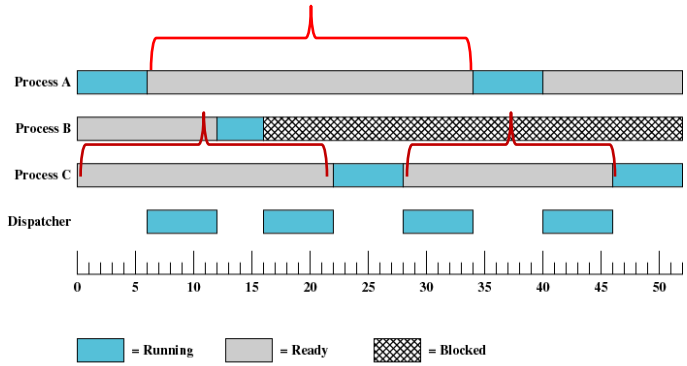
- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo



Métricas

Magnitudes para la valoración de los algoritmos de planificación:

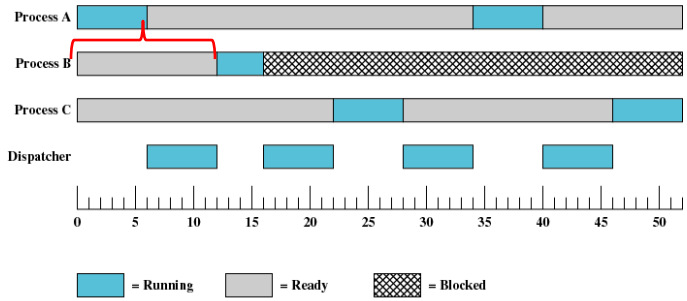
- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo



Métricas

Magnitudes para la valoración de los algoritmos de planificación:

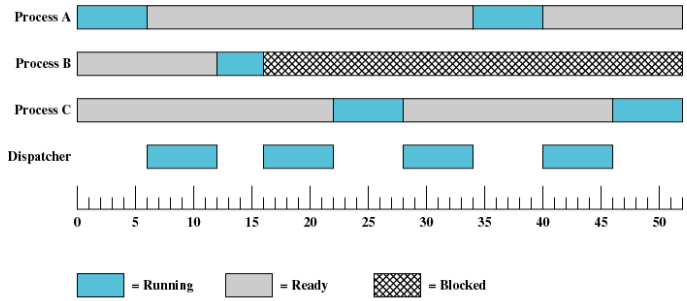
- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo



Métricas

Magnitudes para la valoración de los algoritmos de planificación:

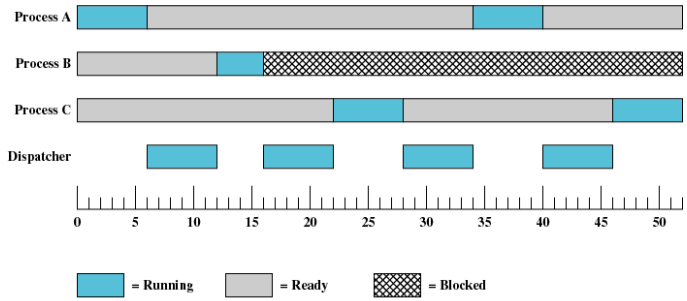
- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo



Métricas

Magnitudes para la valoración de los algoritmos de planificación:

- **Utilización de la CPU:**
  - cuanto por ciento de tiempo que la CPU está ocupada
- **Tiempo de retorno:**
  - tiempo transcurrido entre la llegada de un proceso y su finalización
- **Tiempo de espera:**
  - tiempo que un proceso permanece en la cola de preparados
- **Tiempo de respuesta:**
  - tiempo que un proceso bloqueado tarda en entrar por 1ª vez en CPU
- **Flujo de procesos:**
  - número de procesos terminados por unidad de tiempo



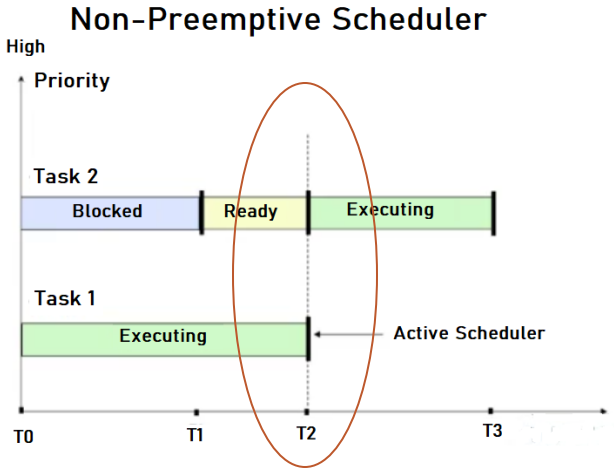
Políticas de planificación

Hay dos grandes grupos según el modo de decisión:

Políticas de planificación

Hay dos grandes grupos según el modo de decisión:

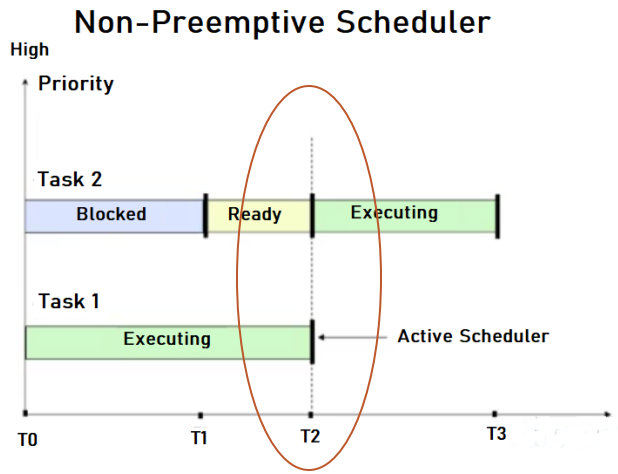
- Políticas no expulsivas/apropiativas (**non preemptive**):
  - una vez que el proceso está en el estado Ejecutando, continuará ejecutando hasta que termina o se bloquea para esperar E/S

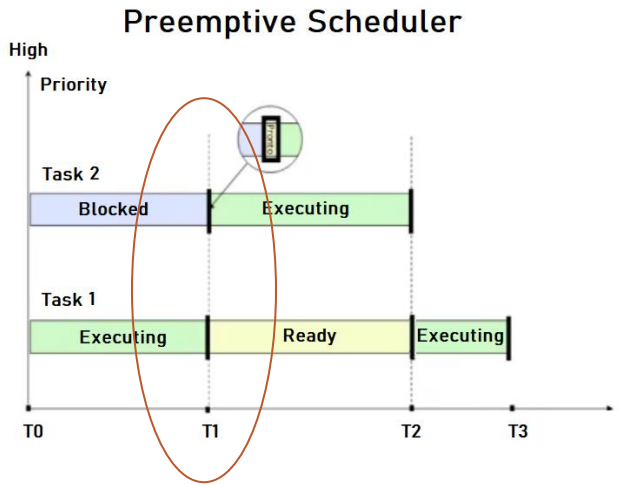


Políticas de planificación

Hay dos grandes grupos según el modo de decisión:

- Políticas no expulsivas/apropiativas (**non preemptive**):
  - una vez que el proceso está en el estado Ejecutando, continuará ejecutando hasta que termina o se bloquea para esperar E/S
- Políticas expulsivas/ apropiativas (**preemptive**):
  - el planificador puede desalojar al proceso que está haciendo uso de la CPU (puede proporcionar mejor servicio al impedir que un proceso monopolice la CPU)





Políticas de planificación

NON-PREEMPTIVE

Servicio por orden de llegada
- FCFS – First Come First Served

Primero la tarea más corta
- SJF – Shortest Job First
- SPN – Shortest Process Next

Planificación con prioridad
- Estática
- Dinámica

PREEMPTIVE

Planificación por turno circular
- RR – Round Robin

Tiempo que queda más corto
- SRT – Shortest Remaining Time

Planificación con colas multinivel
- MLQ – Multi-Level Queues

Planificación con colas multinivel con realimentación
- MLFQ – M.Level Feedback Queues

Planificación con prioridad

FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero en llegar, primero en servirse (FCFS)

A
B
C
D
E

FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

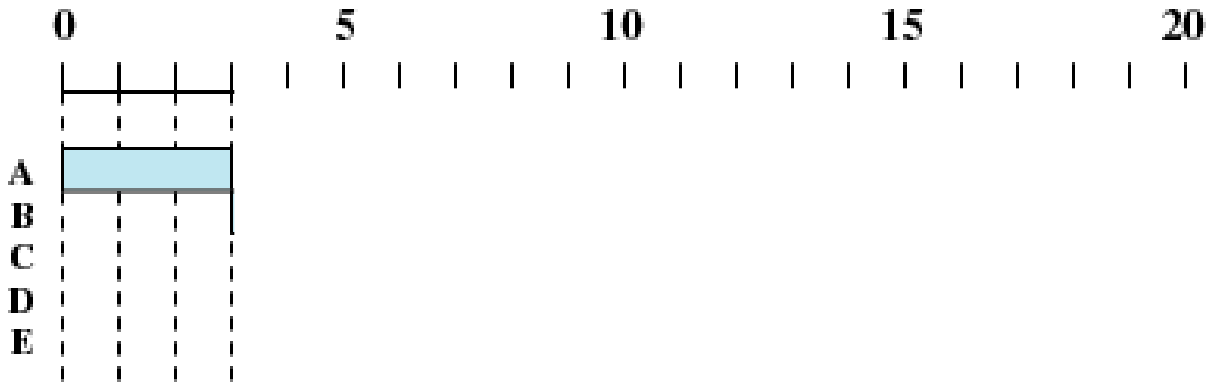
Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Primero en llegar, primero en servirse (FCFS)



FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

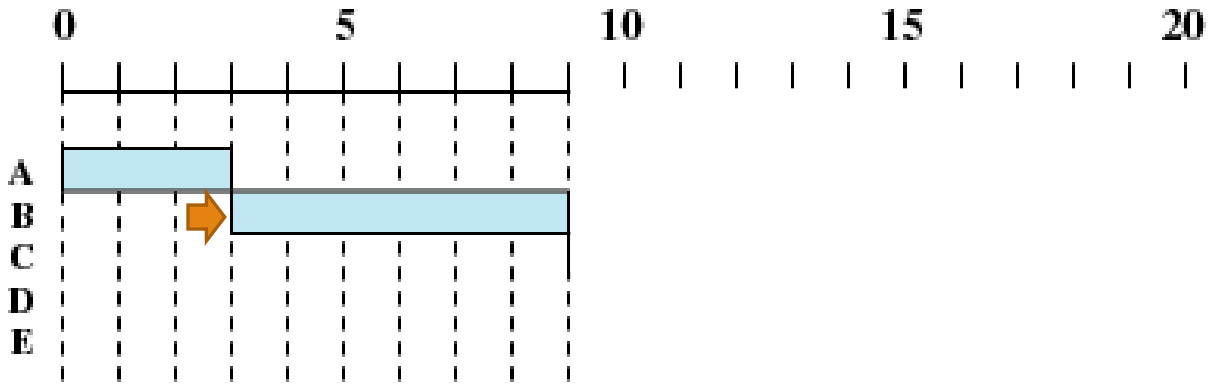
Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Primero en llegar, primero en servirse (FCFS)



FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

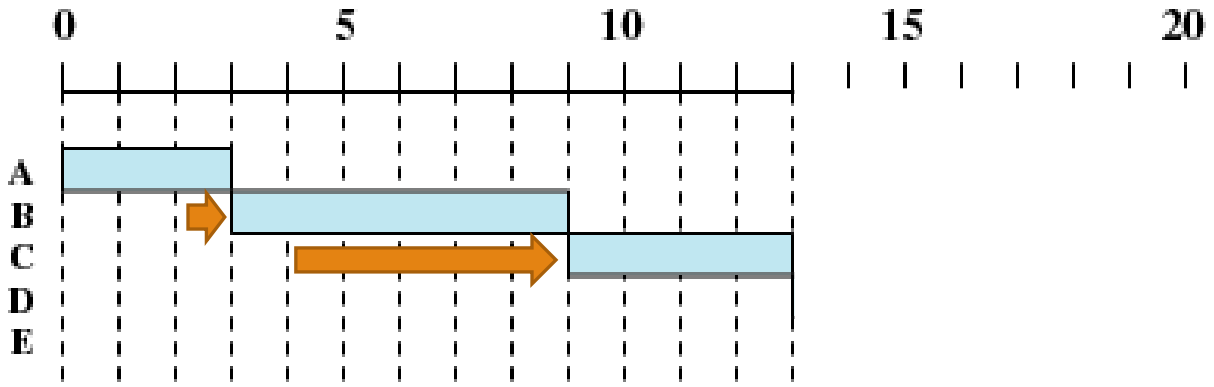
Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Primero en llegar, primero en servirse (FCFS)



FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

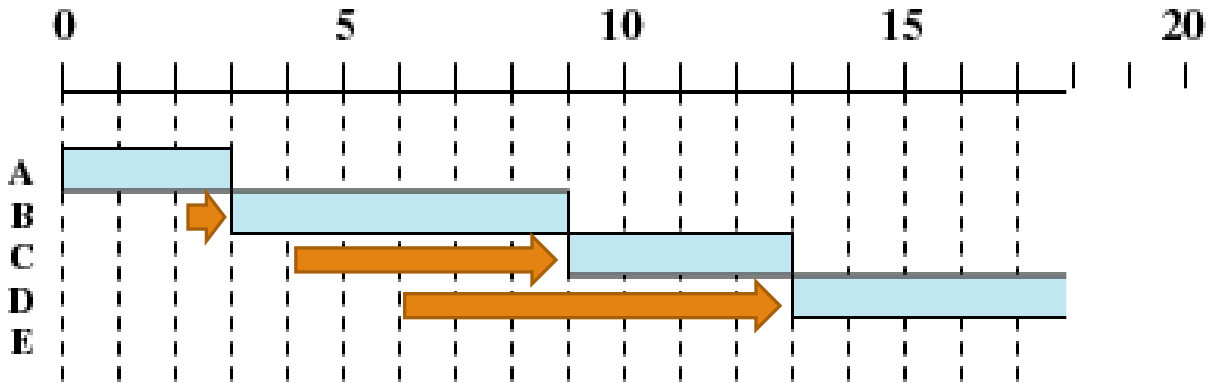
Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Primero en llegar, primero en servirse (FCFS)



FCFS (First Come First Served)

Política **no expulsiva**

Cada proceso que llega se une a la cola de listos

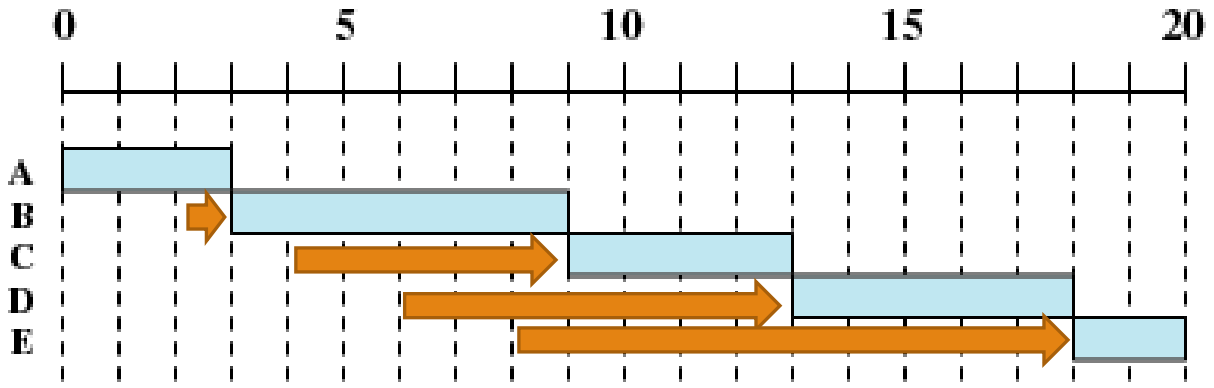
Cuando el proceso actualmente en ejecución deja de ejecutar, se selecciona el proceso que ha estado más tiempo en la cola de listos.

El planificador gestiona la cola de preparados como una cola FIFO (First In First Out) que se ejecutan en función del orden de llegada

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Primero en llegar, primero en servirse (FCFS)



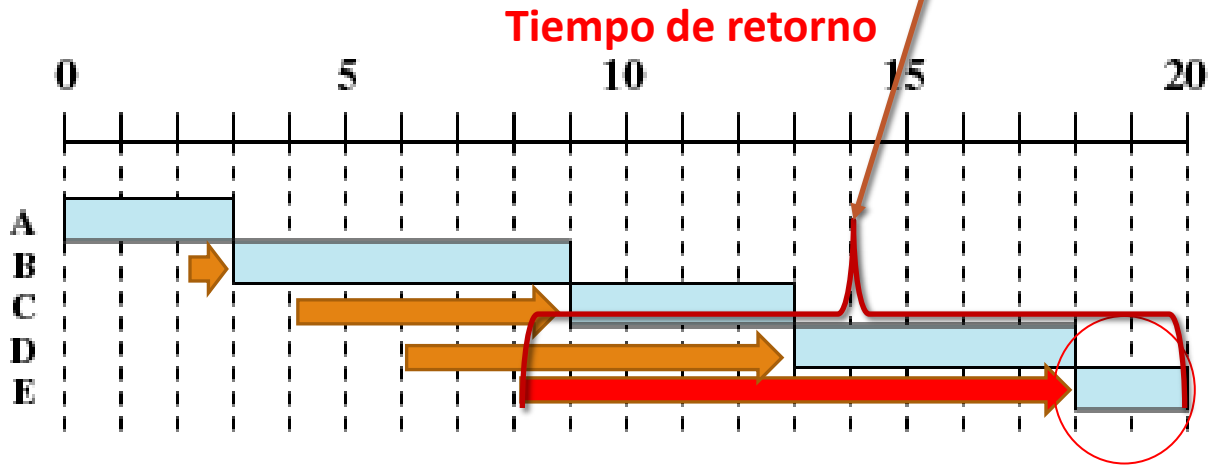
FCFS (First Come First Served)

Un proceso corto tendrá que esperar mucho tiempo

Es muy sensible al orden de llegada de los procesos

	Process	A	B	C	D	E	
	Arrival Time	0	2	4	6	8	
	Service Time (T_s)	3	6	4	5	2	Mean
FCFS	Finish Time	3	9	13	18	20	
	Turnaround Time (T_r)	3	7	9	12	12	8.60
	T_r/T_s	1.00	1.17	2.25	2.40	6.00	2.56

Primero en llegar, primero en servirse (FCFS)

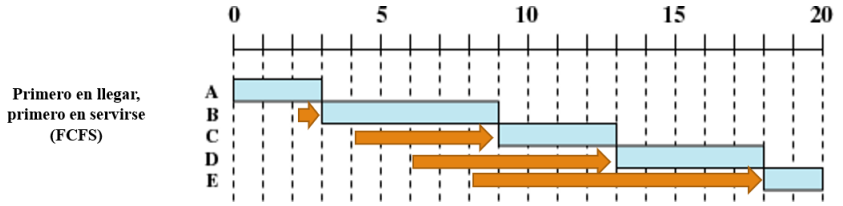


Ejercicio 1
FCFS (First Come First Served)

Sea un sistema con los procesos que se muestran en la tabla.

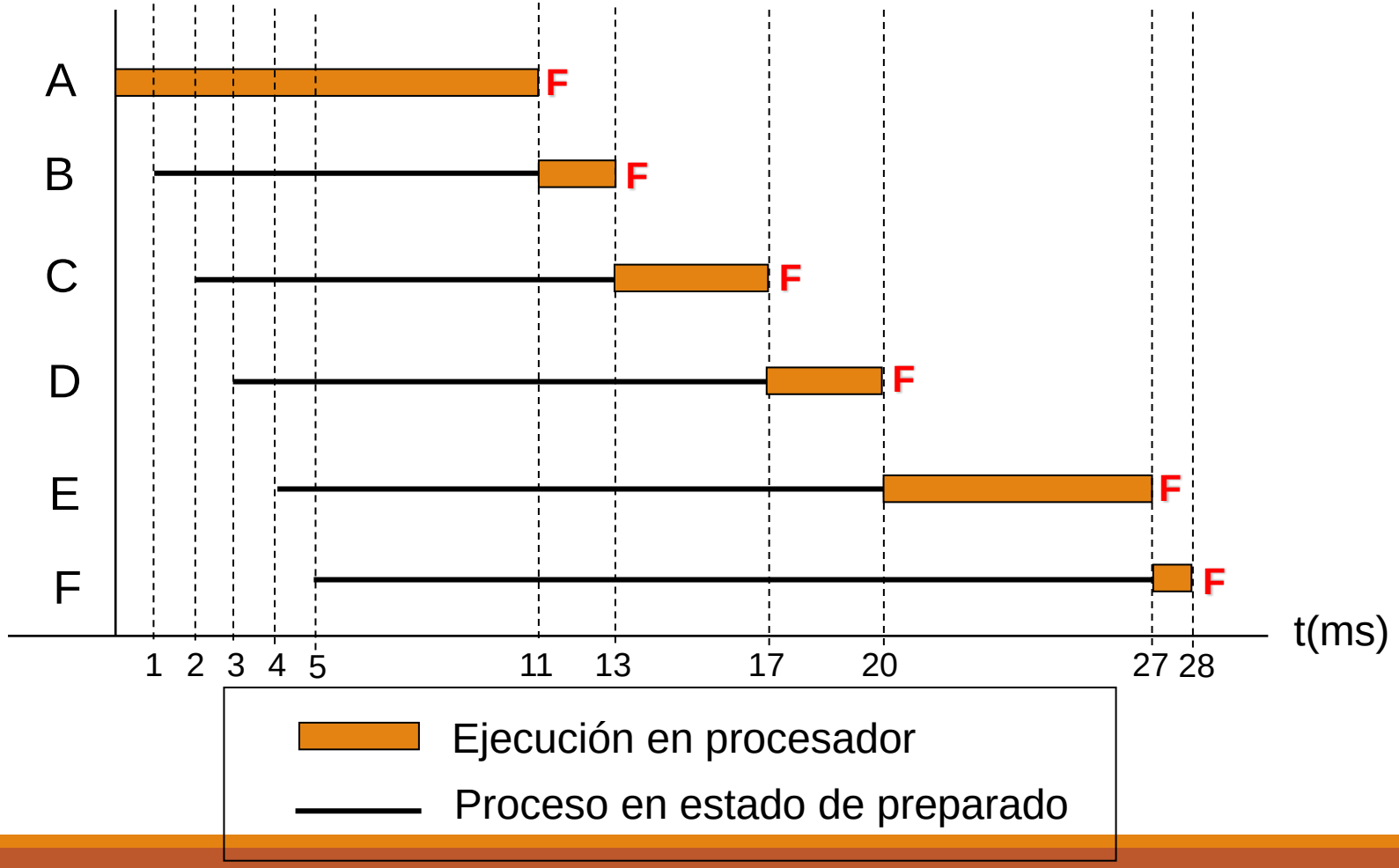
- Representar su evolución temporal
- Calcular para cada uno de los trabajos:
  - el tiempo de retorno,
  - de espera y
  - de respuesta.

Ejemplo

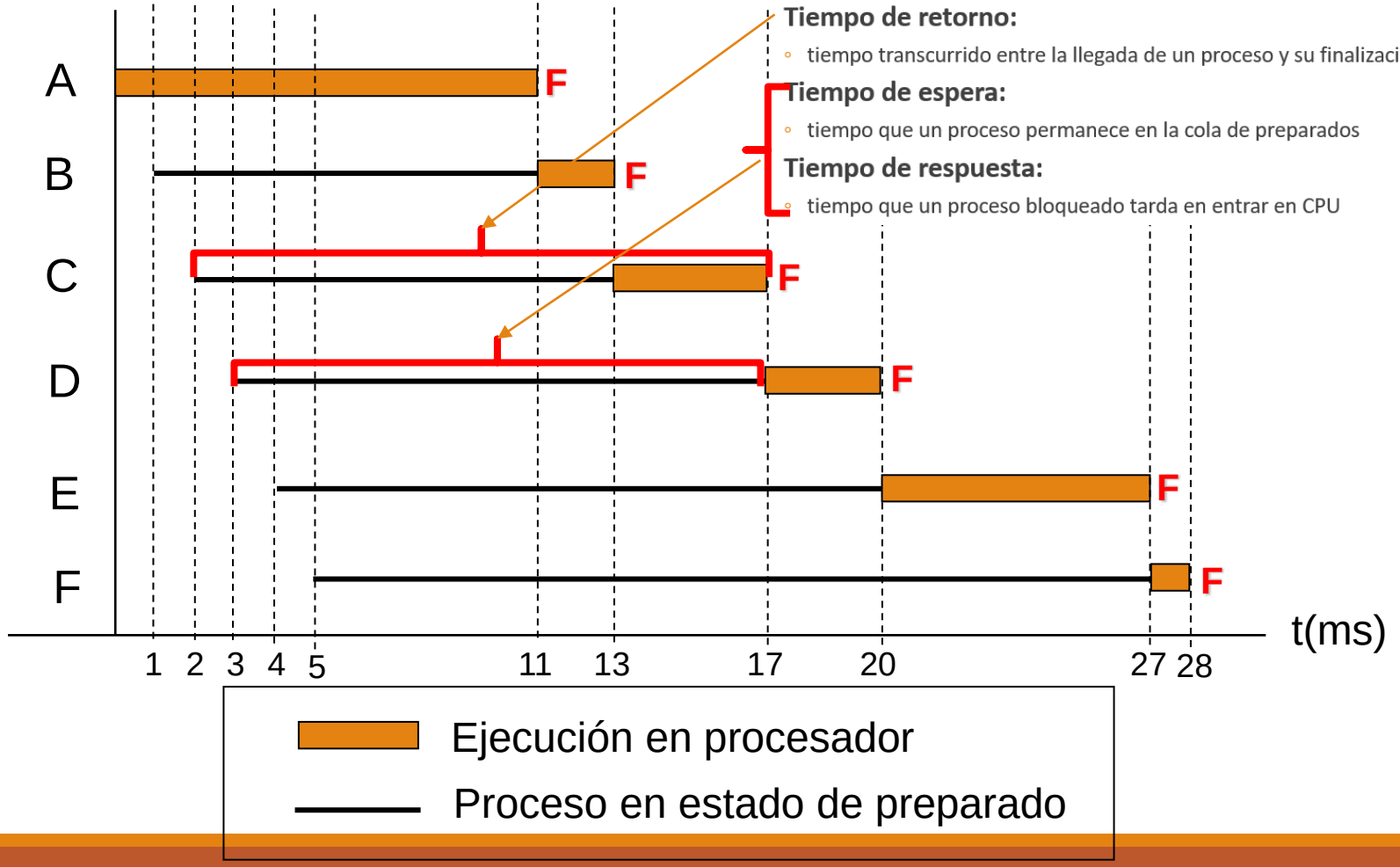


Proceso	Tiempo de llegada (ms)	Tiempo de ejecución (ms)
A	0	11
B	1	2
C	2	4
D	3	3
E	4	7
F	5	1

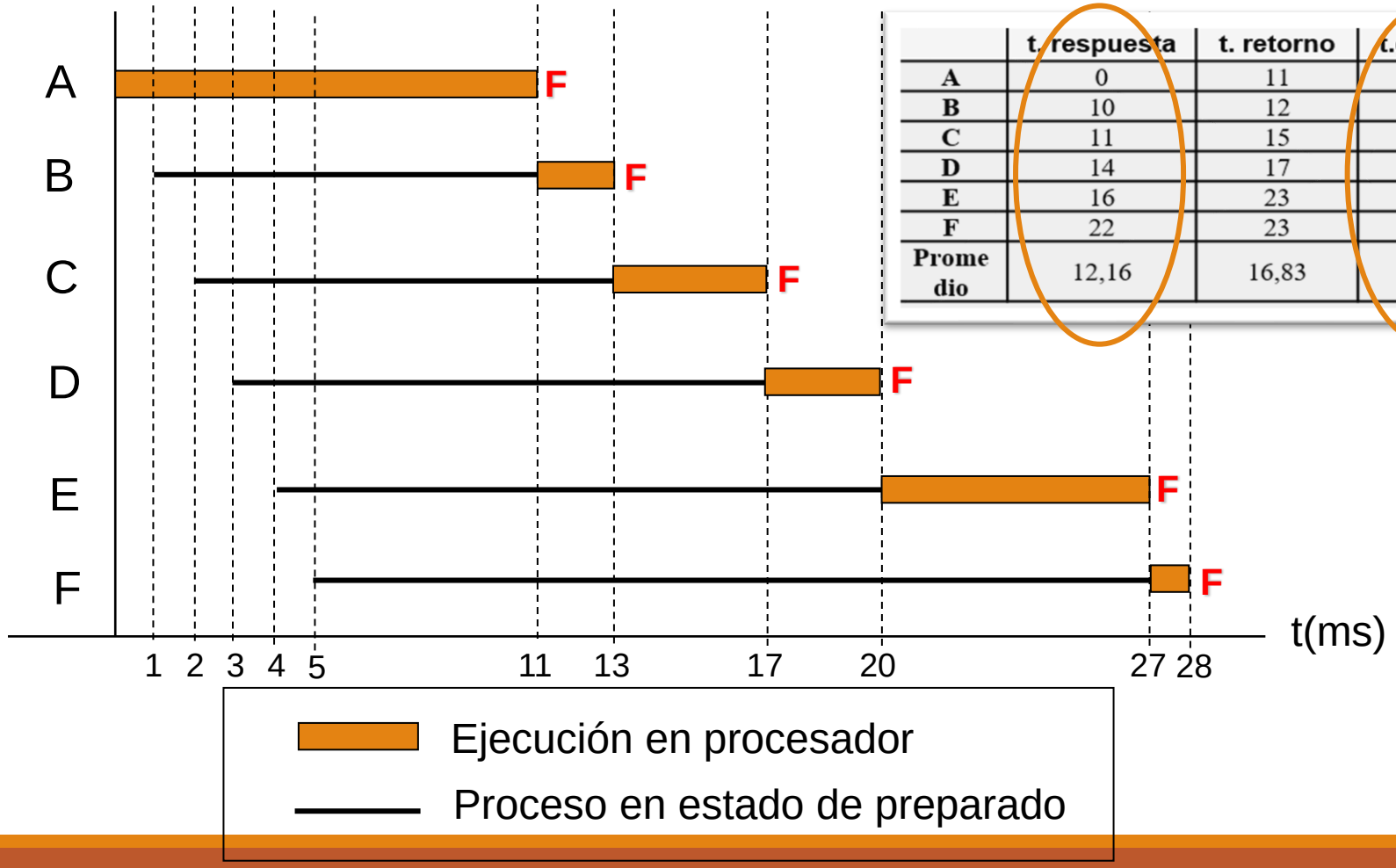
Solución
FCFS (First Come First Served)



Solución
FCFS (First Come First Served)



Solución
FCFS (First Come First Served)



	t. respuesta	t. retorno	t.espera
A	0	11	0
B	10	12	10
C	11	15	11
D	14	17	14
E	16	23	16
F	22	23	22
Promedio	12,16	16,83	12,16

Ejercicio 2
FCFS (First Come First Served)

Sea un sistema con los procesos que se muestran en la tabla.

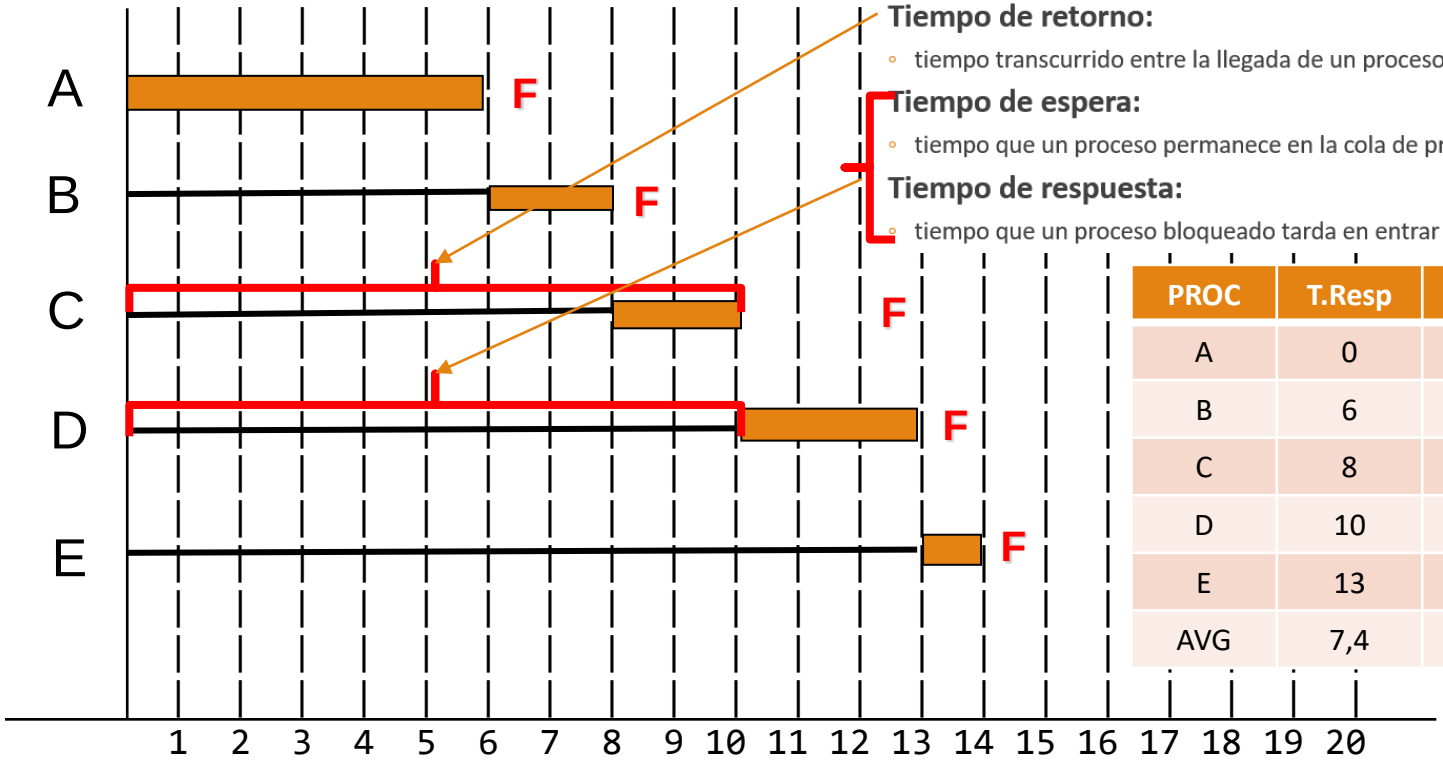
- Representar su evolución temporal
- Calcular para cada uno de los trabajos:
  - el tiempo de retorno,
  - de espera y
  - de respuesta.



Orden alfabético

Proceso	Tiempo de llegada	Tiempo de ejecución
A	0	6
B	0	2
C	0	2
D	0	3
E	0	1

Solución
FCFS (First Come First Served)



PROC	T.Resp	T.Ret	T.Esp
A	0	6	0
B	6	8	6
C	8	10	8
D	10	13	10
E	13	14	13
AVG	7,4	10,2	7,4

Ejecución en procesador

Proceso en estado de preparado

Ordenación por PID

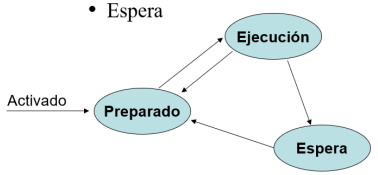
Turno rotatorio [Round Robin (RR)]

Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera

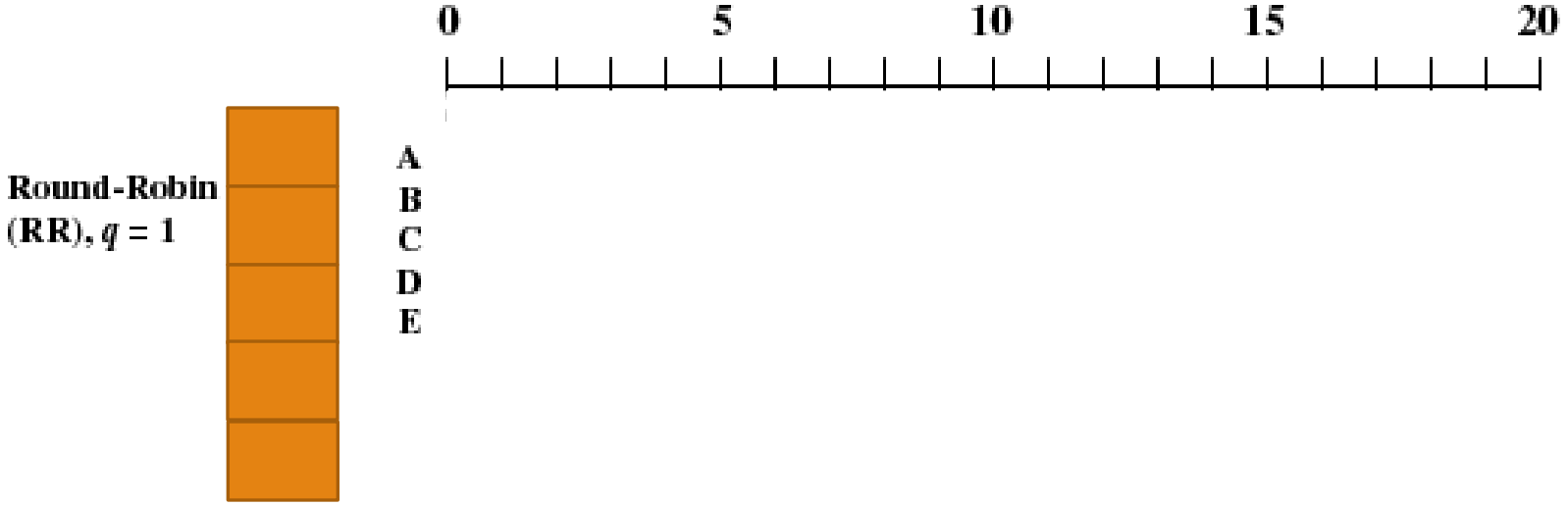


Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Turno rotatorio [Round Robin (RR)]



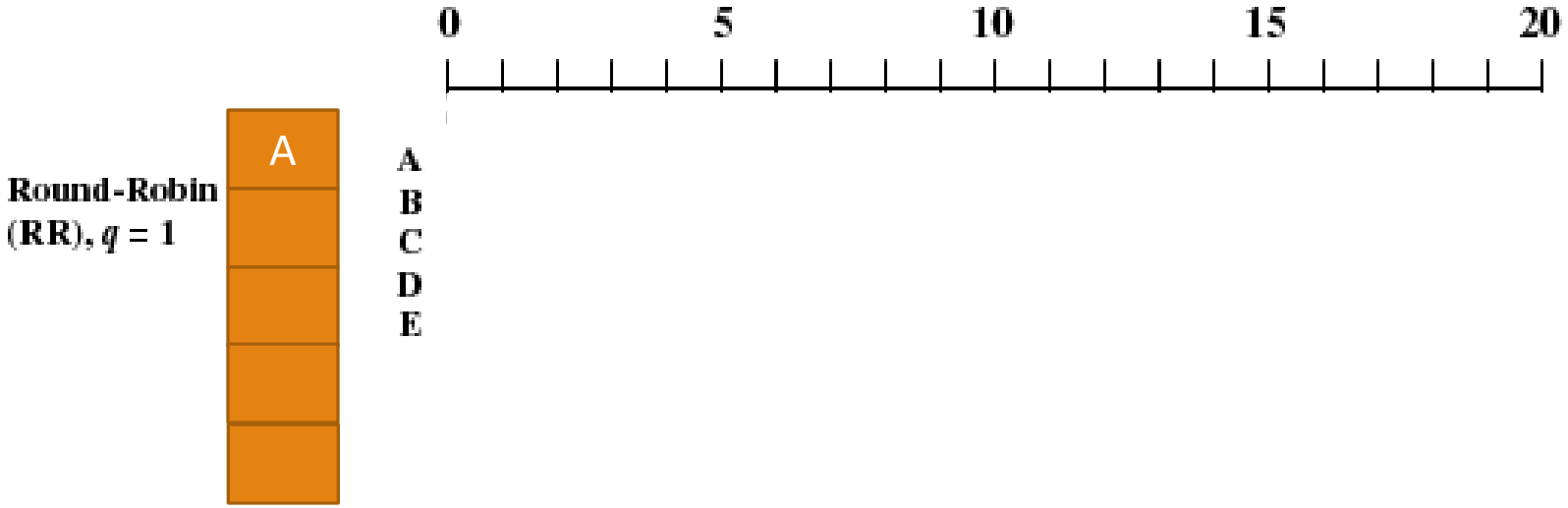
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

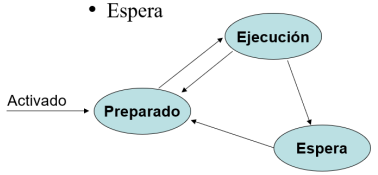
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

3



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



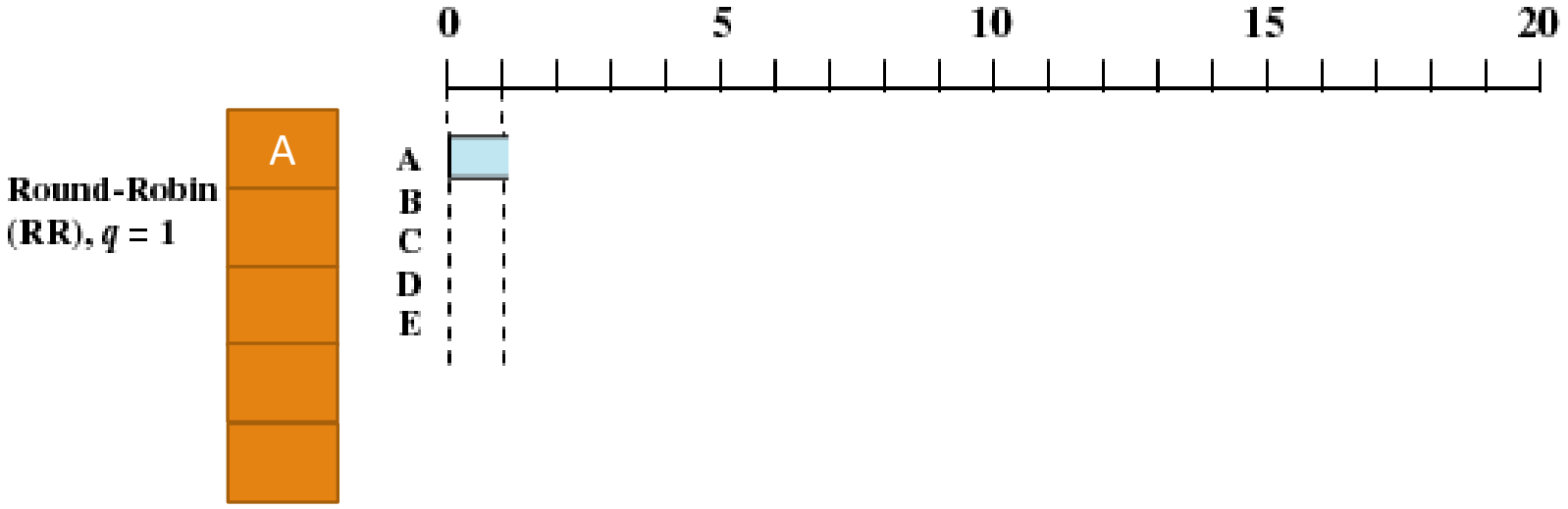
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

2



Turno rotatorio [Round Robin (RR)]



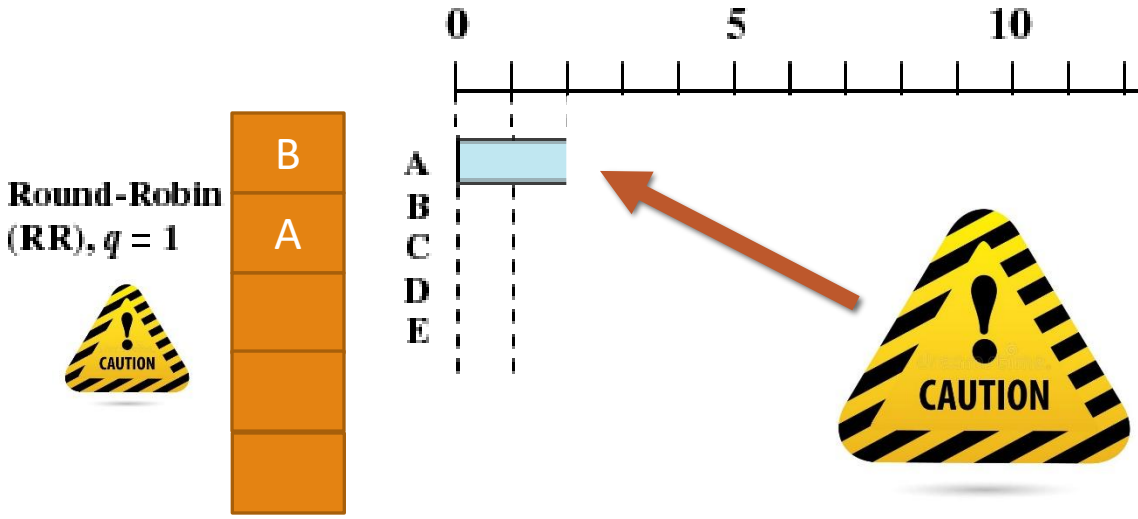
Utiliza la expulsión basándose en el reloj

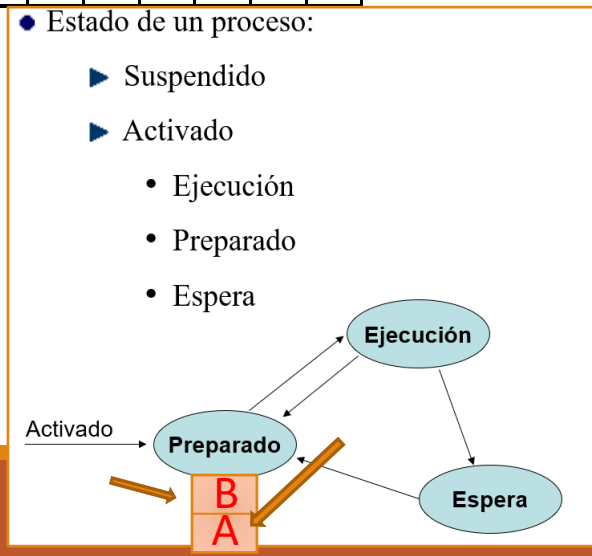
Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

1
6





Turno rotatorio [Round Robin (RR)]



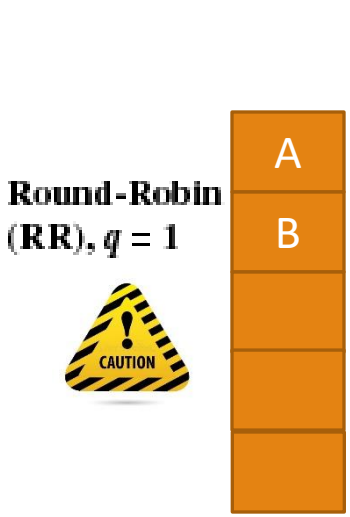
Utiliza la expulsión basándose en el reloj

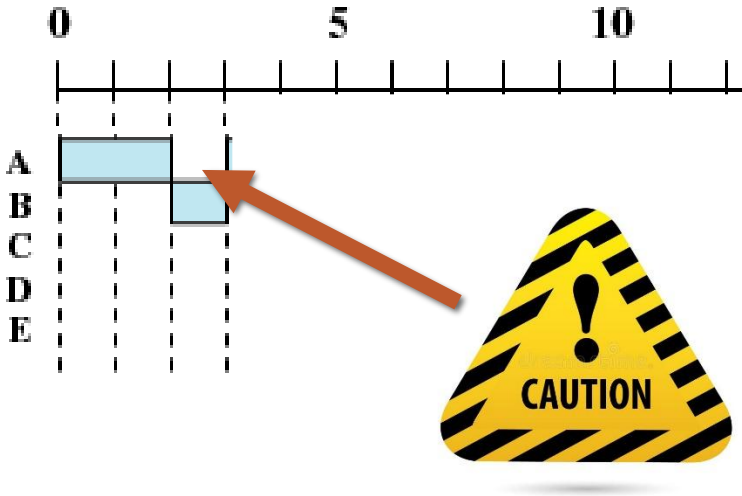
Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

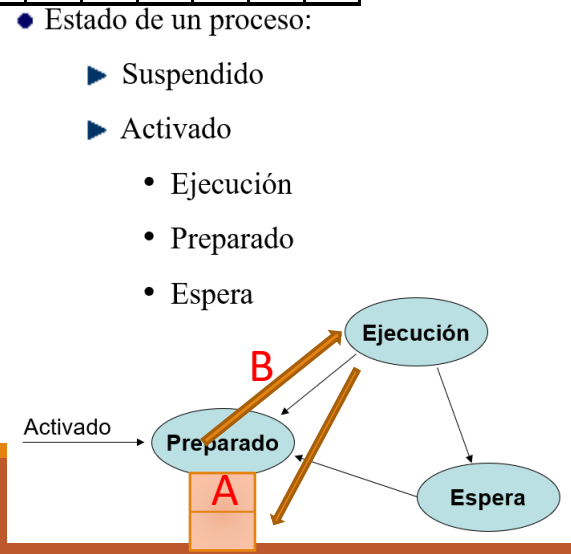
Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

1
5

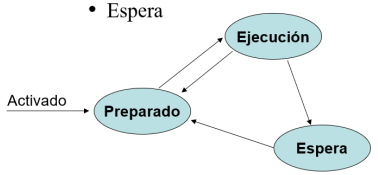






Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



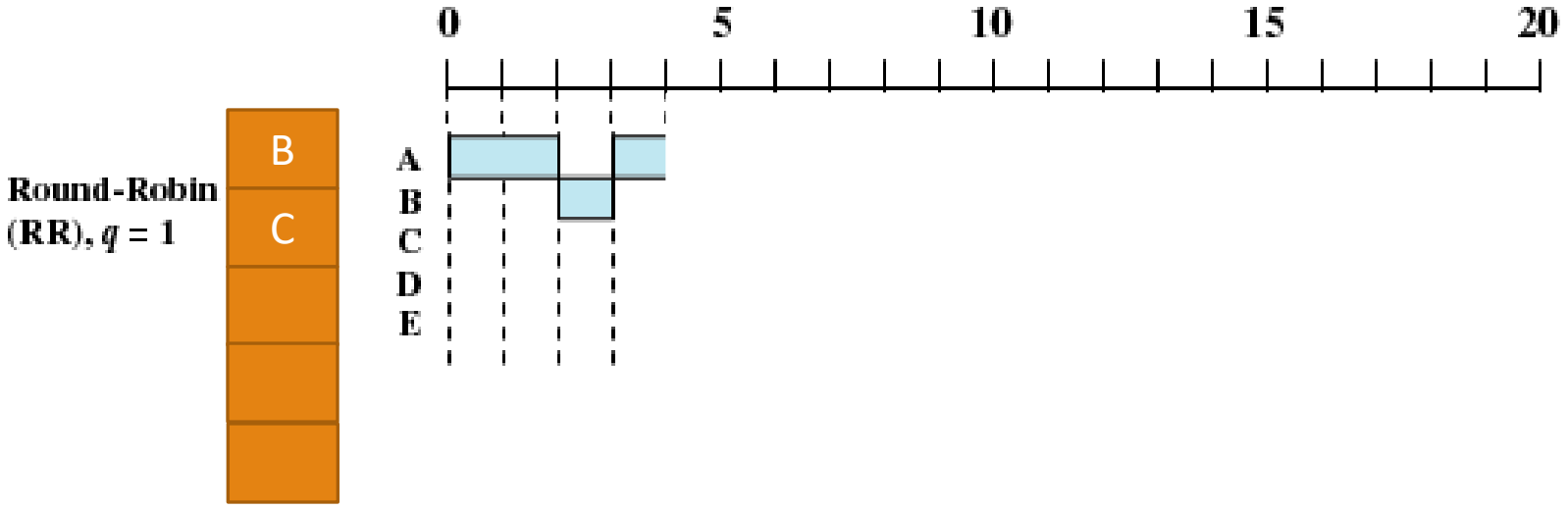
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
5
4



Turno rotatorio [Round Robin (RR)]



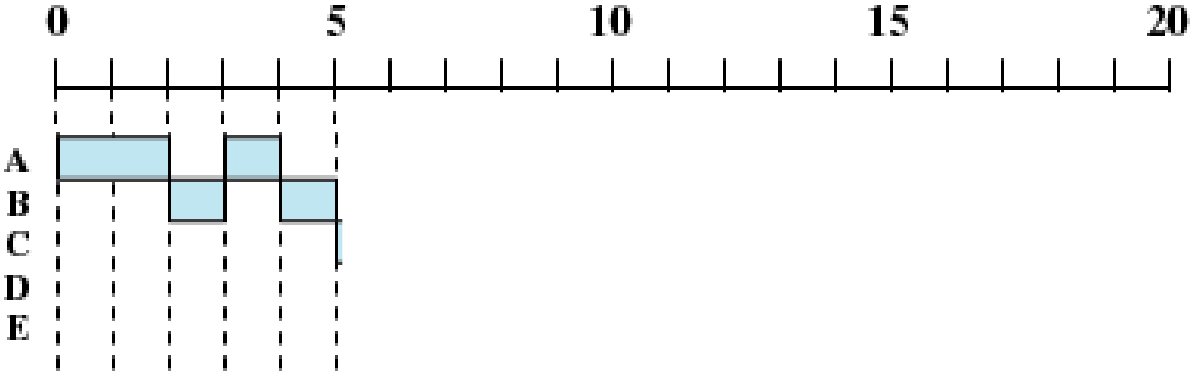
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

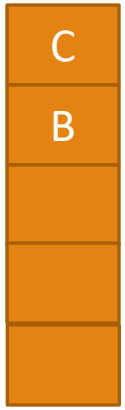
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
4
4



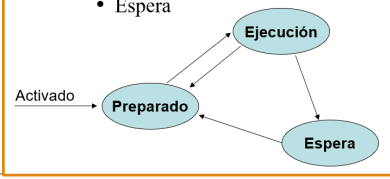
Round-Robin (RR), $q = 1$





Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



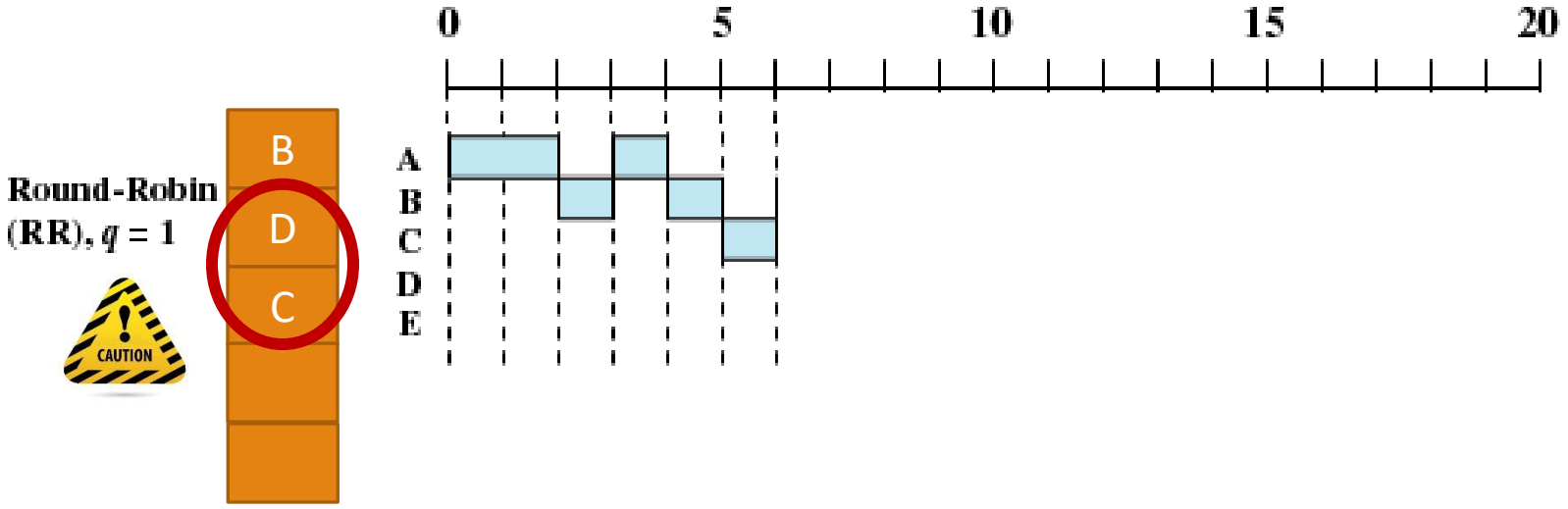
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

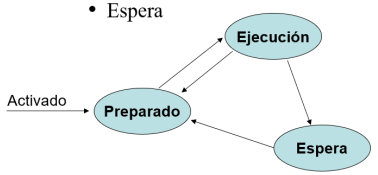
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
4
3
5



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



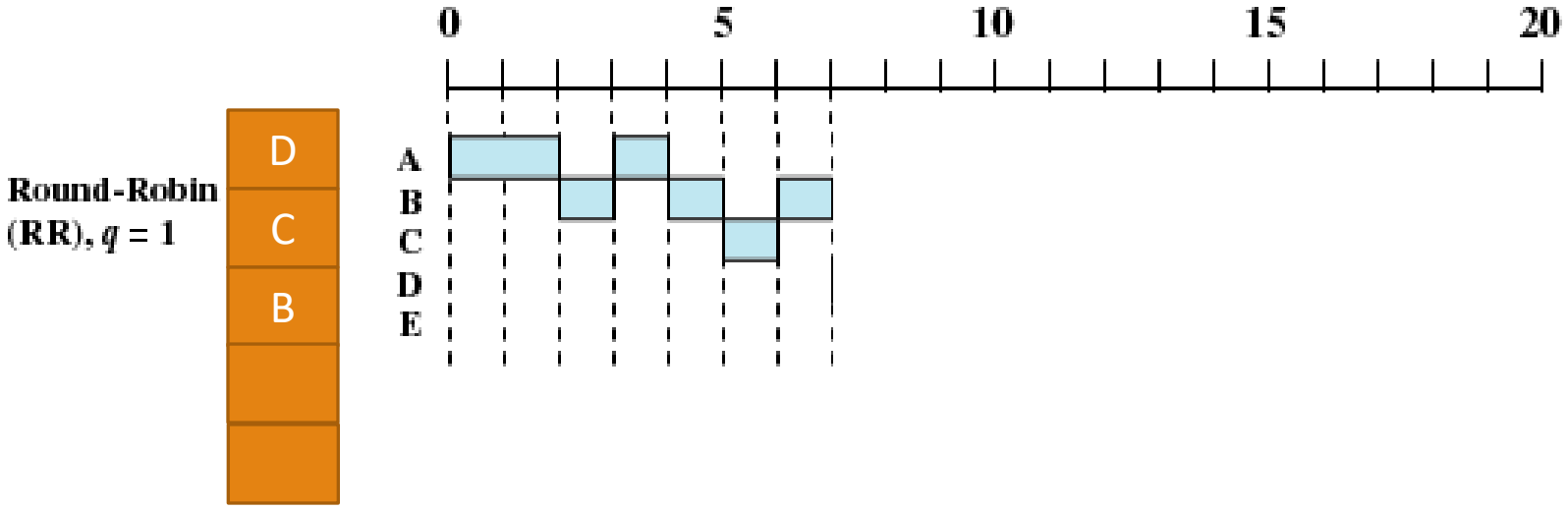
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

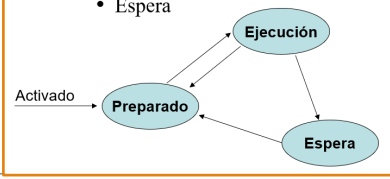
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
3
3
5



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



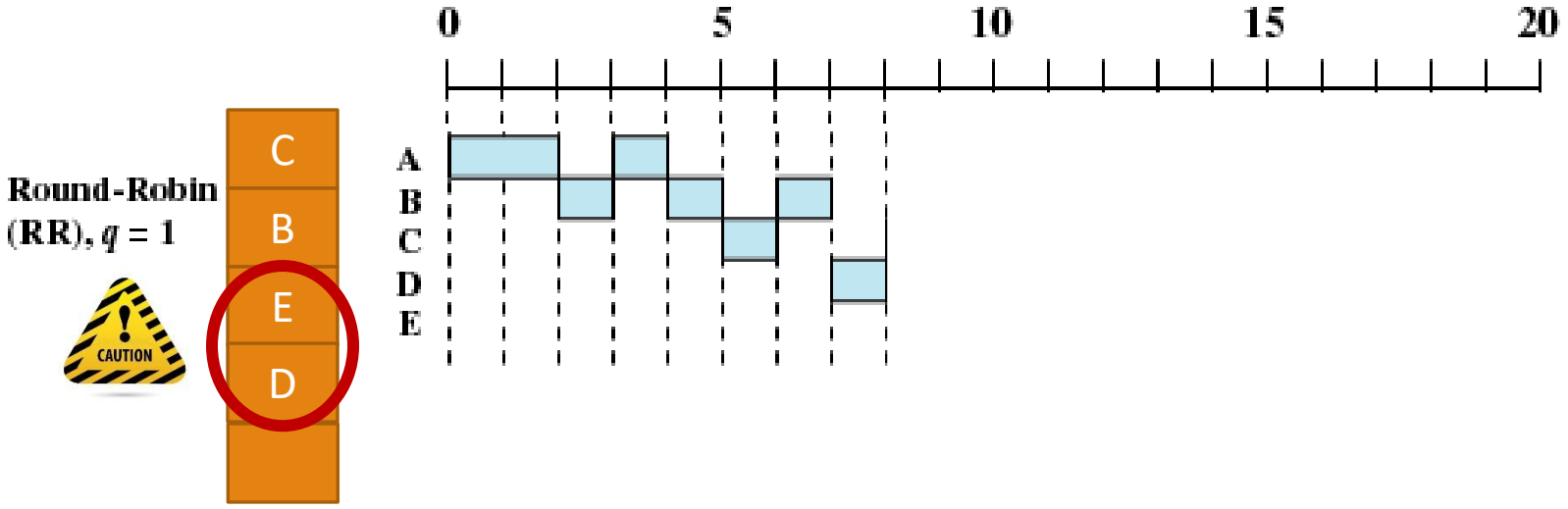
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

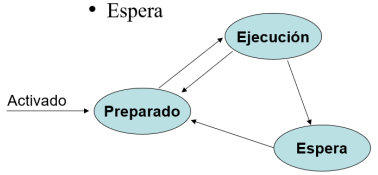
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
3
3
4
2



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



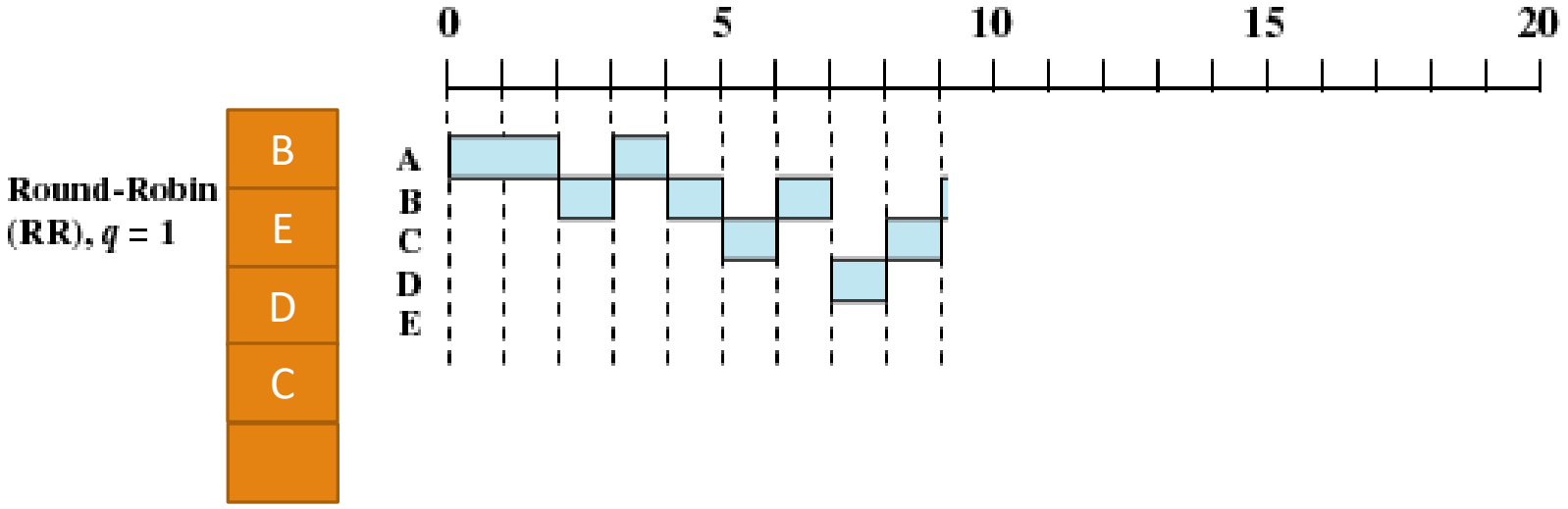
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
3
2
4
2



Turno rotatorio [Round Robin (RR)]



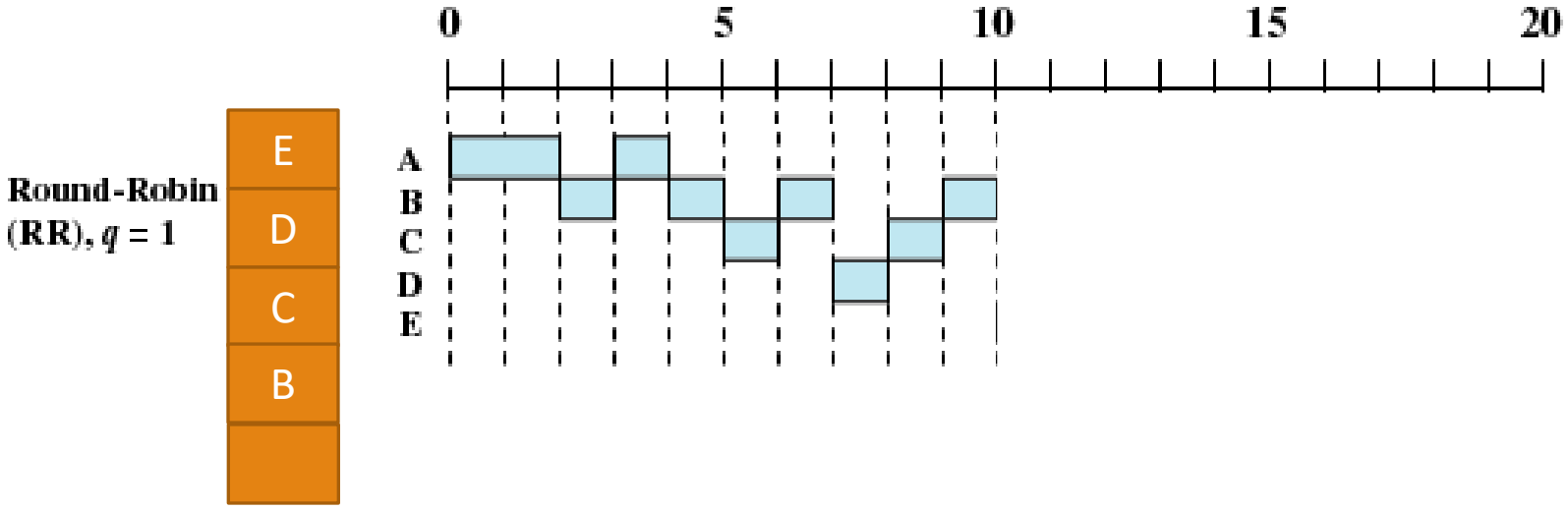
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

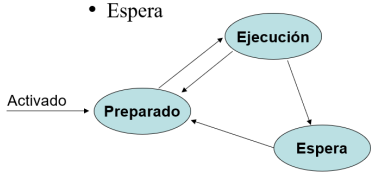
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
2
2
4
2



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



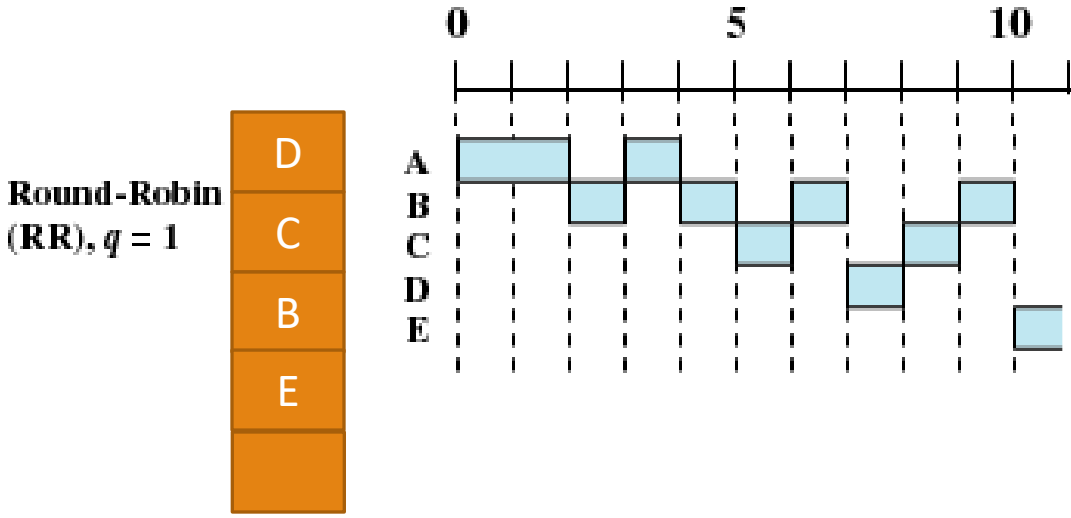
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

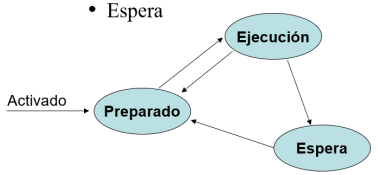
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
2
2
4
1



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



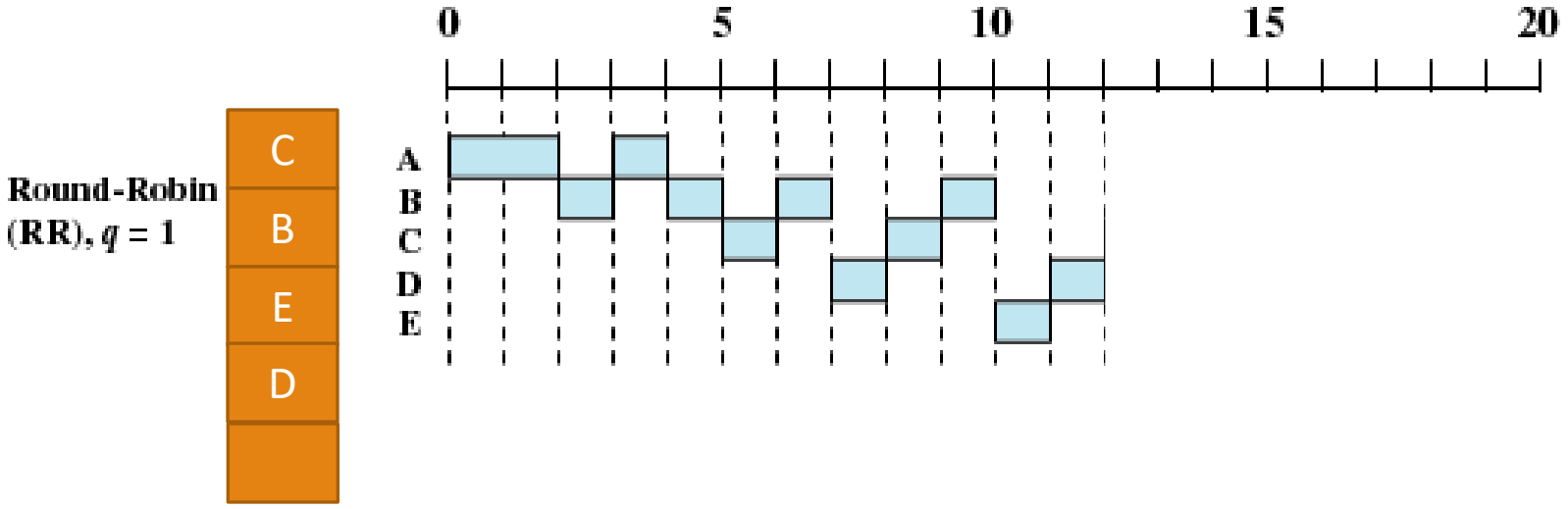
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

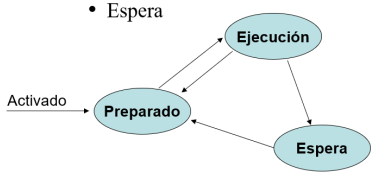
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
2
2
3
1



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



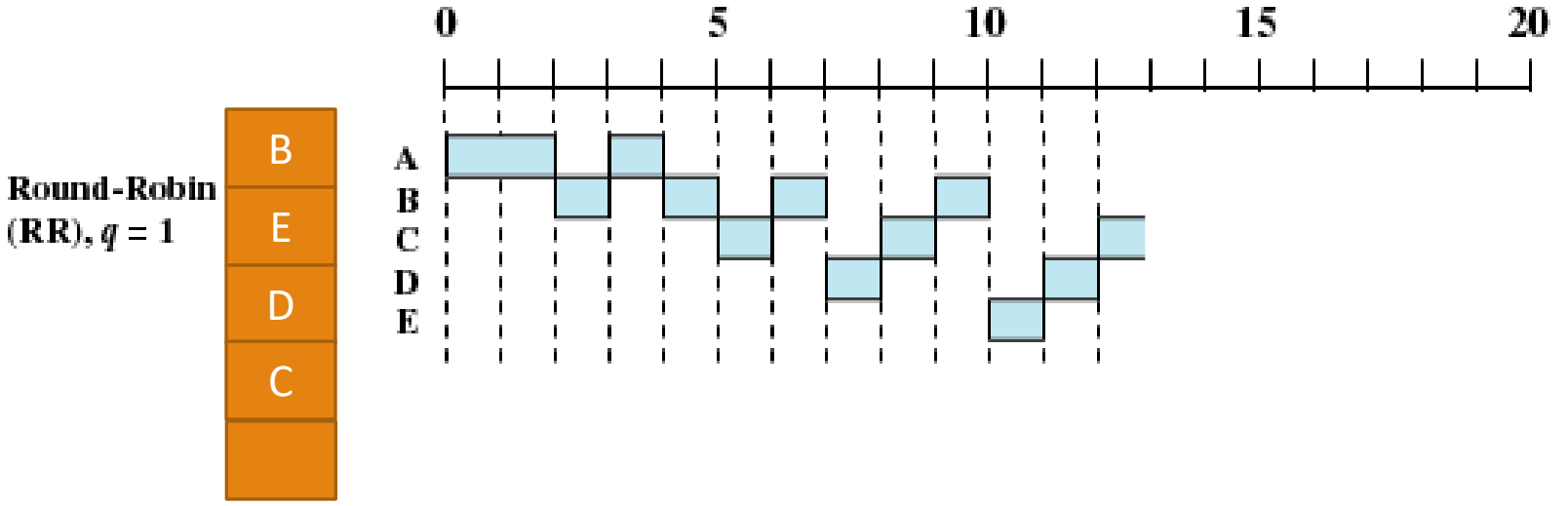
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

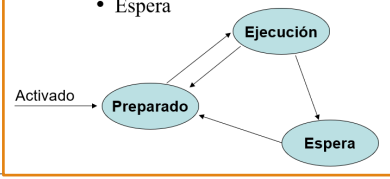
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
2
1
3
1



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



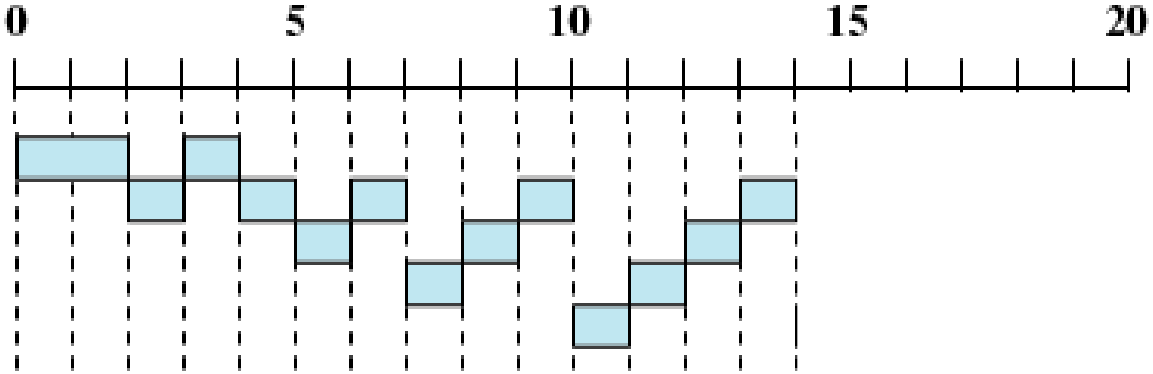
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

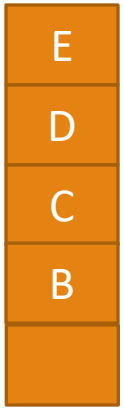
Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
1
1
3
1

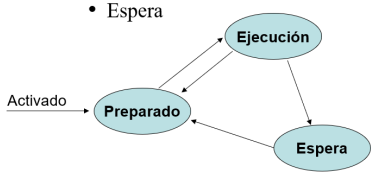


Round-Robin (RR), $q = 1$



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



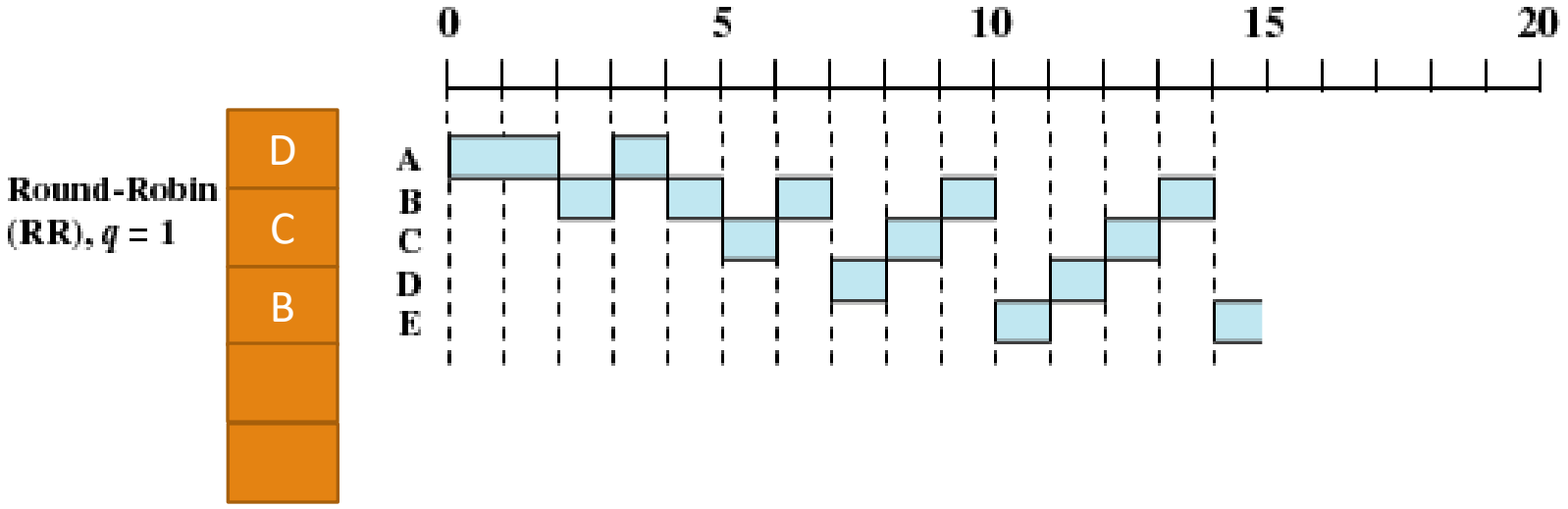
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

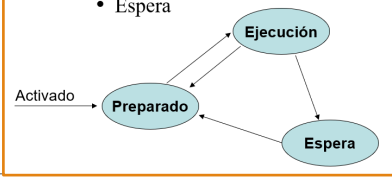
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
1
1
3
0



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



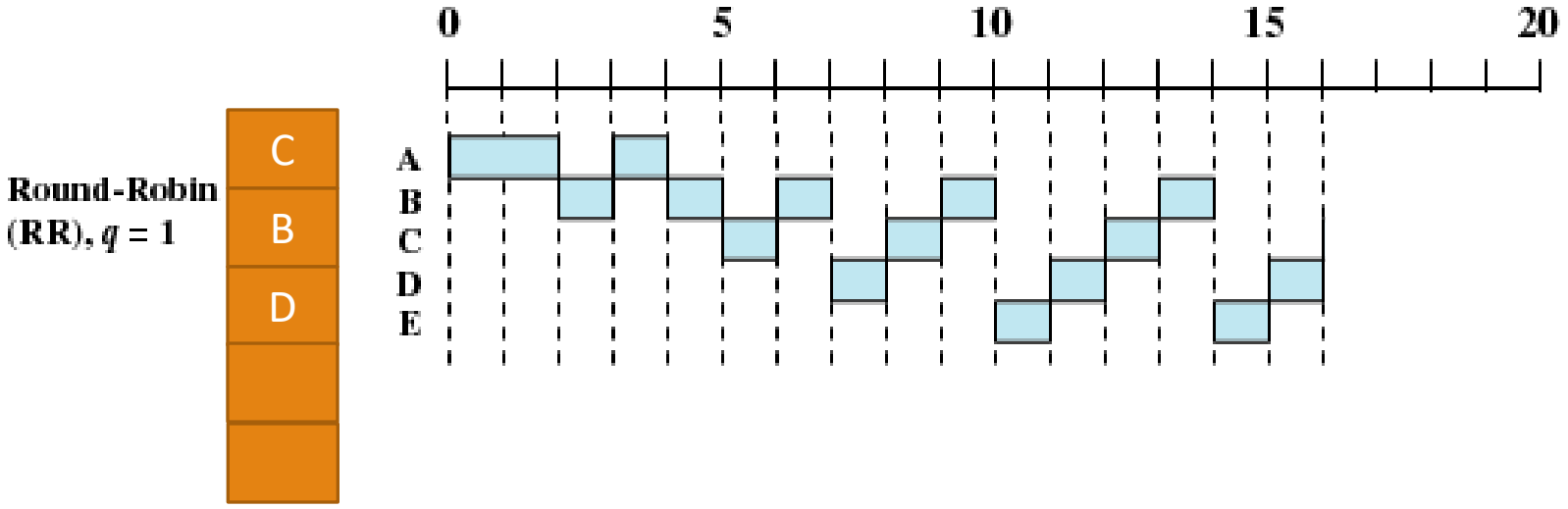
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
1
1
2
0



Turno rotatorio [Round Robin (RR)]



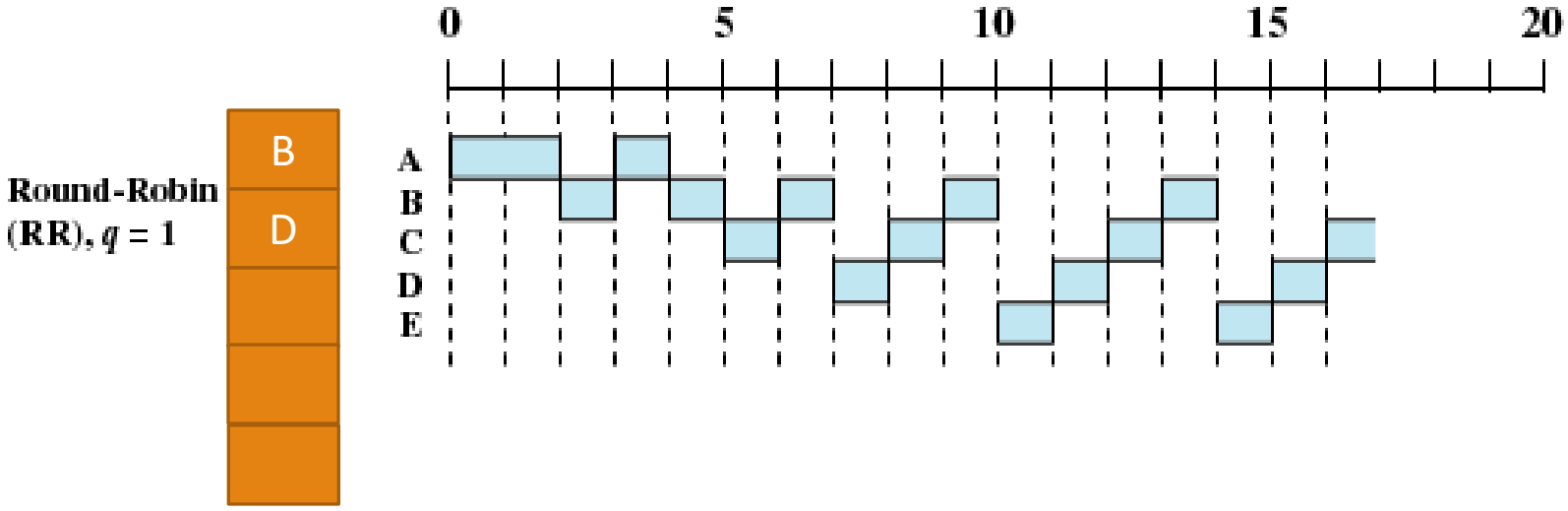
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

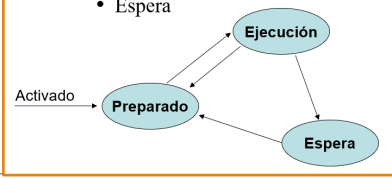
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0 1 0 2 0



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



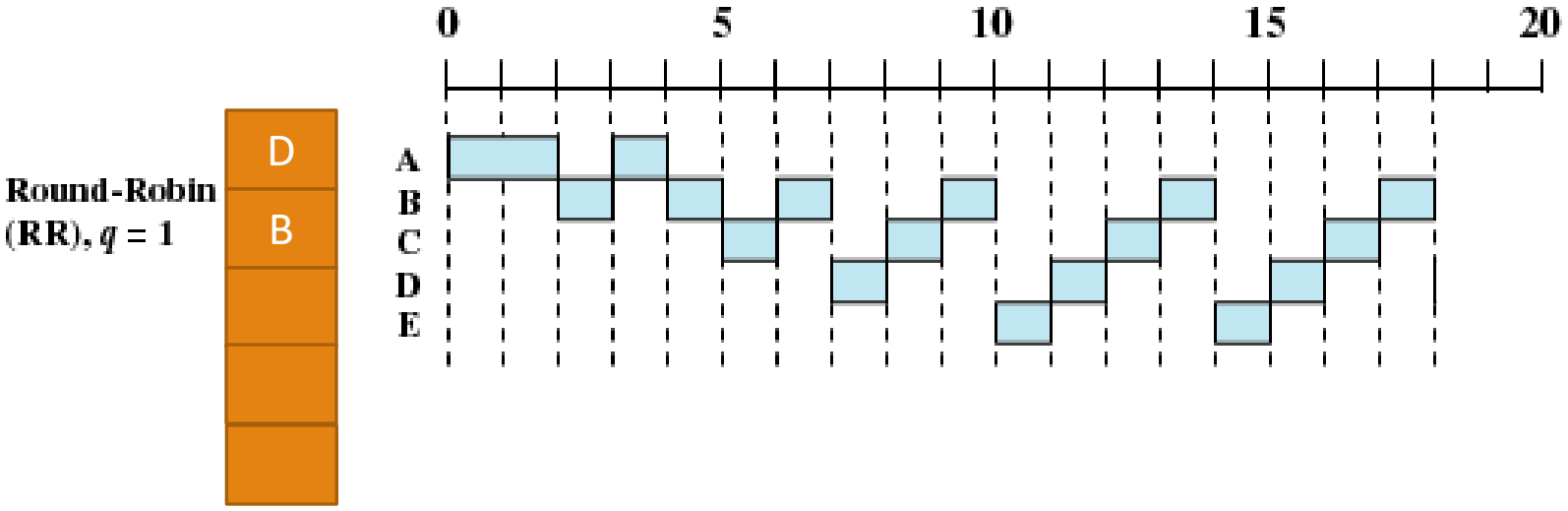
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
0
0
2
0



Turno rotatorio [Round Robin (RR)]



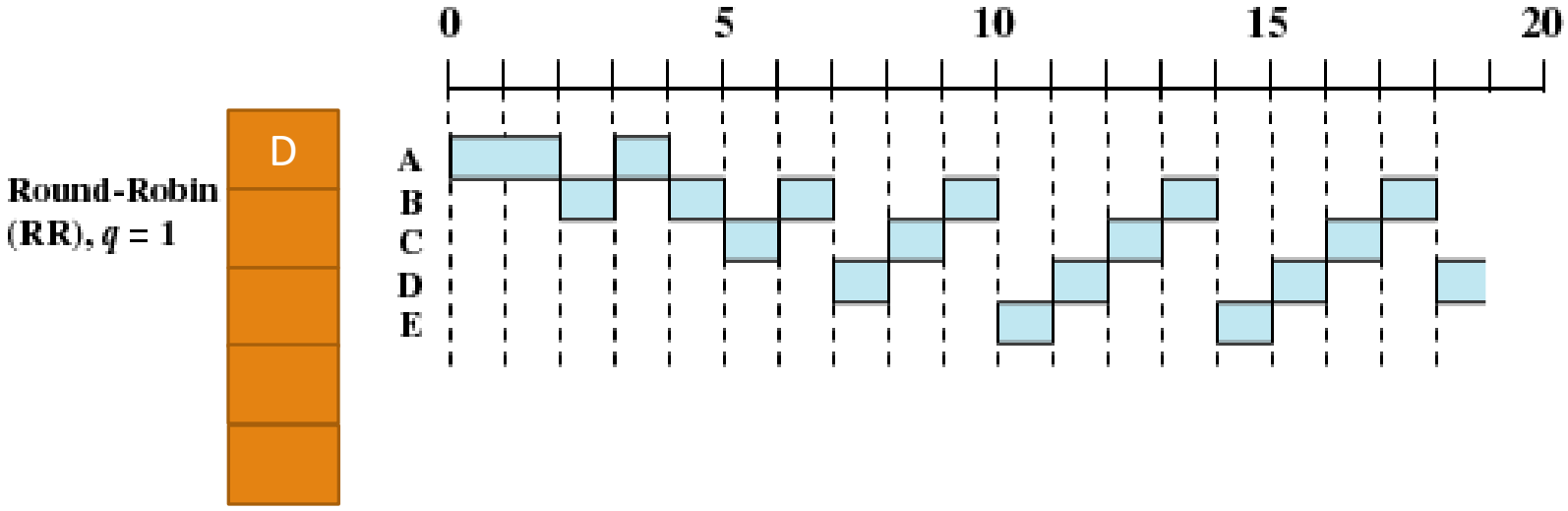
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

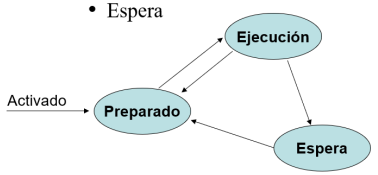
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
0
0
1
0



Turno rotatorio [Round Robin (RR)]

- Estado de un proceso:
  - Suspendido
  - Activado
    - Ejecución
    - Preparado
    - Espera



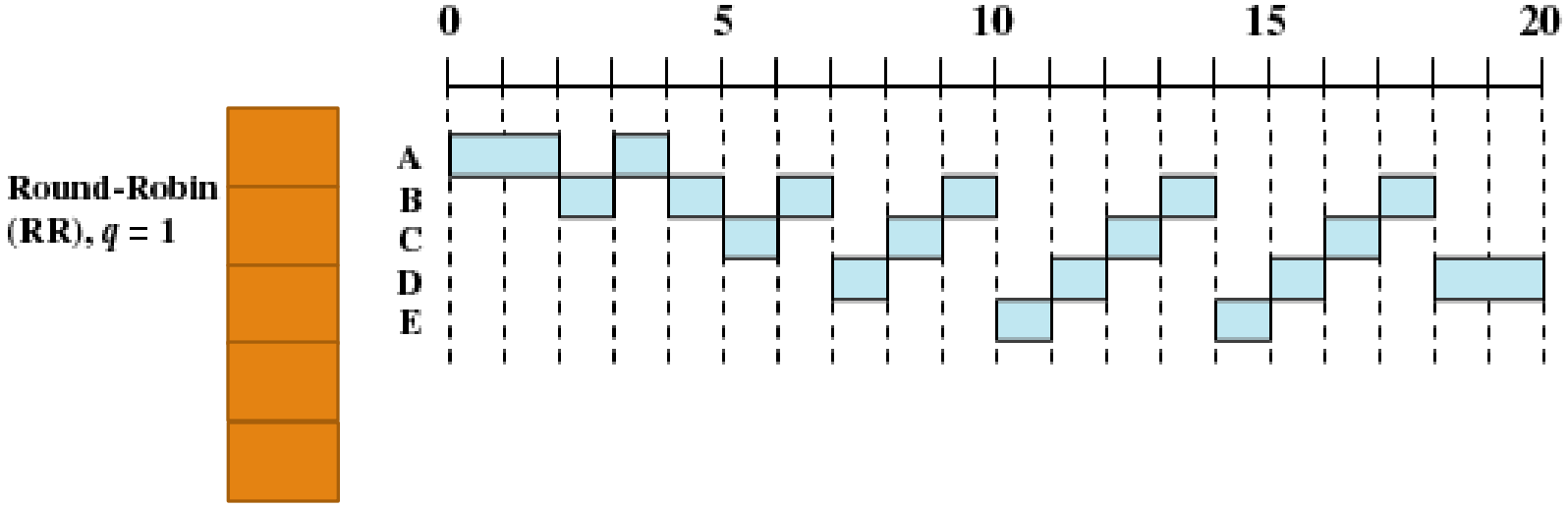
Utiliza la expulsión basándose en el reloj

Se determina una cantidad de tiempo (quantum), lo que permite que cada proceso utilice el procesador durante ese intervalo

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
0
0
0
0



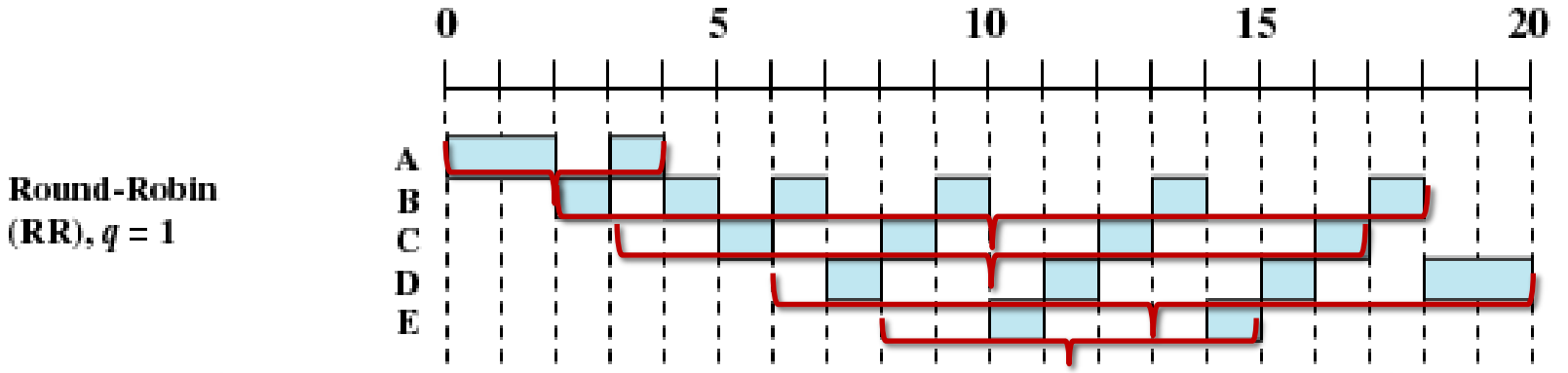
Turno rotatorio [Round Robin (RR)]

Las interrupciones de reloj se suceden cada cierto intervalo de tiempo

- time slicing → Evita la penalización de los procesos cortos

Cuando sucede la interrupción:

- El proceso actual (ejecución) se sitúa en la cola de listos
- Se selecciona el siguiente trabajo



	Process	A	B	C	D	E	
	Arrival Time	0	2	4	6	8	
	Service Time (T_s)	3	6	4	5	2	Mean
RR $q = 1$	Finish Time	4	18	17	20	15	
	Turnaround Time (T_r)	4	16	13	14	7	10.80
	T_r/T_s	1.33	2.67	3.25	2.80	3.50	2.71

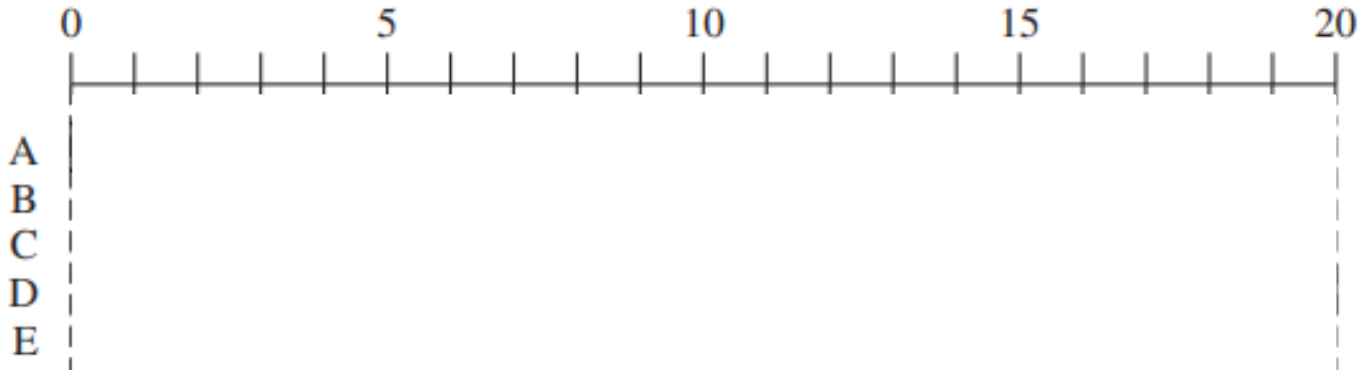
Ejercicio 1
Round Robin (RR)

Repetir el ejercicio con q=4

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round robin (RR), $q = 4$



Solución
Round Robin (RR)

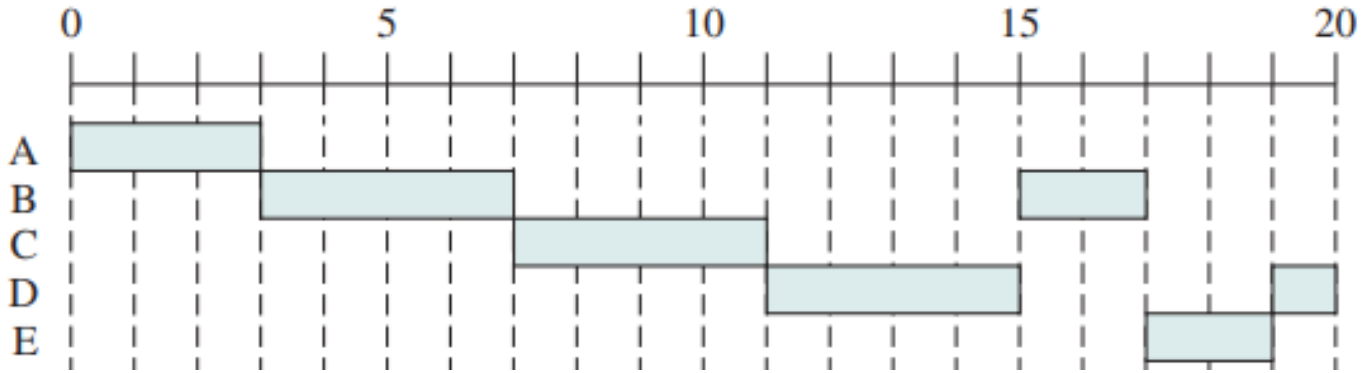
Stallings, p.435

Repetir el ejercicio con q=4

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round robin (RR), $q = 4$



Solución
Round Robin (RR)

Stallings, p.435

Repetir el ejercicio con q=4

Tiempo de retorno:
- tiempo transcurrido entre la llegada de un proceso y su finalización

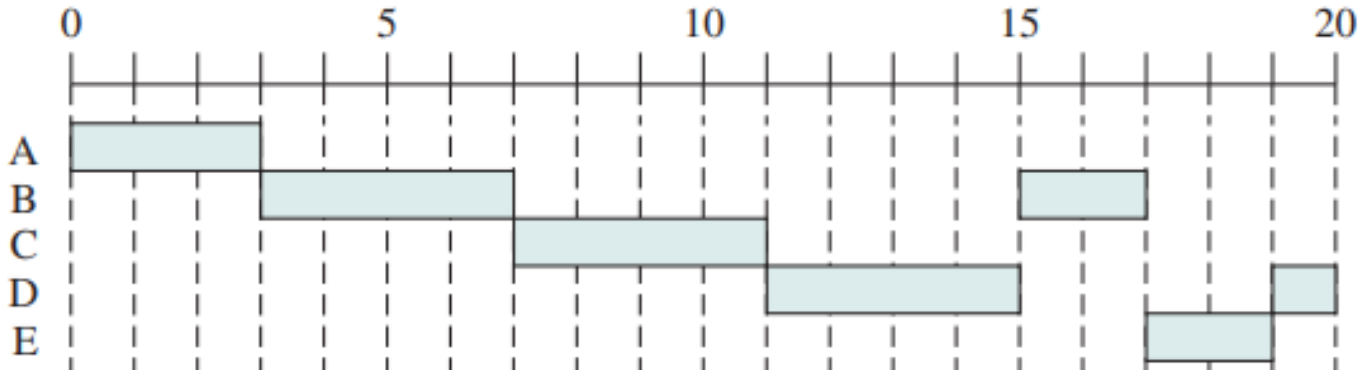
Tiempo de espera:
- tiempo que un proceso permanece en la cola de preparados

Tiempo de respuesta:
- tiempo que un proceso bloqueado tarda en entrar en CPU

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round robin (RR), $q = 4$



Solución
Round Robin (RR)

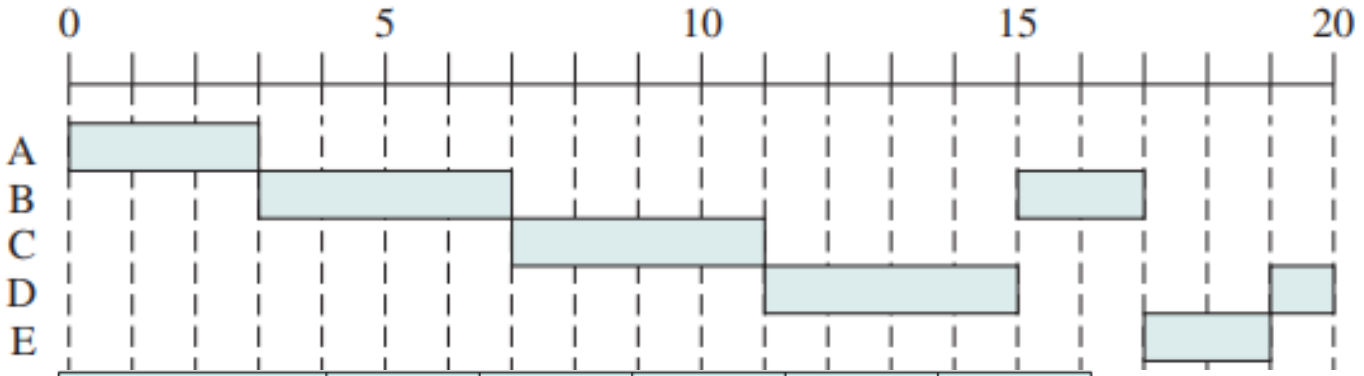
Stallings, p.435

Repetir el ejercicio con q=4

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Round robin (RR), $q = 4$



	Process	A	B	C	D	E	
	Arrival Time	0	2	4	6	8	
	Service Time (T_s)	3	6	4	5	2	Mean
RR $q = 4$	Finish Time	3	17	11	20	19	
	Turnaround Time (T_r)	3	15	7	14	11	10.00
	T_r/T_s	1.00	2.5	1.75	2.80	5.50	2.71

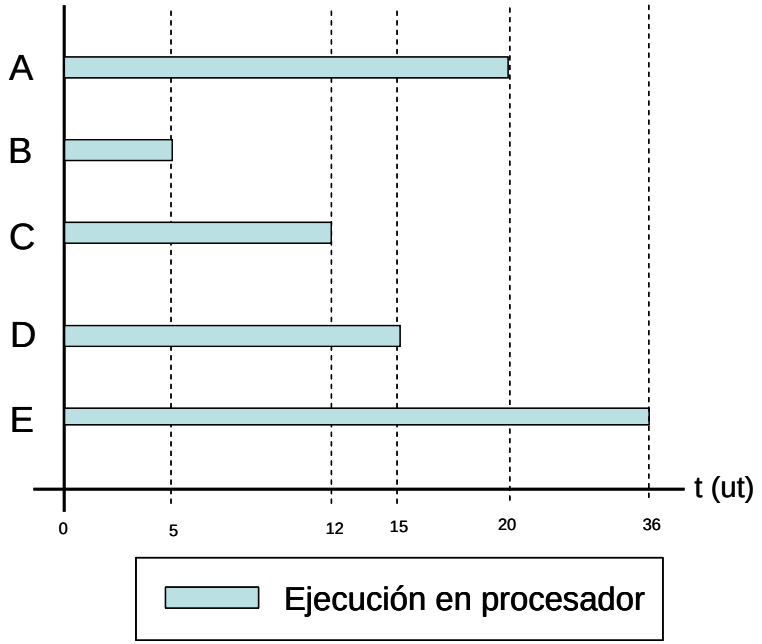
Ejercicio 2
Round Robin (RR)

Se dispone de un sistema monoprocesador con política de planificación del procesador **RR con q = 3 ut.**

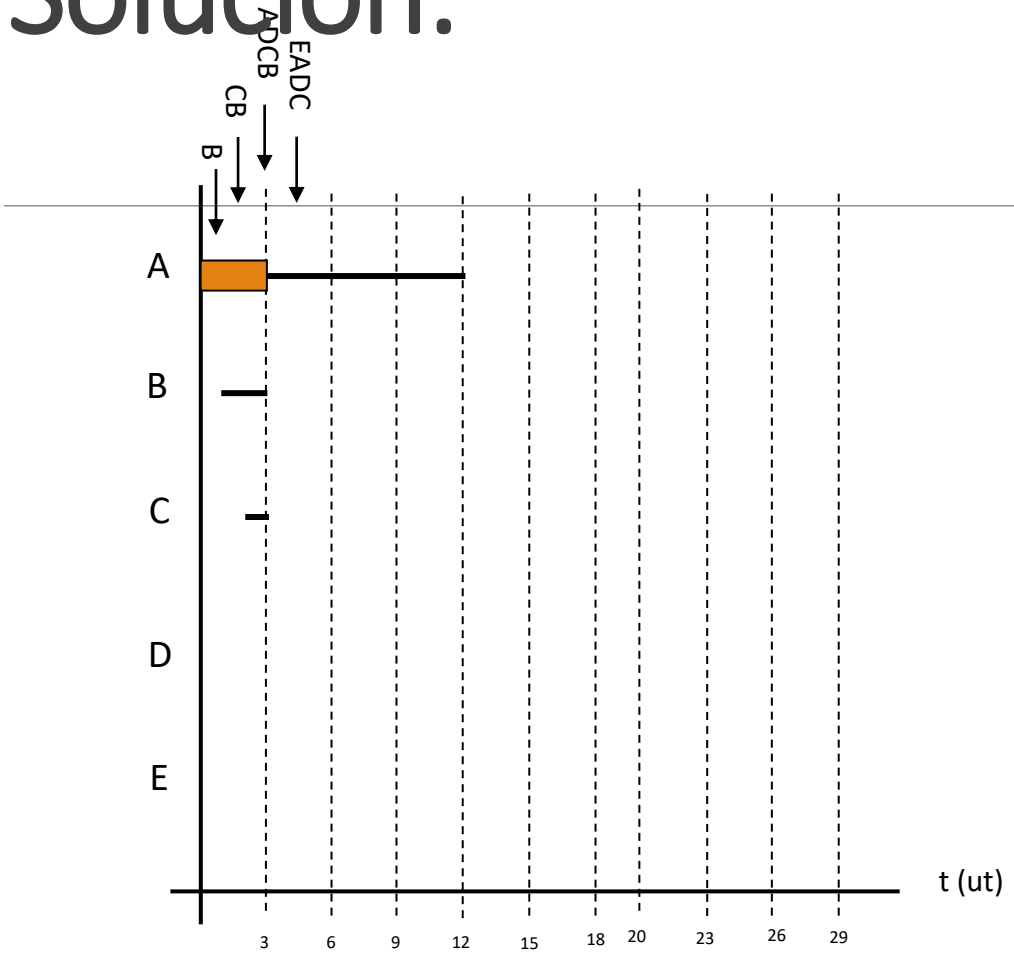
La ejecución de los procesos al sistema sigue el esquema descrito en la figura.

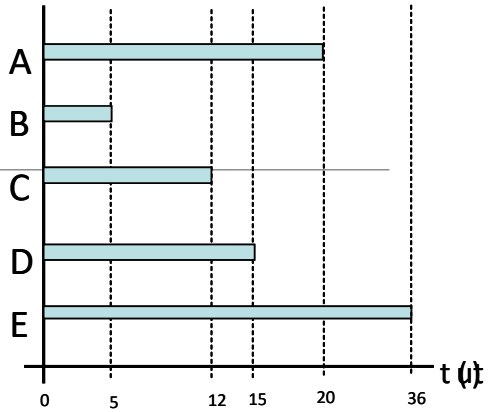
- El proceso A llega al sistema en el instante 0 ut.,
- el proceso B en el instante 1 ut.,
- el proceso C en 2 ut.,
- el proceso D en 3 ut. y
- el proceso E en 4 ut.

Si el quantum de un proceso en ejecución expira a la vez que la llegada de un nuevo proceso, entonces **el nuevo proceso se añade a la cola de procesos en espera** de ejecutarse antes que el proceso que termina.

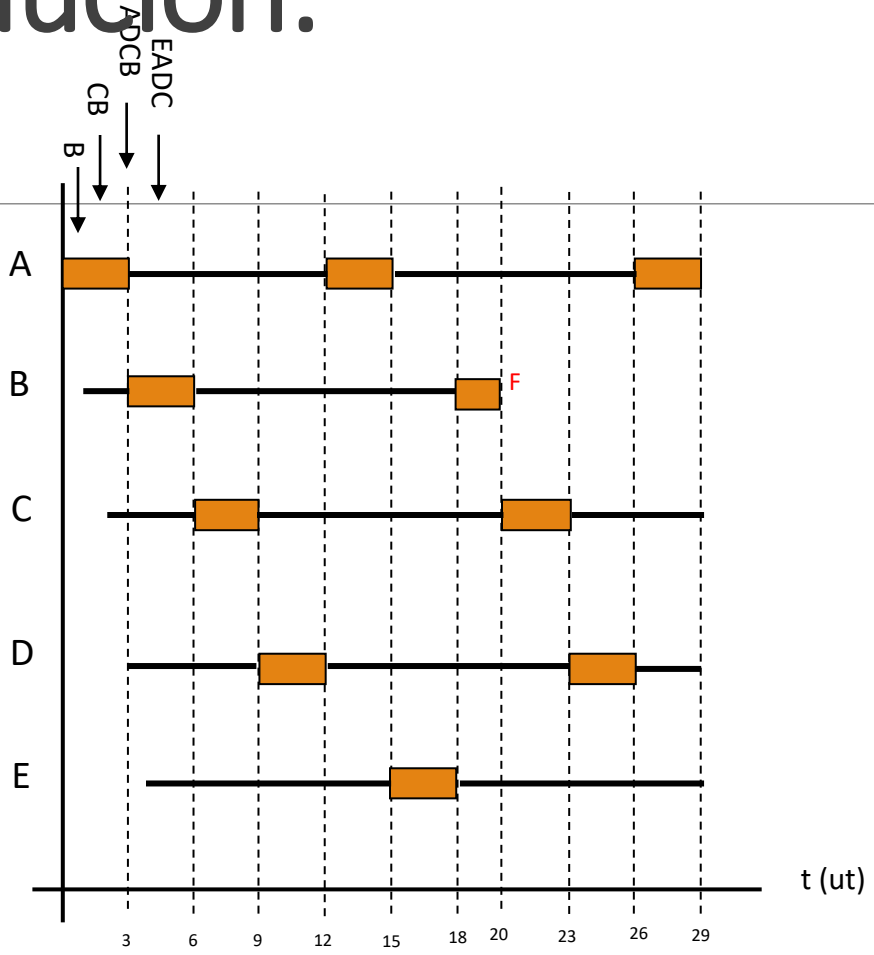


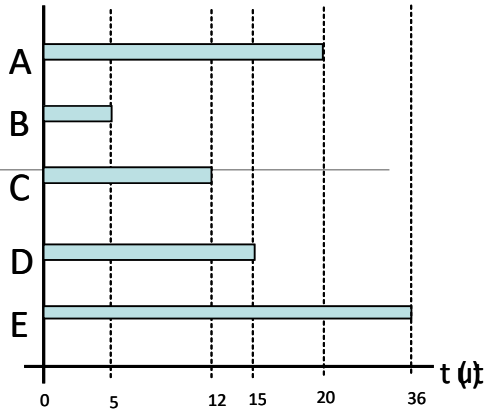
Solución:


EADC
ADCB
CB
B
A
B
C
D
E
t (ut)
3
6
9
12
15
18
20
23
26
29

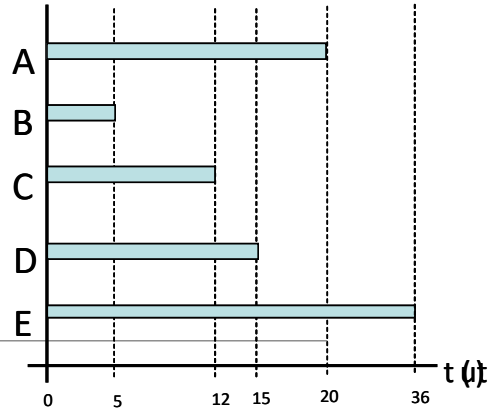

A
B
C
D
E
t (ut)
0
5
12
15
20
36

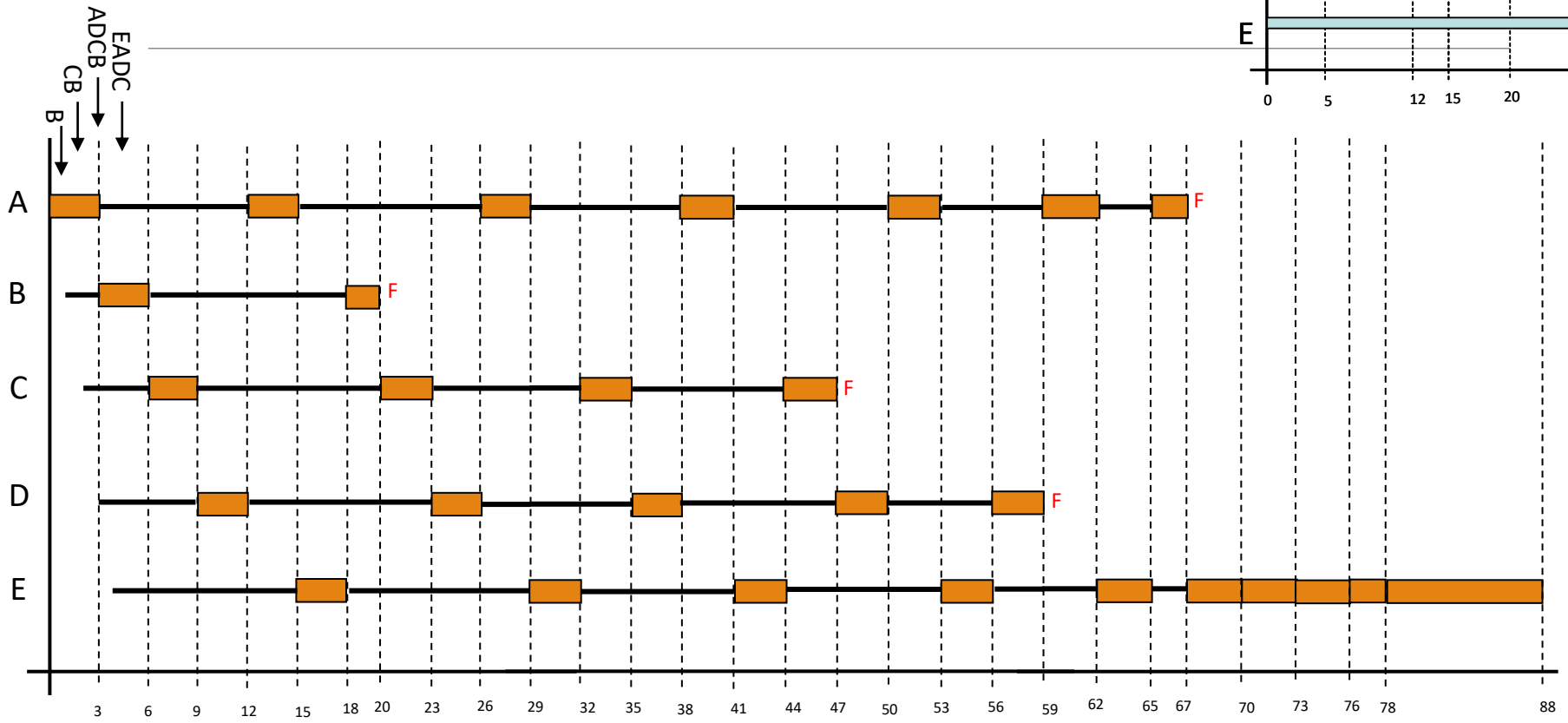
Solución:


EADC
ADCB
CB
B
A
B
C
D
E
F
t (ut)
3
6
9
12
15
18
20
23
26
29

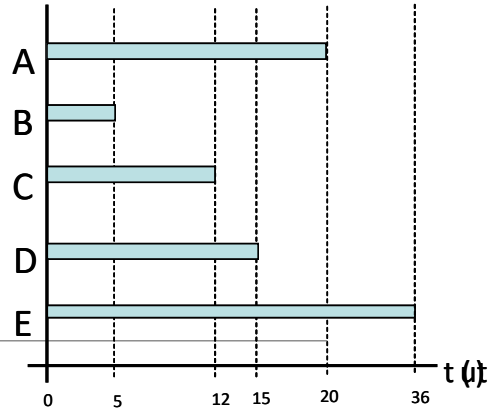

A
B
C
D
E
t (ut)
0
5
12
15
20
36

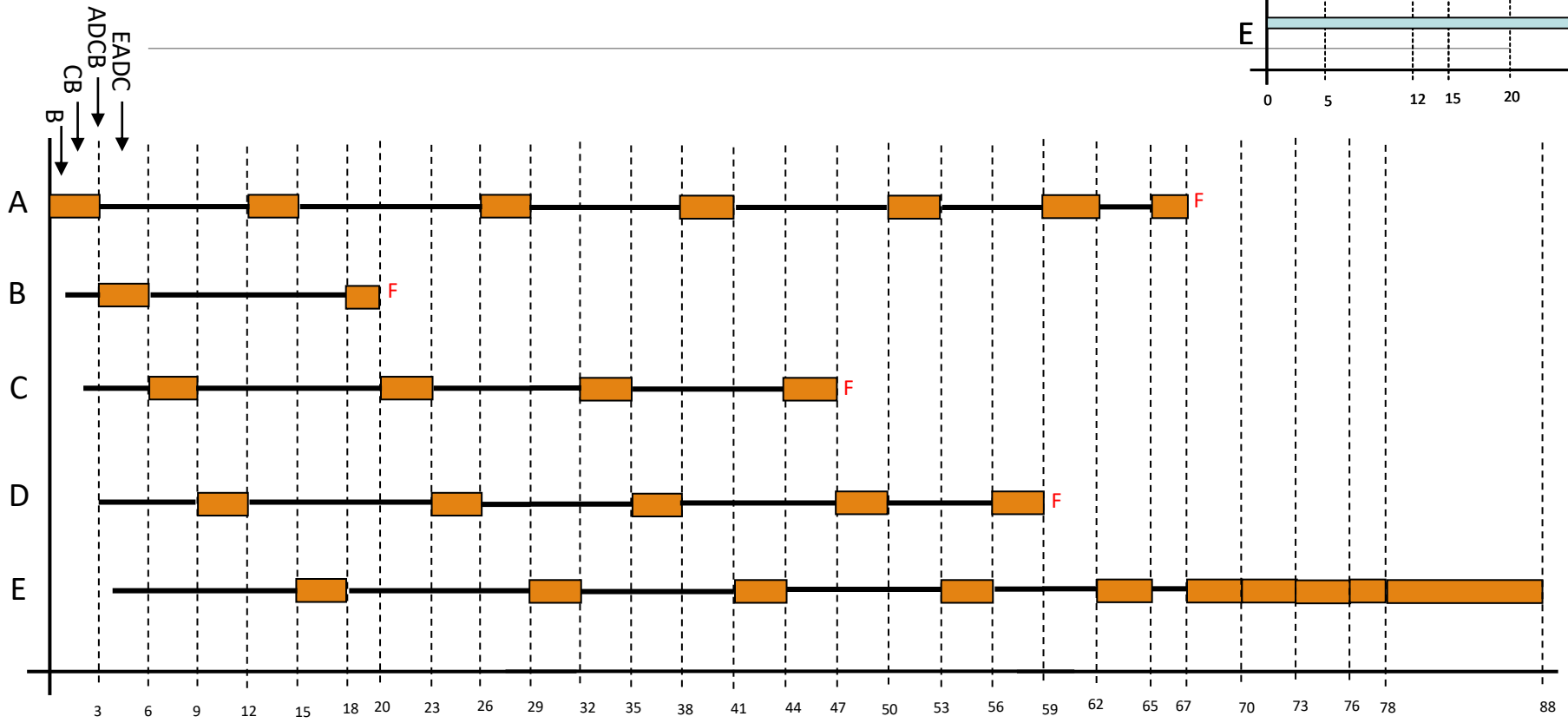
Solución:


A
B
C
D
E
0
5
12
15
20
36
t (ut)


EADC
ADCB
CB
B
A
B
C
D
E
F
F
F
F
F
3
6
9
12
15
18
20
23
26
29
32
35
38
41
44
47
50
53
56
59
62
65
67
70
73
76
78
88
t (ut)

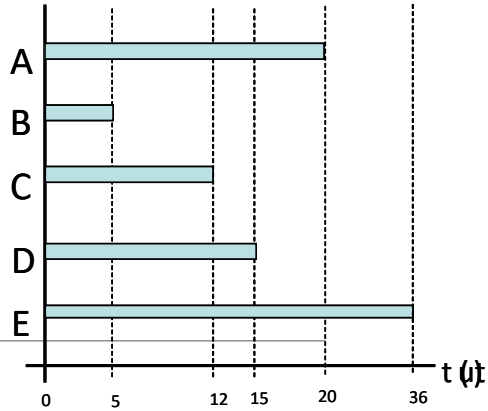
Solución:


A
B
C
D
E
0
5
12
15
20
36
t (ut)


EADC
ADCB
CB
B
A
B
C
D
E
F
F
F
F
F
3
6
9
12
15
18
20
23
26
29
32
35
38
41
44
47
50
53
56
59
62
65
67
70
73
76
78
88
t (ut)

Solución:

	t. retorno	t. respuesta	t. espera
A	67	0	47
B	19	2	14
C	45	4	33
D	56	6	41
E	84	11	48





Ejecución en procesador

Proceso en estado de preparado

Ejercicio 2
Round Robin (RR_q=1)

Sea un sistema con los procesos que se muestran en la tabla.

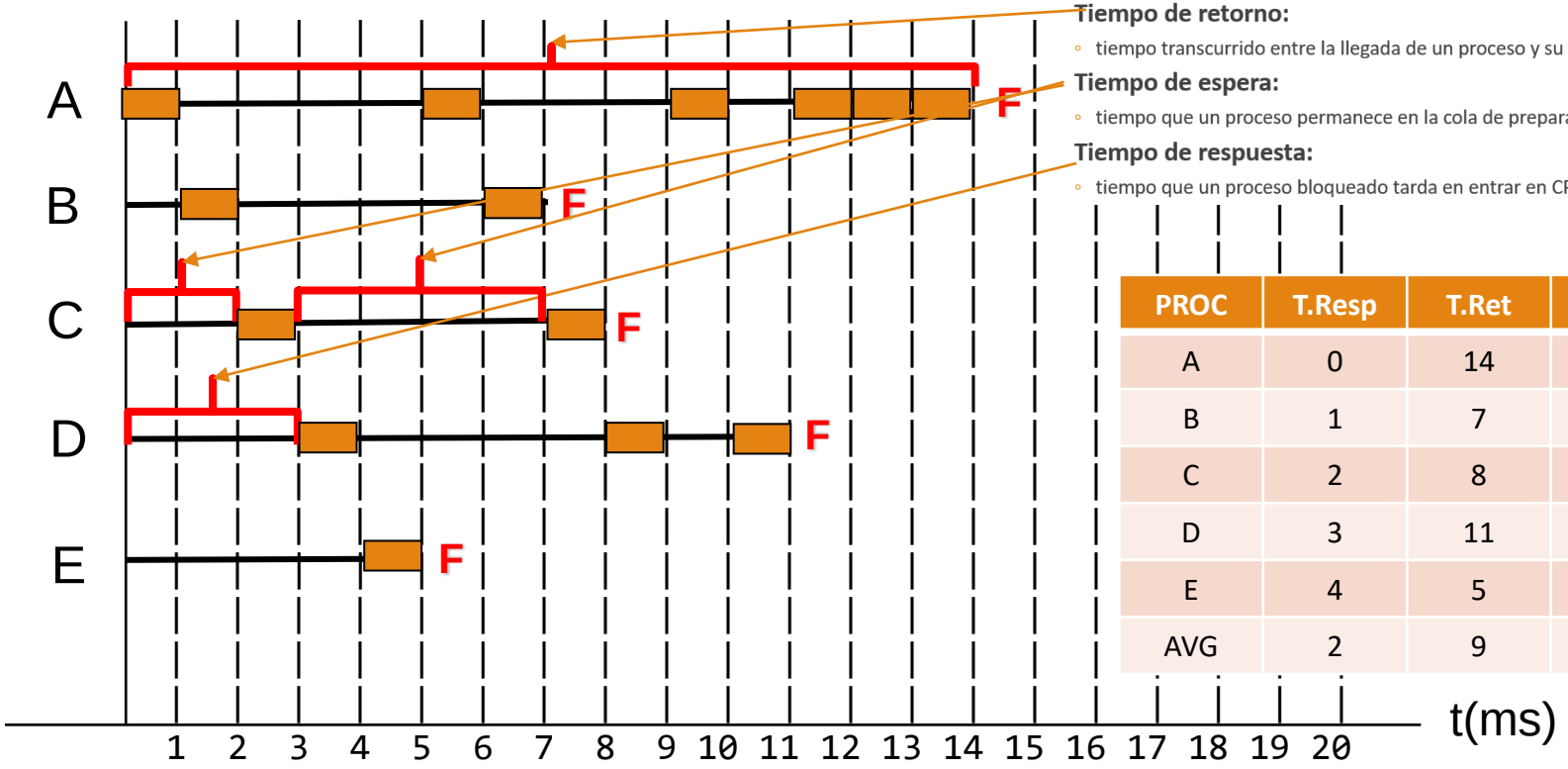
- Representar su evolución temporal
- Calcular para cada uno de los trabajos:
  - el tiempo de retorno,
  - de espera y
  - de respuesta.



Orden alfabético

Proceso	Tiempo de llegada	Tiempo de ejecución
A	0	6
B	0	2
C	0	2
D	0	3
E	0	1

Solución
Round Robin (RR_q=1)



Tiempo de retorno:
- tiempo transcurrido entre la llegada de un proceso y su finalización

Tiempo de espera:
- tiempo que un proceso permanece en la cola de preparados

Tiempo de respuesta:
- tiempo que un proceso bloqueado tarda en entrar en CPU

PROC	T.Resp	T.Ret	T.Esp
A	0	14	8
B	1	7	5
C	2	8	6
D	3	11	8
E	4	5	4
AVG	2	9	6,2

Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

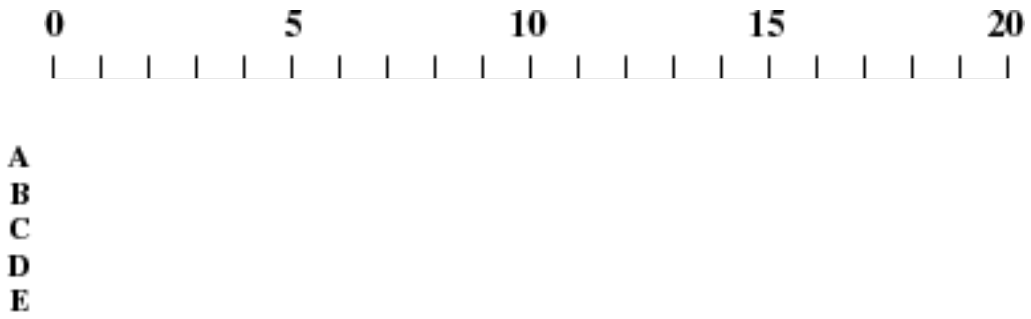
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

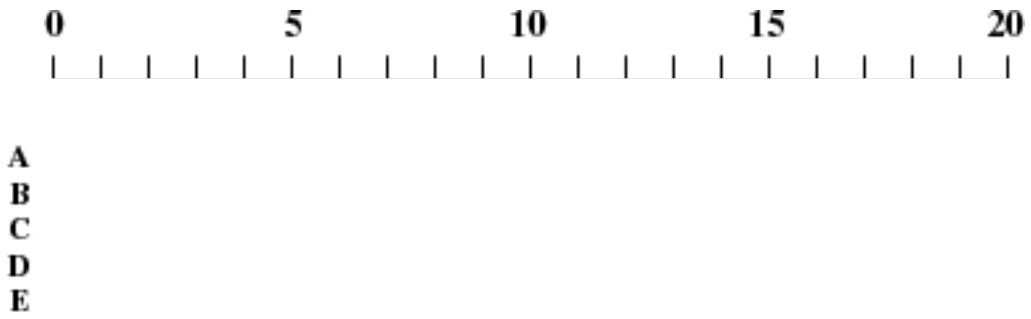
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

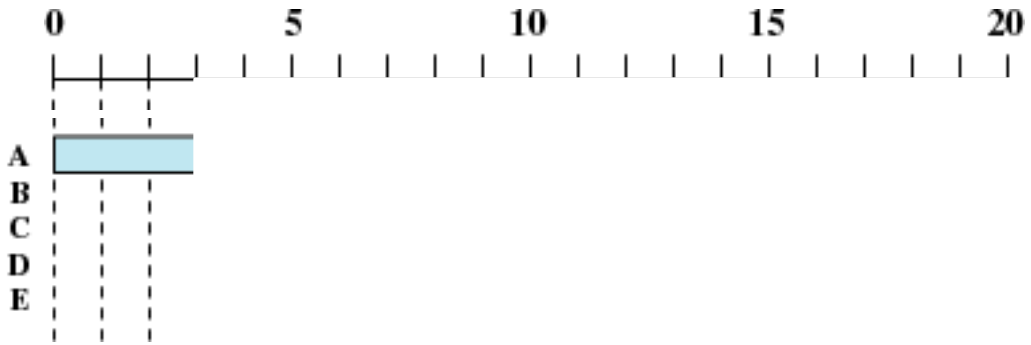
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

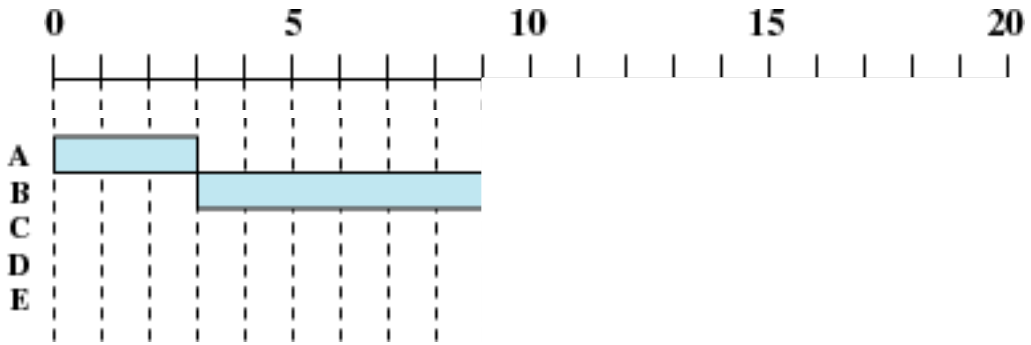
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

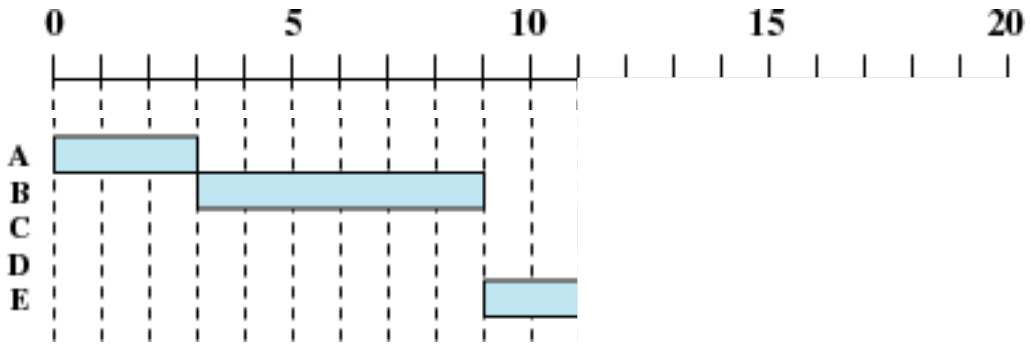
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

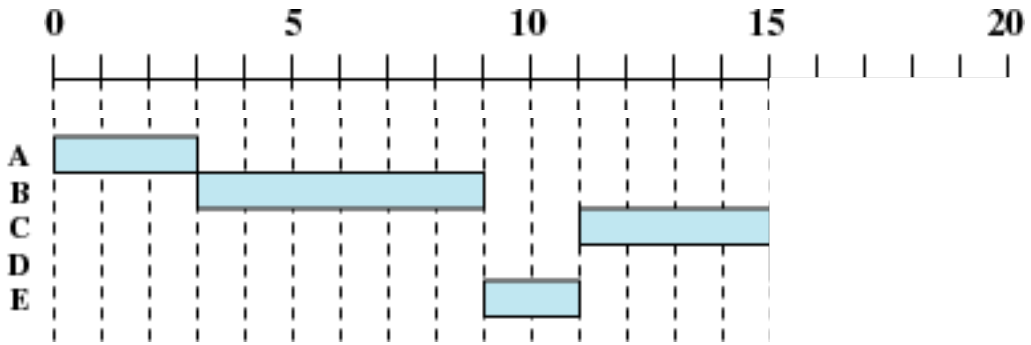
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

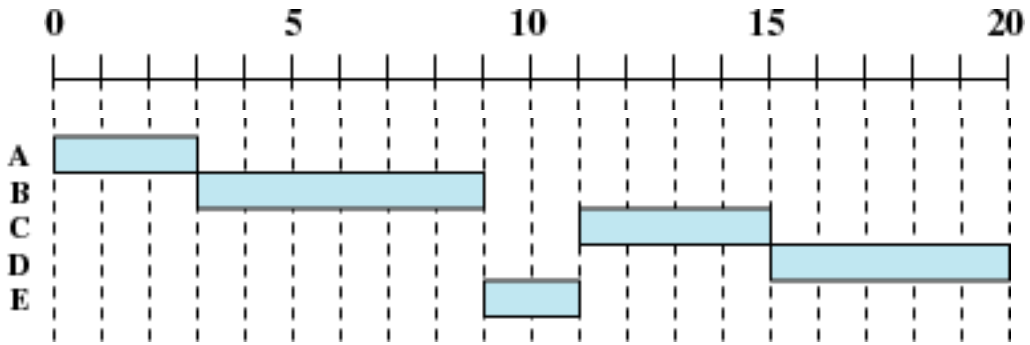
Política **no expulsiva**

Se selecciona el proceso con el tiempo de procesamiento esperado más corto

El proceso más corto se situará a la cabeza de la cola, delante de los procesos más largos

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Primero el proceso más corto [Shortest Process Next (SPN)] o [Shortest Job First (SJF)]

Se reduce la previsibilidad de los procesos más largos

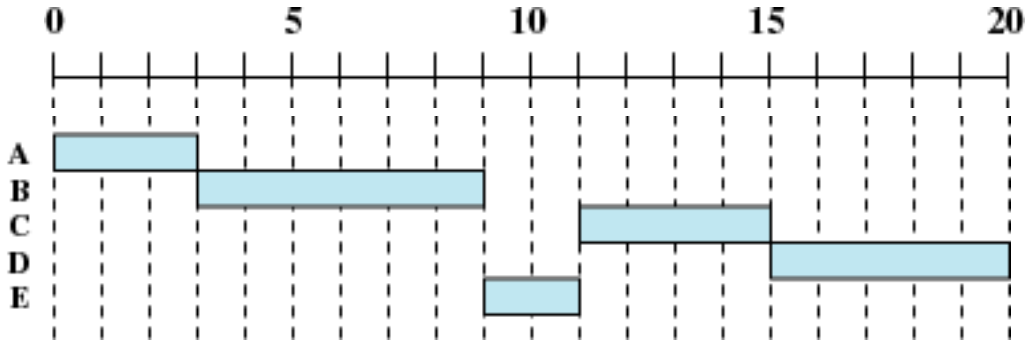
- incrementa la variabilidad en los tiempos de respuesta

Si el tiempo no es el correcto, el SO podría abortar el trabajo

Posibilidad de inanición para los procesos más largos

- Si existe un flujo continuo de procesos cortos

	Process	A	B	C	D	E	
	Arrival Time	0	2	4	6	8	
	Service Time (T_s)	3	6	4	5	2	Mean
SPN	Finish Time	3	9	15	20	11	
	Turnaround Time (T_r)	3	7	11	14	3	7.60
	T_r/T_s	1.00	1.17	2.75	2.80	1.50	1.84



Ejercicio 1
SPN / SJF

Sea un sistema con los procesos que se muestran en la tabla.

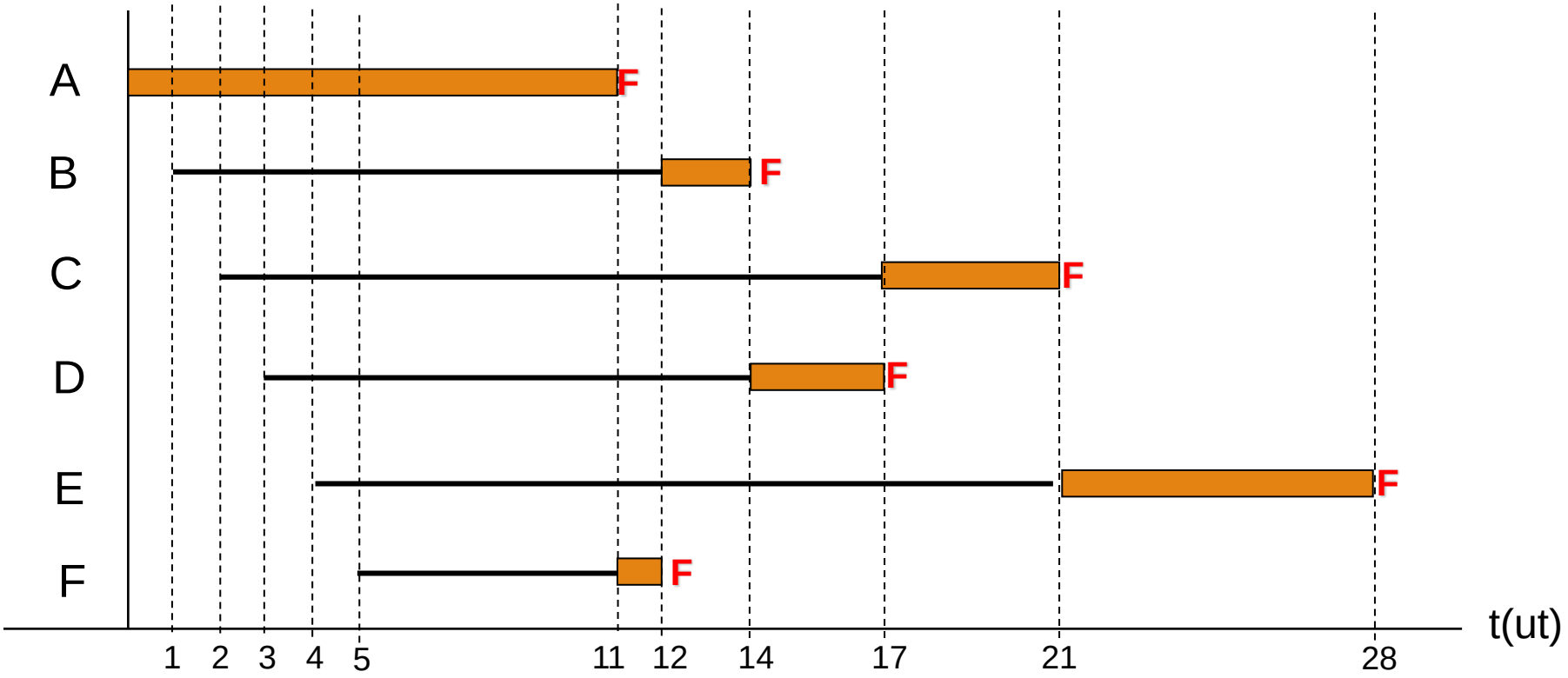
Calcular para cada uno de los trabajos:

- el tiempo de retorno,
- de espera y
- de respuesta.

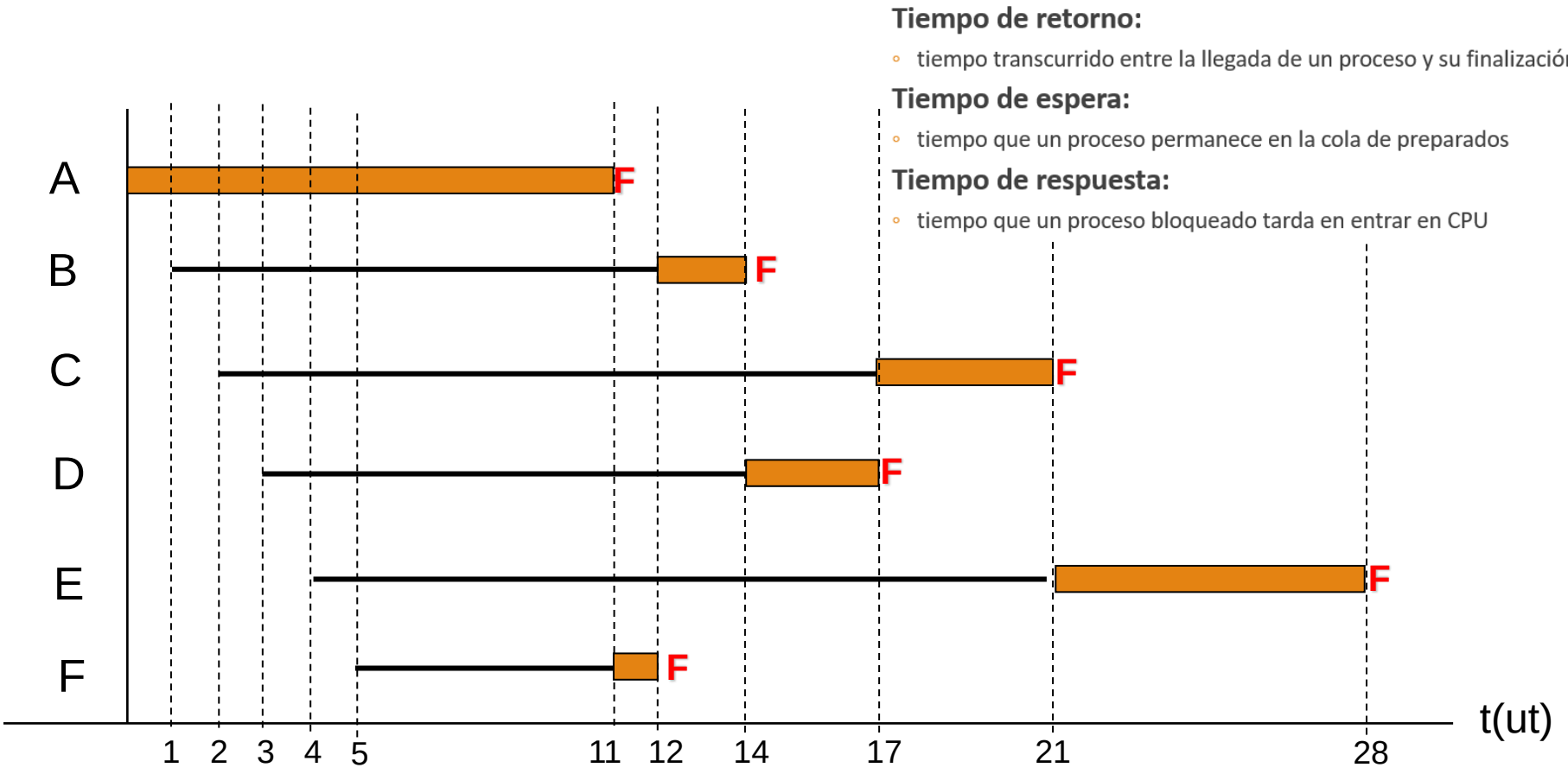
Representar su evolución temporal

Proceso	Tiempo de llegada (ms)	Tiempo de ejecución (ms)
A	0	11
B	1	2
C	2	4
D	3	3
E	4	7
F	5	1

Solución
SPN / SJF



Solución
SPN / SJF



Tiempo de retorno:

- tiempo transcurrido entre la llegada de un proceso y su finalización

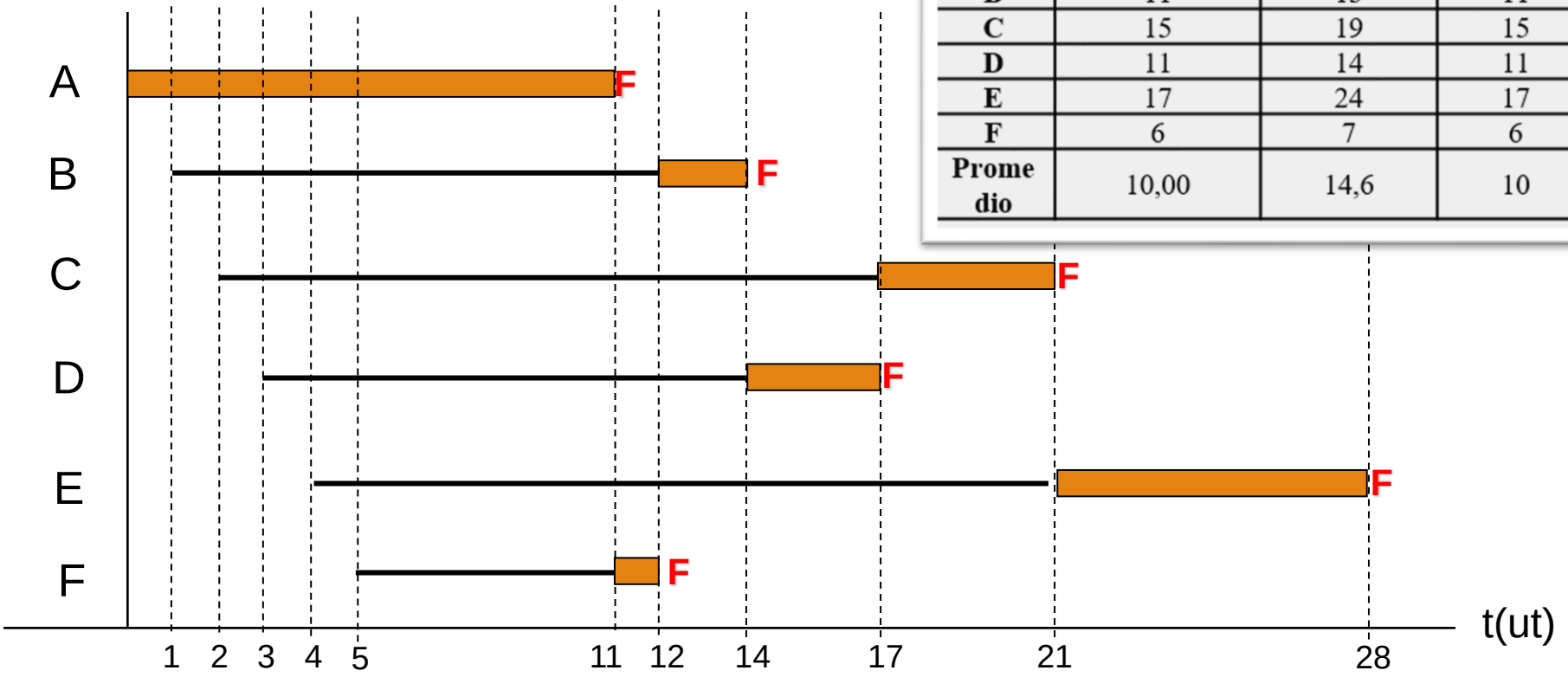
Tiempo de espera:

- tiempo que un proceso permanece en la cola de preparados

Tiempo de respuesta:

- tiempo que un proceso bloqueado tarda en entrar en CPU

Solución
SPN / SJF



	t. respuesta	t. retorno	t.espera
A	0	11	0
B	11	13	11
C	15	19	15
D	11	14	11
E	17	24	17
F	6	7	6
Promedio	10,00	14,6	10

Ejercicio 2
SPN / SJF

Sea un sistema con los procesos que se muestran en la tabla.

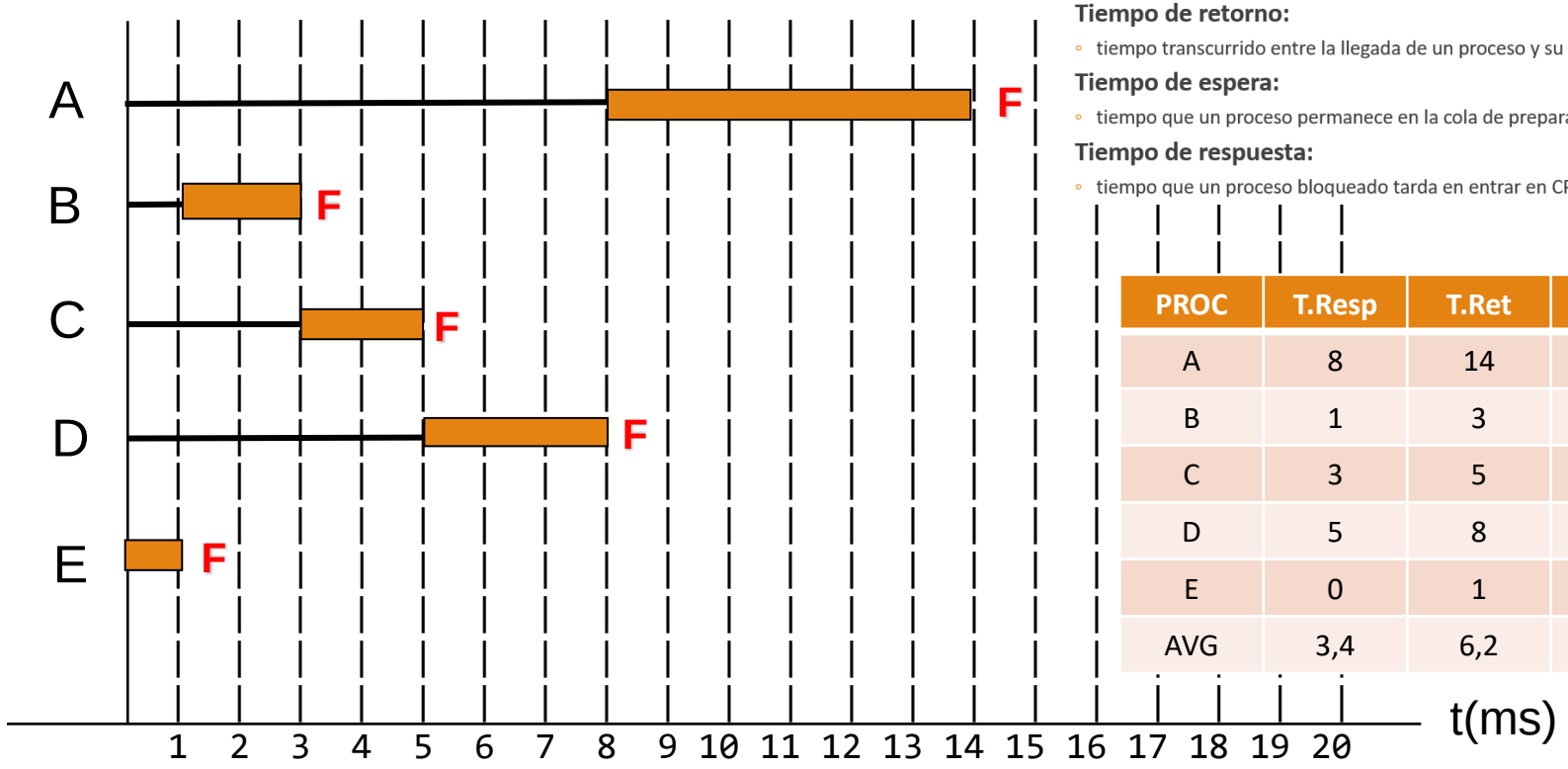
- Representar su evolución temporal
- Calcular para cada uno de los trabajos:
  - el tiempo de retorno,
  - de espera y
  - de respuesta.



Orden alfabético

Proceso	Tiempo de llegada	Tiempo de ejecución
A	0	6
B	0	2
C	0	2
D	0	3
E	0	1

Solución
SPN / SJF



Tiempo de retorno:
- tiempo transcurrido entre la llegada de un proceso y su finalización

Tiempo de espera:
- tiempo que un proceso permanece en la cola de preparados

Tiempo de respuesta:
- tiempo que un proceso bloqueado tarda en entrar en CPU

PROC	T.Resp	T.Ret	T.Esp
A	8	14	8
B	1	3	1
C	3	5	3
D	5	8	5
E	0	1	0
AVG	3,4	6,2	3,4

Política **no expulsiva**

Ejecución en procesador

Proceso en estado de preparado

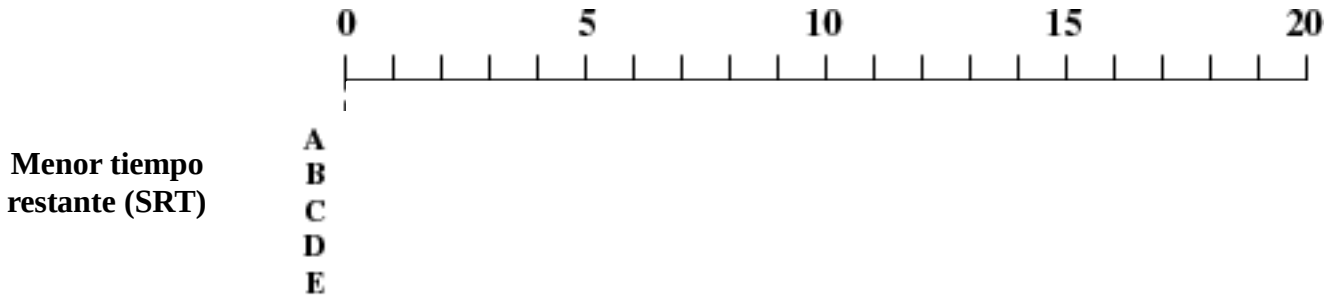
Menor tiempo restante Shortest Remaining Time(SRT)

La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Menor tiempo restante Shortest Remaining Time(SRT)

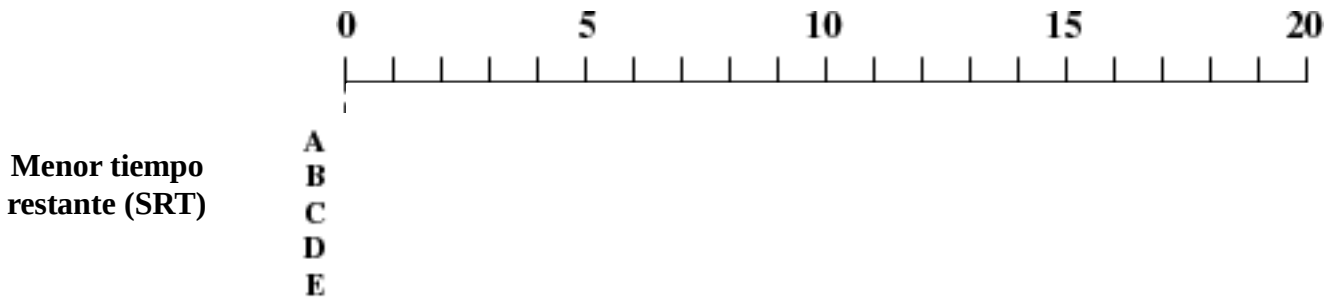
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

3



Menor tiempo restante Shortest Remaining Time(SRT)

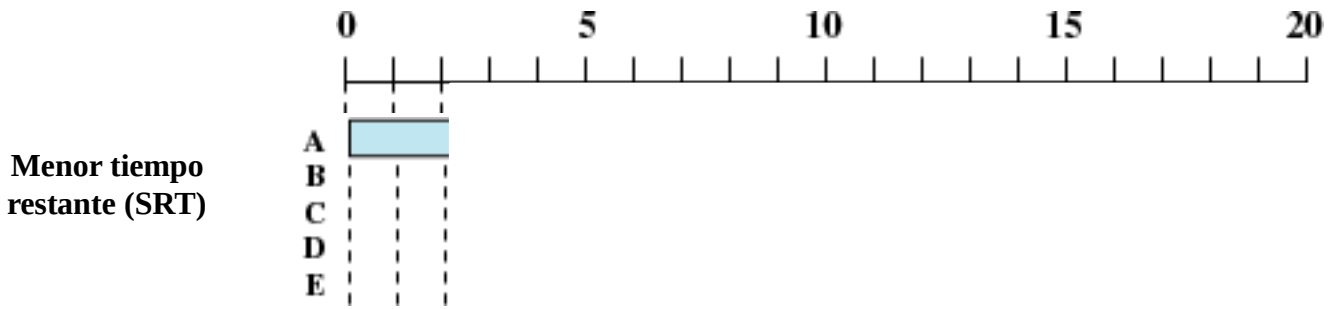
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

1
6



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

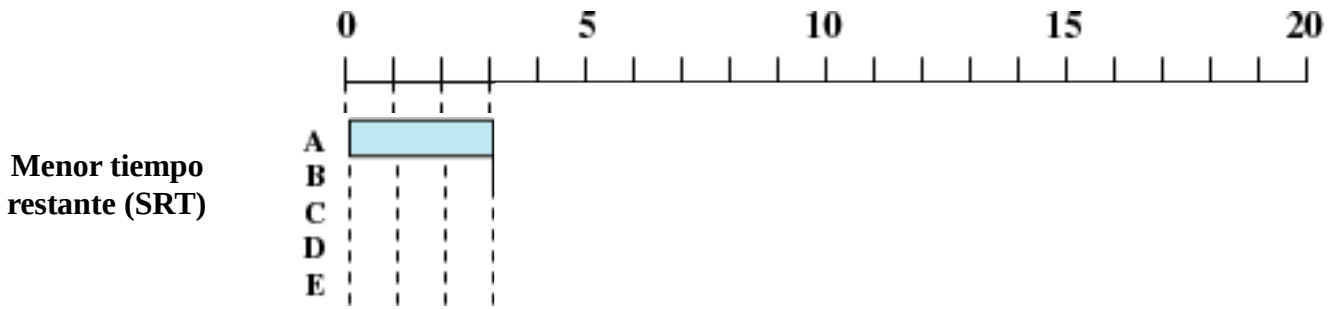
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
6



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

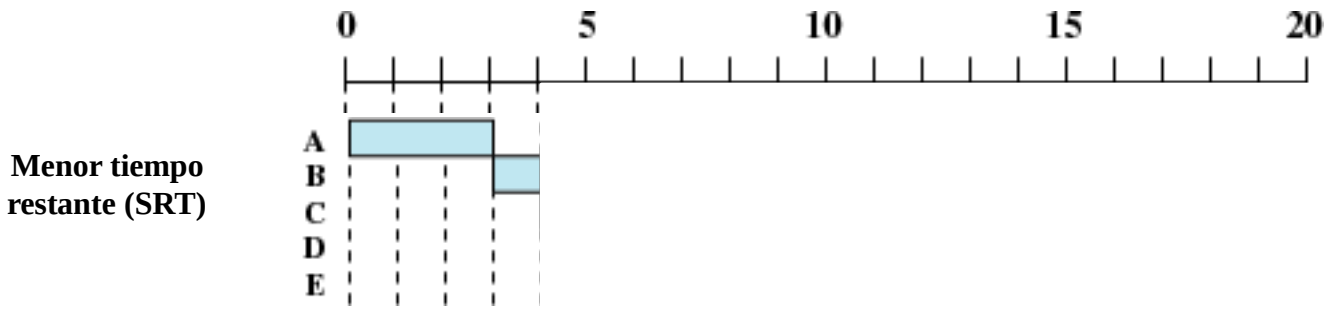
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
5
4



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

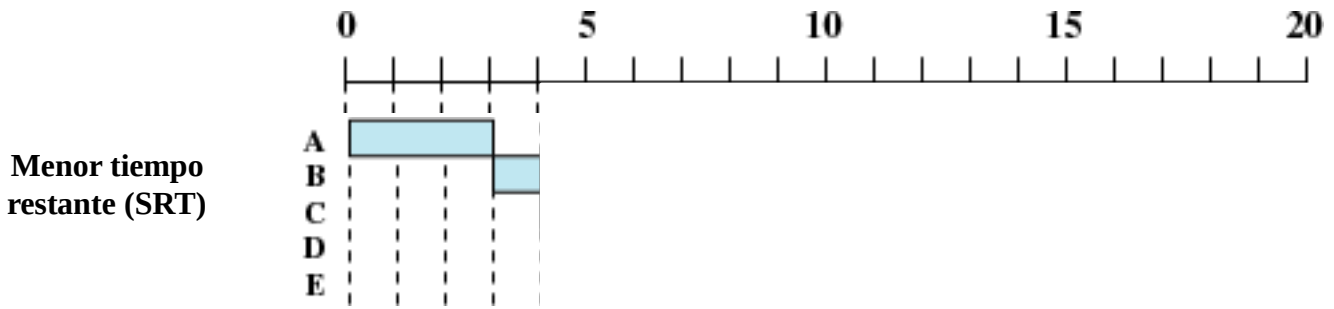
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
5
4



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

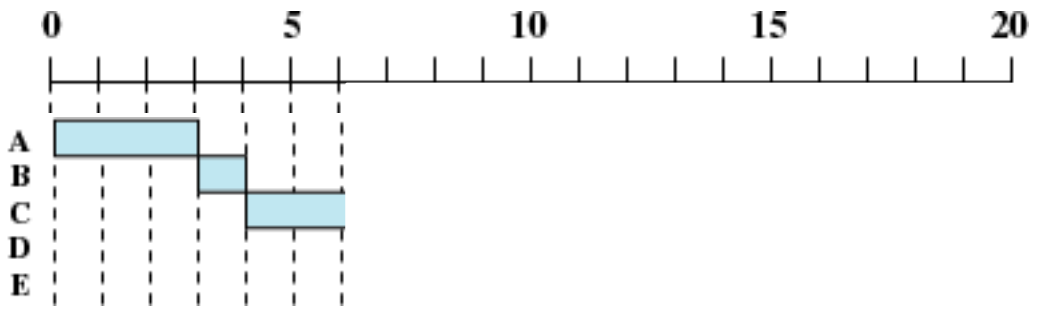
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
5
2
6



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

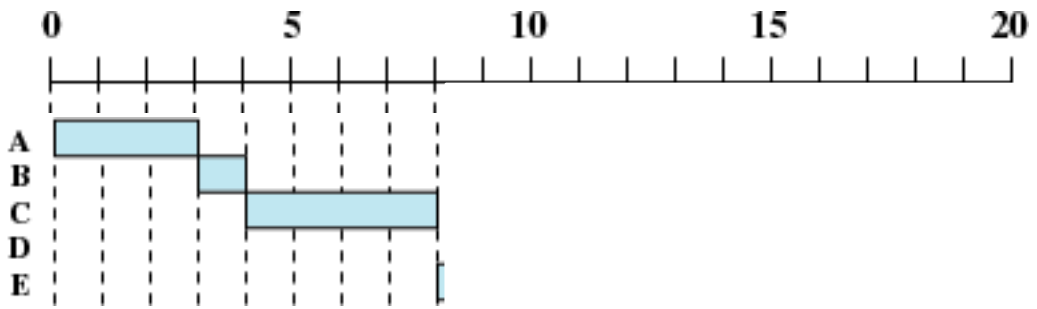
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
5
0
6
2



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

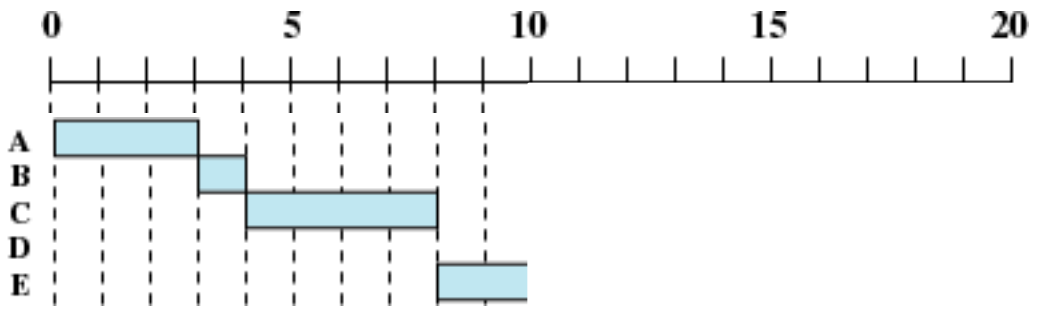
La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0
5
0
5
0



Menor tiempo restante (SRT)

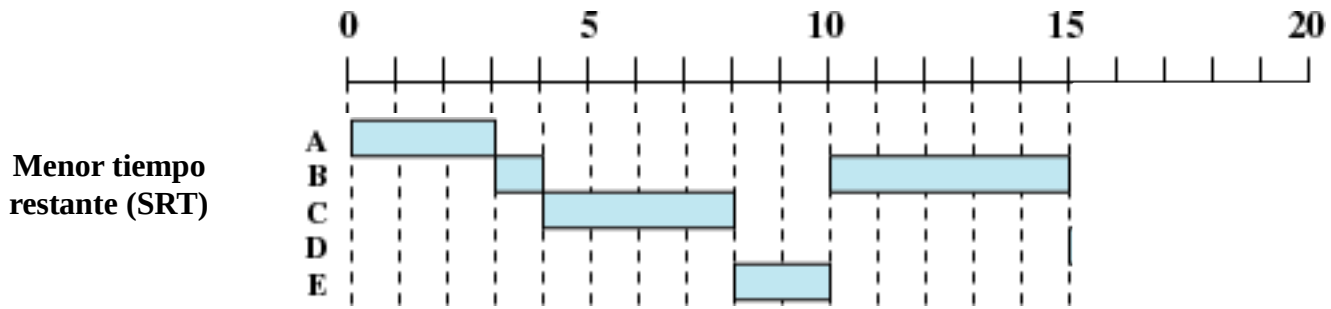
Menor tiempo restante Shortest Remaining Time(SRT)

La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time	
A	0	3	0
B	2	6	0
C	4	4	0
D	6	5	5
E	8	2	0



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

La política del menor tiempo restante es una **versión expulsiva de SPN o SJF**

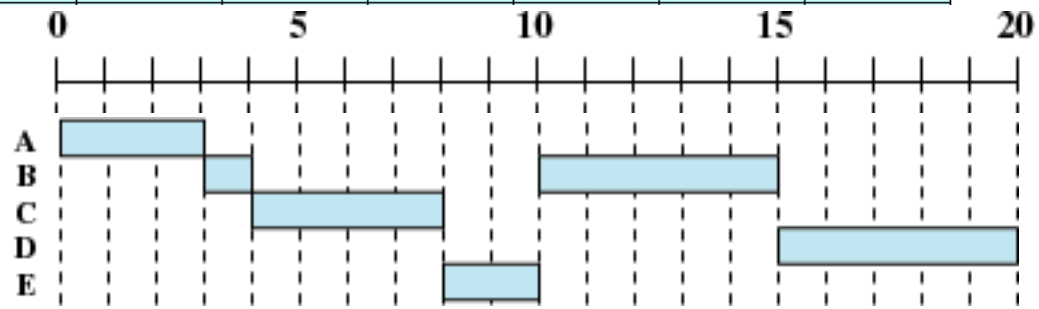
El proceso con menor tiempo restante es el que se procesará el siguiente.

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

0 0 0 0 0

	Process	A	B	C	D	E	
	Arrival Time	0	2	4	6	8	
	Service Time (T_s)	3	6	4	5	2	Mean
SRT	Finish Time	3	15	8	20	10	
	Turnaround Time (T_r)	3	13	4	14	2	7.20
	T_r/T_s	1.00	2.17	1.00	2.80	1.00	1.59



Menor tiempo restante (SRT)

Menor tiempo restante Shortest Remaining Time(SRT)

Debe tener una estimación del tiempo de proceso

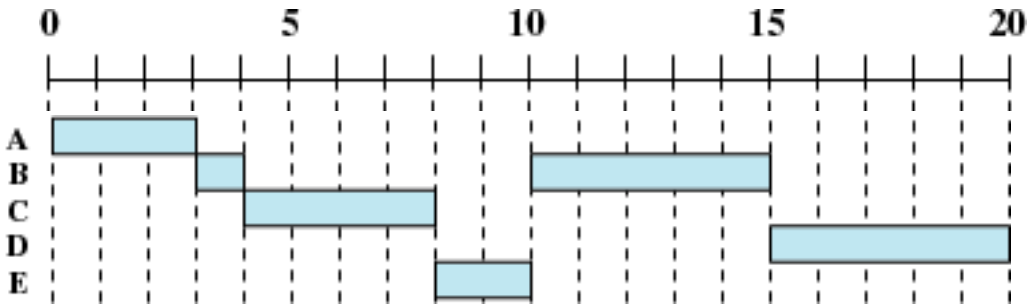
Existe riesgo de inanición para procesos largos

Suele producir tiempos de retorno mejores que SPN (los trabajos cortos reciben atención inmediata y preferente a los procesos largos)

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Menor tiempo restante (SRT)



Ejercicio 1
Shortest Remaining Time(SRT)

Sea un sistema con los procesos que se muestran en la tabla.

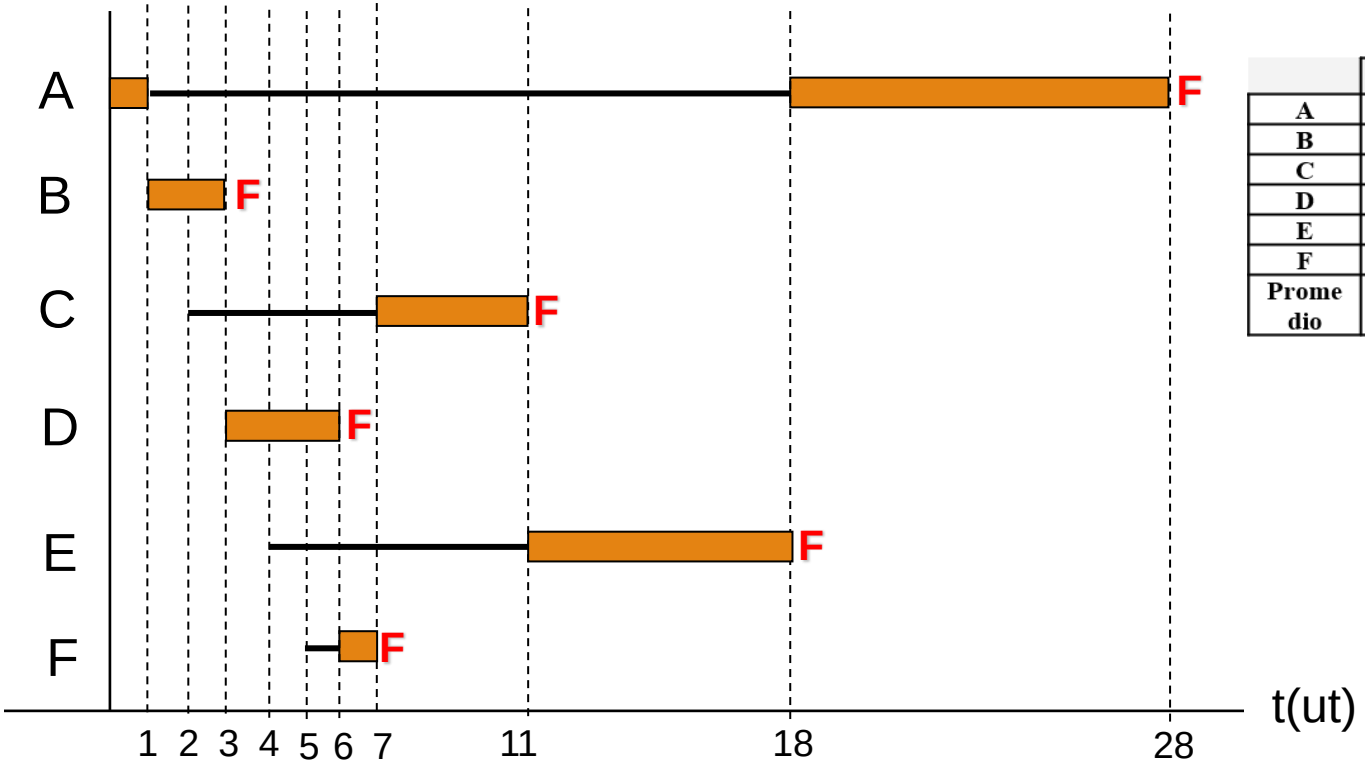
Calcular para cada uno de los trabajos:

- el tiempo de retorno,
- de espera y
- de respuesta.

Representar su evolución temporal

Proceso	Tiempo de llegada (ms)	Tiempo de ejecución (ms)
A	0	11
B	1	2
C	2	4
D	3	3
E	4	7
F	5	1

Solución
Shortest Remaining Time(SRT)



	t. respuesta	t. retorno	t. espera
A	0	28	17
B	0	2	0
C	5	9	5
D	0	3	0
E	7	14	7
F	1	2	1
Promedio	2,16	9,66	5,00

Ejercicio 2
Shortest Remaining Time(SRT)

Sea un sistema con los procesos que se muestran en la tabla.

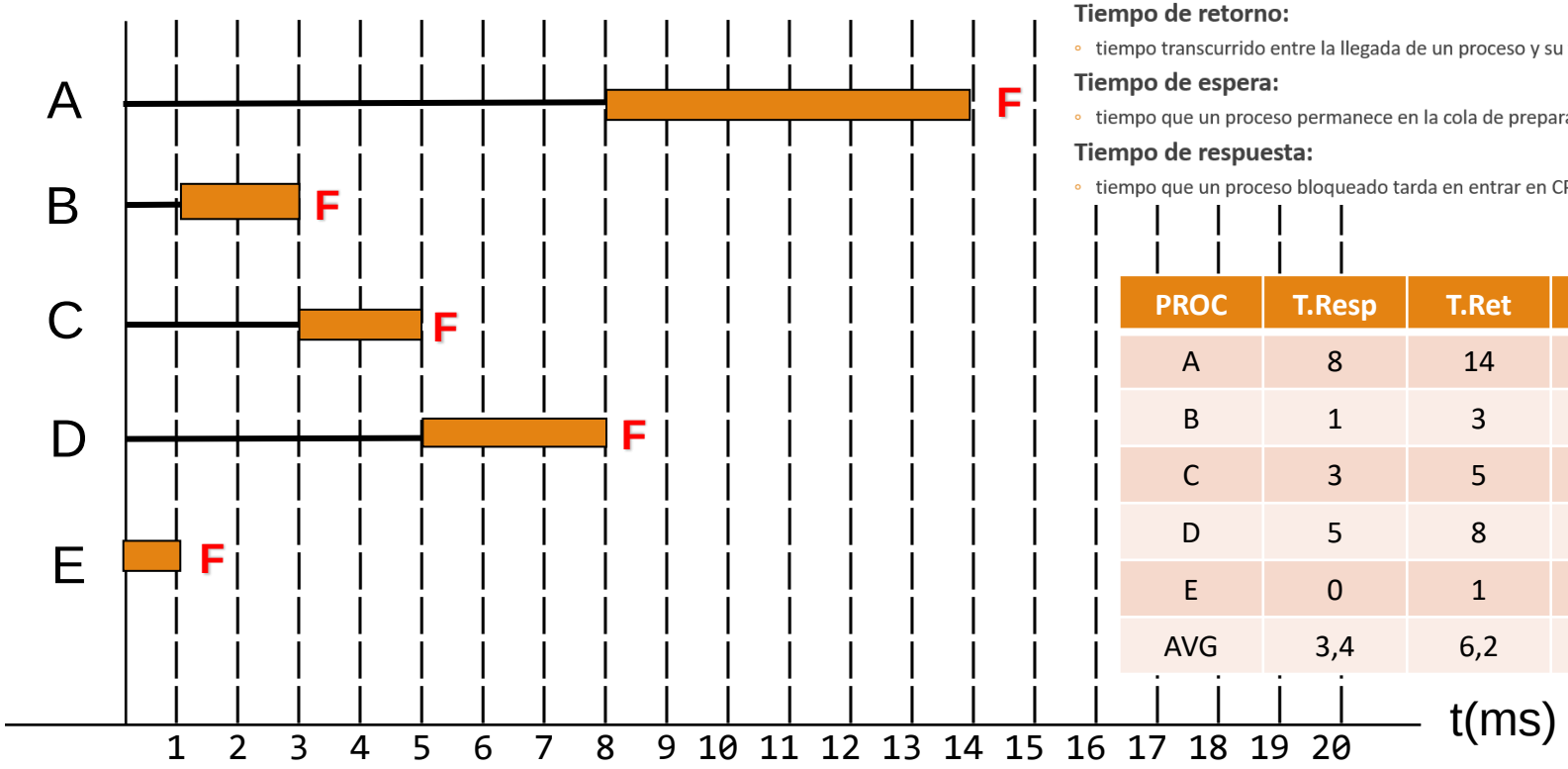
- Representar su evolución temporal
- Calcular para cada uno de los trabajos:
  - el tiempo de retorno,
  - de espera y
  - de respuesta.



Orden alfabético

Proceso	Tiempo de llegada	Tiempo de ejecución
A	0	6
B	0	2
C	0	2
D	0	3
E	0	1

Solución
Shortest Remaining Time(SRT)



Tiempo de retorno:
- tiempo transcurrido entre la llegada de un proceso y su finalización

Tiempo de espera:
- tiempo que un proceso permanece en la cola de preparados

Tiempo de respuesta:
- tiempo que un proceso bloqueado tarda en entrar en CPU

PROC	T.Resp	T.Ret	T.Esp
A	8	14	8
B	1	3	1
C	3	5	3
D	5	8	5
E	0	1	0
AVG	3,4	6,2	3,4

Política **expulsiva**
SPN/SJF???
Llegan a la vez

Ejecución en procesador

Proceso en estado de preparado

Tema 2: Planificación

SISTEMAS OPERATIVOS

RECAP
Políticas de planificación

NON-PREEMPTIVE

Servicio por orden de llegada
- FCFS – First Come First Served

Primero la tarea más corta
- SJF – Shortest Job First
- SPN – Shortest Process Next

Planificación con prioridad
- Estática
- Dinámica

PREEMPTIVE

Planificación por turno circular
- RR – Round Robin

Tiempo que queda más corto
- SRT – Shortest Remaining Time

Planificación con colas multinivel
- MLQ – Multi-Level Queues

Planificación con colas multinivel con realimentación
- MLFQ – M.Level Feedback Queues

Planificación con prioridad

Planificación basada en prioridad

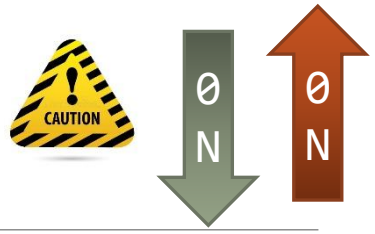
Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

Hay variantes **expulsivas y no expulsivas**

Planificación basada en prioridad



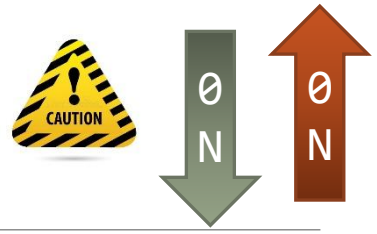
Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

Hay variantes **expulsivas y no expulsivas**

Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

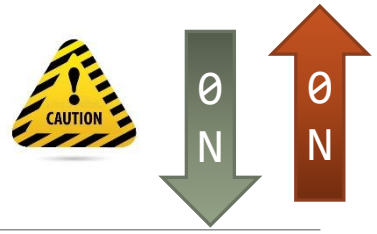
Hay variantes **expulsivas y no expulsivas**

Orden alfabético



Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9

Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

Hay variantes **expulsivas y no expulsivas**

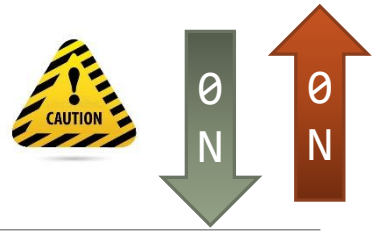
Orden alfabético



Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9



Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

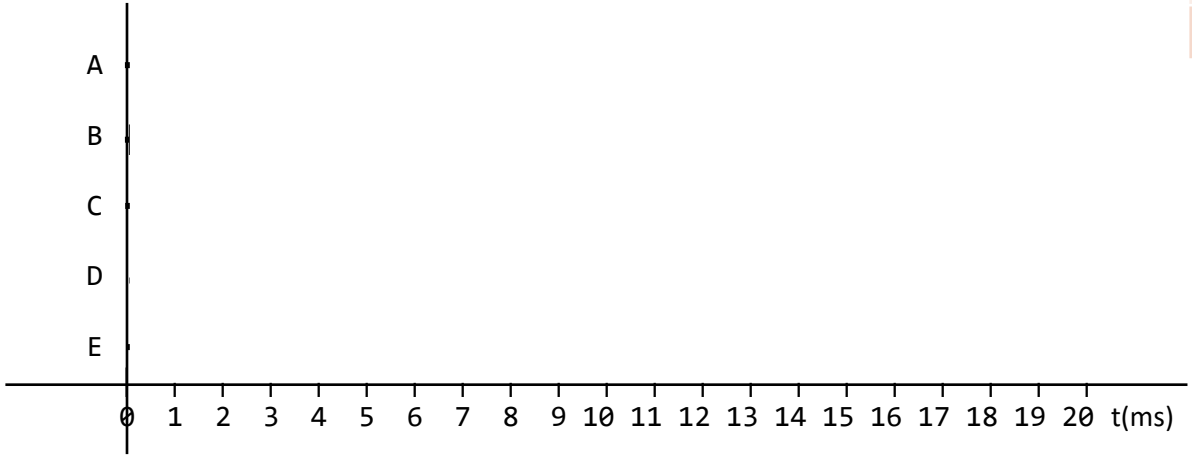
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

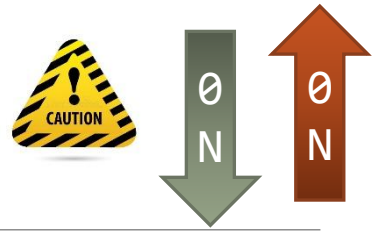


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

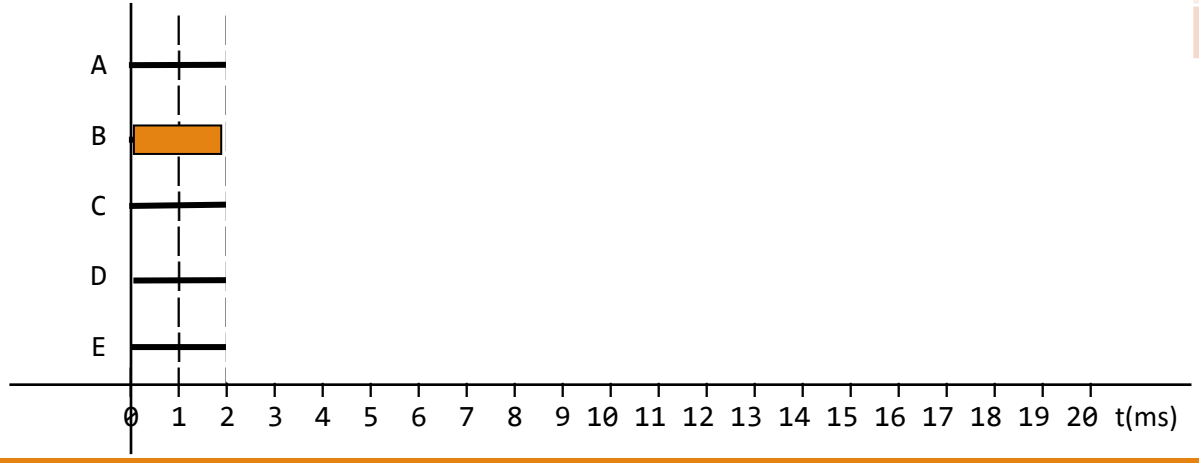
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

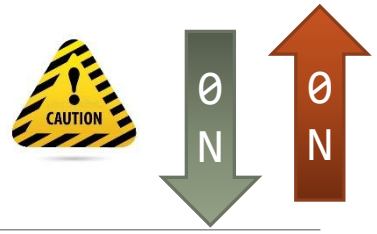


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

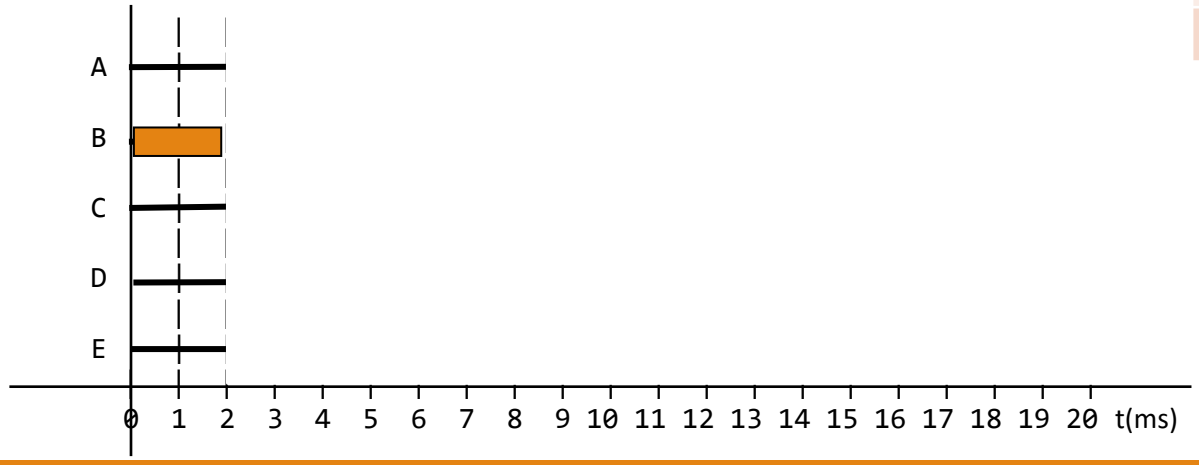
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

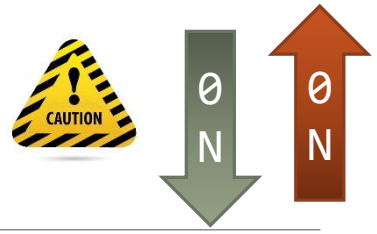


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

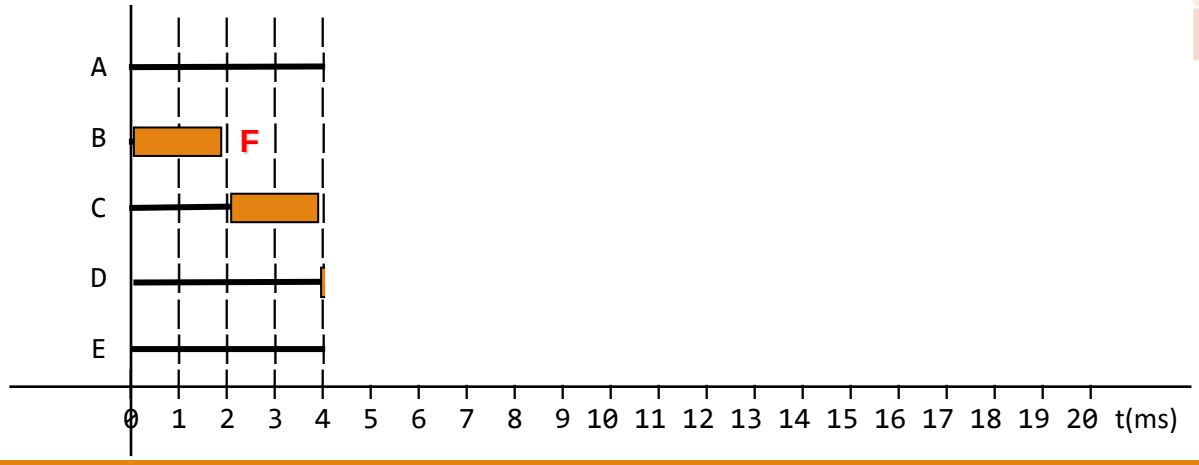
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

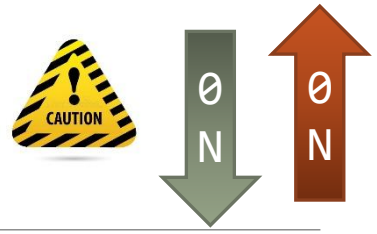


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

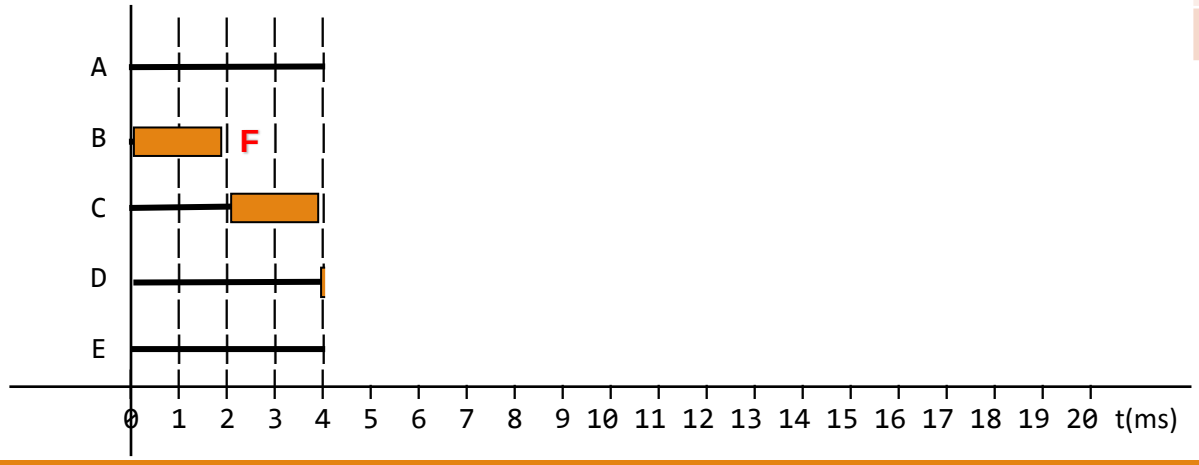
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

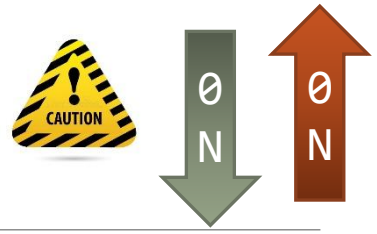


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

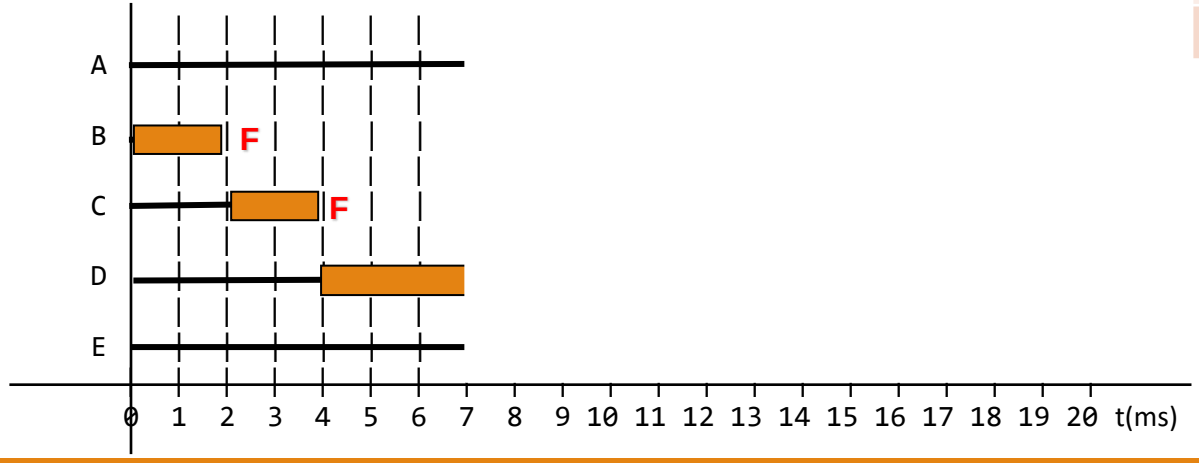
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

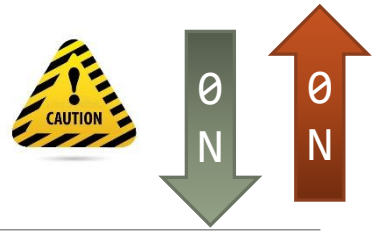


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

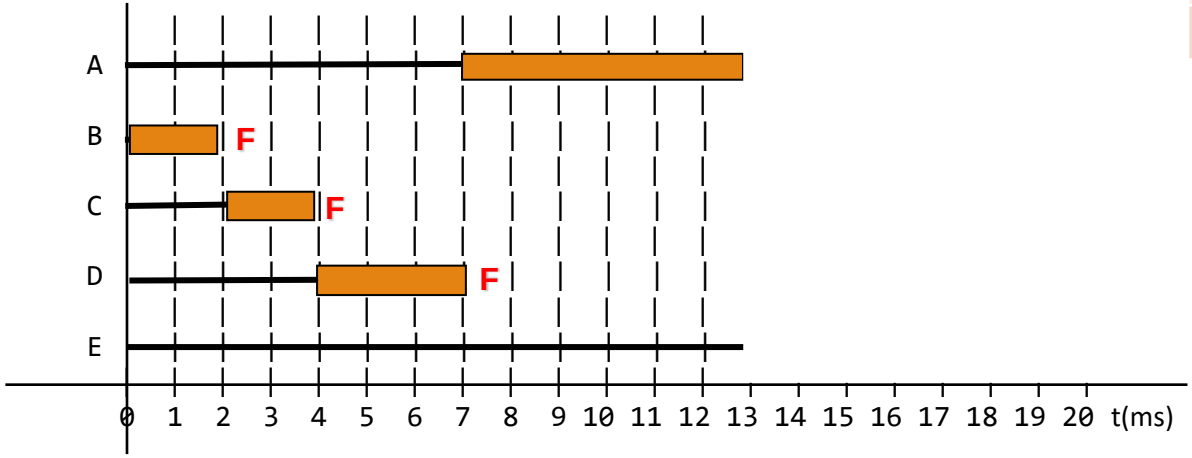
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

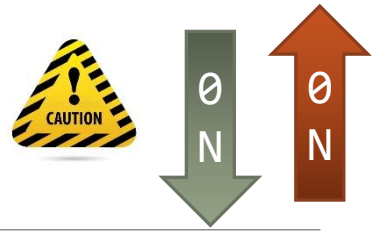


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

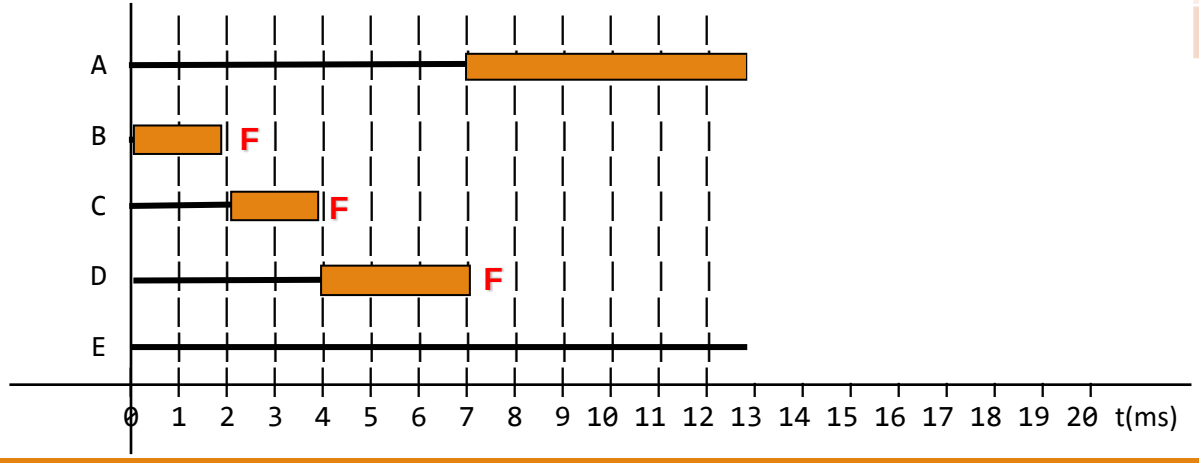
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

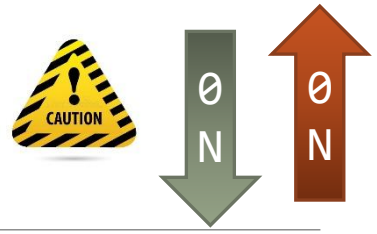


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

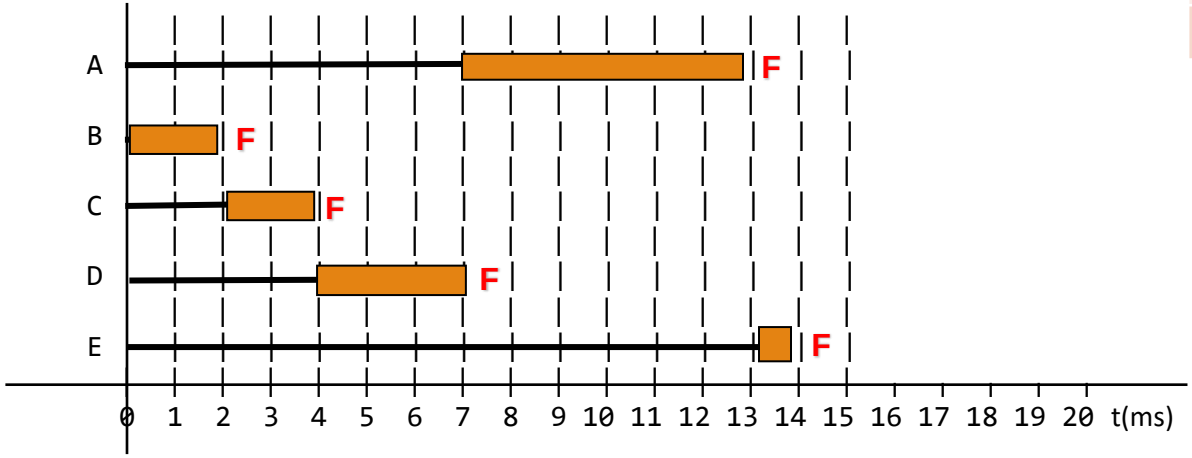
Hay variantes **expulsivas y no expulsivas**

Orden alfabético

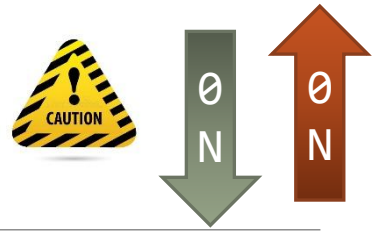


Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9





Planificación basada en prioridad



Se concede CPU en base a una prioridad

Se puede definir la prioridad:

- externamente (el usuario)
- internamente (el sistema)

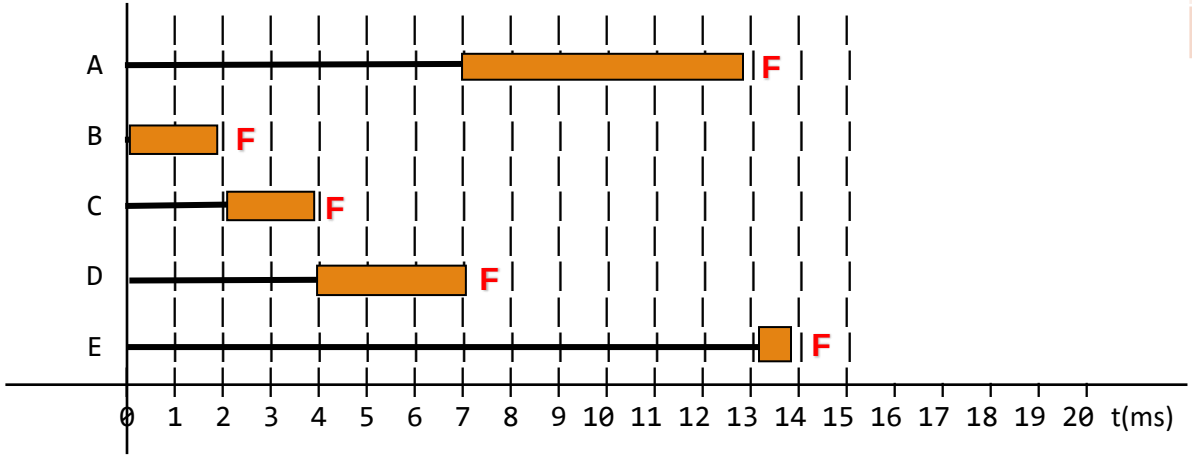
Hay variantes **expulsivas y no expulsivas**

Orden alfabético



Proceso	Tiempo de llegada	Tiempo de ejecución	Prioridad
A	0	6	4
B	0	2	2
C	0	2	2
D	0	3	3
E	0	1	9

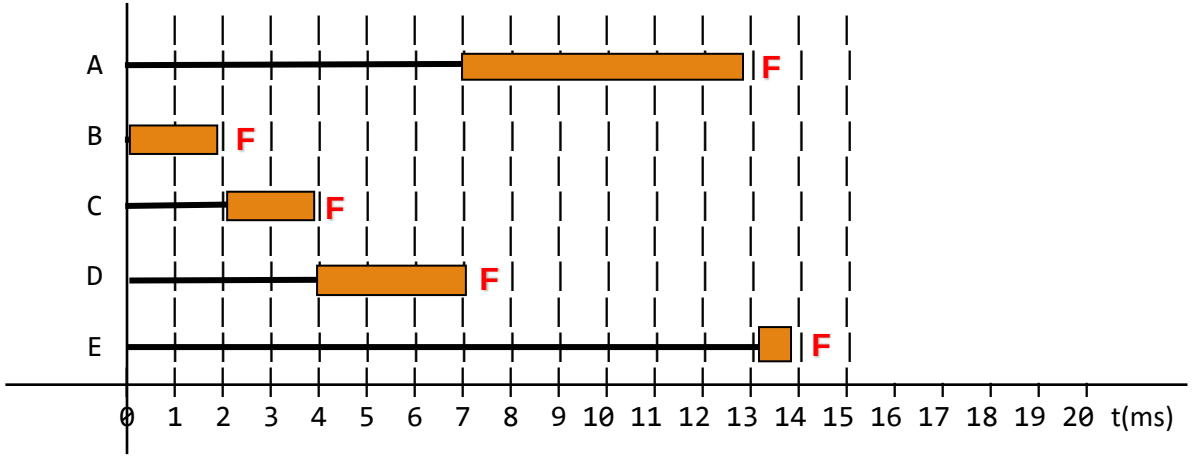




Planificación basada en prioridad

Posibilidad de **inanición** para los **procesos con poca prioridad**

Solución: aumentar la prioridad de los procesos a medida que aumenta el tiempo de espera



PROC	T.Resp	T.Ret	T.Esp
A	7	13	7
B	0	2	0
C	2	4	2
D	4	7	4
E	13	14	13
AVG	5,2	8	5,2

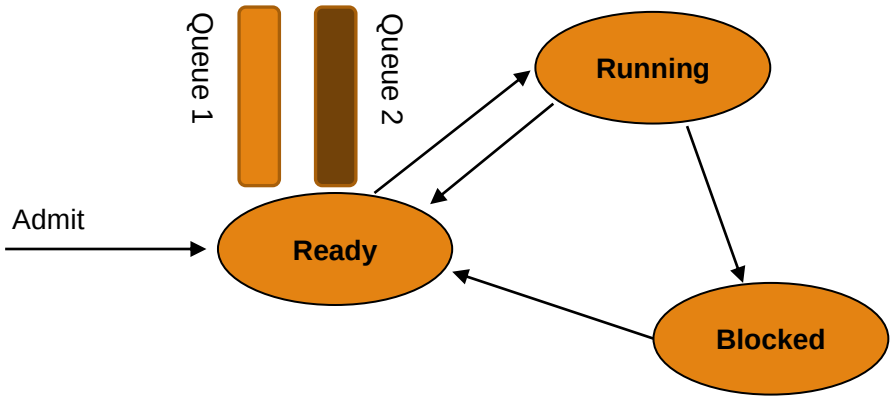
Planificación multicolas (MLQ)

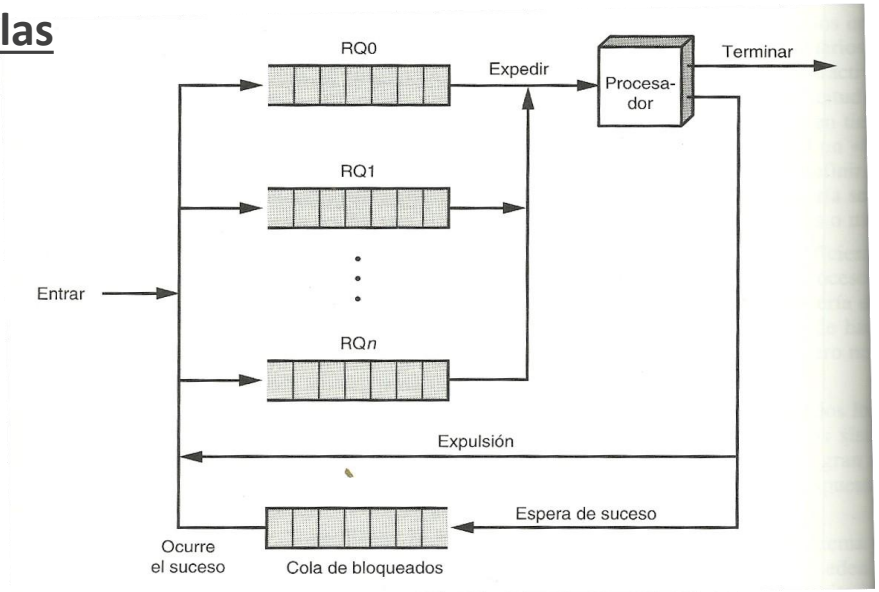
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

Es determinante:

- la política de **ordenación de los procesos** en las diferentes colas
- la política de **ordenación entre las colas**





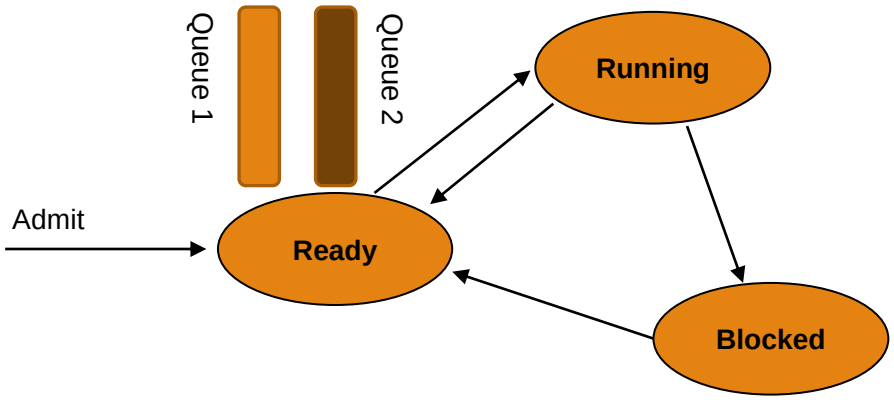
Planificación multicolas (MLQ)

El planificador gestiona varias colas de preparados

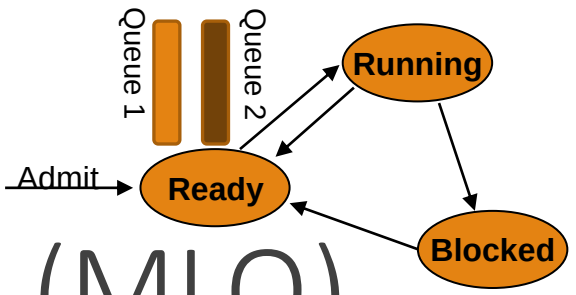
Cada cola puede gestionarse mediante una política diferente

Es determinante:

- la política de **ordenación de los procesos** en las diferentes colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.



Planificación multicolas (MLQ)

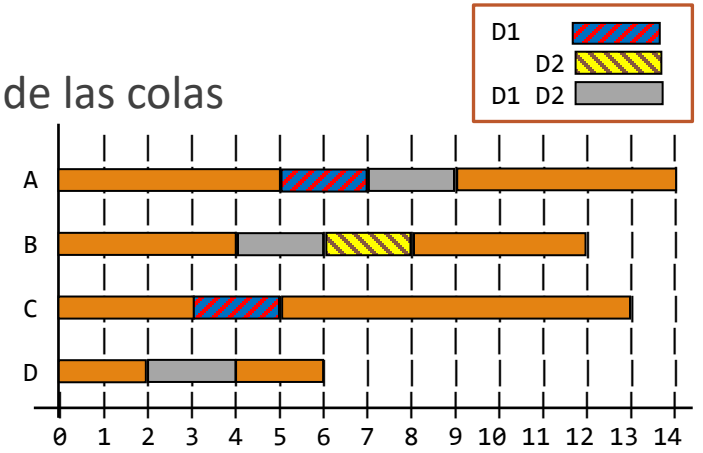
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

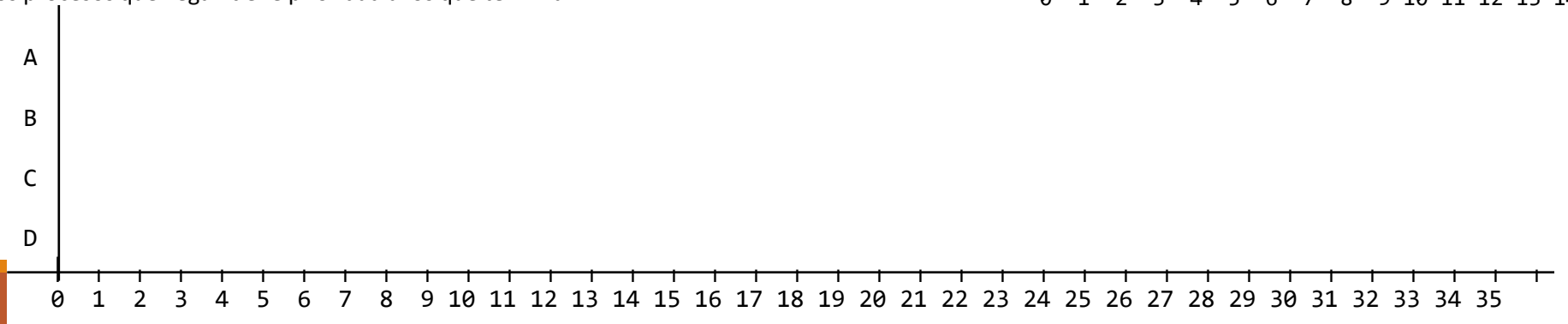
Es determinante:

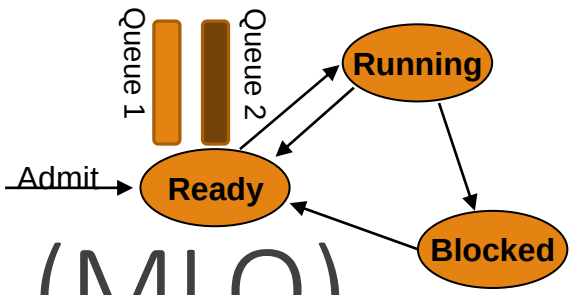
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

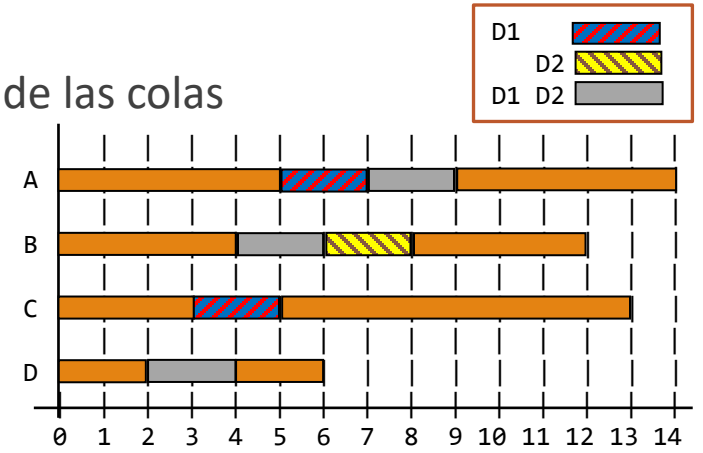
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

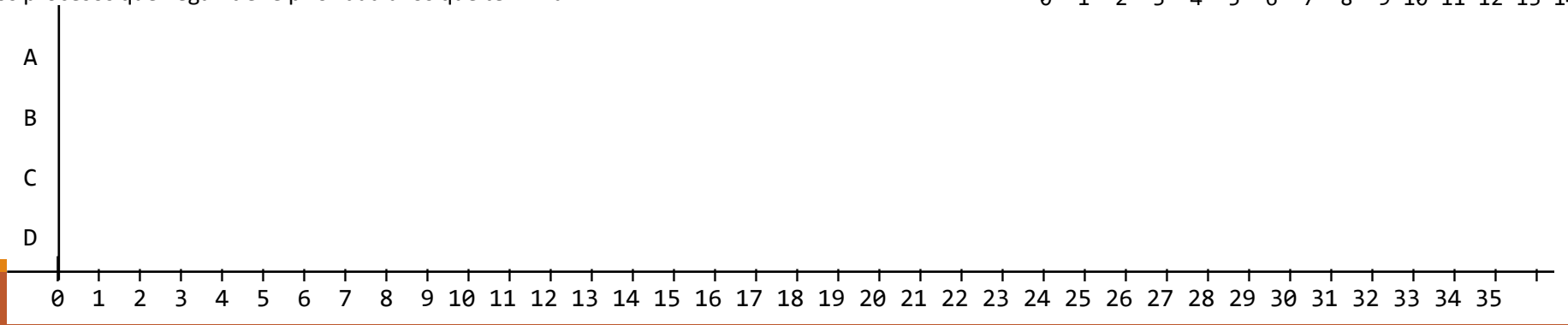
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

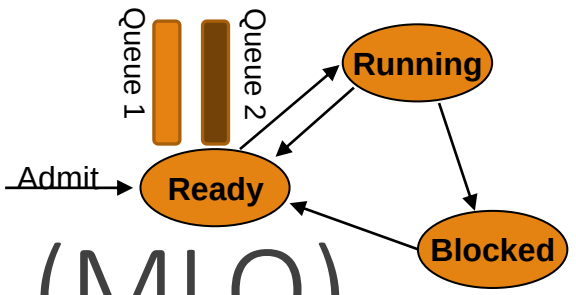
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

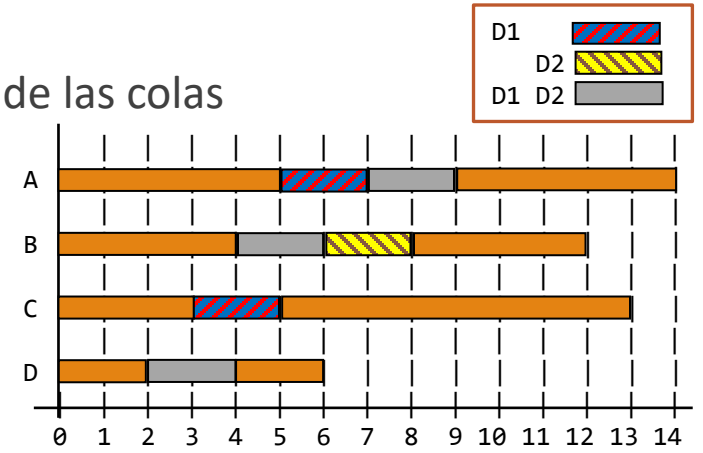
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

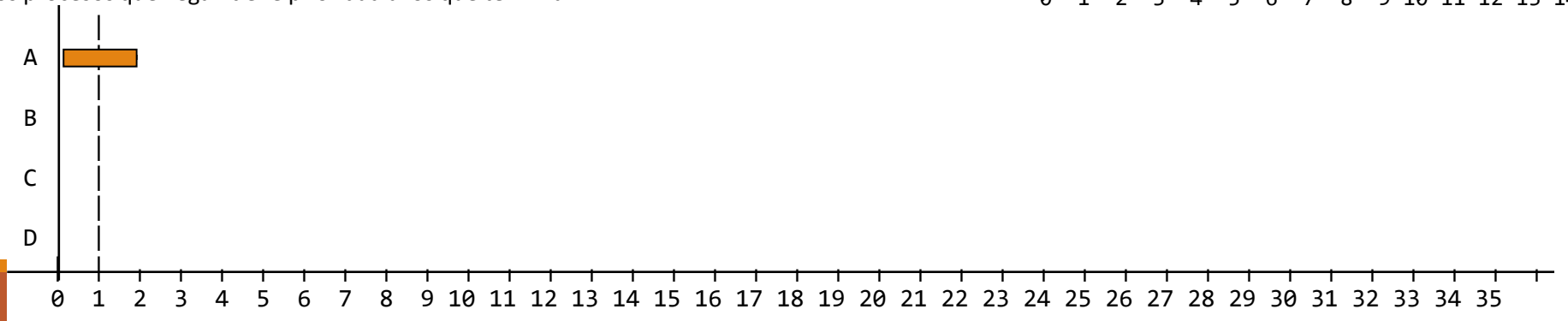
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

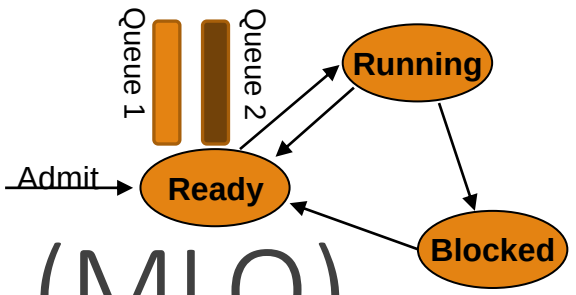
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

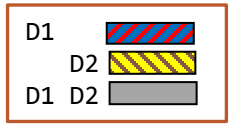
El planificador gestiona varias colas de preparados

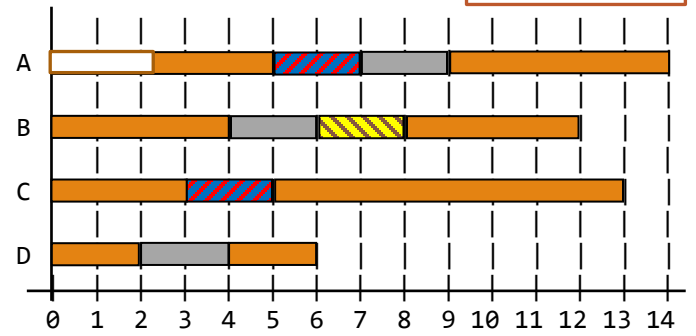
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

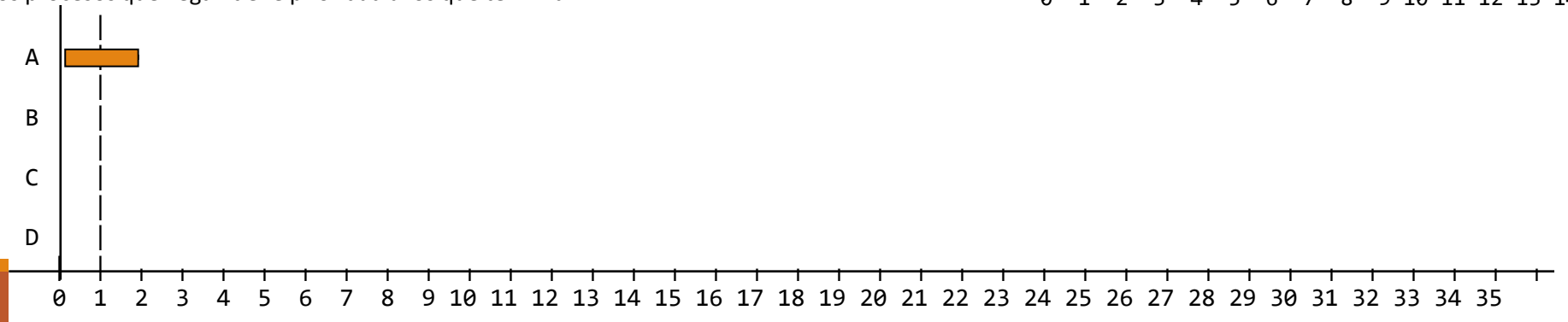
Es determinante:

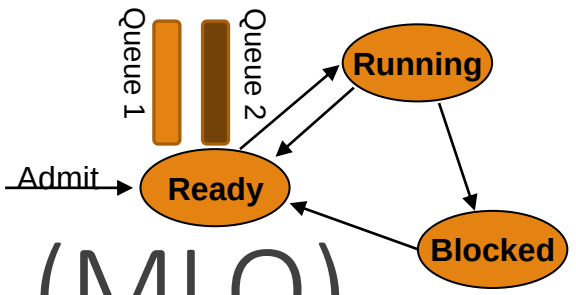
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

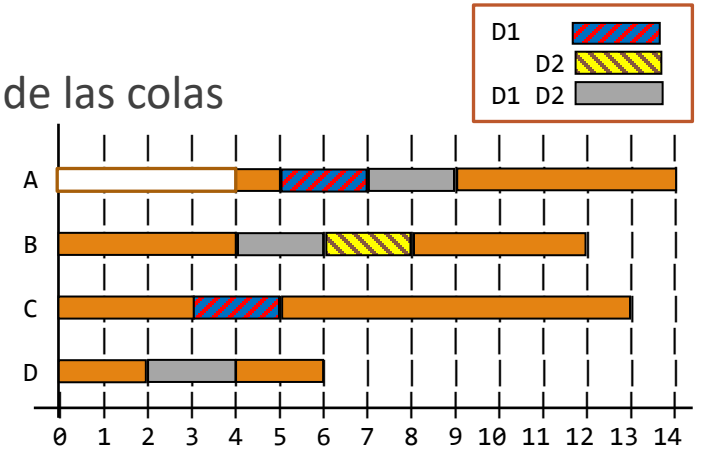
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

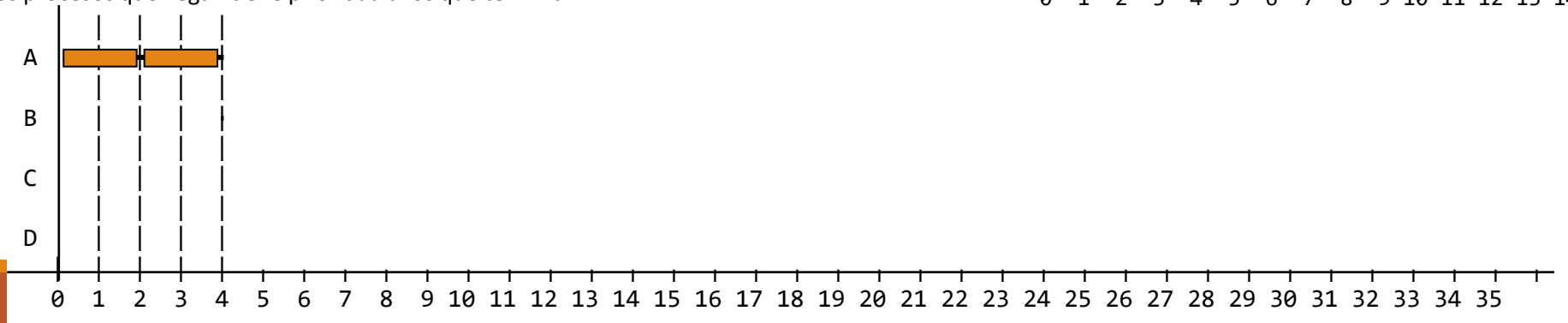
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

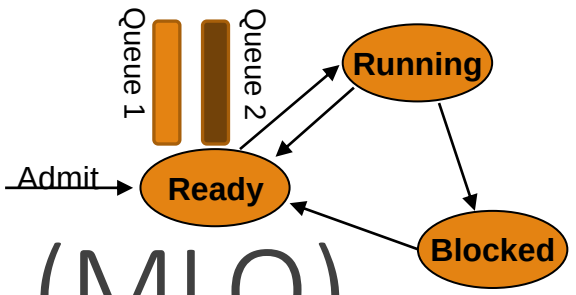
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

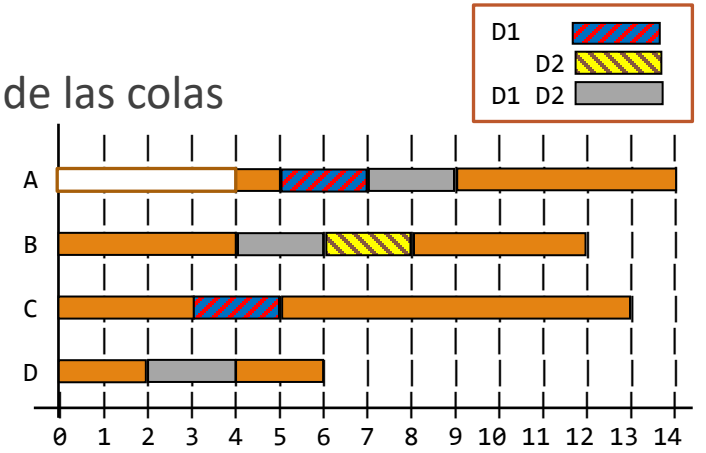
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

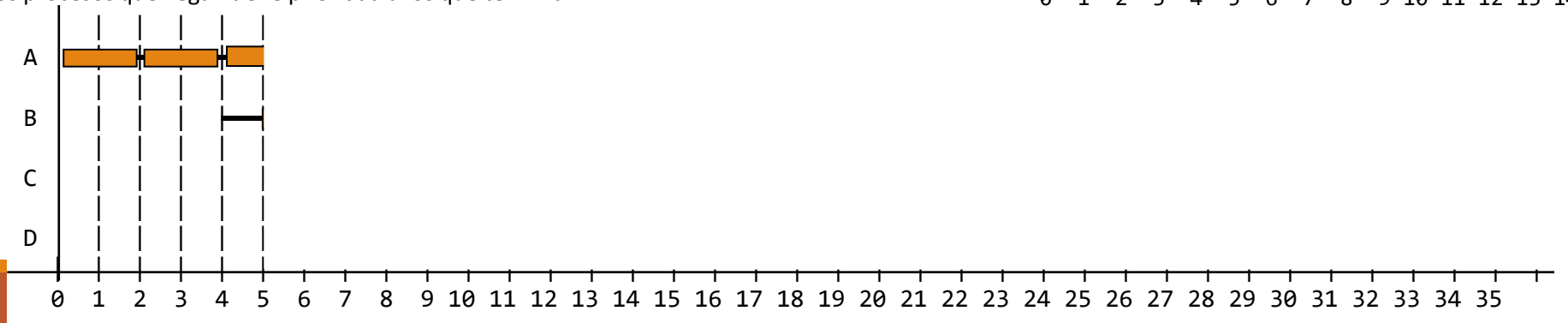
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

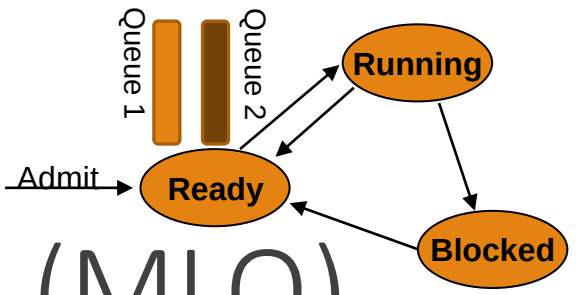
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

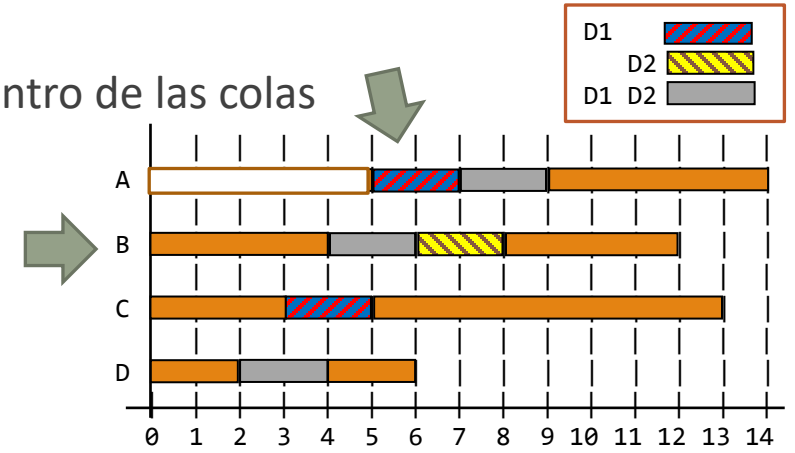
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

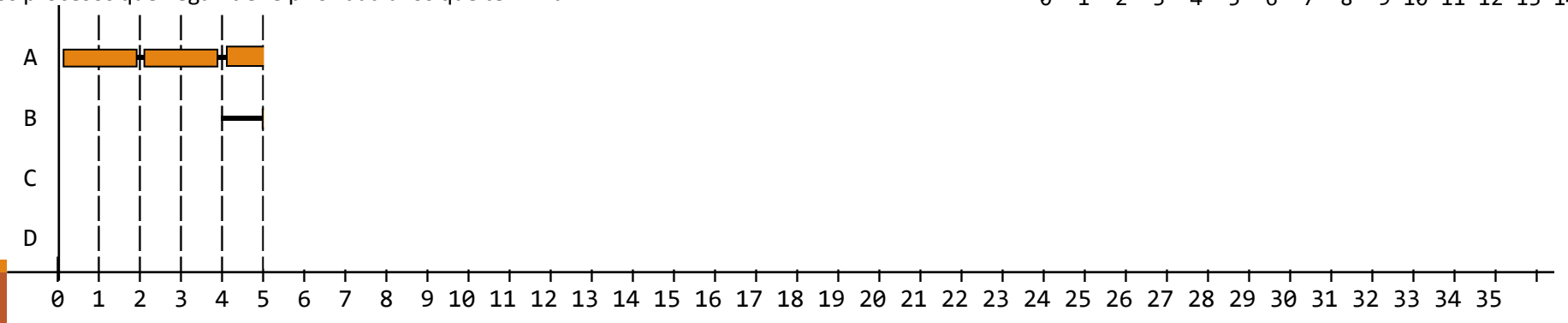
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

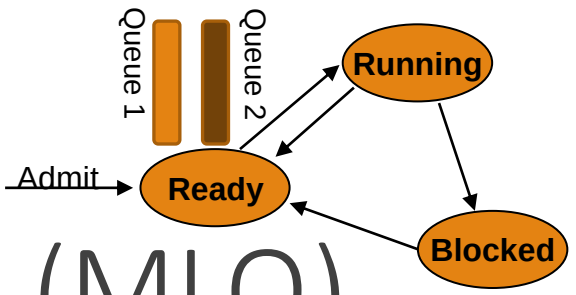
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

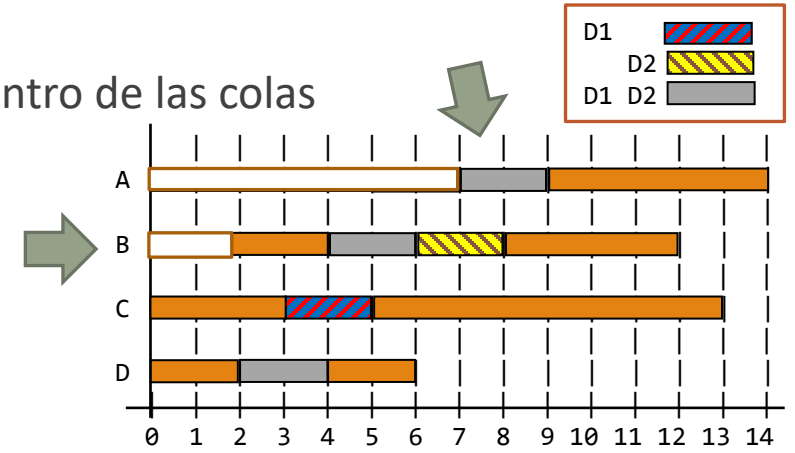
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

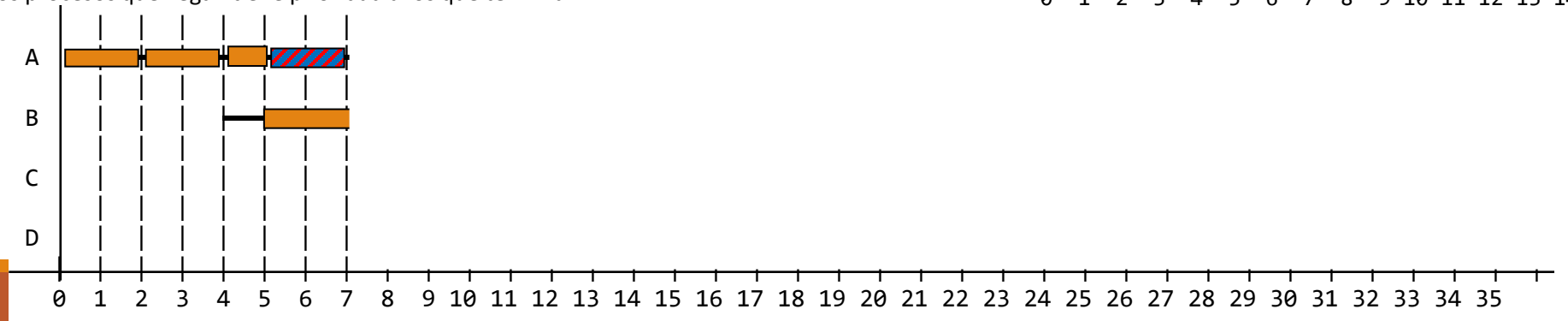
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

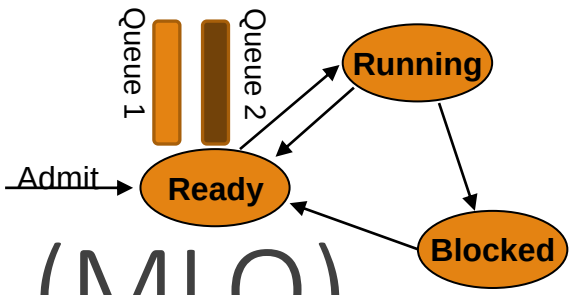
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

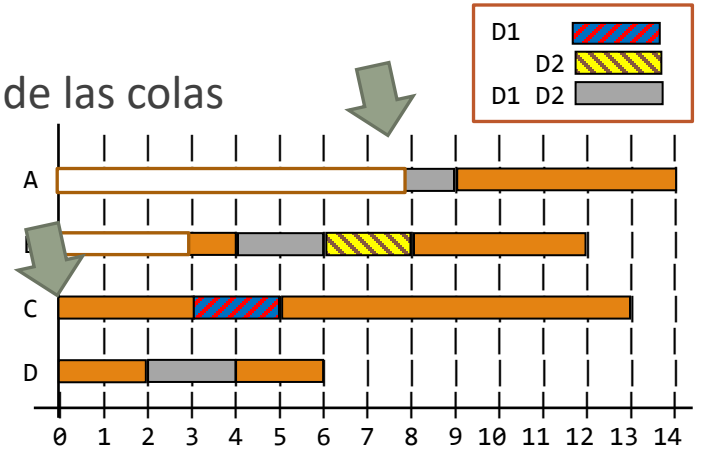
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

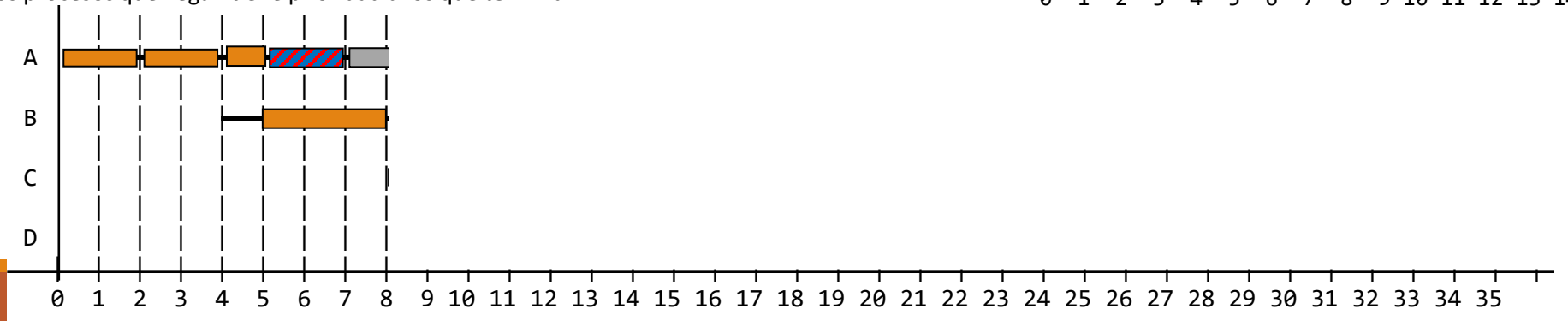
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

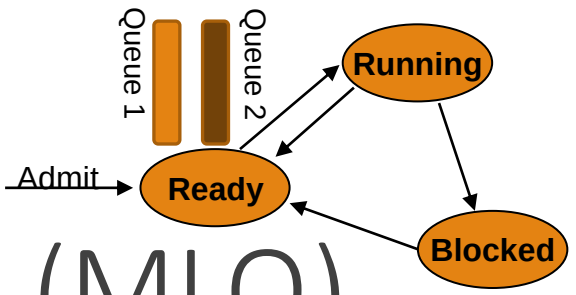
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

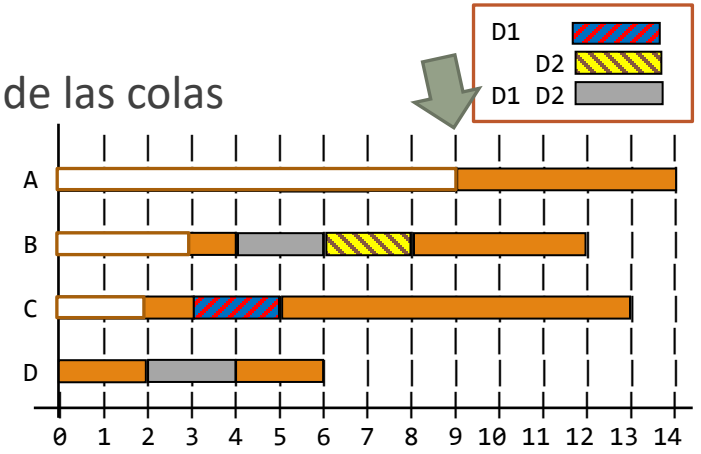
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

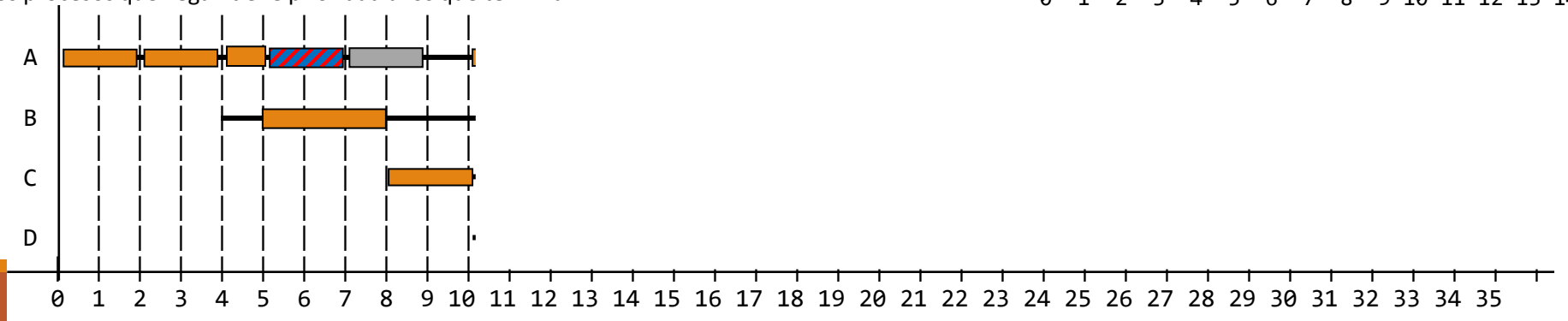
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

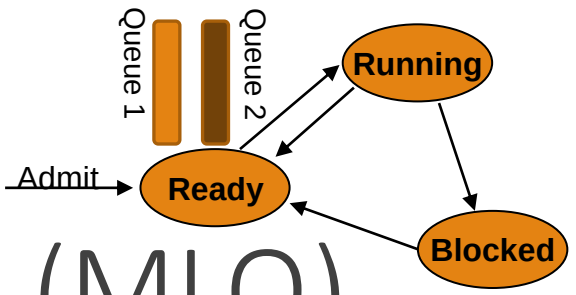
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

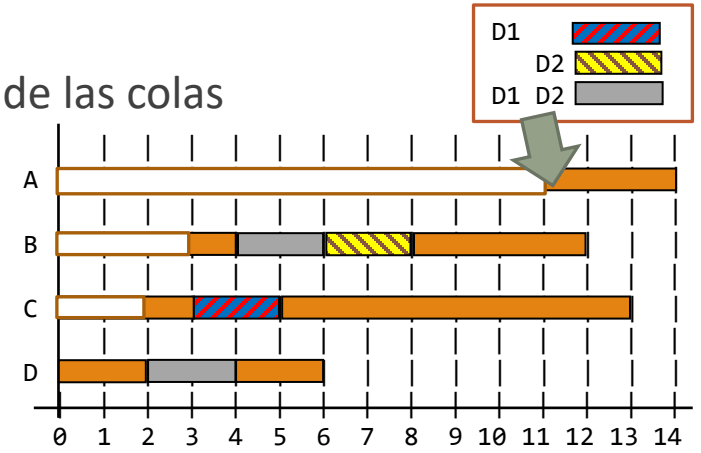
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

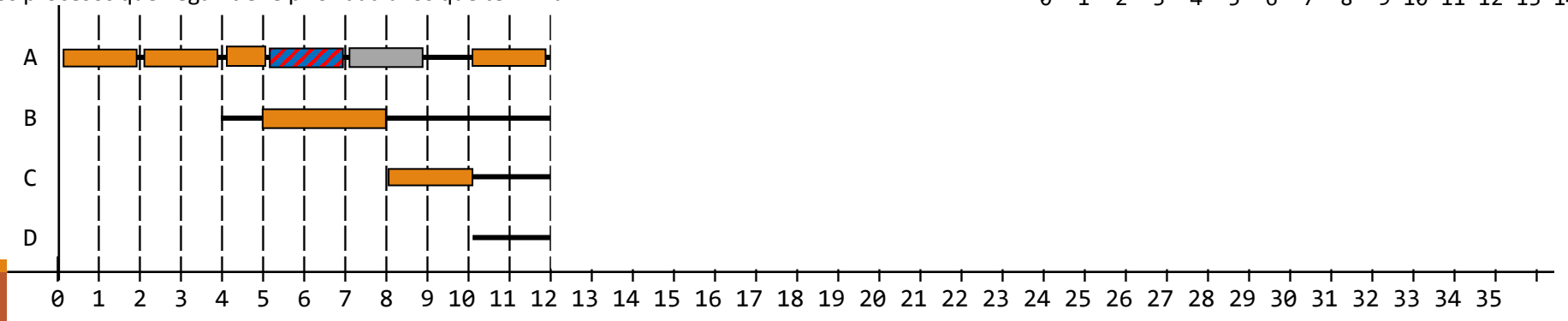
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

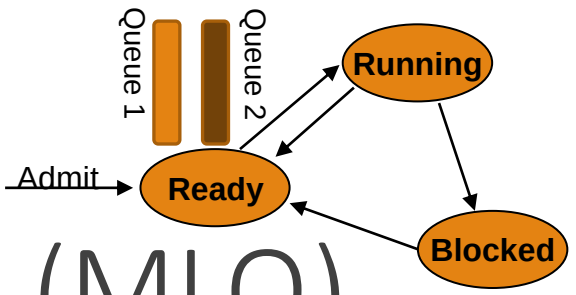
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

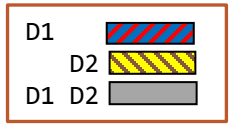
El planificador gestiona varias colas de preparados

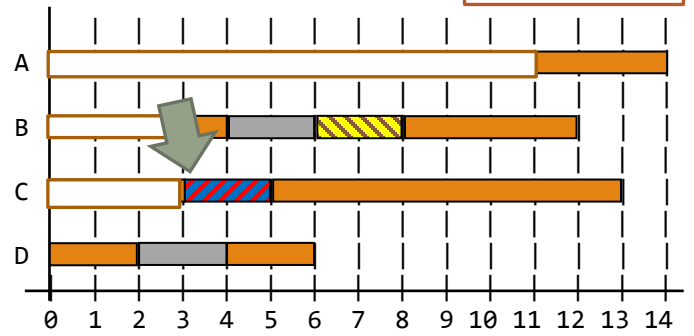
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

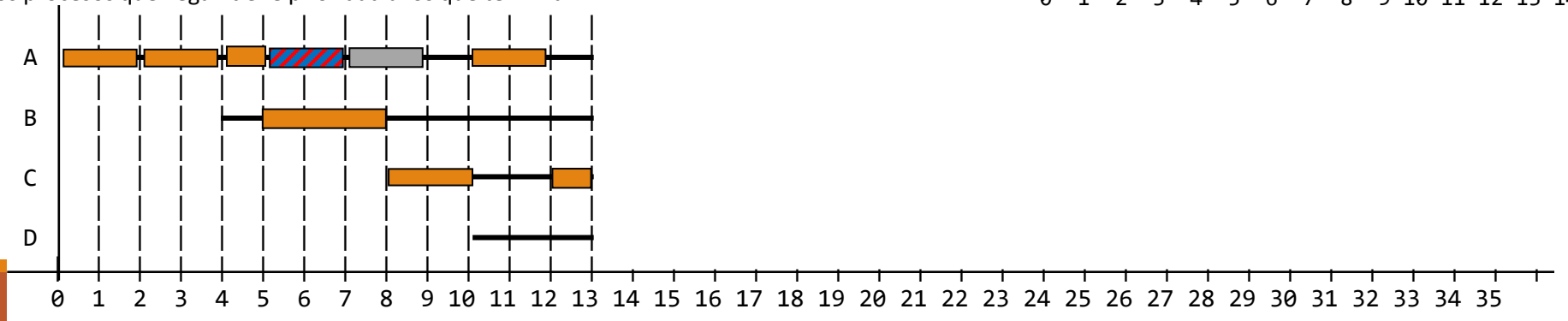
Es determinante:

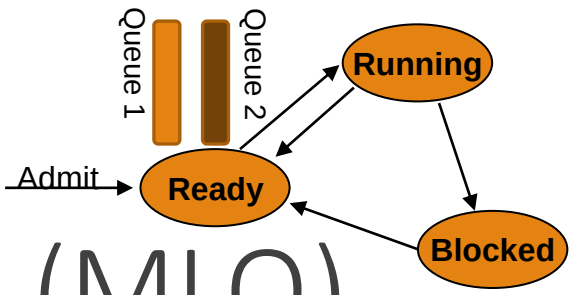
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

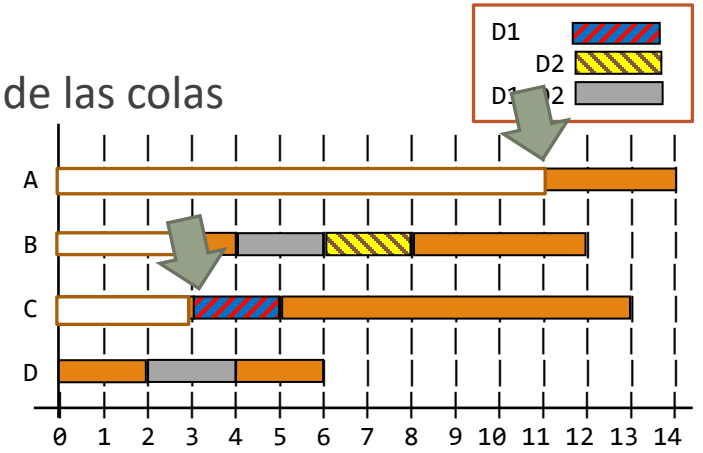
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

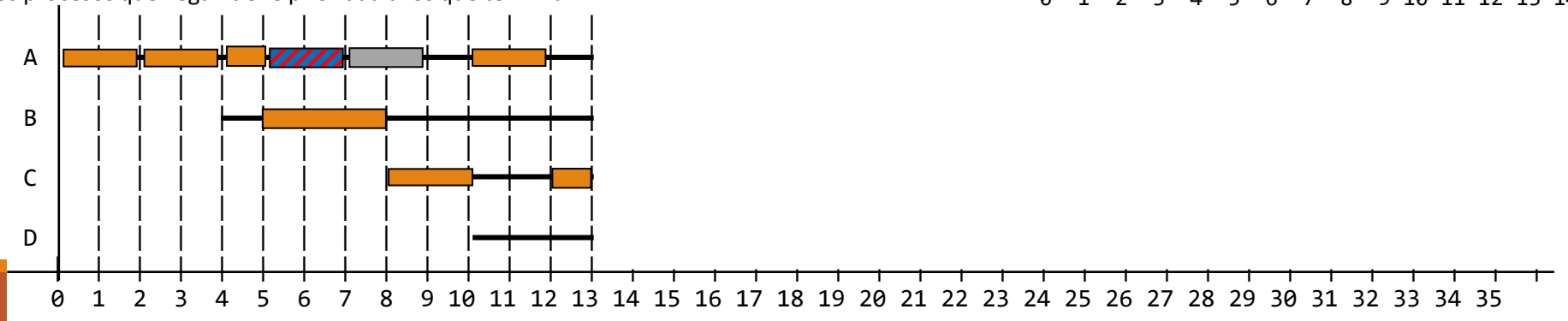
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

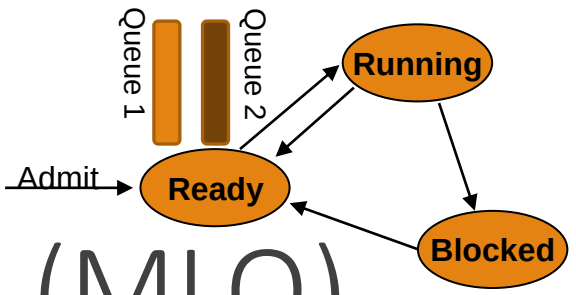
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

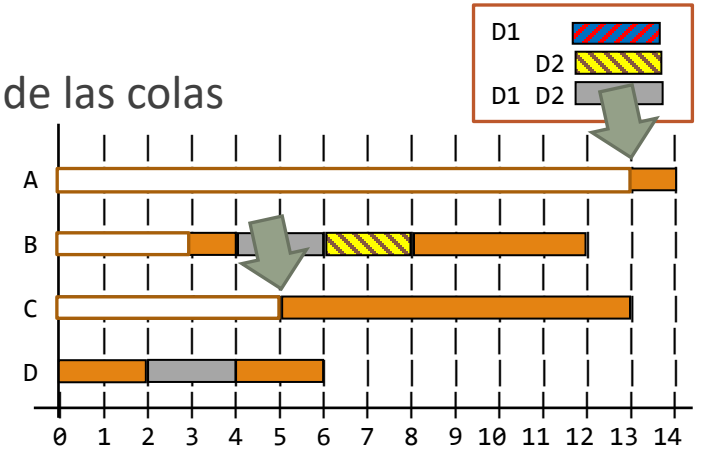
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

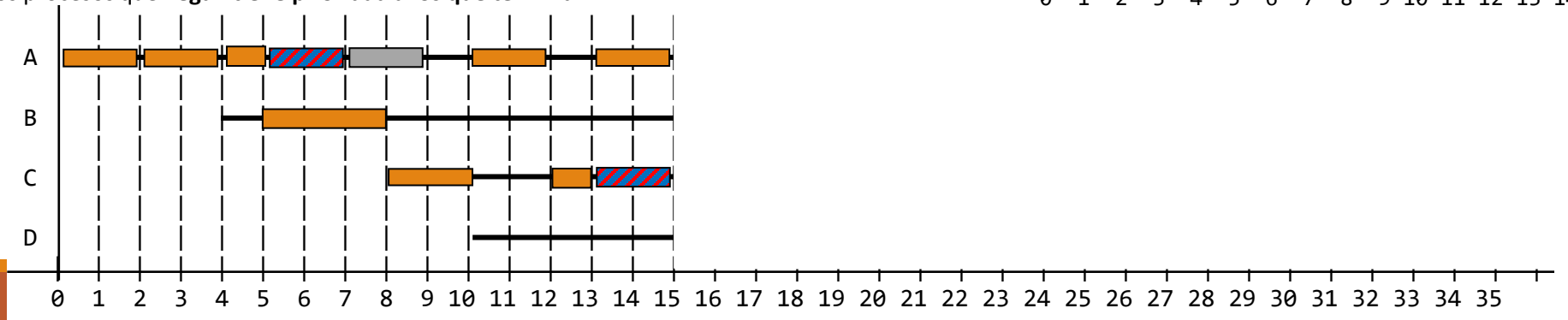
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

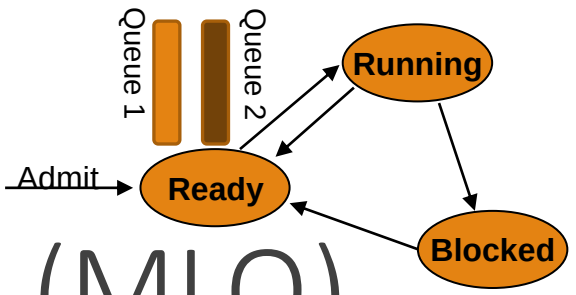
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que **llegan tiene prioridad a los que terminan.**





Planificación multicolas (MLQ)

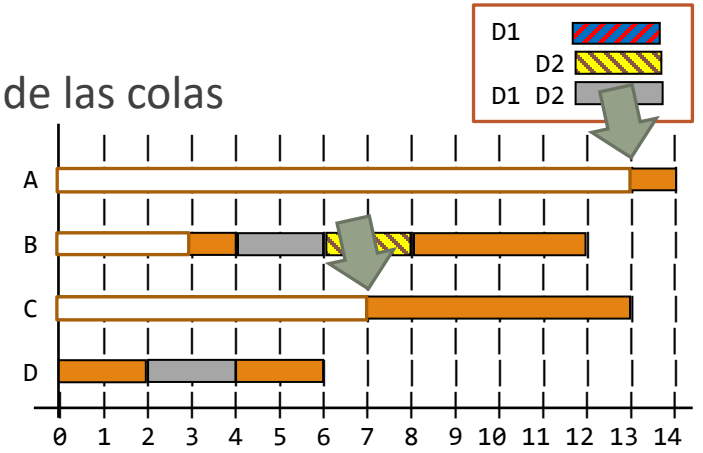
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

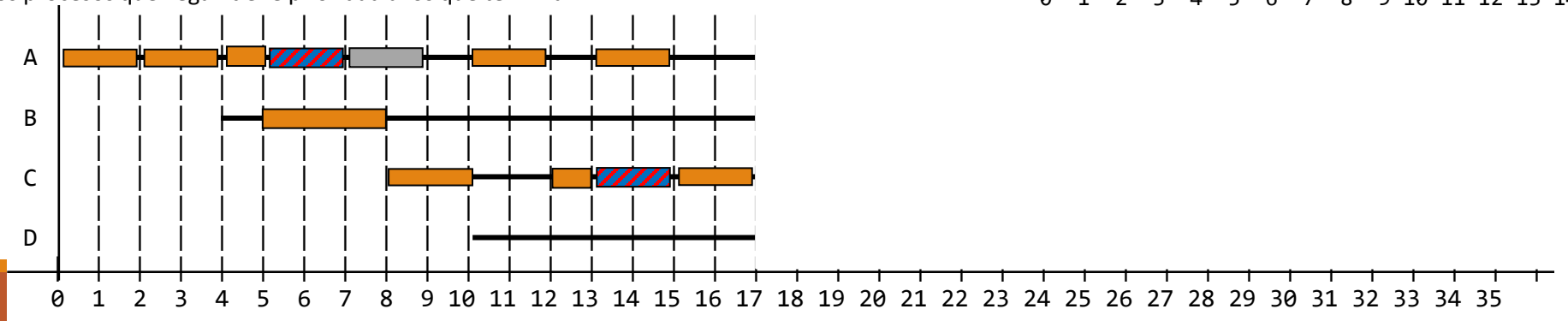
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

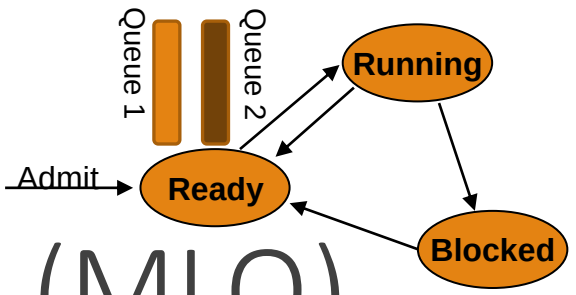
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

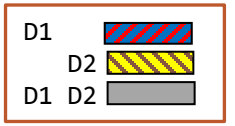
El planificador gestiona varias colas de preparados

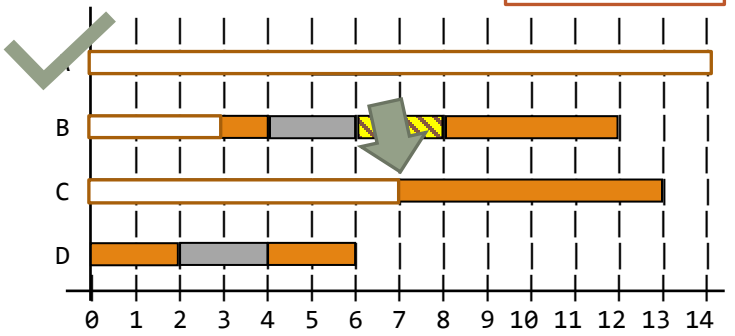
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

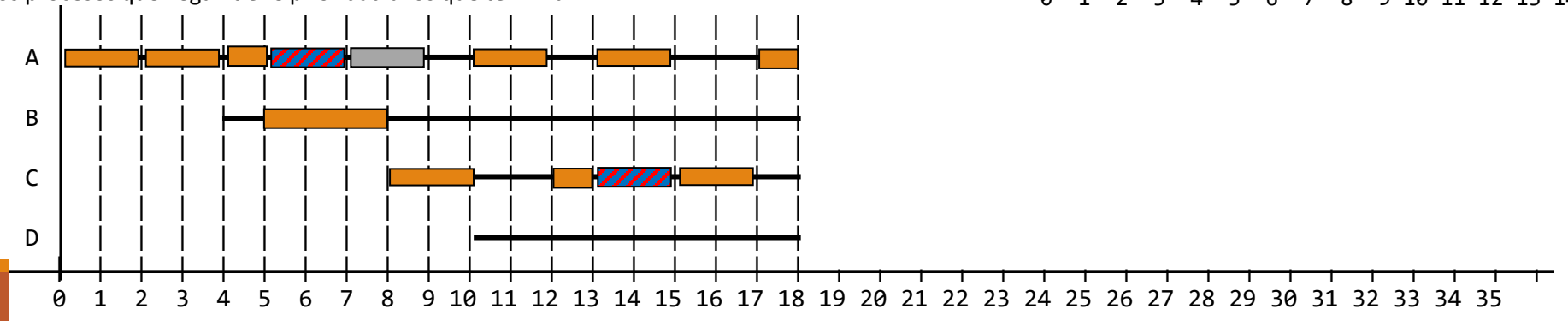
Es determinante:

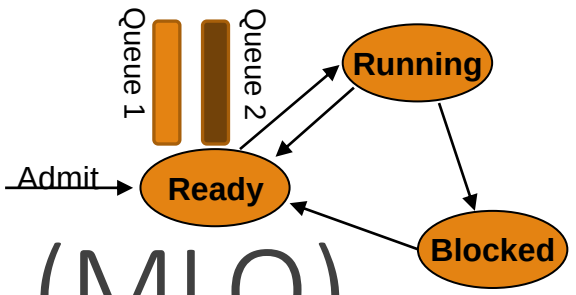
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

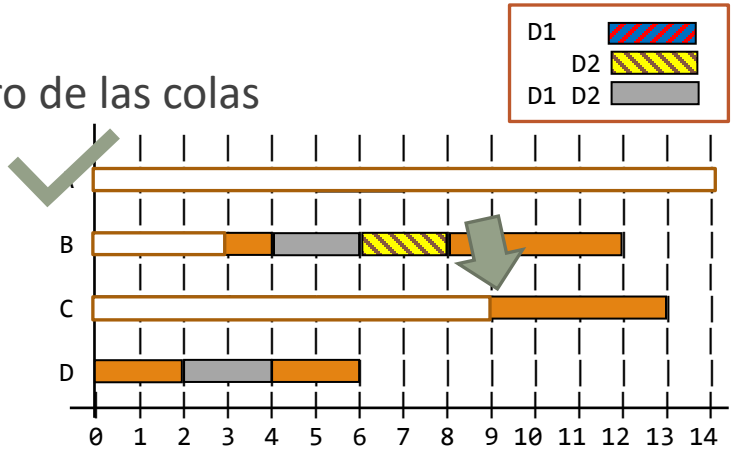
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

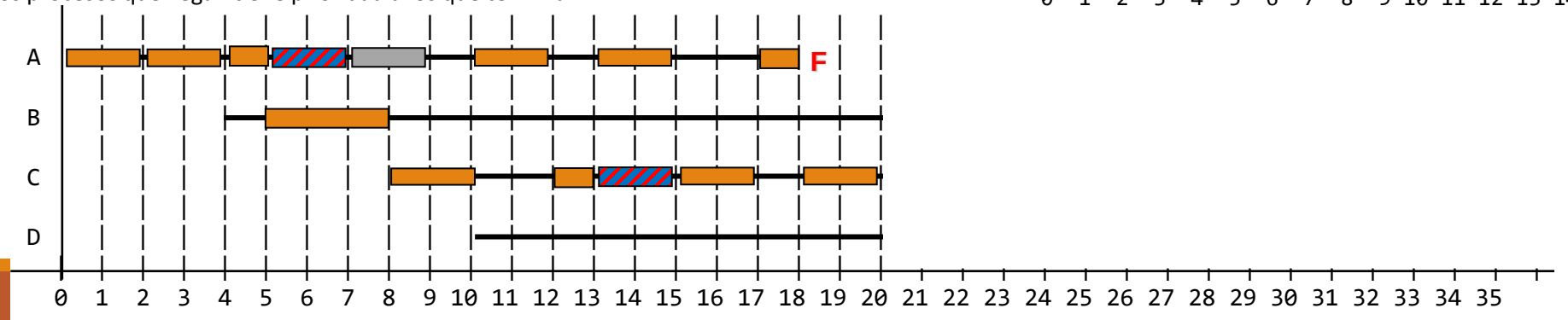
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

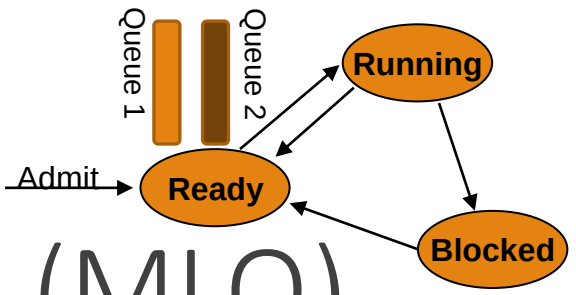
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

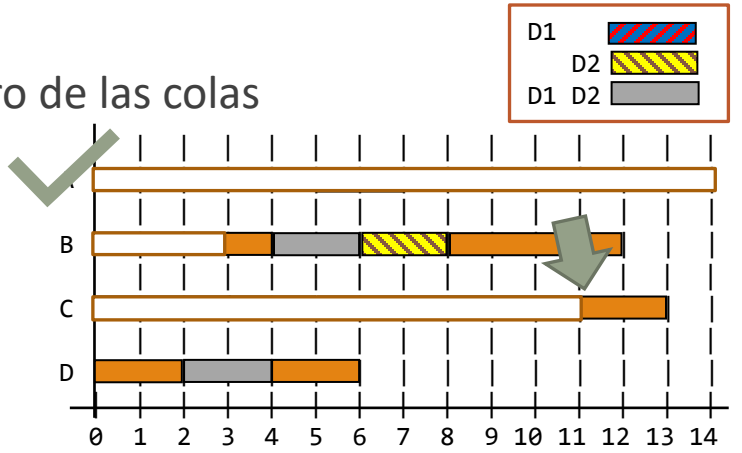
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

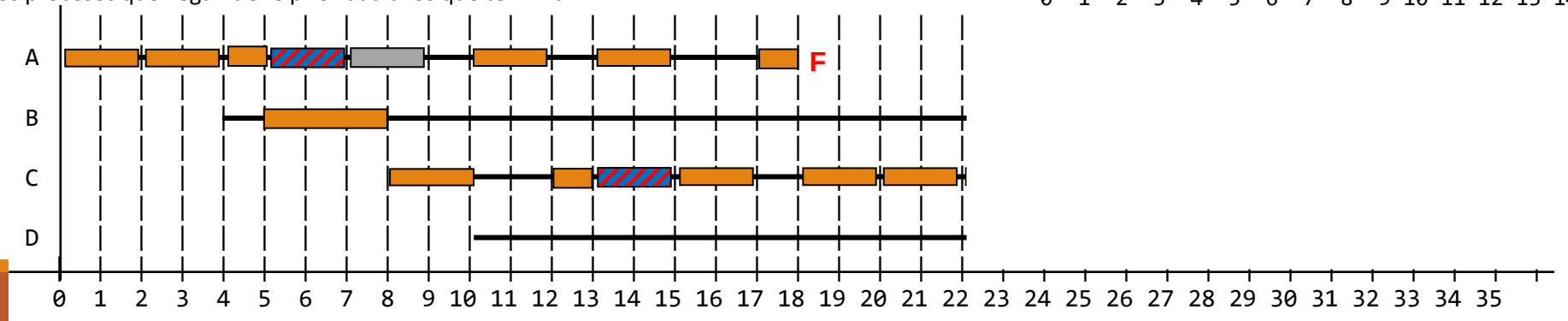
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

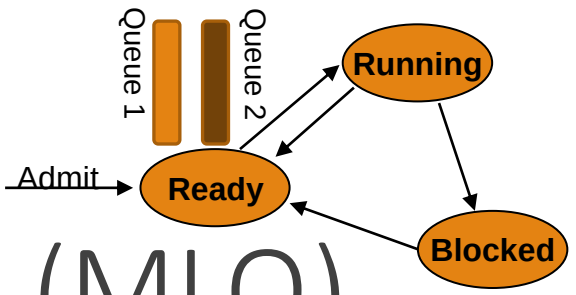
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad **1 es mas prioritaria que 2** (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

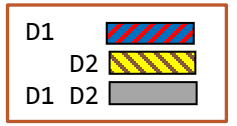
El planificador gestiona varias colas de preparados

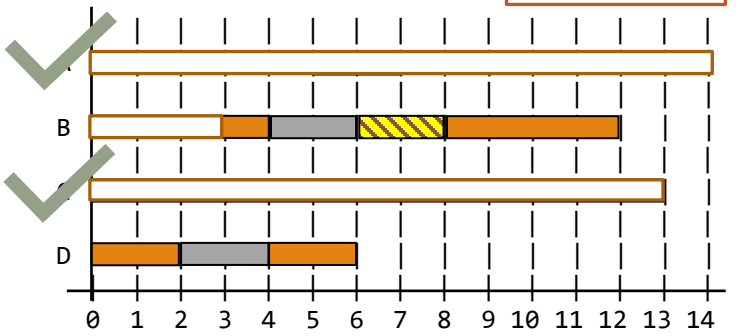
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

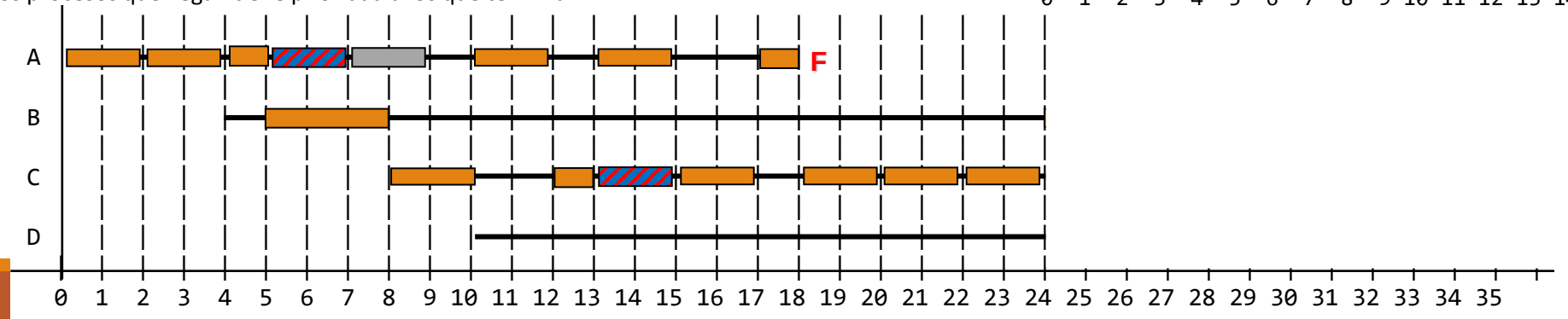
Es determinante:

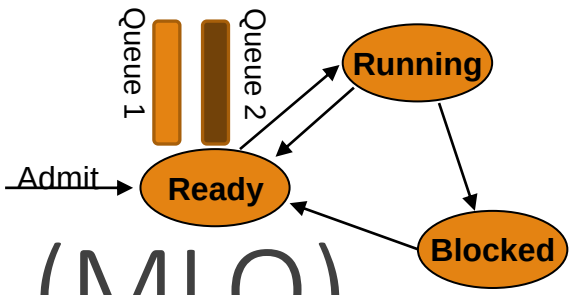
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

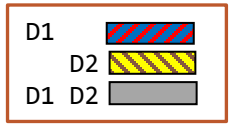
El planificador gestiona varias colas de preparados

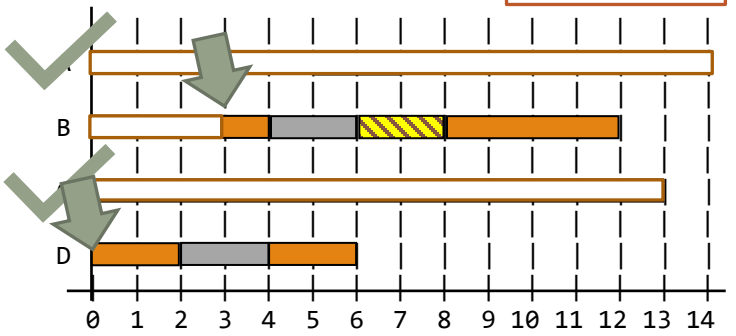
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

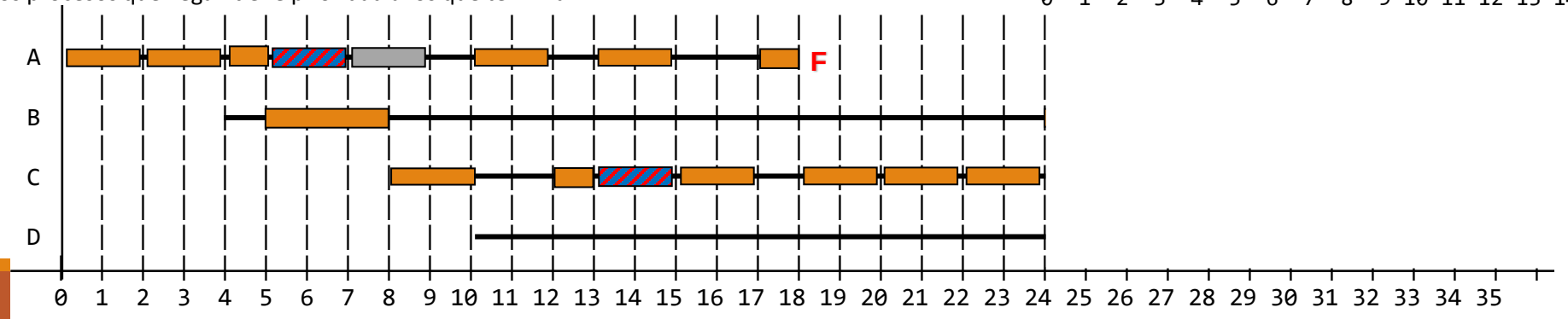
Es determinante:

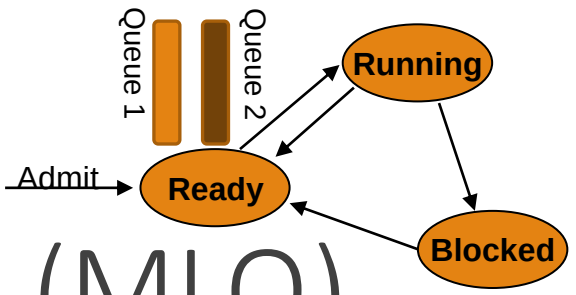
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

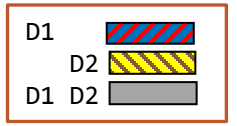
El planificador gestiona varias colas de preparados

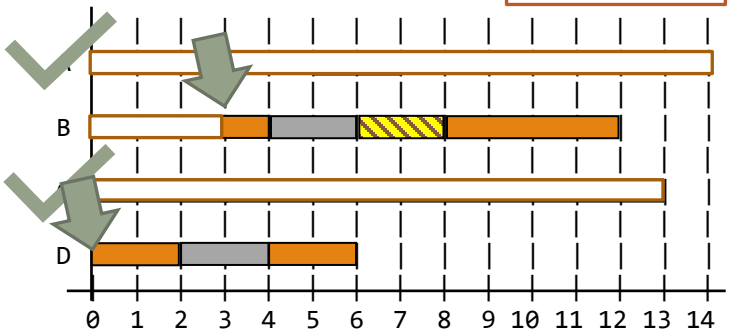
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

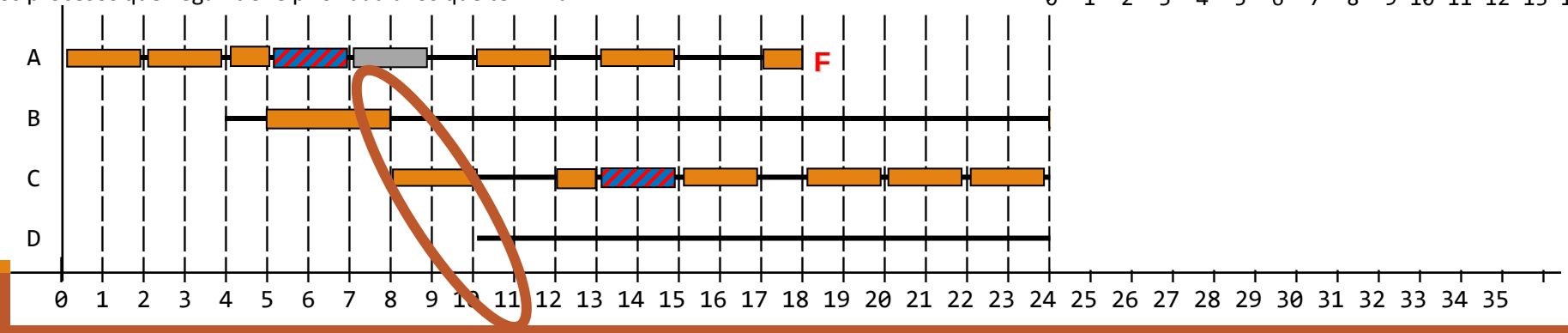
Es determinante:

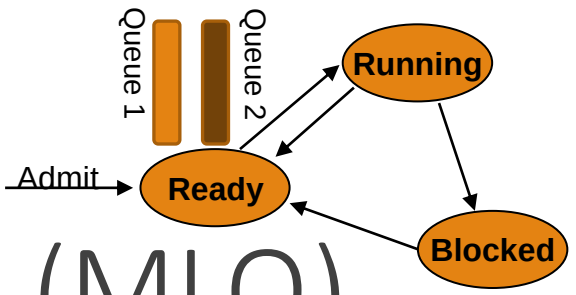
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

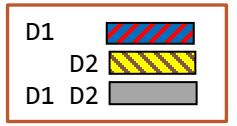
El planificador gestiona varias colas de preparados

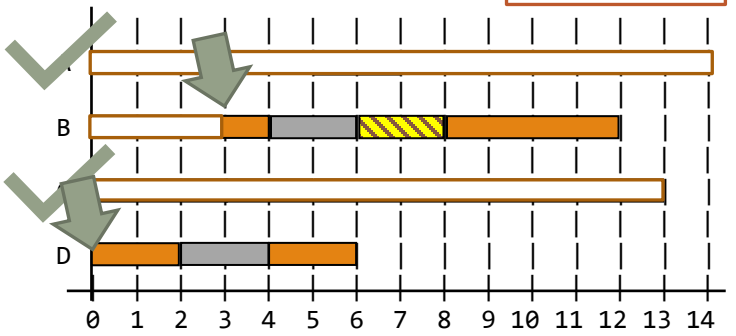
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

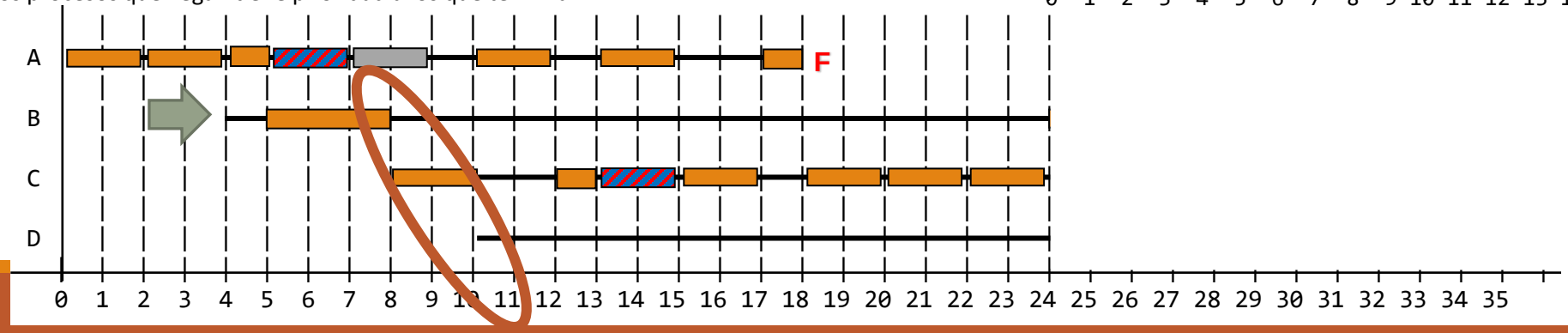
Es determinante:

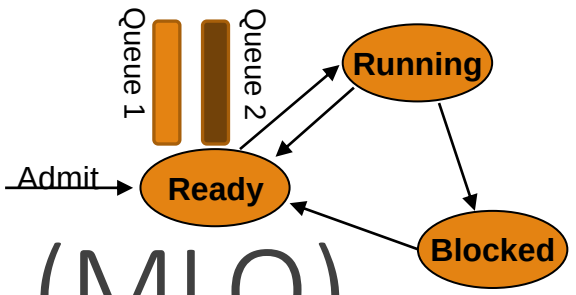
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

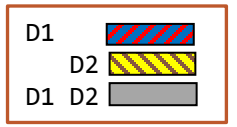
El planificador gestiona varias colas de preparados

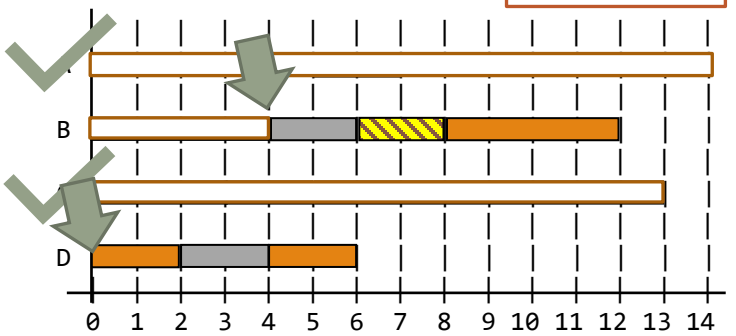
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

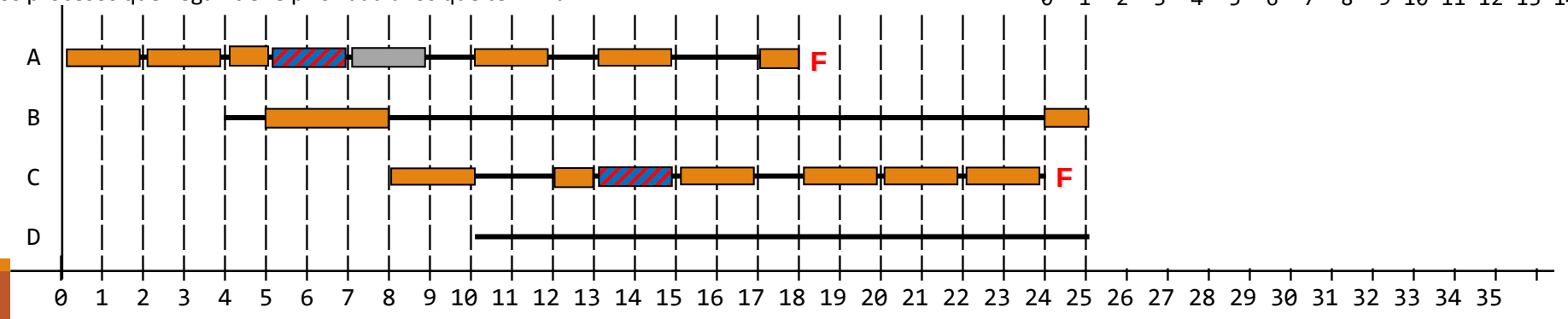
Es determinante:

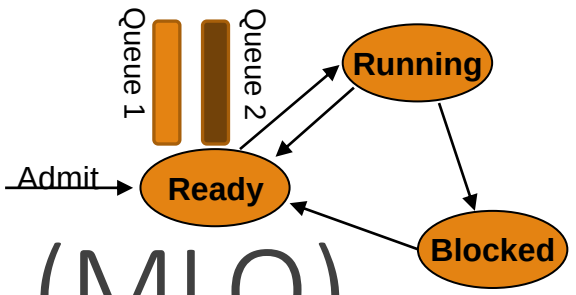
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

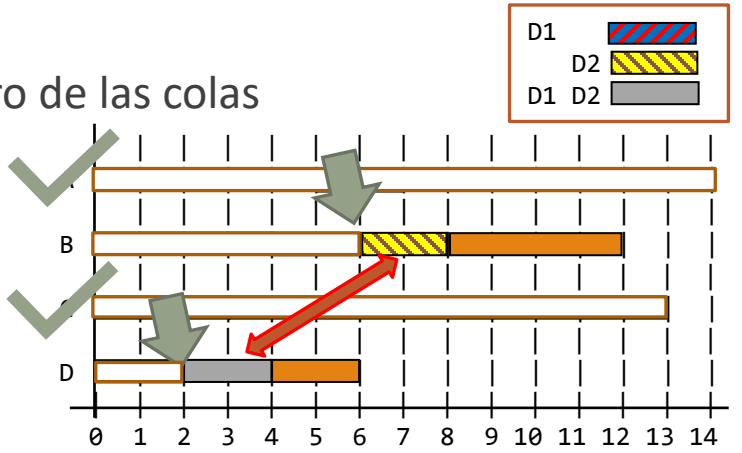
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

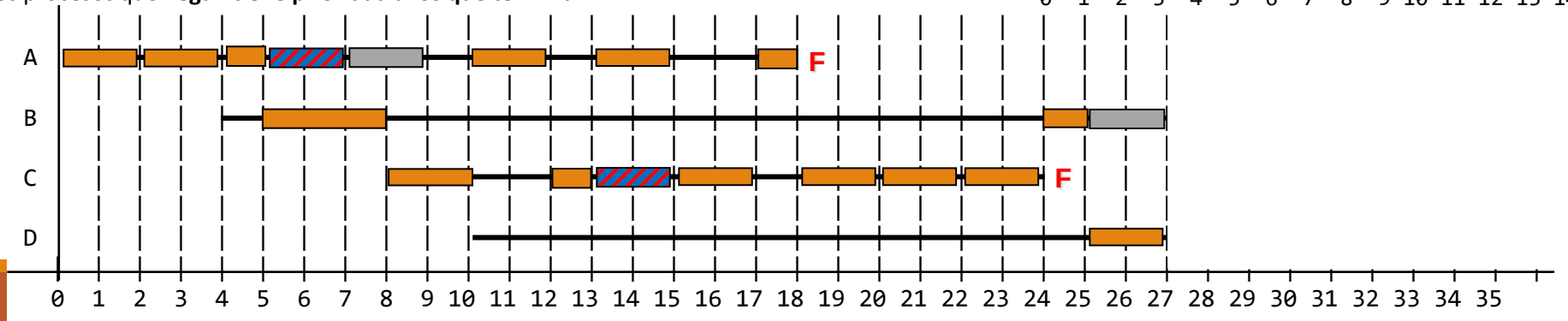
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

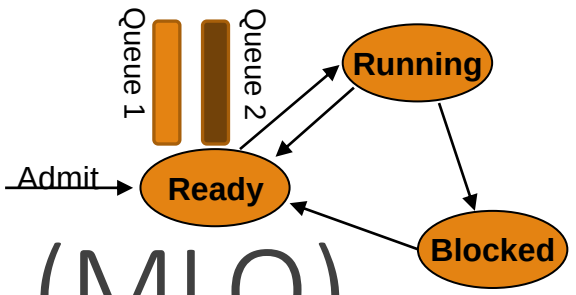
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que **llegan tiene prioridad a los que terminan.**





Planificación multicolas (MLQ)

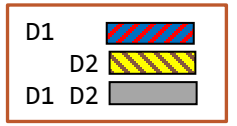
El planificador gestiona varias colas de preparados

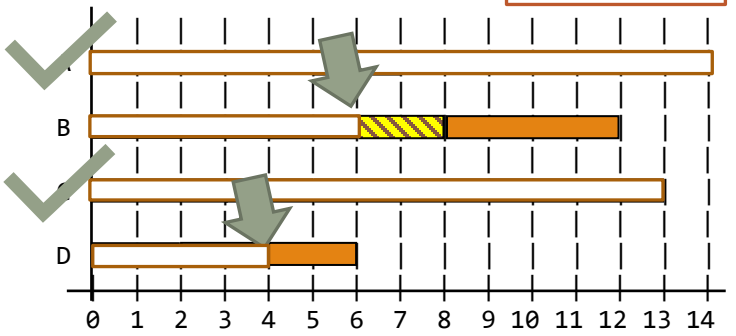
Cada cola puede gestionarse mediante una política diferente

Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

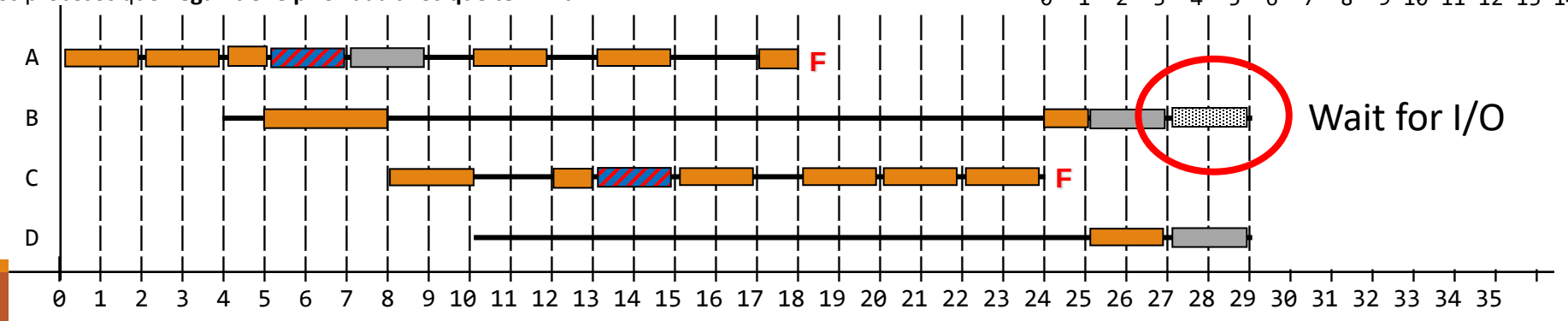
Es determinante:

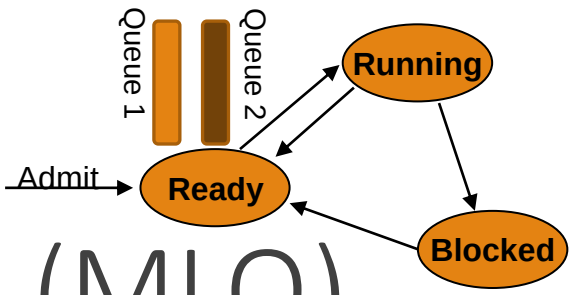
- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**





Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos **de acceso a los dispositivos** acceso se debe realizar usando **FCFS**.
Los procesos que **llegan tiene prioridad a los que terminan.**





Planificación multicolas (MLQ)

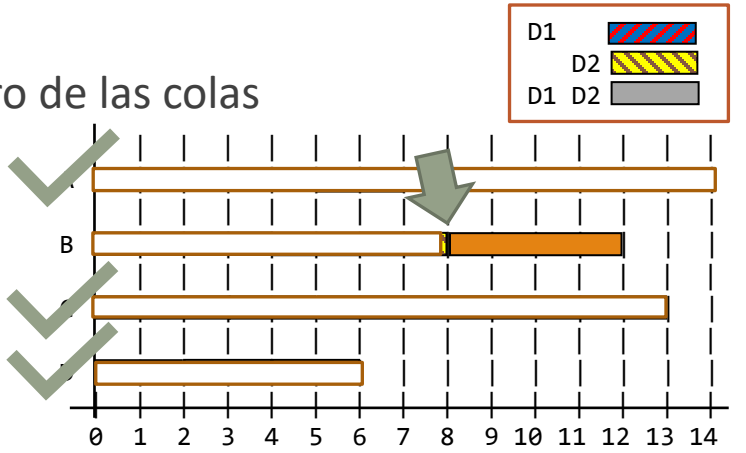
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

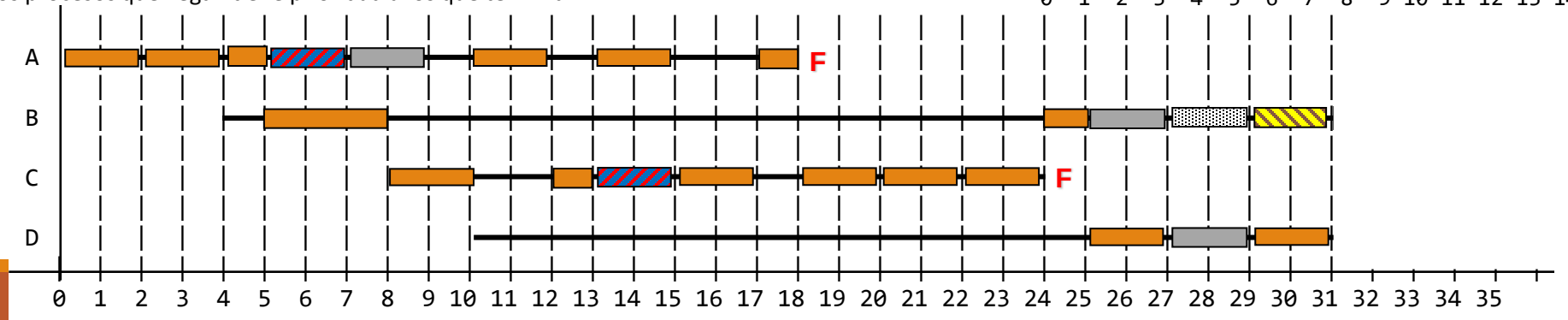
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

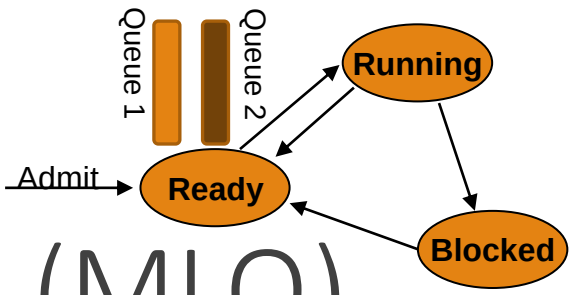
Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.





Planificación multicolas (MLQ)

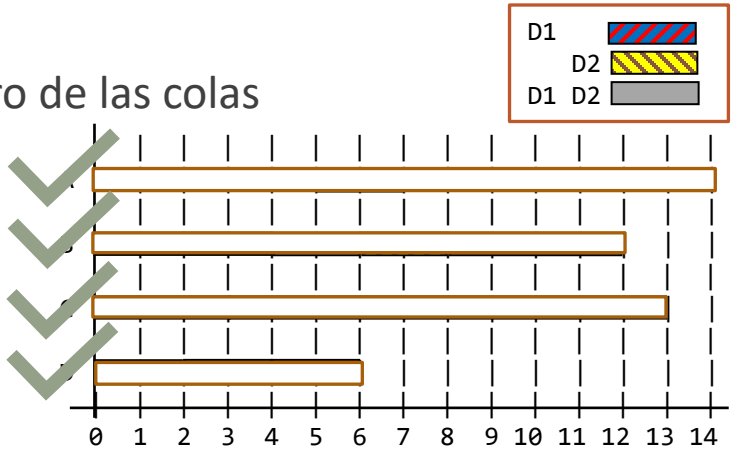
El planificador gestiona varias colas de preparados

Cada cola puede gestionarse mediante una política diferente

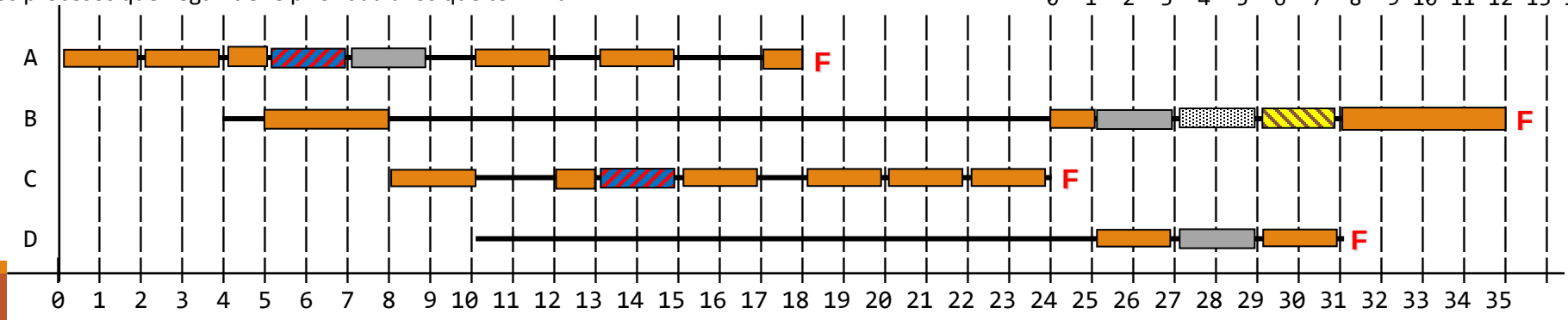
Proceso	Tª ini	Cola
A	0	1
B	4	2
C	8	1
D	10	2

Es determinante:

- la política de **ordenación de los procesos** dentro de las colas
- la política de **ordenación entre las colas**



Ejercicio 15:
Sistema monoprocesador con sistema de gestión multicolas sin realimentación.
Dos colas con distinta prioridad 1 es mas prioritaria que 2 (preemtible)
La cola 1 es gestionada con RRq=2
La cola 2 es gestionada con FCFS
Los procesos tienen requerimientos de acceso a los dispositivos acceso se debe realizar usando FCFS.
Los procesos que llegan tiene prioridad a los que terminan.

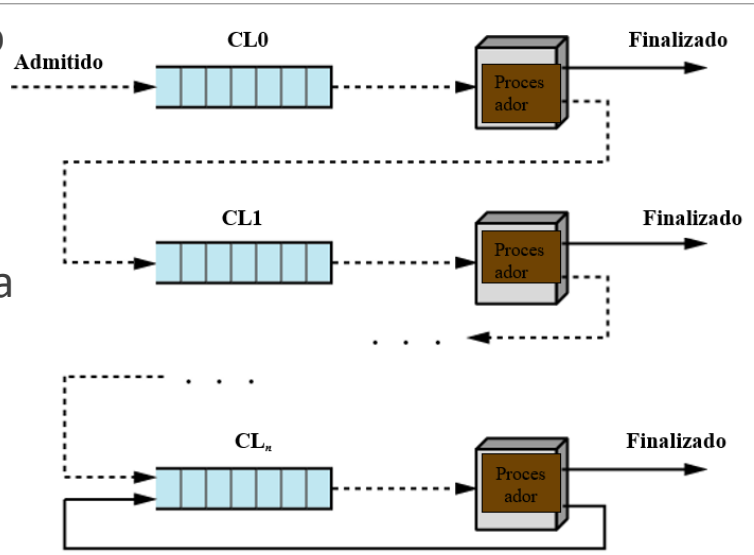


Feedback (Retroalimentación)

Se penaliza a los trabajos que han estado ejecutando más tiempo

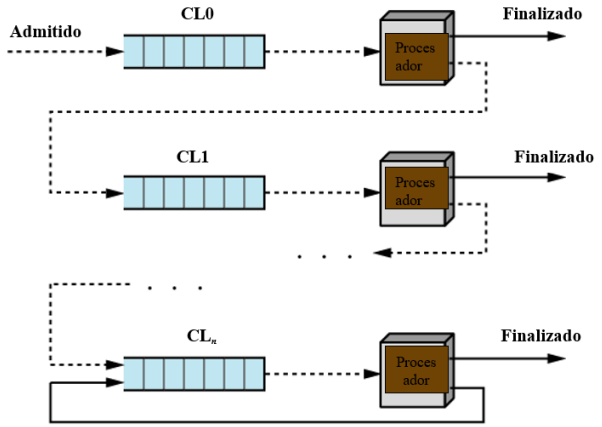
Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

Los procesos van descendiendo a colas inferiores después de cada ejecución



Planificación retroalimentada

Feedback MQ



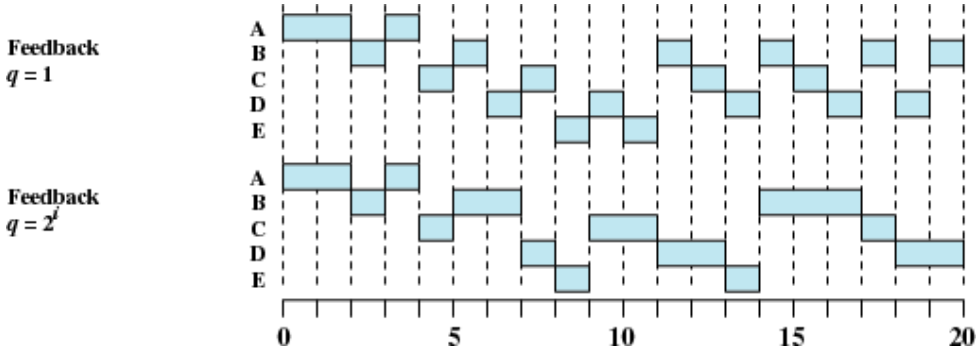
Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

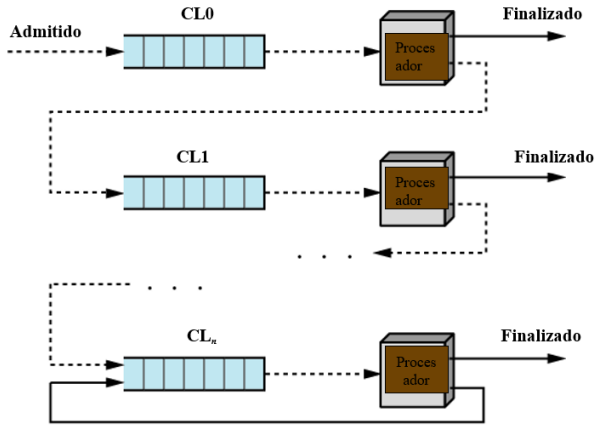
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

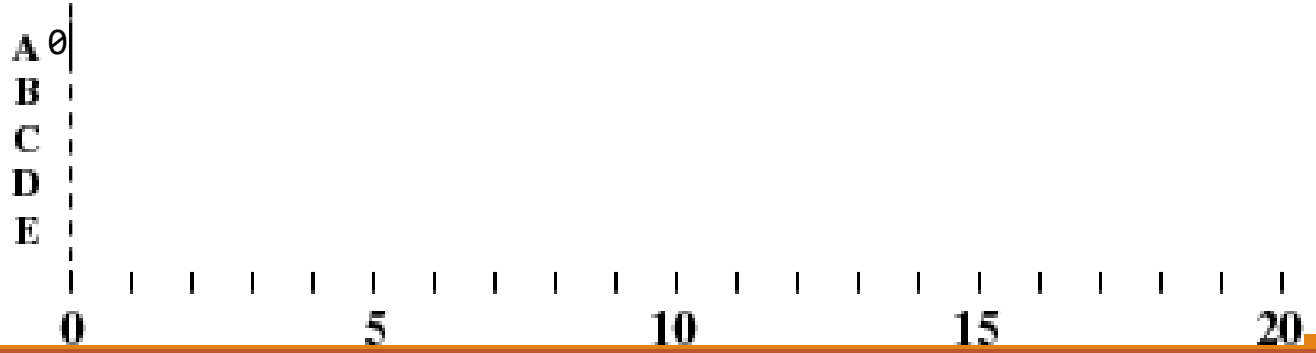
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

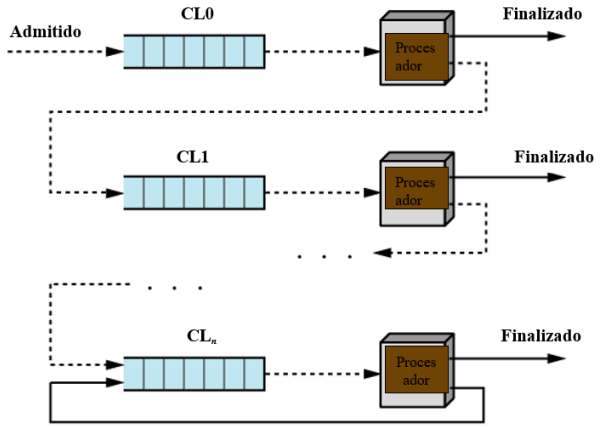
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

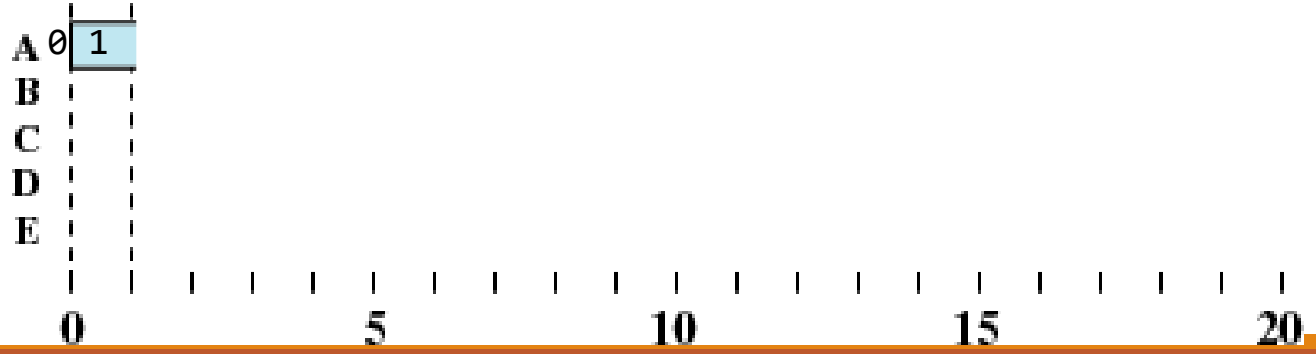
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

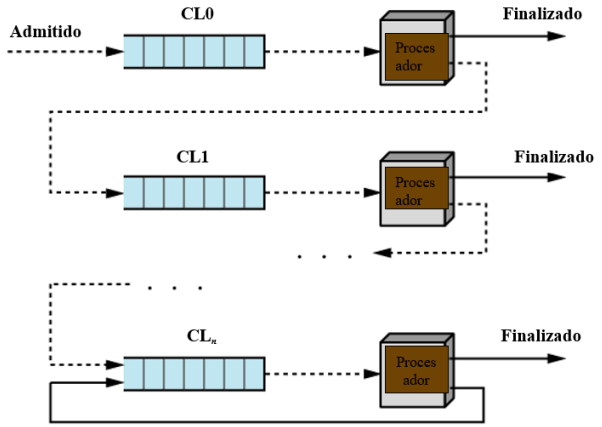
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

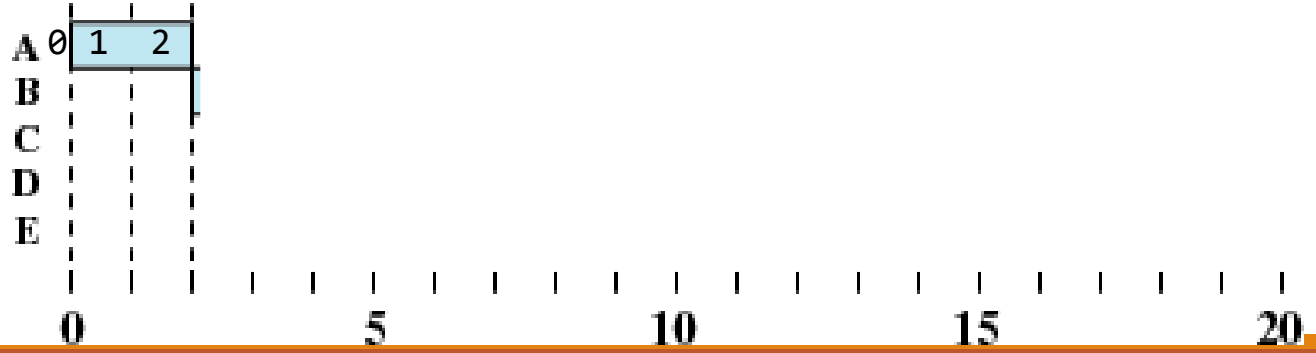
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

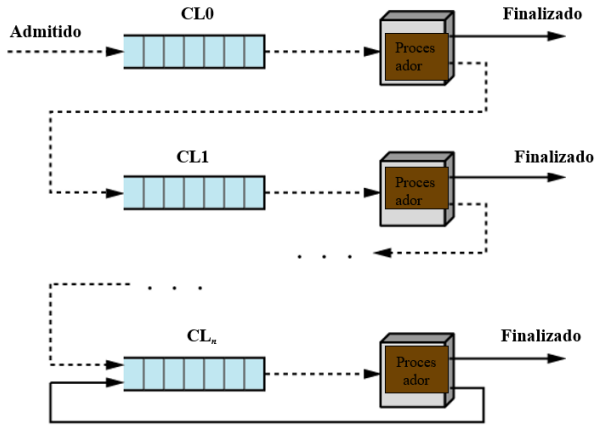
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

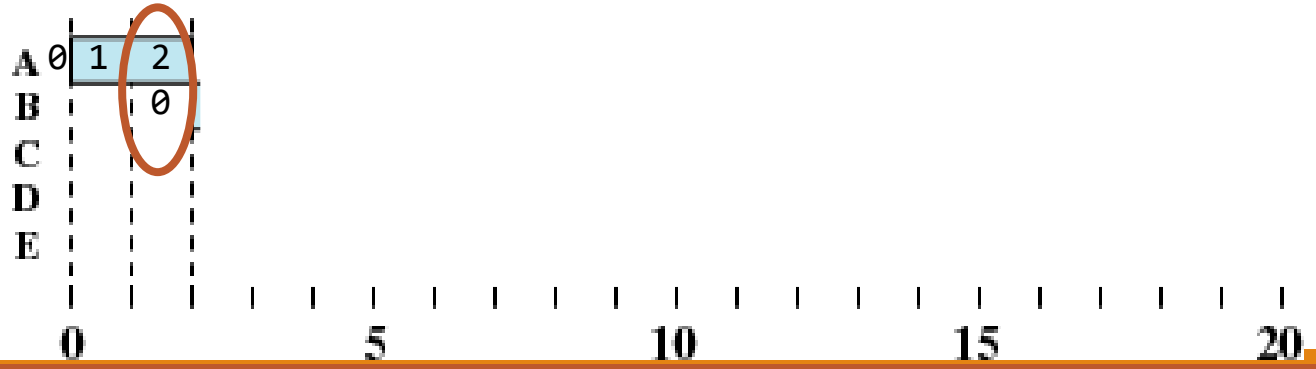
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

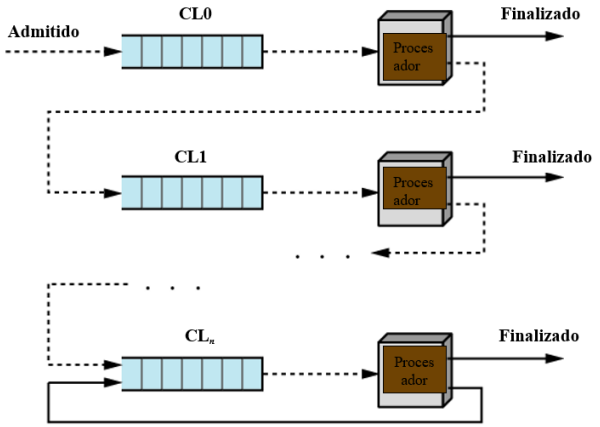
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

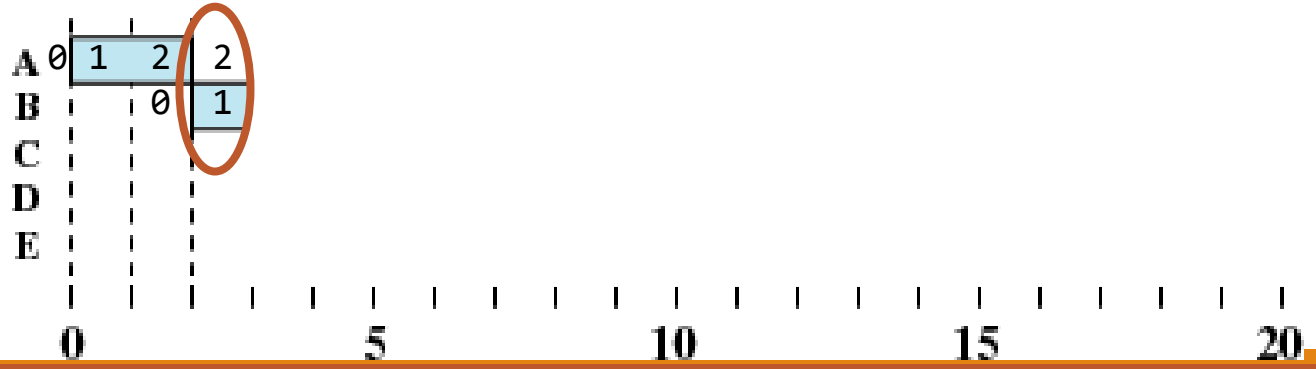
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

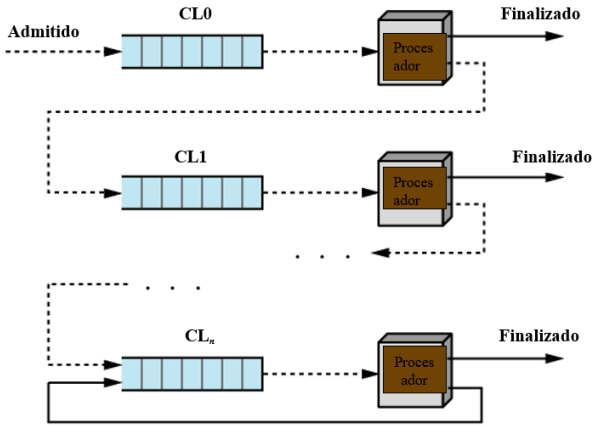
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

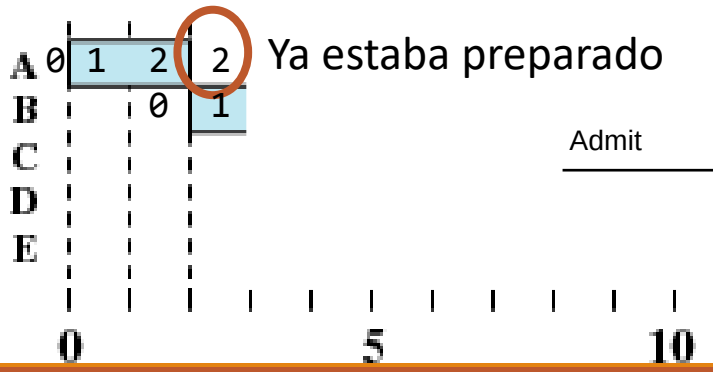
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

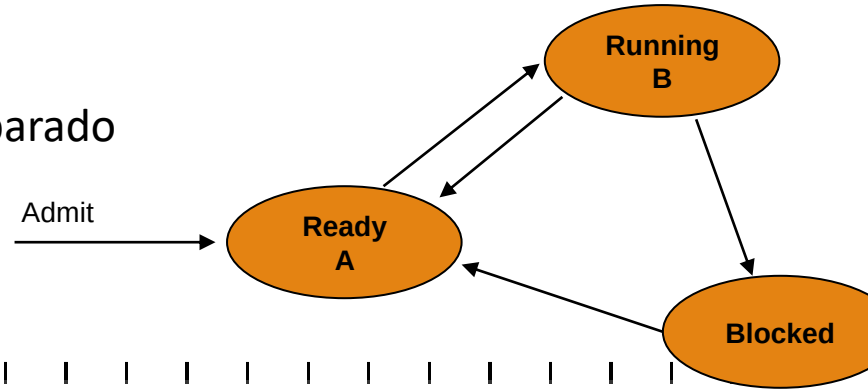
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

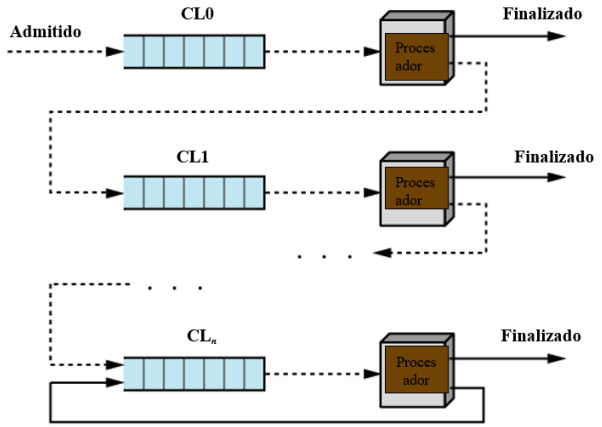
Todas las colas tienen el mismo quantum



Ya estaba preparado



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

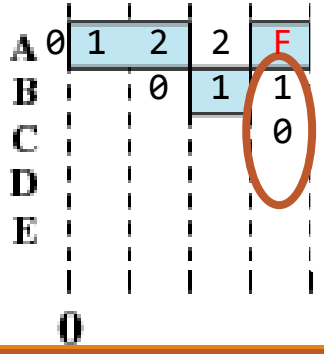
Los procesos van descendiendo a colas inferiores después de cada ejecución

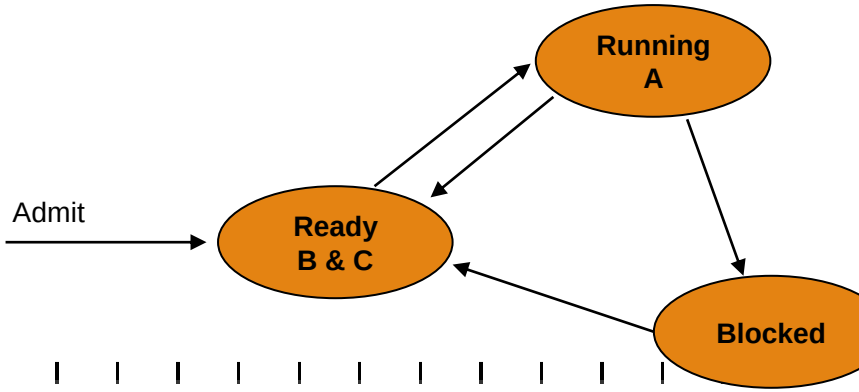
Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

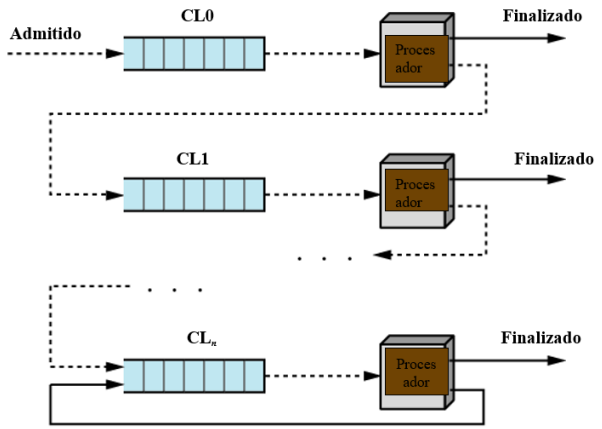
Feedback
$q = 1$

Todas las colas tienen el mismo quantum





Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

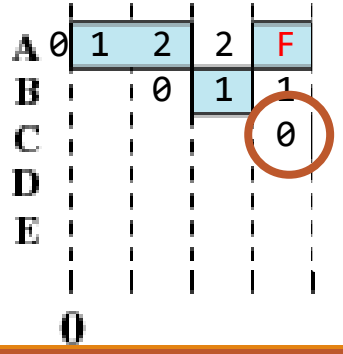
Los procesos van descendiendo a colas inferiores después de cada ejecución

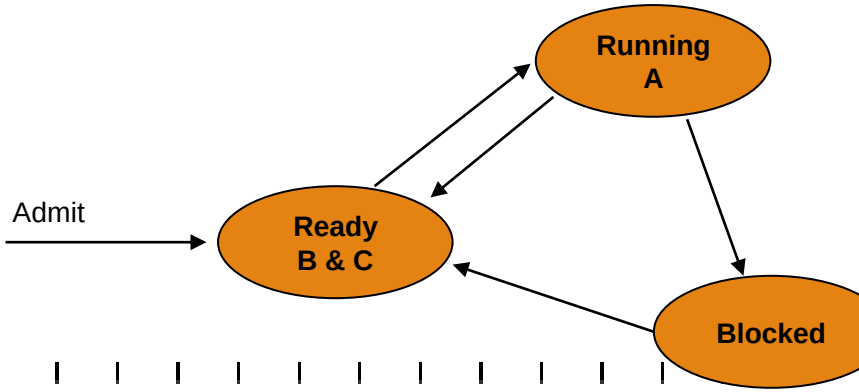
Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

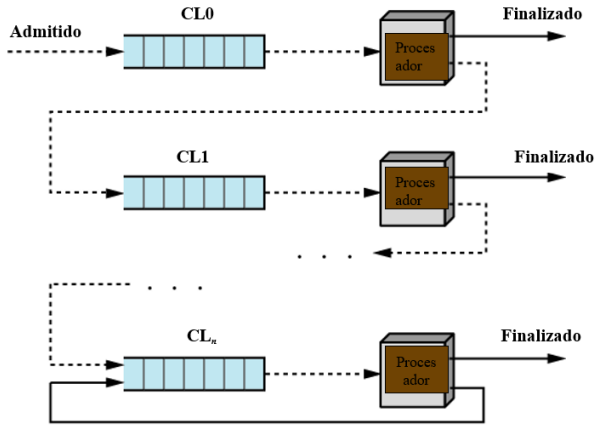
Feedback
$q = 1$

Todas las colas tienen el mismo quantum





Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

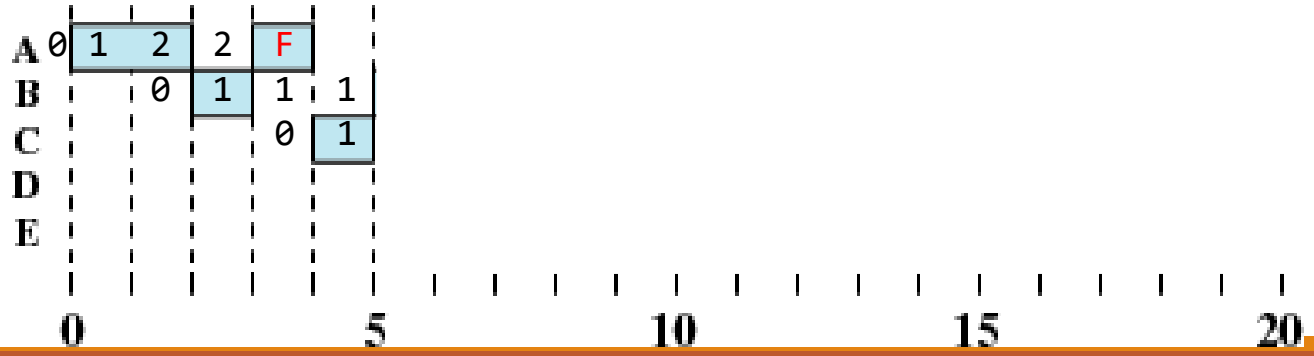
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

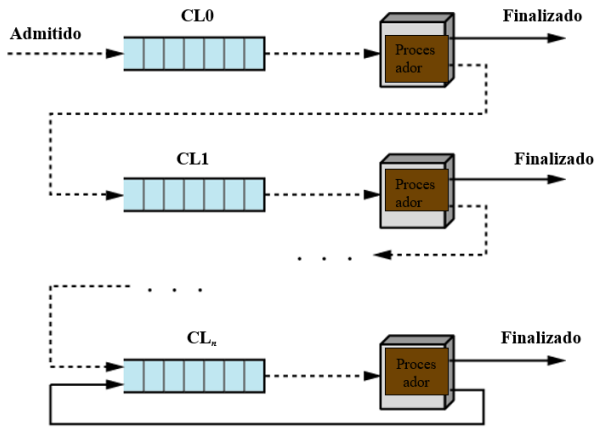
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

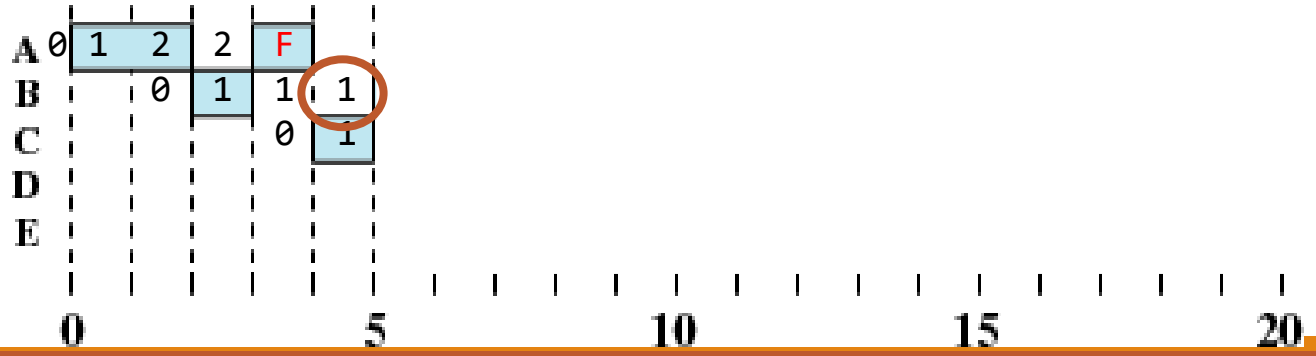
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

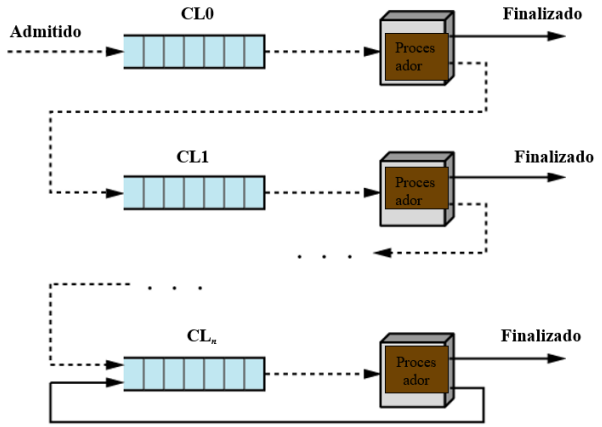
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

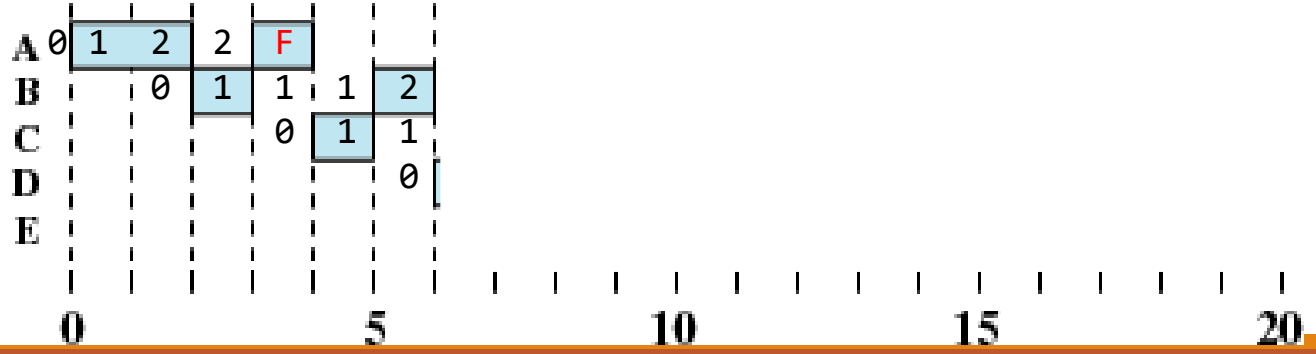
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

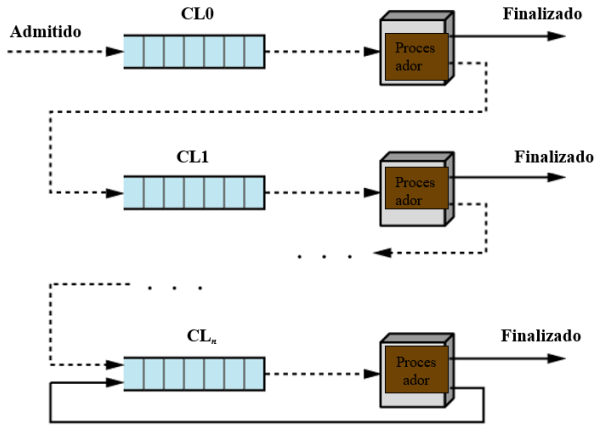
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

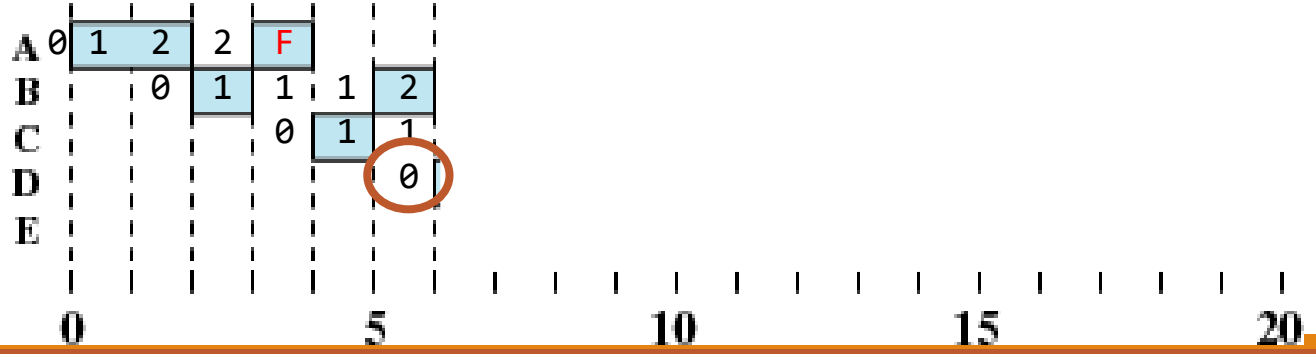
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

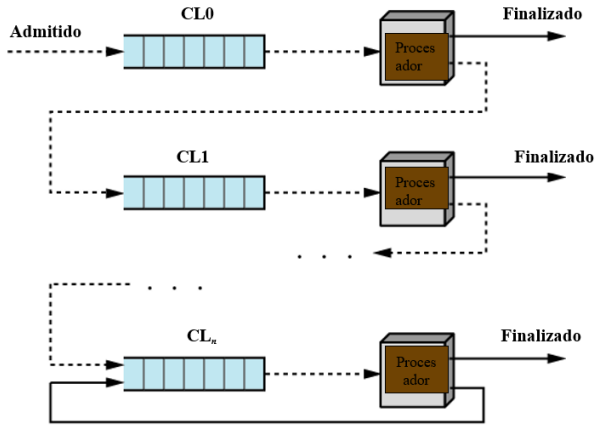
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

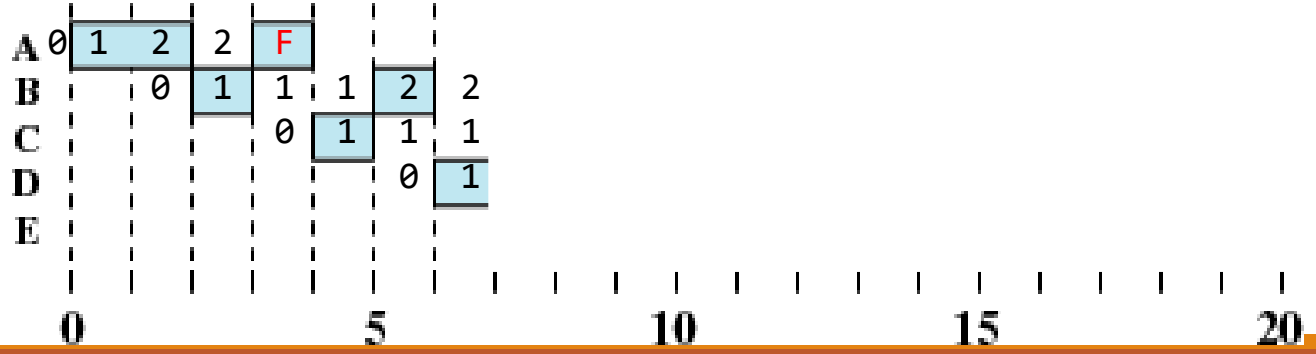
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

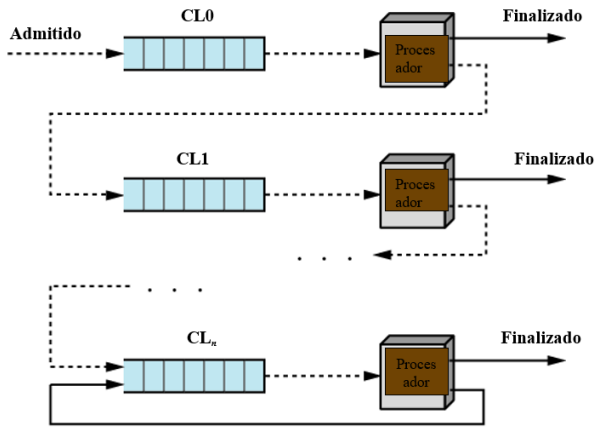
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

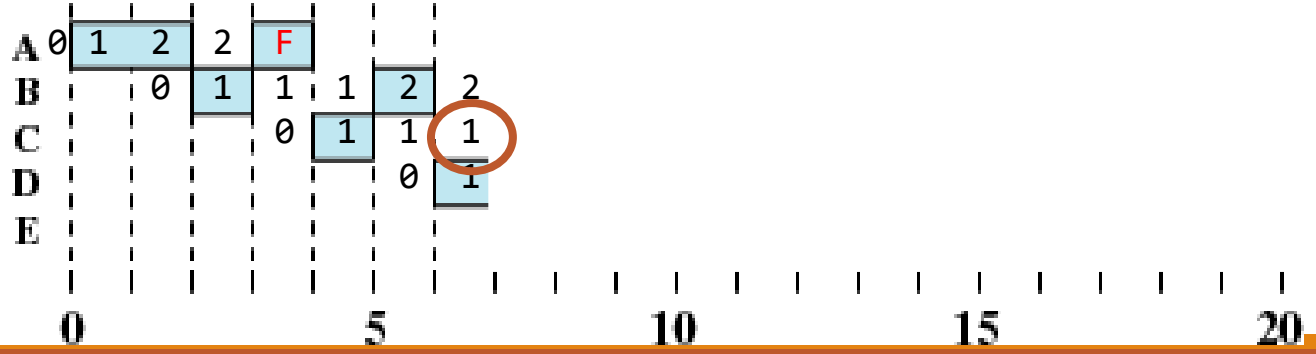
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

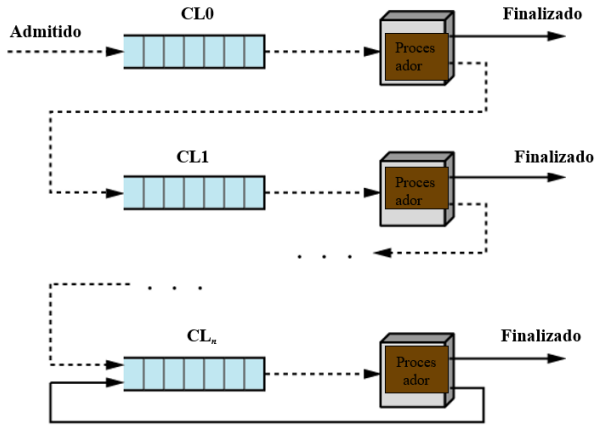
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

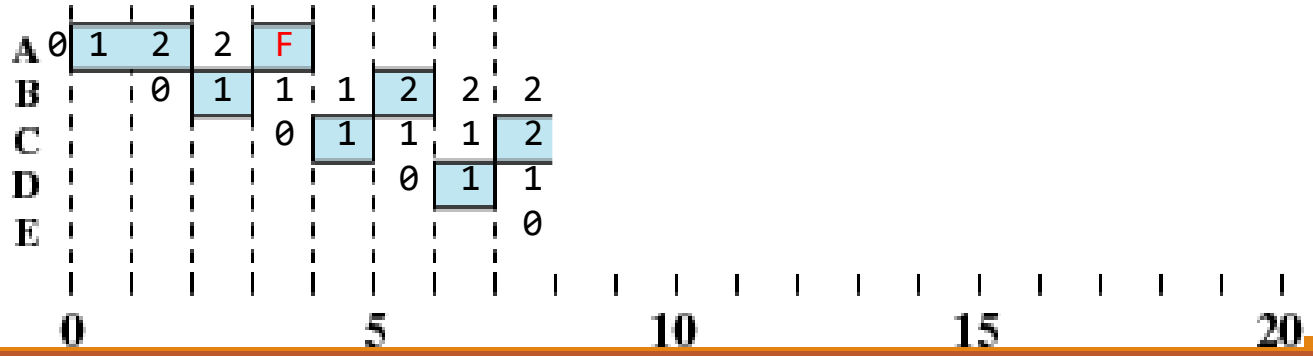
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

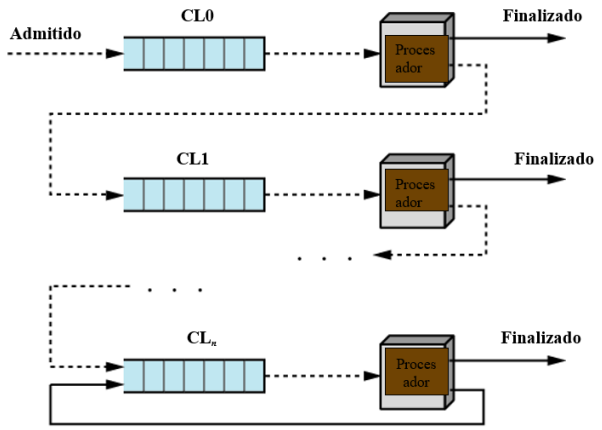
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

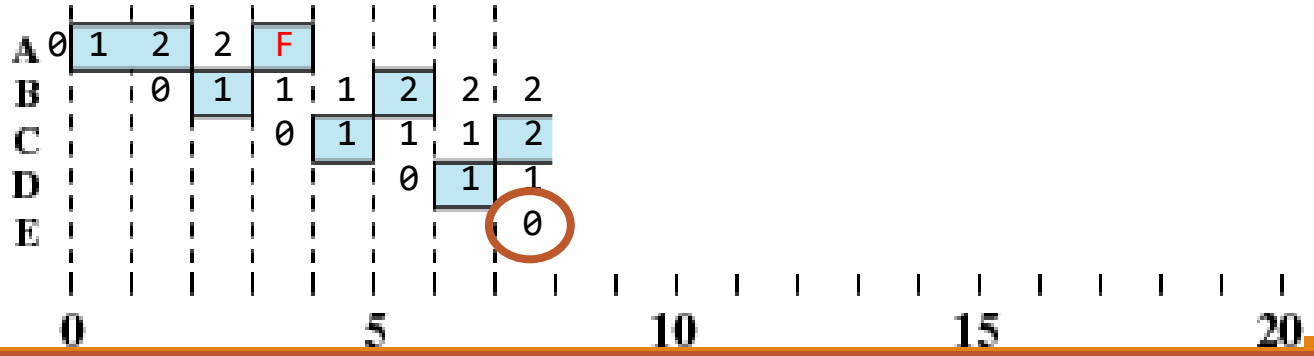
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

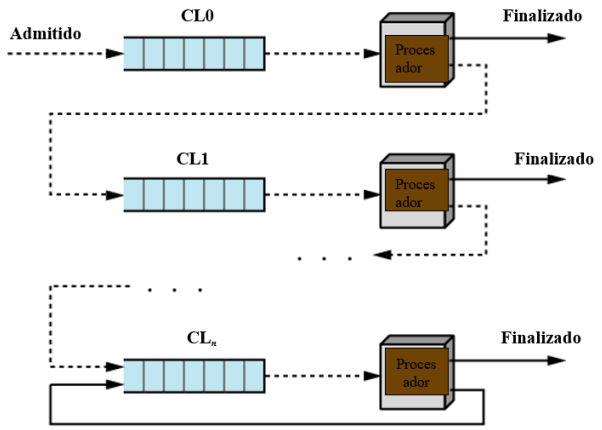
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

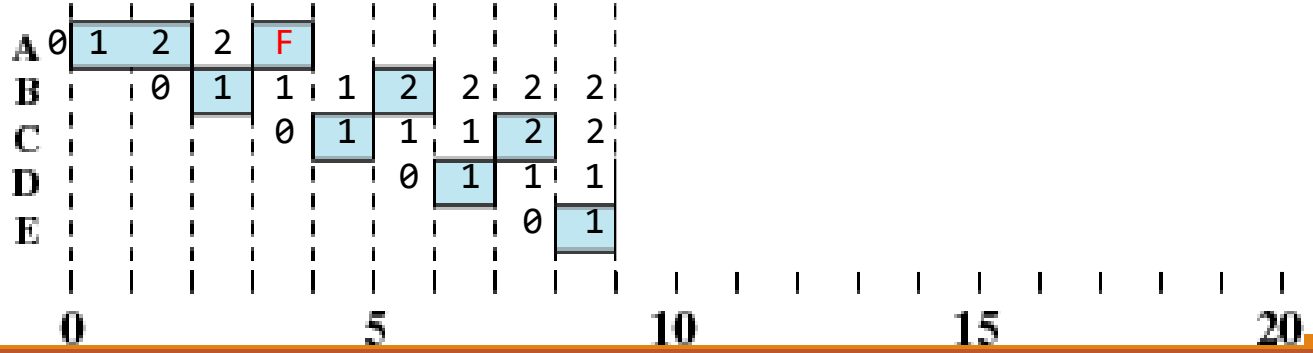
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

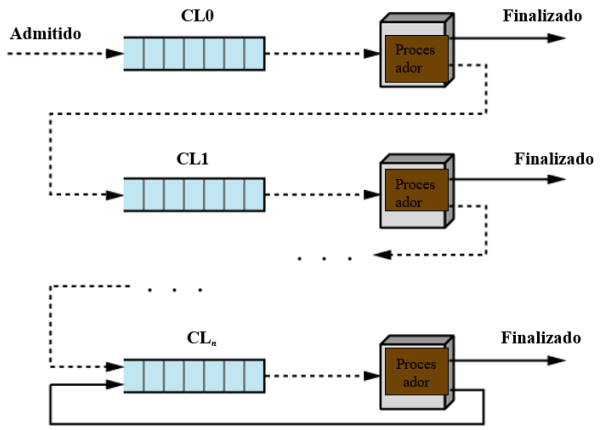
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

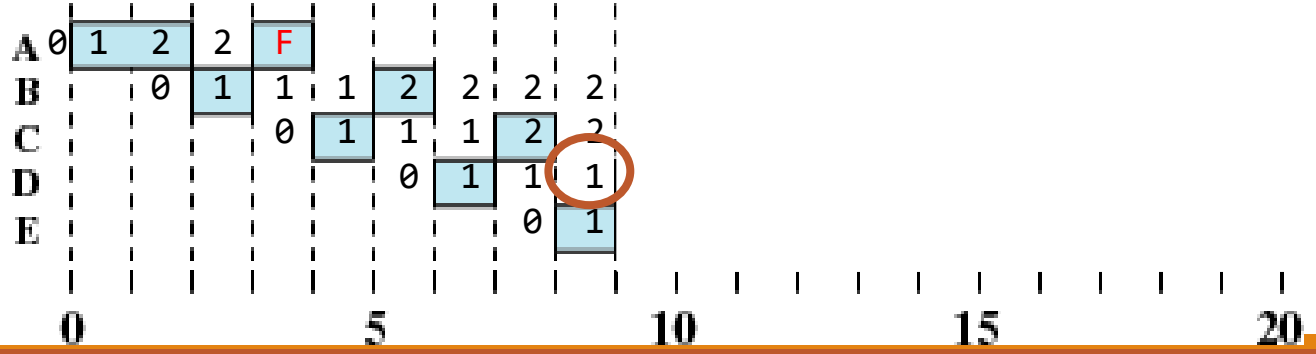
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

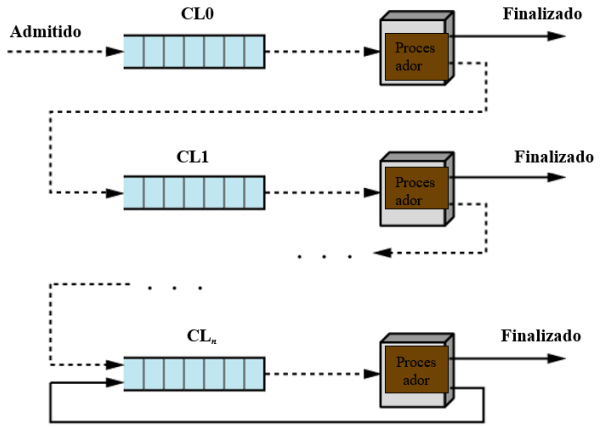
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

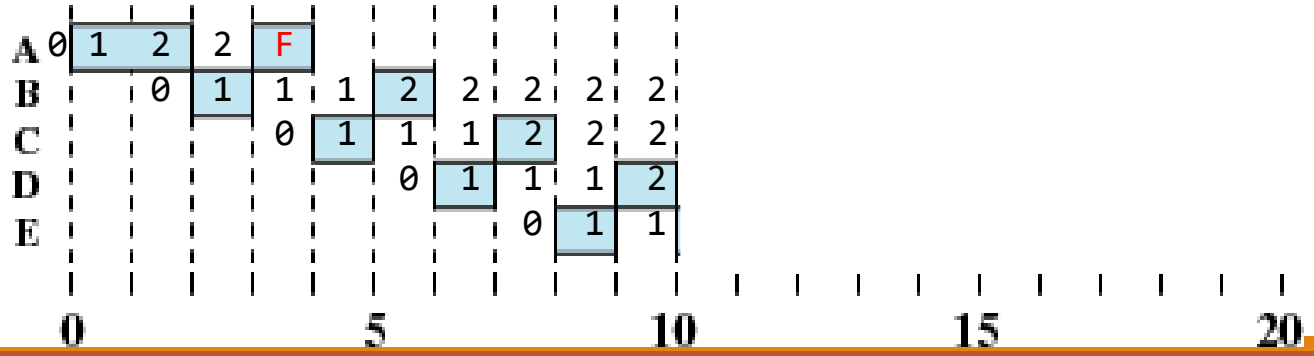
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

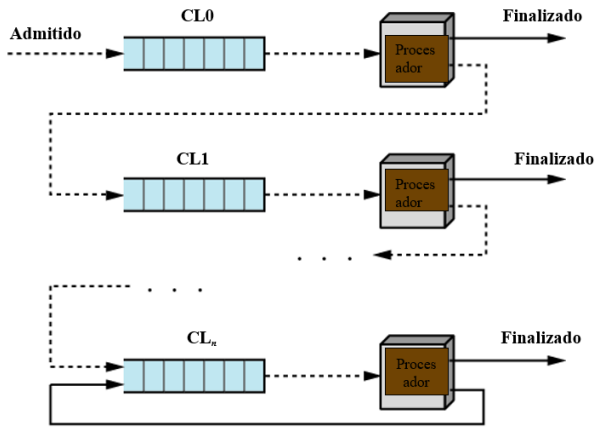
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

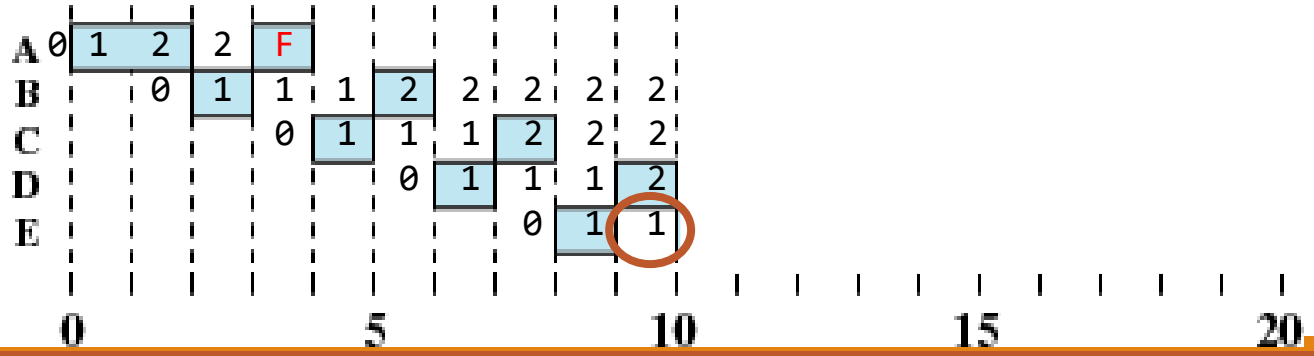
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

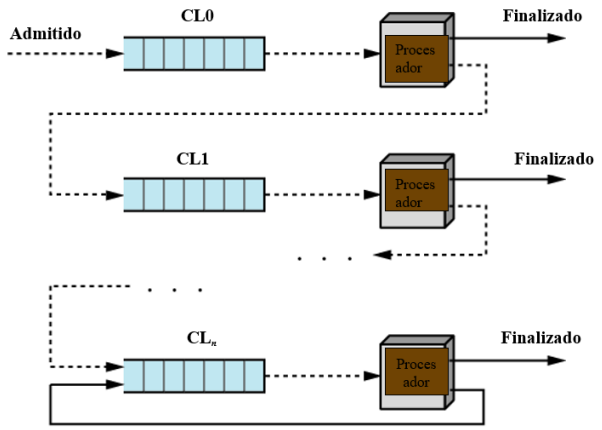
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

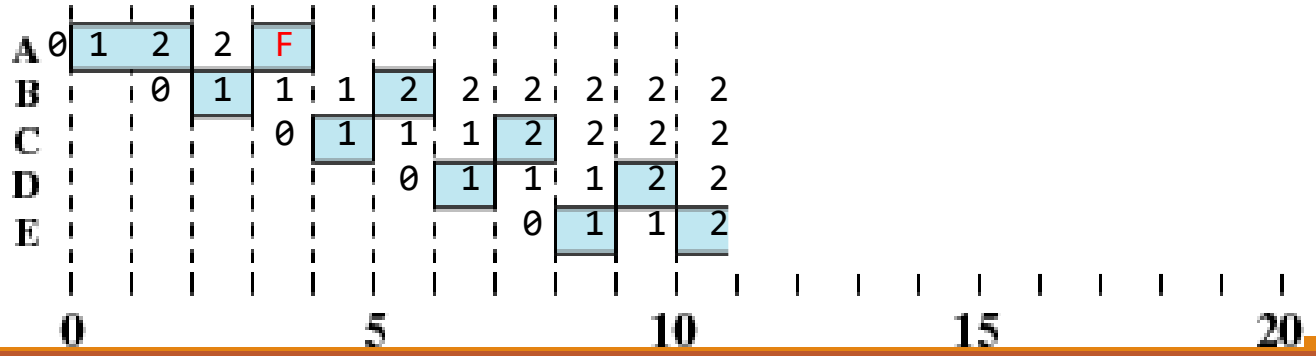
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

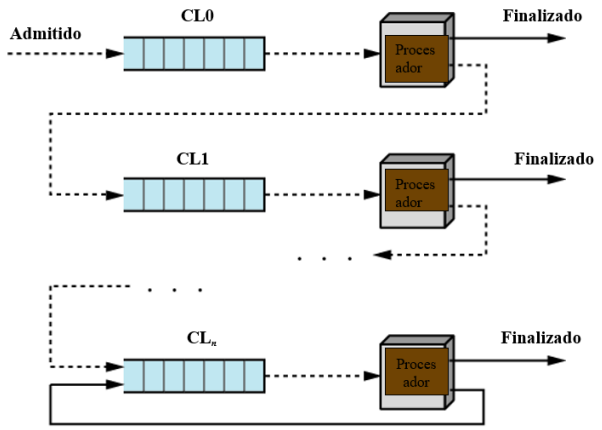
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

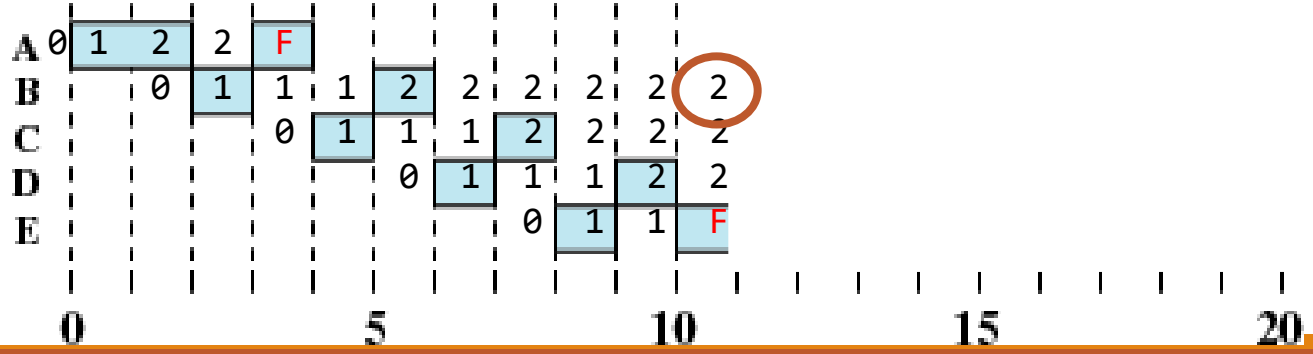
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

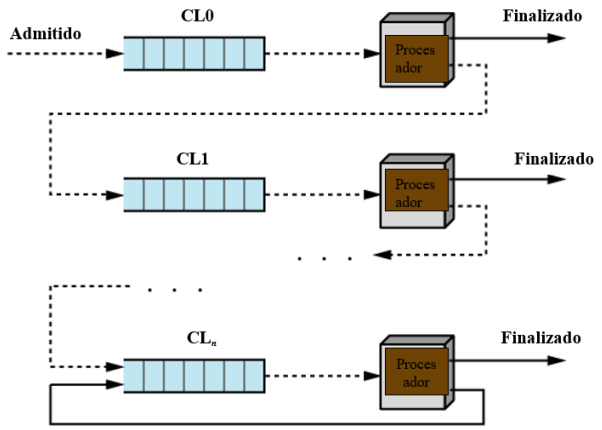
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

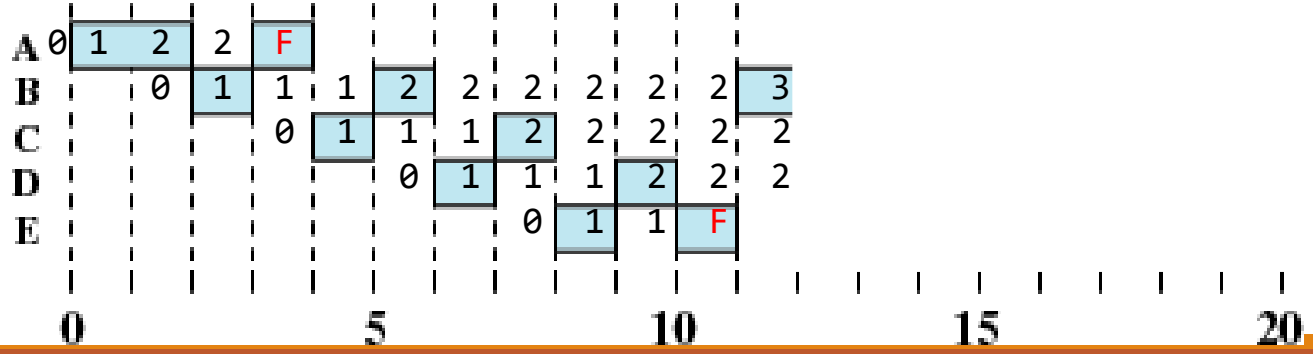
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

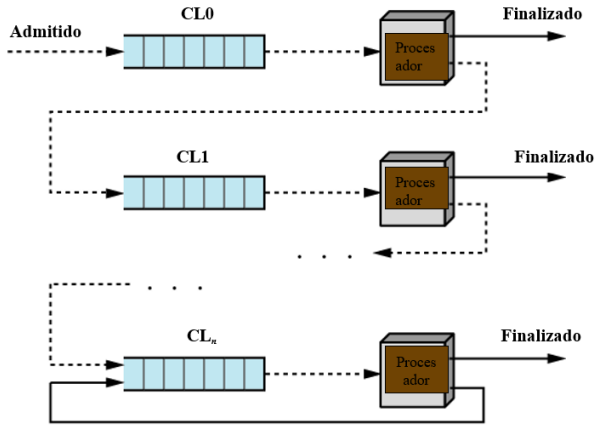
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

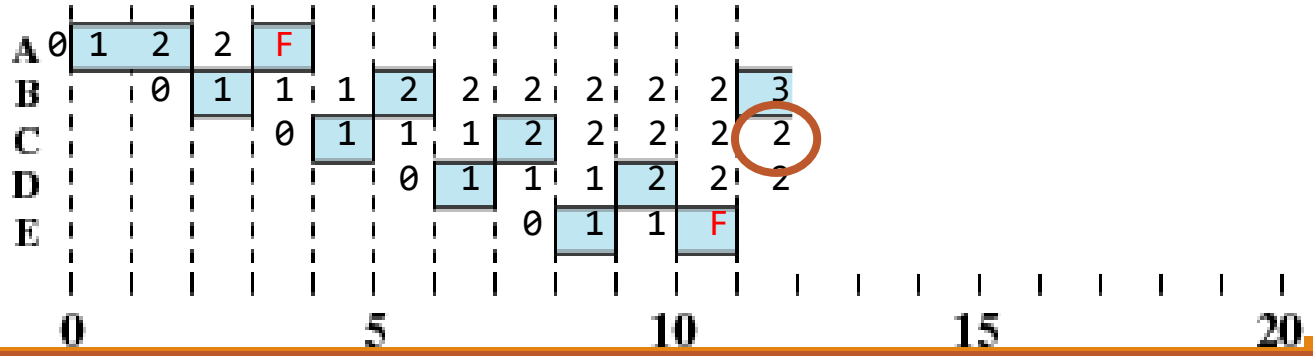
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

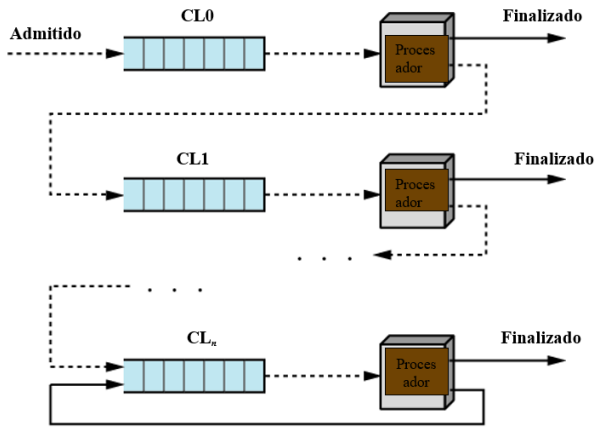
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

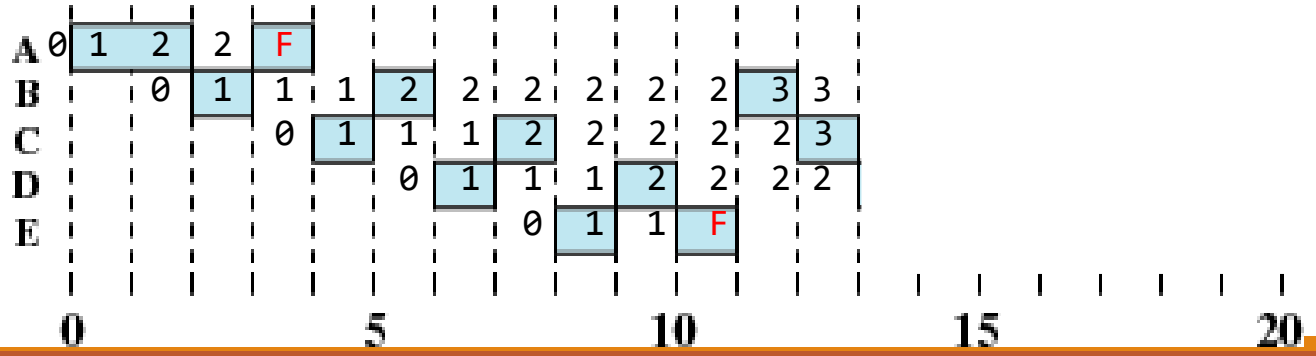
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

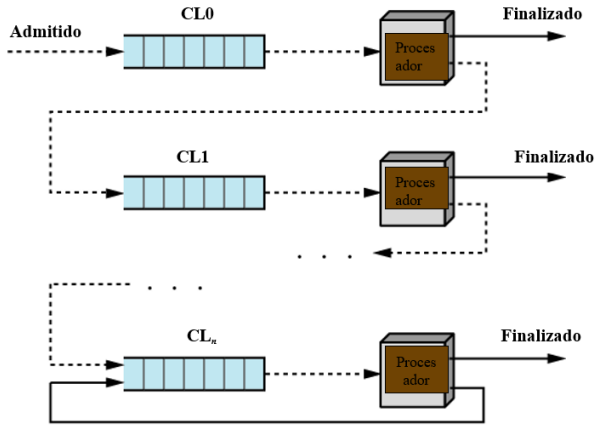
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

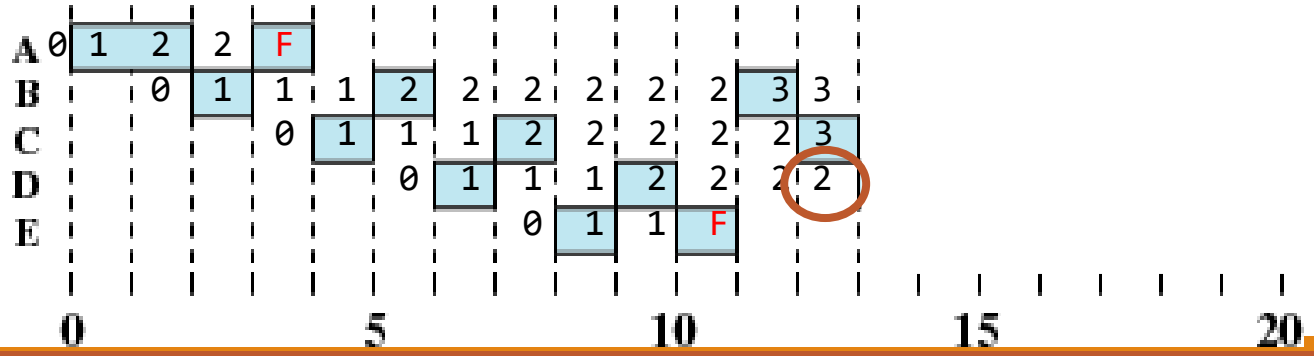
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

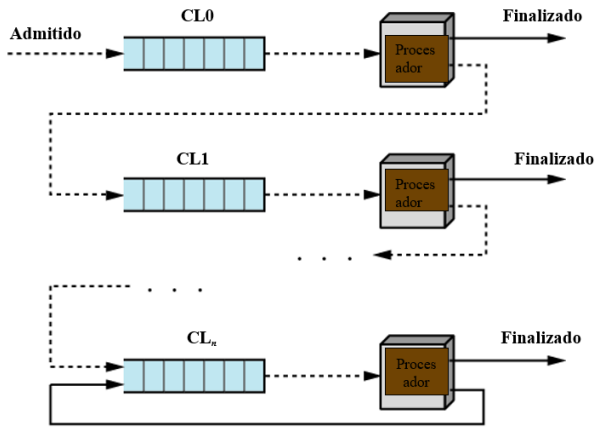
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

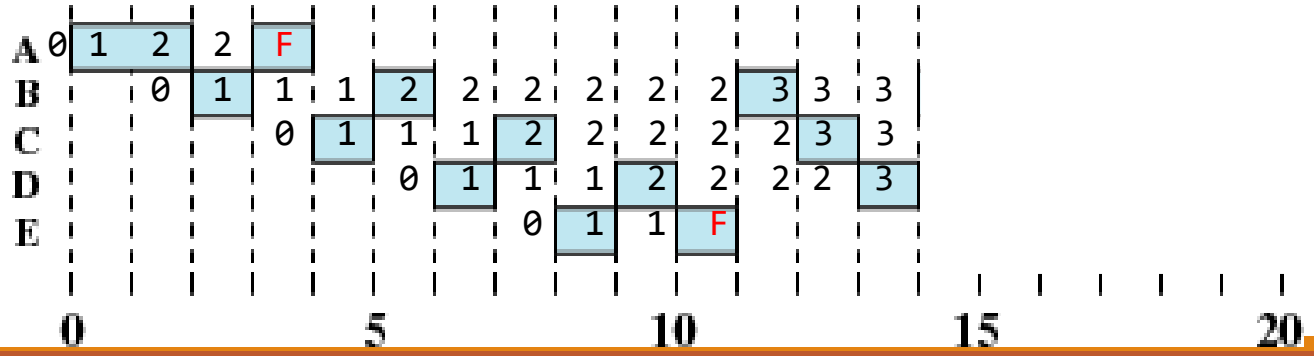
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

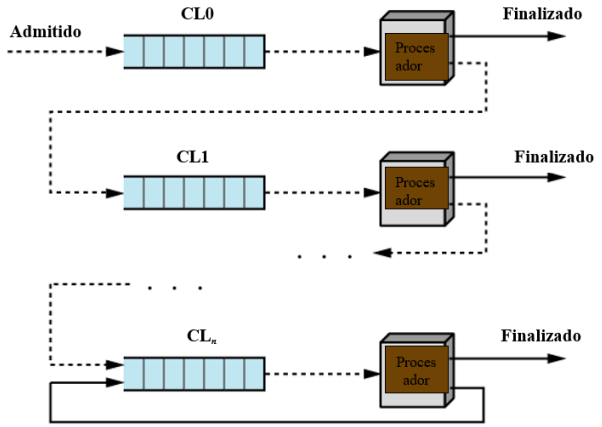
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

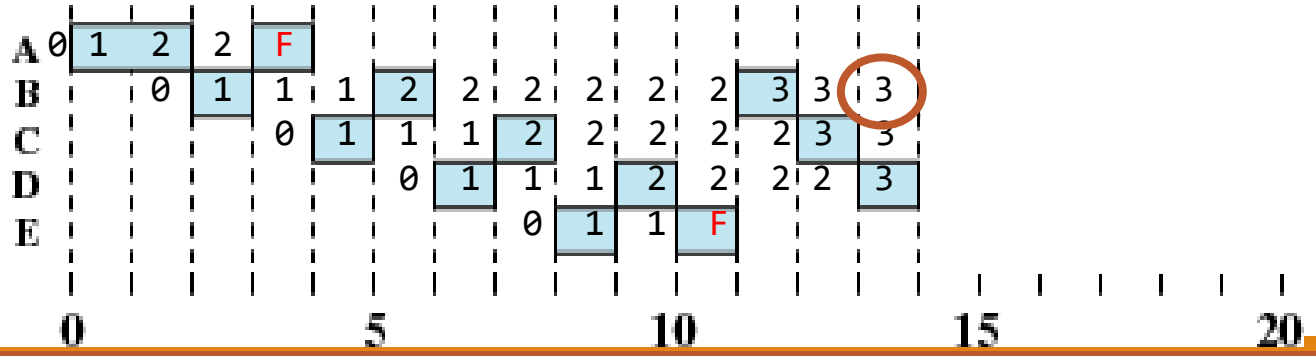
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

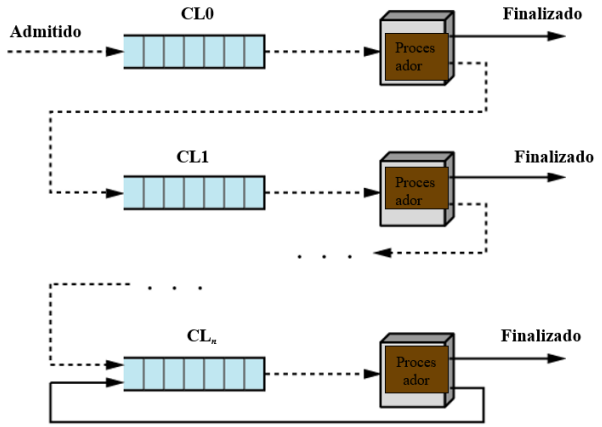
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

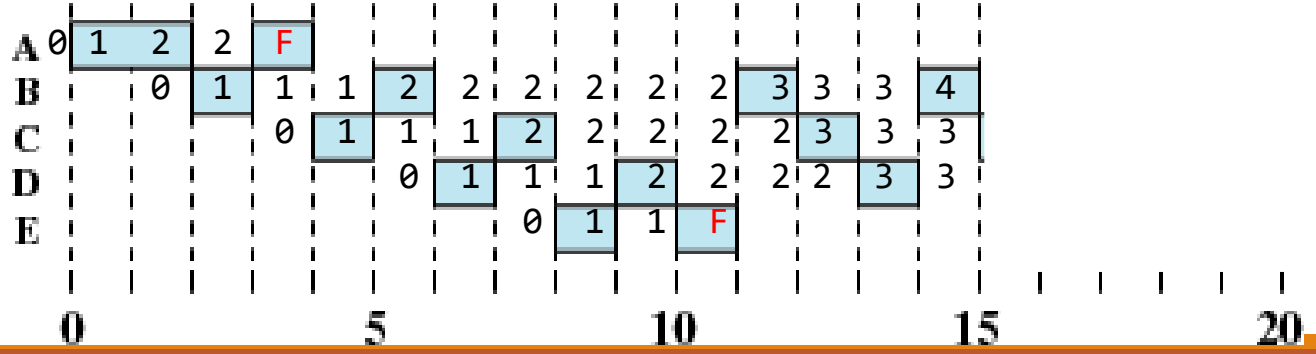
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

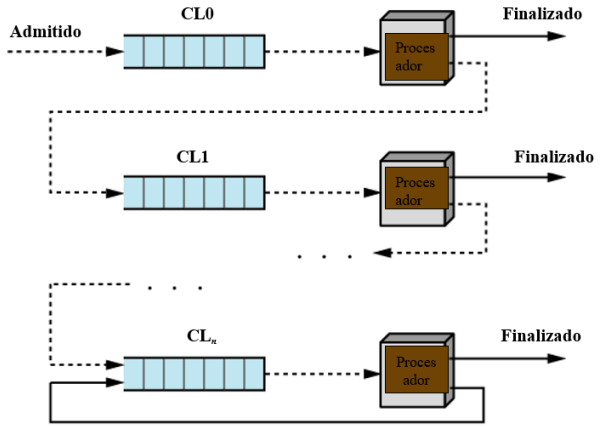
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

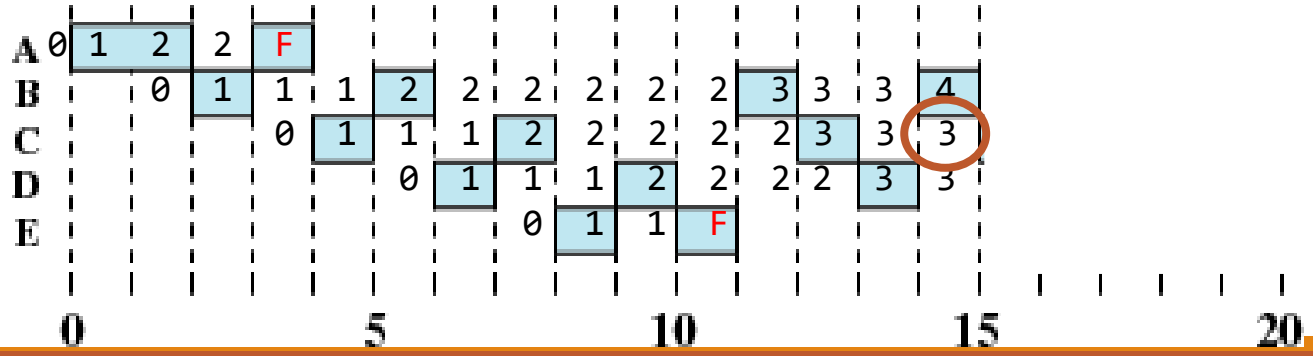
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

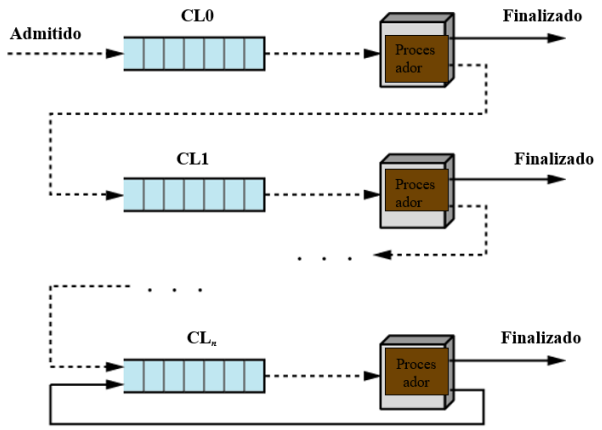
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

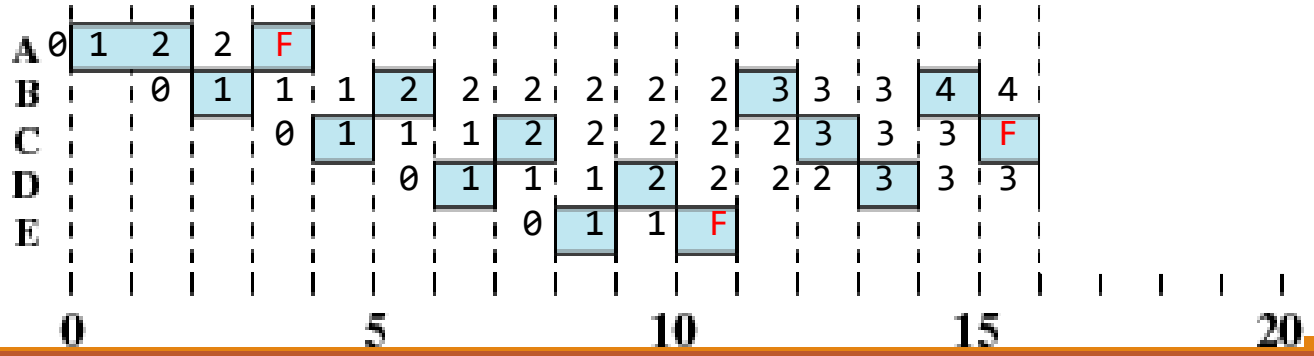
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

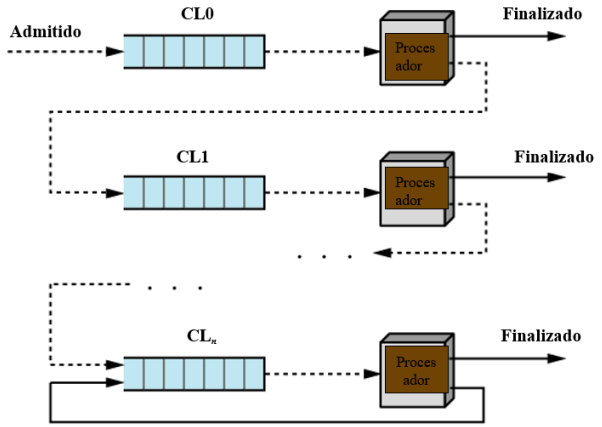
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

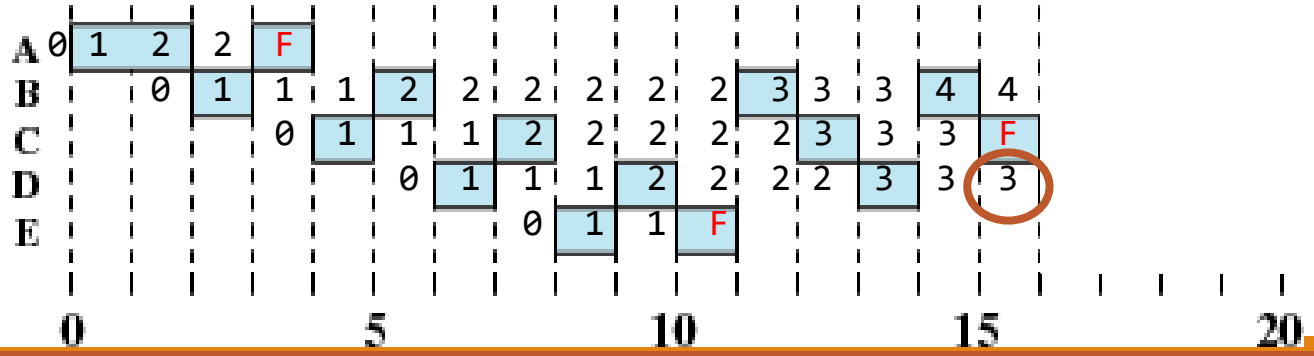
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

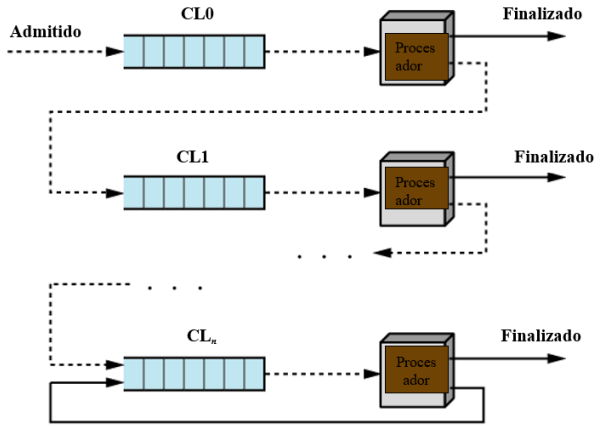
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

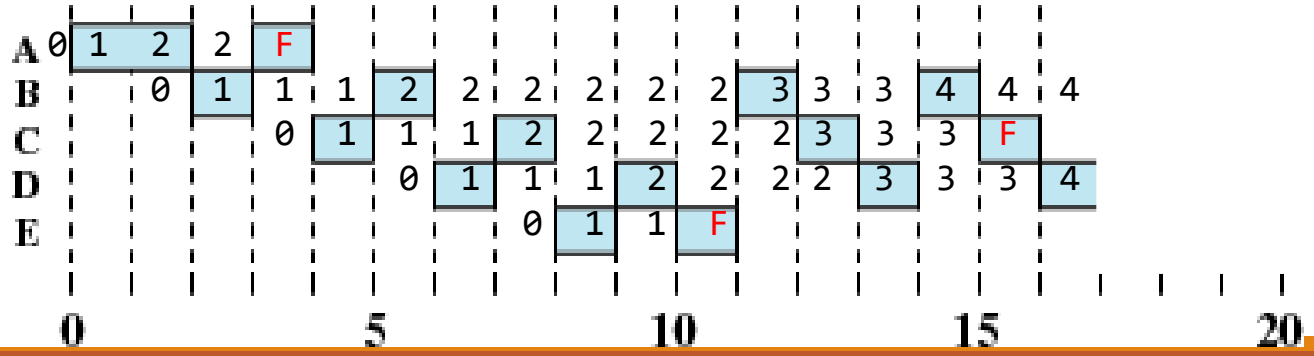
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

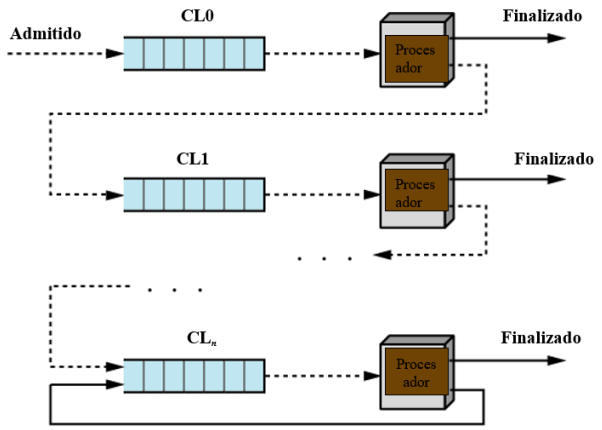
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

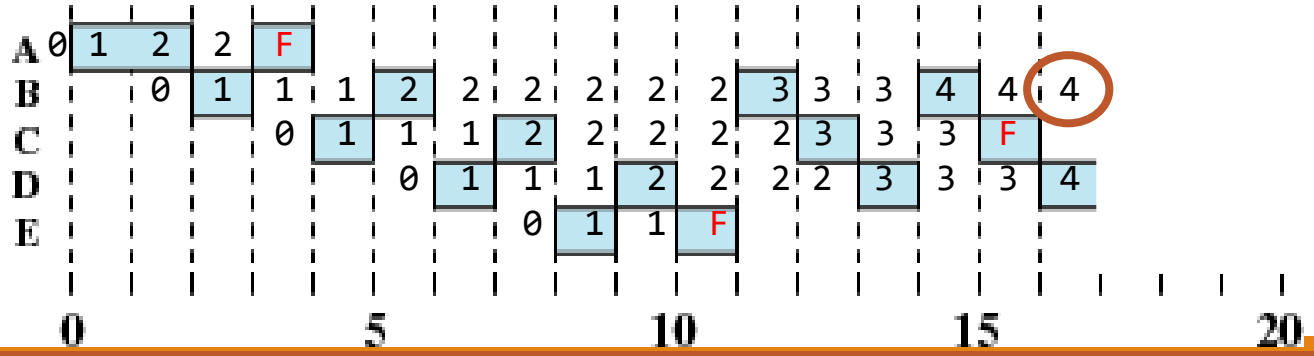
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

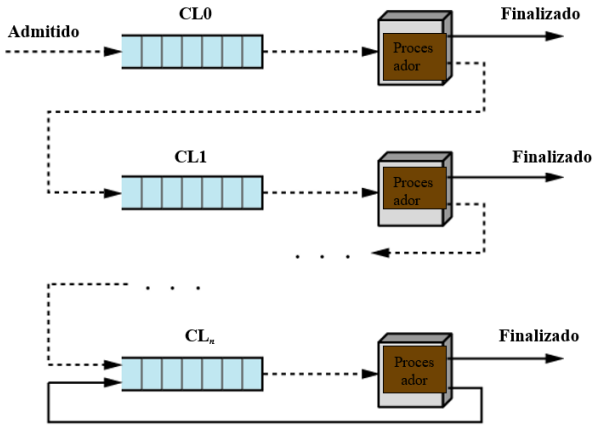
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

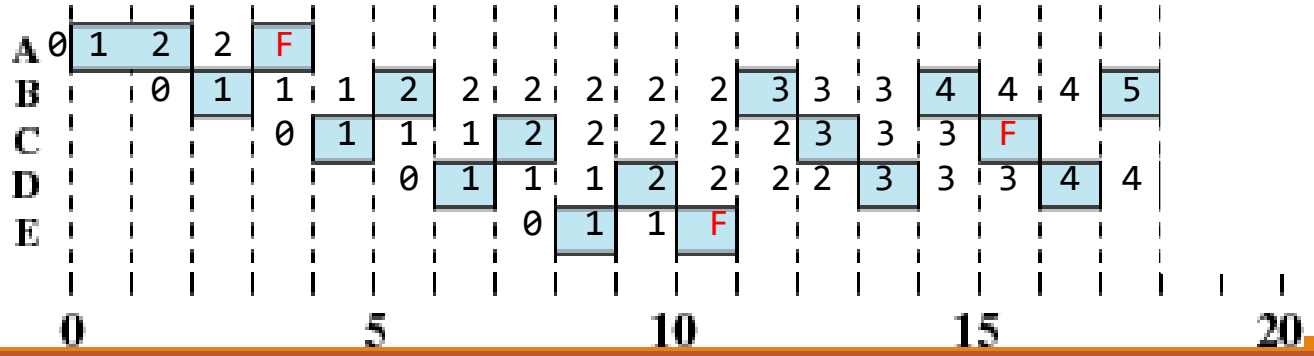
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

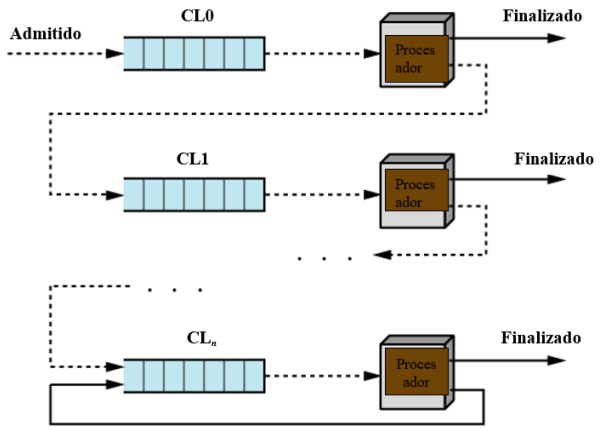
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

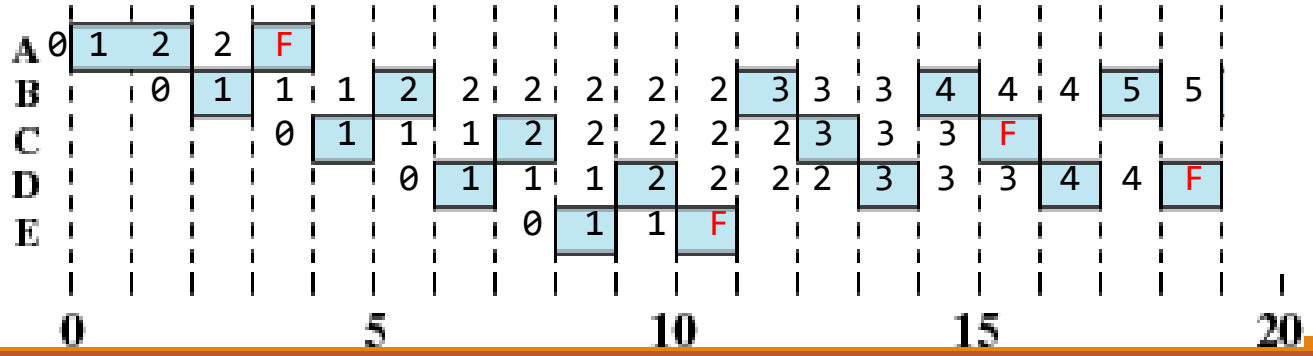
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

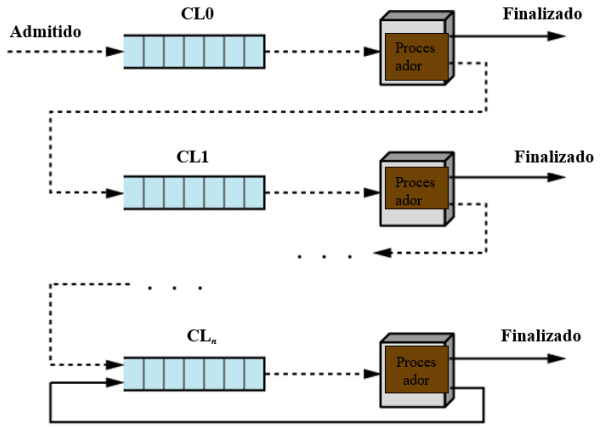
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

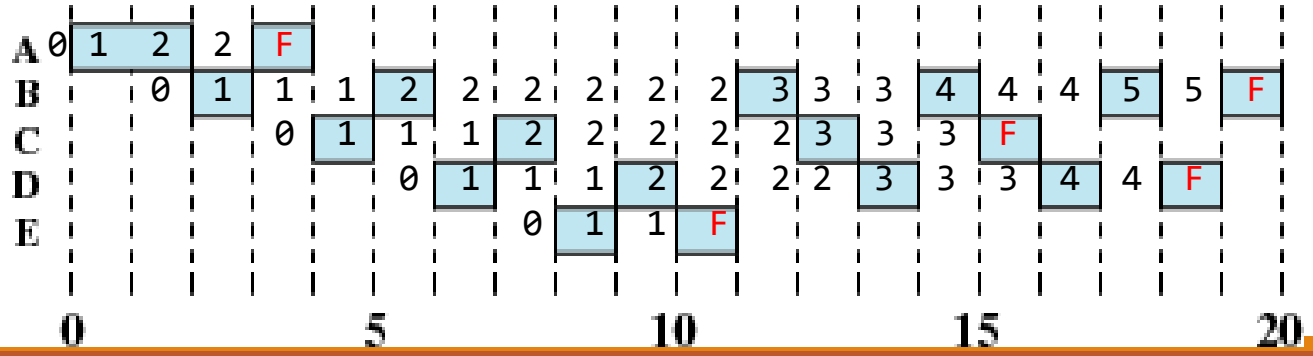
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

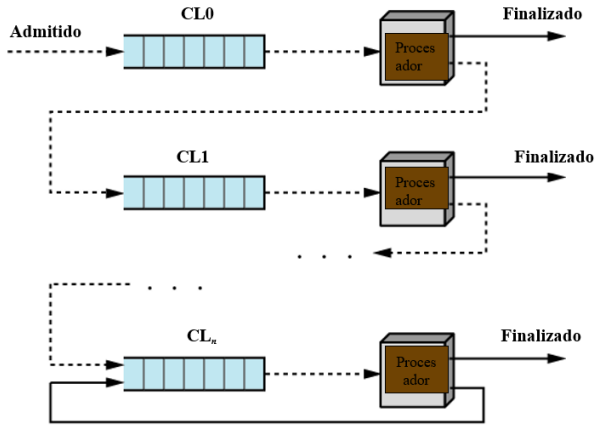
Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 1$

Todas las colas tienen el mismo quantum



Feedback MQ



Se penaliza a los trabajos que han estado ejecutando más tiempo

Si no podemos basarnos en el tiempo de ejecución restante, nos podemos basar en el tiempo de ejecución utilizado hasta el momento

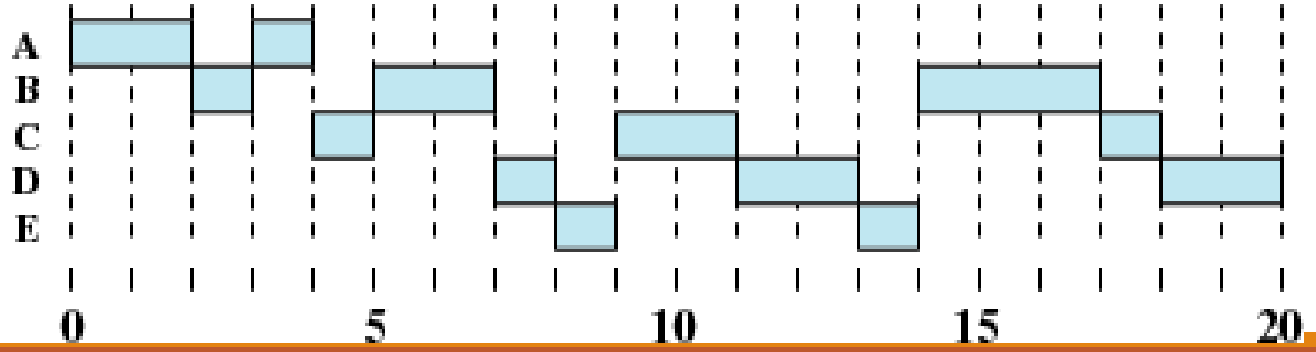
Los procesos van descendiendo a colas inferiores después de cada ejecución

Table 9.4 Process Scheduling Example

Process	Arrival Time	Service Time
A	0	3
B	2	6
C	4	4
D	6	5
E	8	2

Feedback
$q = 2^i$

Todas las colas tienen **distinto** quantum



Ejercicio 1 (16)
Feedback MQ

Nota: 3500 ut

Se dispone de un sistema **monoprocesador** con política de planificación **MLFQ**.

Cola 0: Planificación **FCFS**.
Cola 1: Planificación **RR** con quantum = 100 ut.
Cola 2: Planificación **RR** con quantum = 200 ut.
Cola 3: Planificación **RR** con quantum = 300 ut.

La prioridad de las colas es decreciente.

Los procesos de sistema entran directamente a la cola 0 (cola más prioritaria).

Los procesos de usuario entran a la cola 1; cuando abandonan el procesador pasan a la cola siguiente.

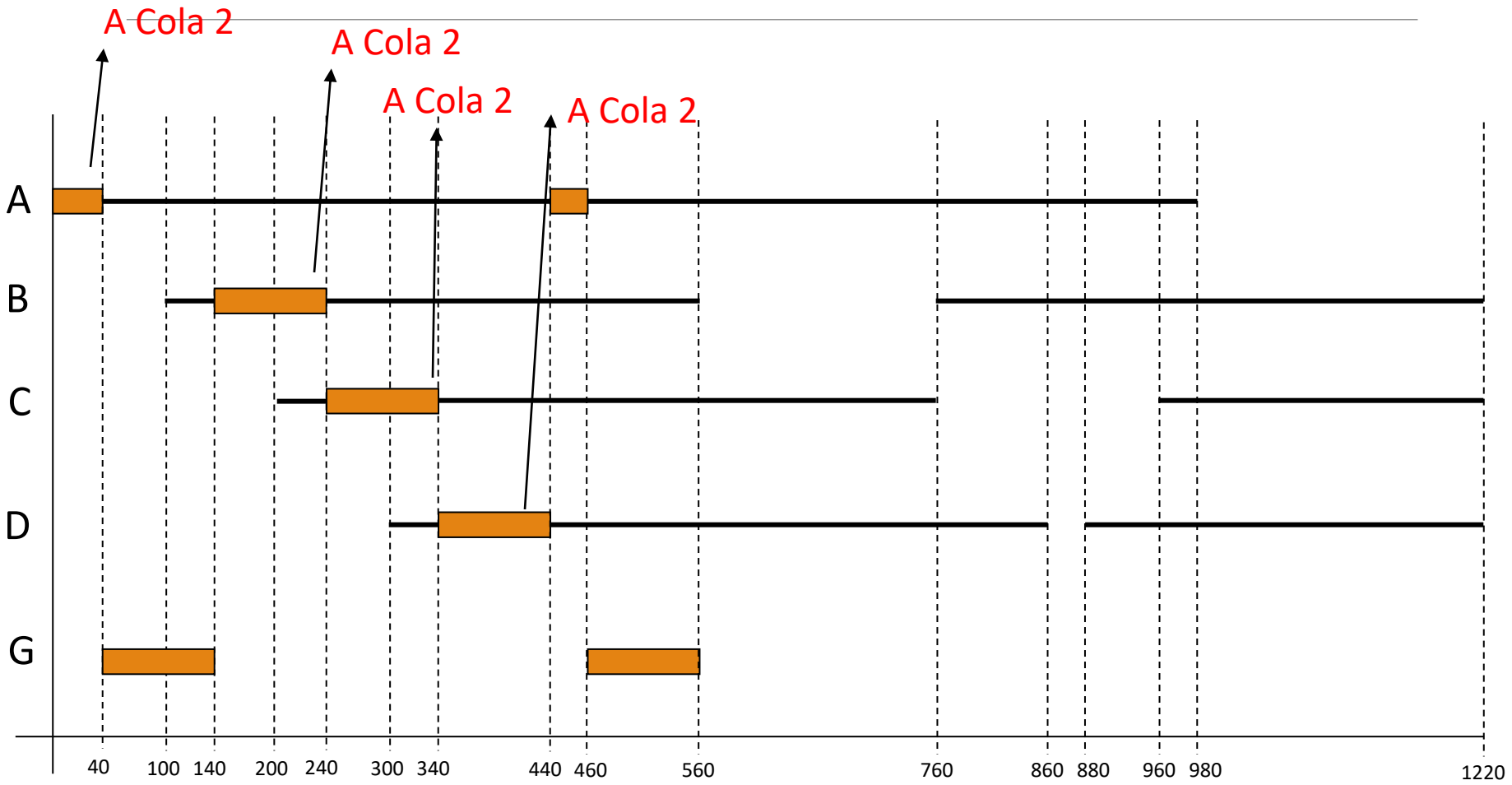
Mostrar la evolución temporal de los procesos del sistema señalando el estado en el que se encuentra cada proceso, así como la ocupación temporal de la CPU y de los dispositivos de E/S.

Calcular los tiempos de respuesta, de retorno y de espera para cada uno de los procesos

Instante de llegada	Tipo	Duración
0 ut.	A de usuario	700 ut. cada 300 ut. hace E/S de 100 ut.
100 ut.	B de usuario	500 ut. cada 500 ut. hace E/S de 100 ut.
200 ut.	C de usuario	600 ut. cada 200 ut. hace E/S de 100 ut.
300 ut.	D de usuario	700 ut. cada 600 ut. hace E/S de 100 ut.
40 ut. Se reinvoca automáticamente cada 320 ut. después de finalizar su ejecución.	**G de sistema**	100 ut..

Solución 1
Feedback MQ

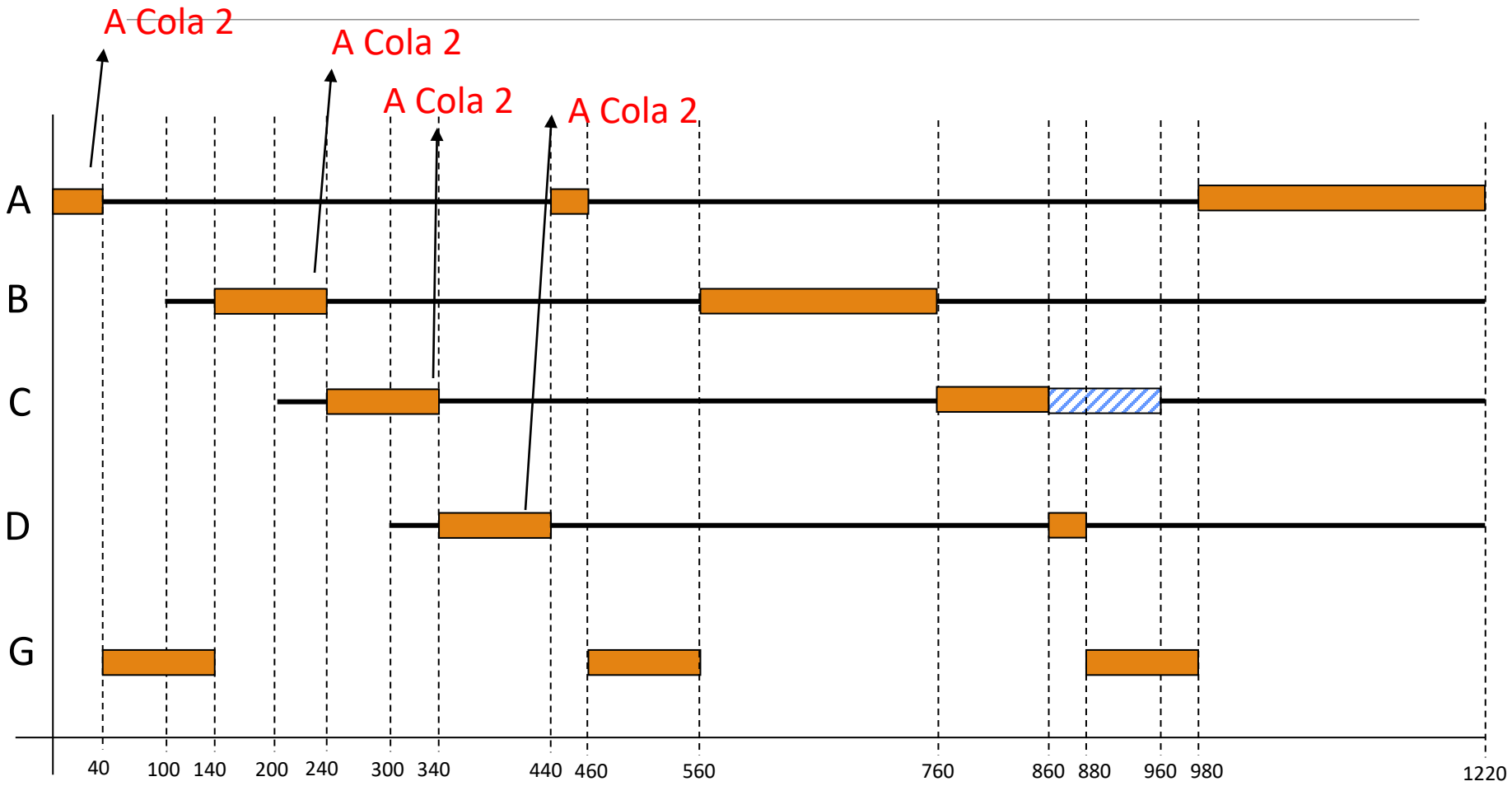
Instante de llegada	Tipo	Duración
0 ut.	A de usuario	700 ut. cada 300 ut. hace E/S de 100 ut.
100 ut.	B de usuario	500 ut. cada 500 ut. hace E/S de 100 ut.
200 ut.	C de usuario	600 ut. cada 200 ut. hace E/S de 100 ut.
300 ut.	D de usuario	700 ut. cada 600 ut. hace E/S de 100 ut.
40 ut. Se reinvoca automáticamente cada 320 ut. después de finalizar su ejecución.	**G de sistema**	100 ut..



Primero dibujar los del sistema

Solución 1
Feedback MQ

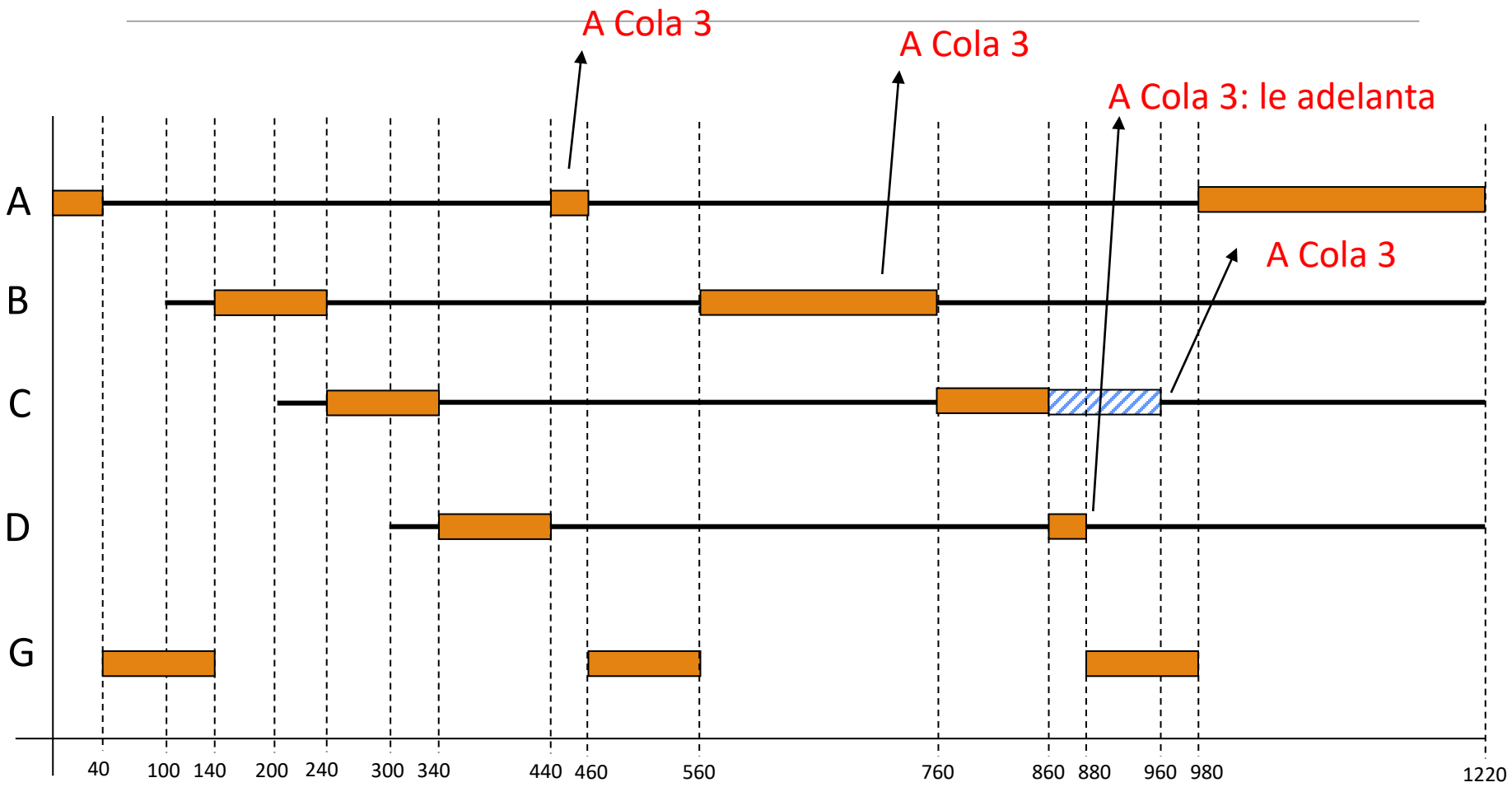
Instante de llegada	Tipo	Duración
0 ut.	A de usuario	700 ut. cada 300 ut. hace E/S de 100 ut.
100 ut.	B de usuario	500 ut. cada 500 ut. hace E/S de 100 ut.
200 ut.	C de usuario	600 ut. cada 200 ut. hace E/S de 100 ut.
300 ut.	D de usuario	700 ut. cada 600 ut. hace E/S de 100 ut.
40 ut. Se reinvoca automáticamente cada 320 ut. después de finalizar su ejecución.	**G de sistema**	100 ut..



Primero dibujar los del sistema

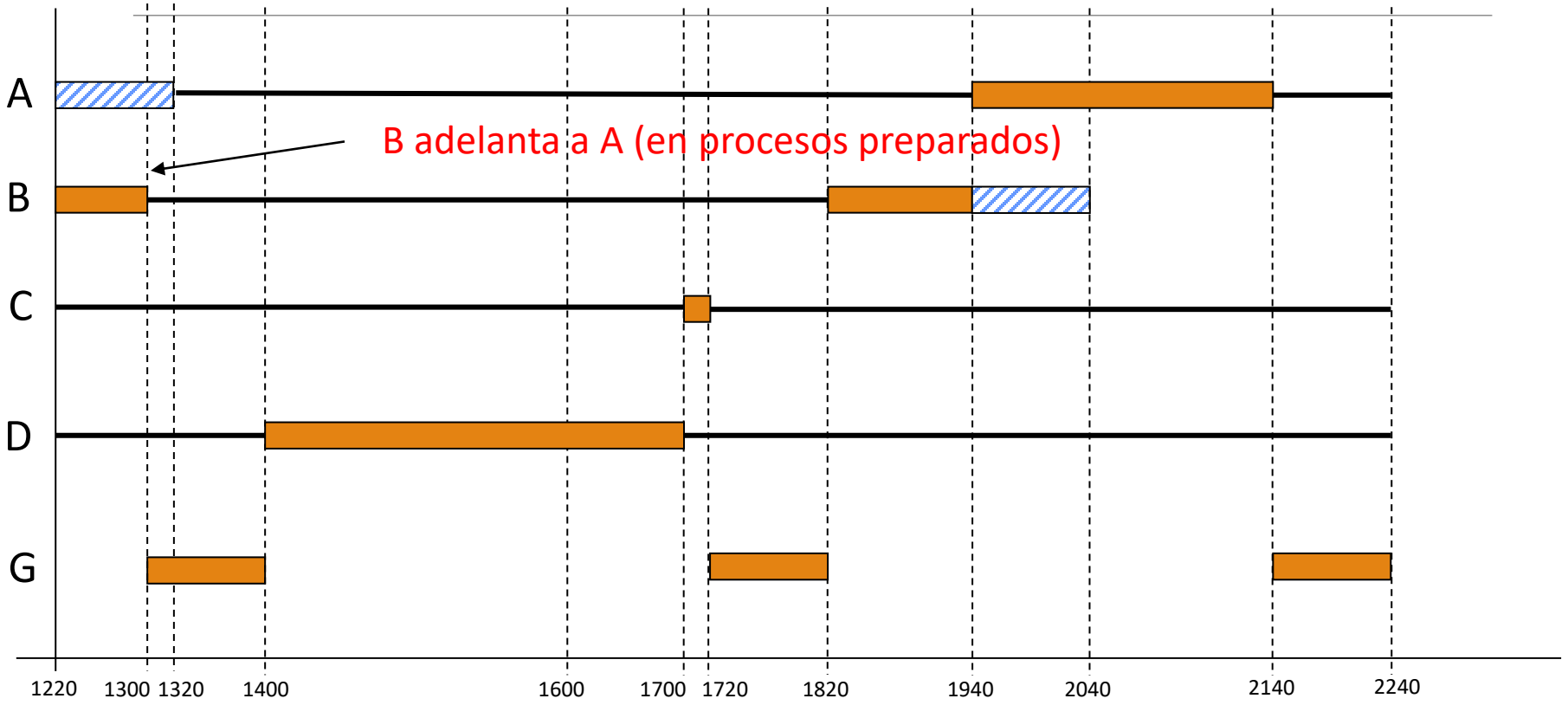
Solución 1
Feedback MQ

Instante de llegada	Tipo	Duración
0 ut.	A de usuario	700 ut. cada 300 ut. hace E/S de 100 ut.
100 ut.	B de usuario	500 ut. cada 500 ut. hace E/S de 100 ut.
200 ut.	C de usuario	600 ut. cada 200 ut. hace E/S de 100 ut.
300 ut.	D de usuario	700 ut. cada 600 ut. hace E/S de 100 ut.
40 ut. Se reinvoca automáticamente cada 320 ut. después de finalizar su ejecución.	G de sistema	100 ut..



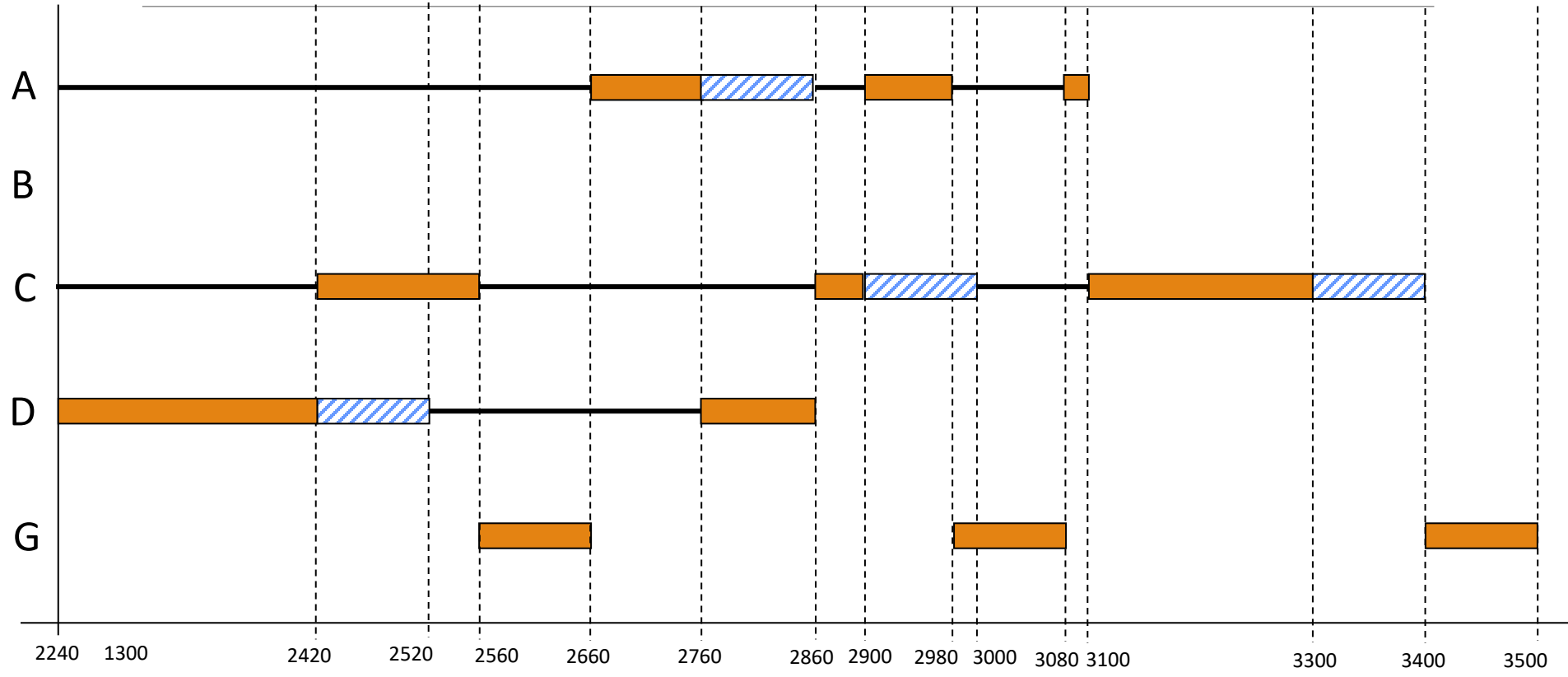
Solución 1
Feedback MQ

Instante de llegada	Tipo	Duración
0 ut.	A de usuario	700 ut. cada 300 ut. hace E/S de 100 ut.
100 ut.	B de usuario	500 ut. cada 500 ut. hace E/S de 100 ut.
200 ut.	C de usuario	600 ut. cada 200 ut. hace E/S de 100 ut.
300 ut.	D de usuario	700 ut. cada 600 ut. hace E/S de 100 ut.
40 ut. Se reinvoca automáticamente cada 320 ut. después de finalizar su ejecución.	**G de sistema**	100 ut..



Solución 1
Feedback MQ

Instante de llegada	Tipo	Duración
0 ut.	A de usuario	700 ut. cada 300 ut. hace E/S de 100 ut.
100 ut.	B de usuario	500 ut. cada 500 ut. hace E/S de 100 ut.
200 ut.	C de usuario	600 ut. cada 200 ut. hace E/S de 100 ut.
300 ut.	D de usuario	700 ut. cada 600 ut. hace E/S de 100 ut.
40 ut. Se reinvoca automáticamente cada 320 ut. después de finalizar su ejecución.	G de sistema	100 ut..



Políticas de planificación Características

	Función de Selección	Modo de Decisión	Rendimiento	Tiempo de respuesta	Rendimiento	Efecto sobre los Procesos	Inanición
FCFS	max[w]	No expulsiva	No especificado	Puede ser alto especialmente si hay mucha diferencia entre los tiempos de ejecución de los procesos	Mínimo	Penaliza procesos cortos; penaliza procesos con mucha E/S	No
Turno Rotatorio (round robin)	constante	Expulsiva (por rodajas de tiempo)	Puede ser mucho si la rodaja es demasiado pequeña	Proporciona buen tiempo de respuesta para procesos cortos	Mínimo	Tratamiento justo	No
SPN	min[s]	No expulsiva	Alto	Proporciona buen tiempo de respuesta para procesos cortos	Puede ser alto	Penaliza procesos largos	Posible
SRT	min[s-e]	Expulsiva (a la llegada)	Alto	Proporciona buen tiempo de respuesta	Puede ser alto	Penaliza procesos largos	Posible
HRRN	max(w+s/s)	No expulsiva	Alto	Proporciona buen tiempo de respuesta	Puede ser alto	Buen equilibrio	No
Feedback	(ver texto)	Expulsiva (por rodajas de tiempo)	No especificado	No especificado	Puede ser alto	Puede favorecer procesos con mucha E/S	Posible

w = tiempo de espera; *e* = tiempo de ejecución hasta el momento; *s* = tiempo total de servicio requerido por el proceso, incluyendo *e*

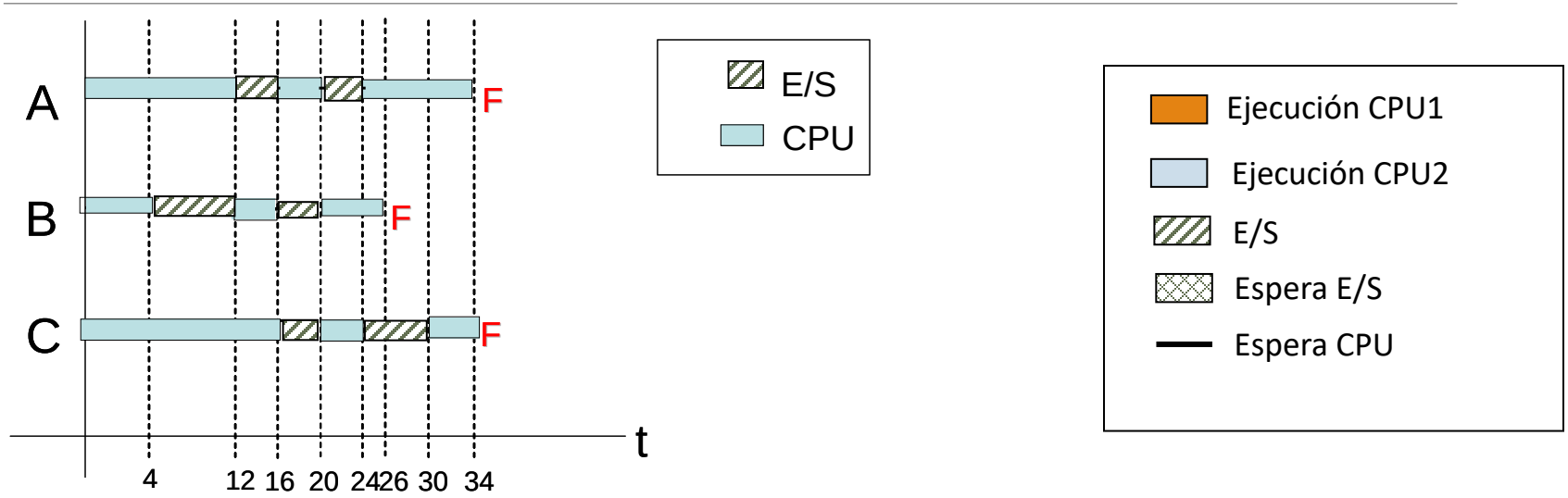
Políticas de planificación Comparación

	Process	A	B	C	D	E	
	Arrival Time	0	2	4	6	8	
	Service Time (T_s)	3	6	4	5	2	Mean
FCFS	Finish Time	3	9	13	18	20	
	Turnaround Time (T_r)	3	7	9	12	12	8.60
	T_r/T_s	1.00	1.17	2.25	2.40	6.00	2.56
RR $q = 1$	Finish Time	4	18	17	20	15	
	Turnaround Time (T_r)	4	16	13	14	7	10.80
	T_r/T_s	1.33	2.67	3.25	2.80	3.50	2.71
RR $q = 4$	Finish Time	3	17	11	20	19	
	Turnaround Time (T_r)	3	15	7	14	11	10.00
	T_r/T_s	1.00	2.5	1.75	2.80	5.50	2.71
SPN	Finish Time	3	9	15	20	11	
	Turnaround Time (T_r)	3	7	11	14	3	7.60
	T_r/T_s	1.00	1.17	2.75	2.80	1.50	1.84
SRT	Finish Time	3	15	8	20	10	
	Turnaround Time (T_r)	3	13	4	14	2	7.20
	T_r/T_s	1.00	2.17	1.00	2.80	1.00	1.59
HRRN	Finish Time	3	9	13	20	15	
	Turnaround Time (T_r)	3	7	9	14	7	8.00
	T_r/T_s	1.00	1.17	2.25	2.80	3.5	2.14
FB $q = 1$	Finish Time	4	20	16	19	11	
	Turnaround Time (T_r)	4	18	12	13	3	10.00
	T_r/T_s	1.33	3.00	3.00	2.60	1.5	2.29
FB $q = 2^i$	Finish Time	4	17	18	20	14	
	Turnaround Time (T_r)	4	15	14	14	6	10.60
	T_r/T_s	1.33	2.50	3.50	2.80	3.00	2.63

Ejercicio Completo: 2 procesadores (24)

Se dispone de un sistema con dos procesadores: el primer procesador tiene política de planificación RR con q = 10 u.t. y ejecuta cualquier tipo de proceso; el segundo procesador tiene política FCFS y sólo ejecuta procesos de sistema. Se dispone de un recurso de E/S con gestión FCFS. El sistema recibe para su ejecución **procesos de usuario y de sistema.**

La ejecución de **los procesos de usuario** sigue el esquema descrito en la figura 1.



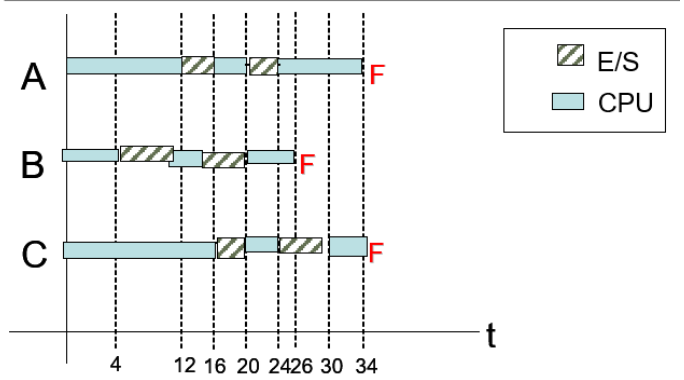
La ejecución de los **procesos de sistema** sigue el esquema que se muestra en la tabla. Los tiempos de ejecución periódica de estos procesos se miden en relación al reloj del sistema sin tener en consideración el instante en que terminaron.

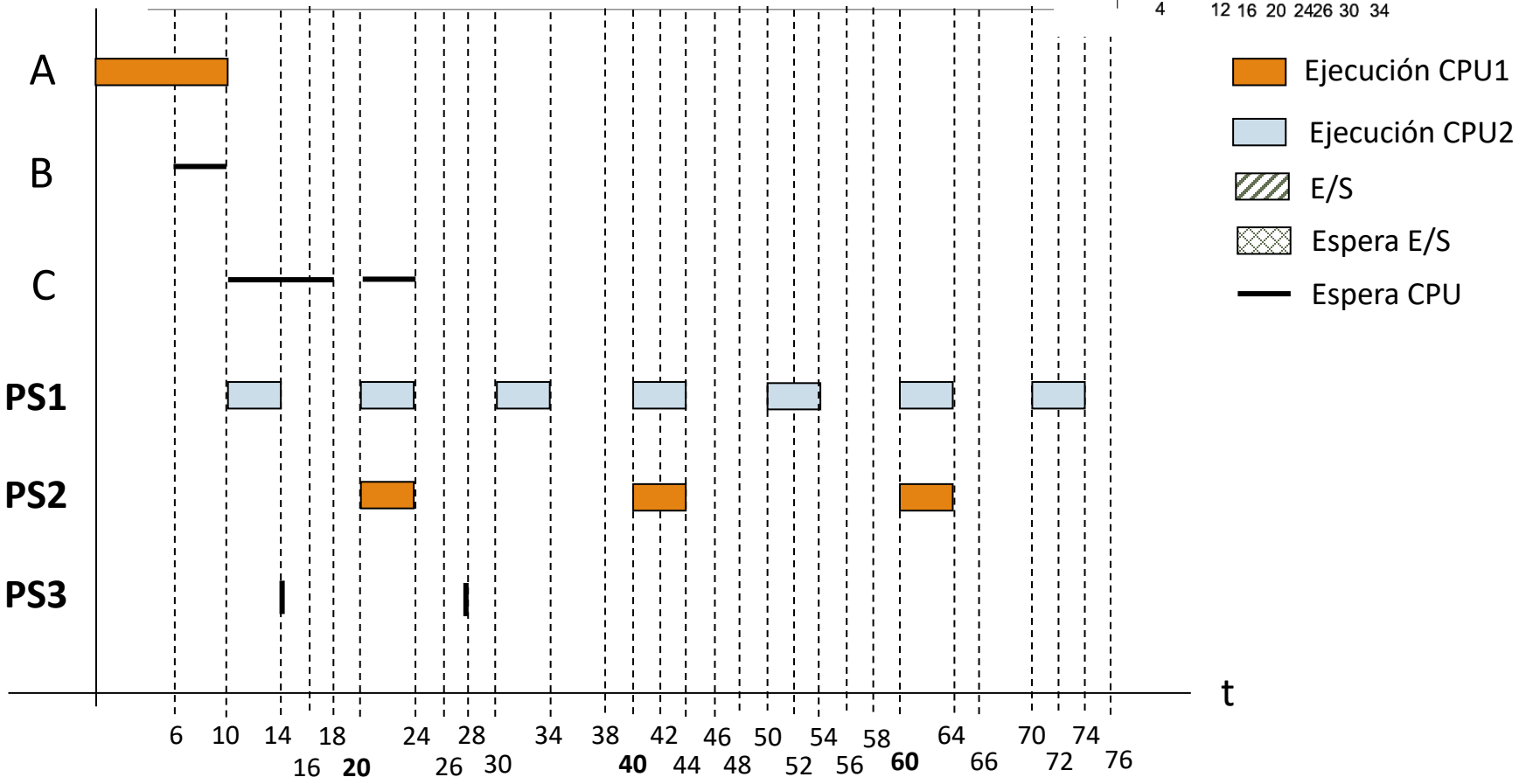
PS1	Ejecución en CPU de duración 4 u.t.	Se ejecuta periódicamente cada 10 u.t
PS2	Ejecución en CPU de duración 4 u.t.	Se ejecuta periódicamente cada 20 u.t
PS3	Realiza E/S de duración 4 u.t.	Se ejecuta periódicamente cada 14 u.t

El instante de llegada de los procesos de usuario se muestra en la tabla:

A	0 u.t.
B	6 u.t.
C	10 u.t.

Los procesos de sistema tienen prioridad expropiativa sobre los procesos de usuario, salvo que haya algún procesador libre. No es posible expropiar recursos a procesos. Si el quantum de un proceso en ejecución expira a la vez que un proceso termina una operación de E/S o que la llegada de un nuevo proceso, entonces el proceso que abandona el procesador se añade a la cola de procesos en preparado antes que el proceso que termina la E/S o el que llega. Mostrar la evolución temporal de los procesos del sistema señalando el estado en el que se encuentra cada proceso, así como la ocupación temporal de la CPU y del dispositivo de E/S. Calcular los tiempos de respuesta, de retorno y de espera para cada uno de los procesos. Considera un tiempo de cambio de contexto despreciable.





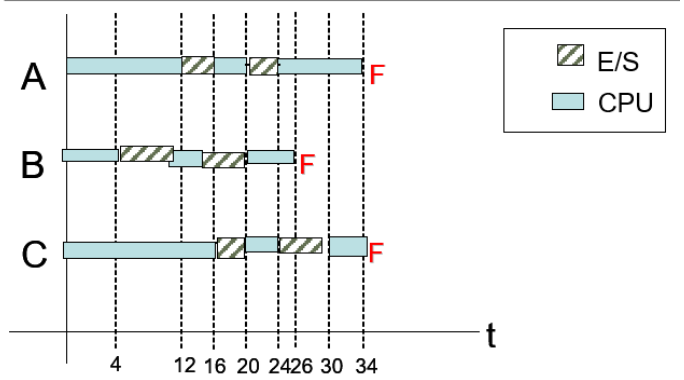
Ejecución CPU1

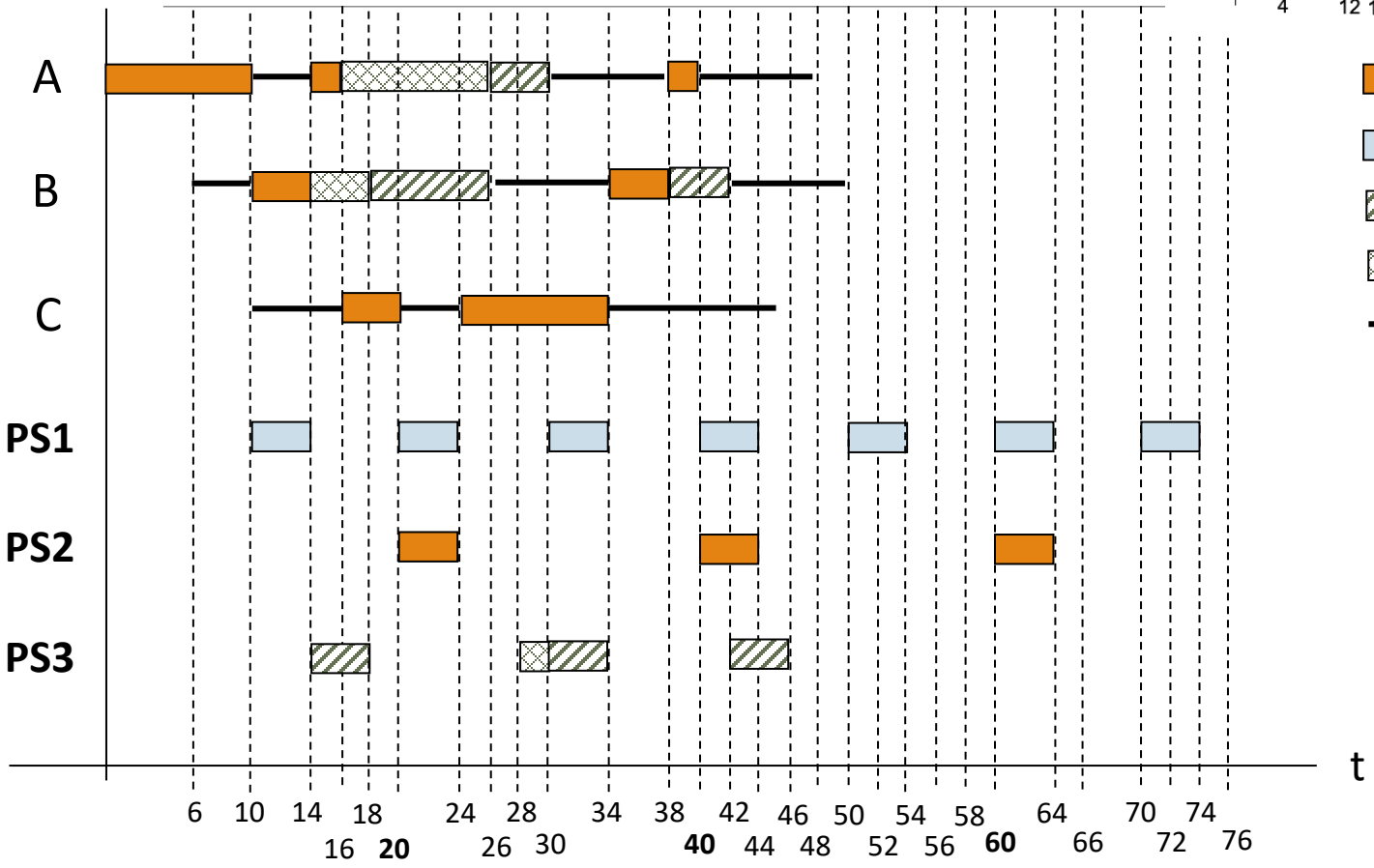
Ejecución CPU2

E/S

Espera E/S

Espera CPU





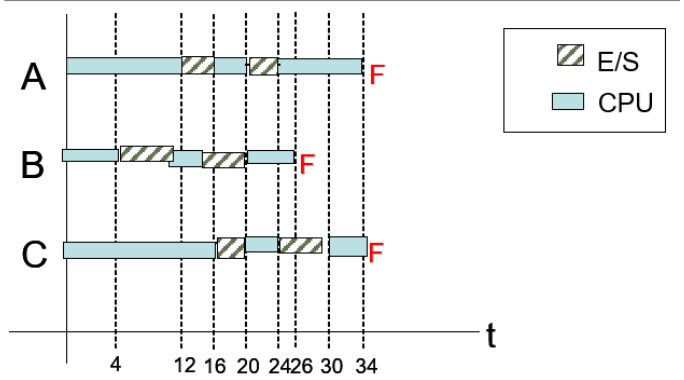
Ejecución CPU1

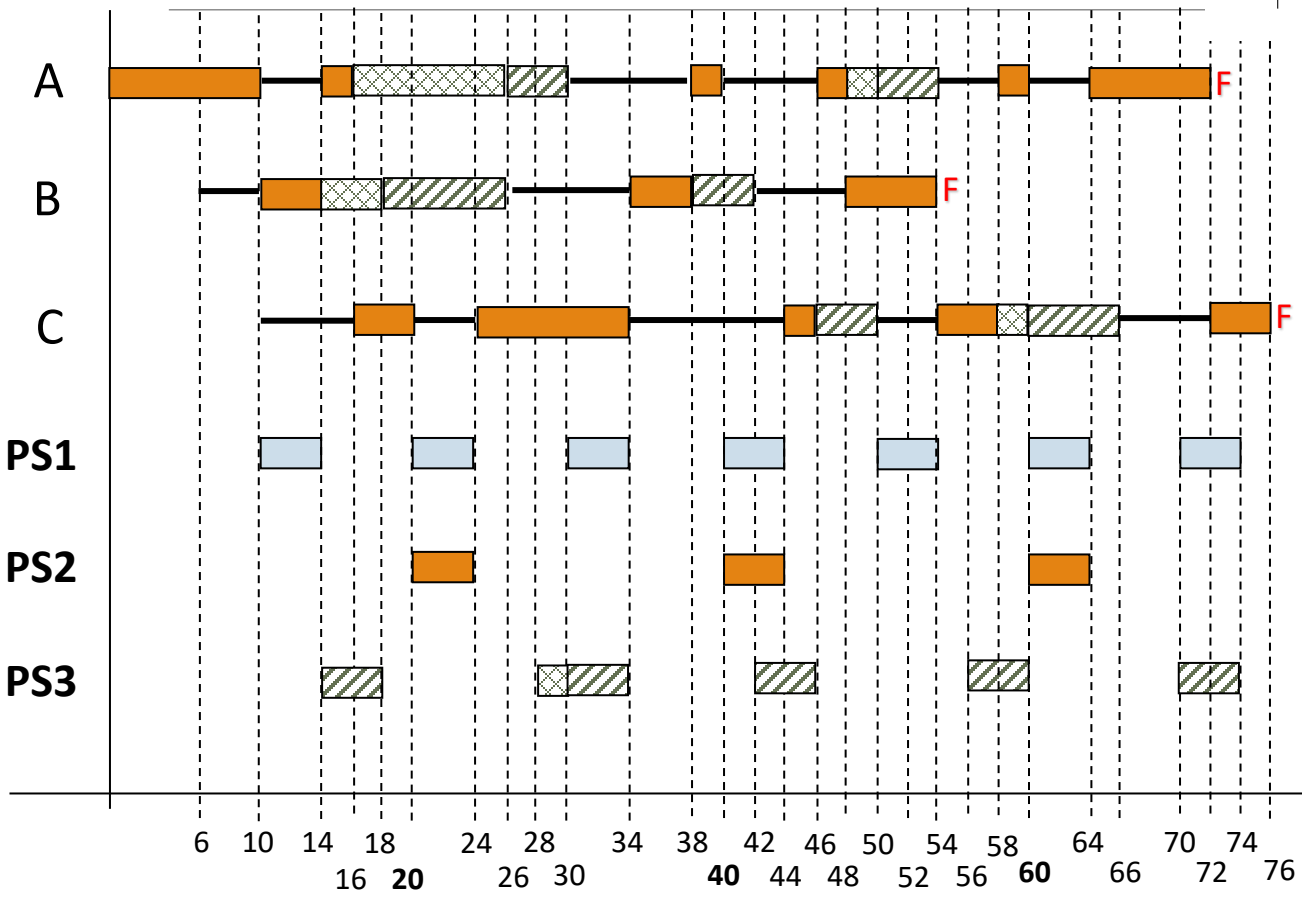
Ejecución CPU2

E/S

Espera E/S

Espera CPU





Ejecución CPU1

Ejecución CPU2

E/S

Espera E/S

Espera CPU

P	T.Resp	T.Ret	T.Esp
A	0	72	26+12
B	4	48	18+4
C	6	66	30+2

Utilización CPU 1 = 100%

Utilización CPU 2 = 28%